

A Program Logic for Byzantine-Fault-Tolerant Protocols

YUWEN KUANG, HONGJIN LIANG, and XINYU FENG, State Key Laboratory for Novel Software Technology, Nanjing University, China

Byzantine-fault-tolerant (BFT) protocols allow distributed systems to function correctly even when some of the nodes act maliciously. Byzantine linearizability, as an extension of linearizability in the presence of faulty nodes, is often used as the correctness criterion for BFT protocols. In this paper, we propose a program logic for verifying Byzantine linearizability. We reason about the effects of faulty nodes using a helping mechanism which instruments the protocol code with abstract operations that model the faulty nodes' behaviors in the views of correct nodes. We prove the soundness of our logic by establishing a contextual refinement that is equivalent to Byzantine linearizability. We apply our logic to verify two typical BFT protocols.

ACM Reference Format:

Yuwen Kuang, Hongjin Liang, and Xinyu Feng. 2025. A Program Logic for Byzantine-Fault-Tolerant Protocols. 1, 1 (November 2025), 124 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Byzantine-fault-tolerant (BFT) protocols are distributed protocols that can keep functioning correctly in the presence of faulty nodes which may exhibit arbitrary behaviors (i.e., Byzantine faults). BFT protocols are increasingly important. They serve as key components in decentralized systems with untrusted nodes, such as blockchains, and have also been used in distributed file systems, firewalls, key-value stores, etc.

BFT protocols are notoriously complex and error-prone. They usually contain multiple asynchronous rounds of message passing, with sophisticated mechanisms to handle the possibly malicious messages sent from faulty nodes. To ensure correctness of the protocols, their designers struggle to analyze various concurrent execution scenarios, but subtle errors may be lurking. For instance, the famous Zyzzyva protocol [Kotla et al. 2010] was found to be buggy by Abraham et al. [2017].

It is even unclear what it means for a BFT protocol to be correct. On the one hand, protocol designers often create their own definitions of correctness, which are tied to specific functionality and not generally applicable. On the other hand, linearizability [Herlihy and Wing 1990] is commonly accepted as a standard correctness criterion for concurrent objects. It relates the concurrent execution of object implementations (i.e. the *concrete* level) to the execution of their sequential specifications (i.e. the *abstract* level), but the executions assume the absence of faulty clients (that is, all clients access the shared object through the object methods only).

As a result, people propose correctness criteria [Hendricks 2009; Liskov and Rodrigues 2006; Malkhi et al. 1998] that extend linearizability to take faulty nodes into account. At the core of their definitions is the safety requirement, which we will call “*Byzantine linearizability*” in this paper. It says that 1) linearizability holds in the absence of faulty nodes; and 2) the abstract effects of operations from faulty nodes are *harmless* from the view of correct nodes. Harmlessness means, what correct nodes observe can be explained as if all nodes were correct. It requires the abstract effects of operations from faulty nodes appear to follow the sequential specification.

The main challenge in verifying Byzantine linearizability is to reason about the effects of faulty nodes. On the one hand, the abstract effects of faulty nodes cannot be generated out of thin air. Instead, they must be traceable back to some concrete actions of the faulty nodes. On the other

Authors' Contact Information: Yuwen Kuang, kyw@smail.nju.edu.cn; Hongjin Liang, hongjin@mail.nju.edu.cn; Xinyu Feng, xyfeng@mail.nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China.

2025. ACM XXXX-XXXX/2025/11-ART
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

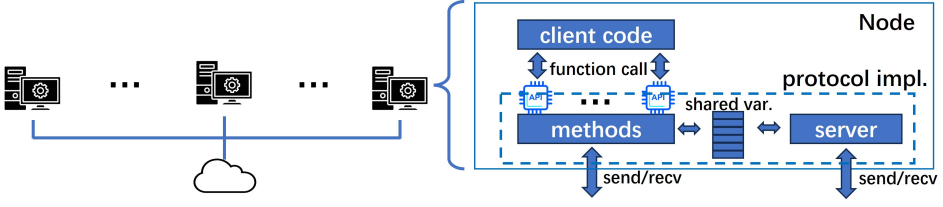


Fig. 1. System model

hand, not all the concrete actions of faulty nodes correspond to abstract effects. Even if there is correspondence, this correspondence may be indirect, in the sense that the abstract operations may take place much later and in a different order than the concrete actions. The reason is that, the messages sent from faulty nodes must be received and processed by correct nodes before they can take effect, but the network may lose messages or transmit them out of order.

In this paper, we propose the first program logic for modular verification of Byzantine linearizability. To address the above challenge, our key idea is to let the correct nodes *help* the faulty nodes. When the correct nodes see the concrete effects of the faulty nodes, they can decide what abstract operations should be used to capture the faulty nodes' effects, and when the abstract operations should be completed. We formalize the idea by introducing new auxiliary commands called **flin**, which are instrumented in the protocol code, allowing the correct nodes to help execute specific abstract operations for faulty nodes.

Our work is inspired by earlier program logics for verifying linearizability of concurrent objects with helping mechanisms, but makes the following new contributions:

- We develop a rely-guarantee-based program logic for verifying Byzantine linearizability of BFT protocols. The key novelty is the helping mechanism for reasoning about the effects of faulty nodes. We introduce auxiliary commands called **flin**, which can be instrumented in the protocol code, so that the correct nodes executing the protocol can help simulate the effects of the faulty nodes. We provide new logic rules for the auxiliary commands.
- We propose a contextual refinement that takes into account faulty clients, and prove that it is equivalent to Byzantine linearizability. Then we prove the soundness of our program logic by establishing the contextual refinement via simulations. As a consequence, our logic enables us to do refinement reasoning in the presence of Byzantine faulty nodes.
- We apply our logic to verify two BFT protocols, including the no-duplicity broadcast algorithm [Toueg 1984], and the single-writer multiple-reader register algorithm (abbreviated as SWMR) [Malkhi et al. 1998].

Outline. We explain our ideas informally in Sec. 2. Then we introduce the language settings, define Byzantine linearizability and prove its equivalence to a contextual refinement in Sec. 3. We present the instrumentation mechanism in Sec. 4 and our program logic in Sec. 5, and use them to verify the broadcast and SWMR protocols in Sec. 6 and Sec. 7. We discuss related work in Sec. 8. This technical report (TR) also contains the full formal details of this work, including semantic rules, logic rules, equivalence proofs, logic soundness proofs and example proofs.

```

1: var MailBox :=  $\emptyset$ , SentTs :=  $\emptyset$ , Initd :=  $\emptyset$ ;
2: operation broadcast( $ts, v$ ) {
3:   if ( $ts \notin \text{SentTs}$ ) then
4:     SentTs := SentTs  $\cup$  { $ts$ };
5:     send(Nodes, (INIT, ( $t$ ,  $ts, v$ )));
6:     return ok;
7:   else
8:     return bad;
9:   end if
10: }
11: operation deliver( $t', ts$ ) {
12:   res := None;
13:   while (res = None) do
14:     res := getEnough(MailBox,  $t', ts, TH$ );
15:   end while
16:   MailBox := setFlag(MailBox, ( $t', ts, res$ ));
17:   return res;
18: }

19: process server {
20:   while (true) do
21:     msg := recv();
22:     if (msg.tag = INIT) then
23:       ( $\_$ , ( $t', ts', v'$ )) := msg;
24:       if (( $t', ts'$ )  $\notin$  Initd) then
25:         Initd := Initd  $\cup$  {( $t', ts'$ )};
26:         send(Nodes, (ECHO, ( $t', ts', v'$ ),  $t$ id));
27:       end if
28:     else if (msg.tag = ECHO) then
29:       ( $\_$ , cont,  $t'$ ) := msg;
30:       if (cont  $\notin$  dom(MailBox)) then
31:         MailBox := MailBox{cont  $\rightsquigarrow$  ( $\emptyset$ , false)};
32:       end if
33:       MailBox := addVote(MailBox, cont,  $t'$ );
34:     end if
35:   end while
36: }

```

Fig. 2. The broadcast protocol

2 Informal Development

2.1 System Model

We first give an overview of the system model. As shown in Fig. 1, there is a set of nodes in the network, some of which are faulty. The nodes communicate through message passing. Messages sent on the network can be lost or duplicated. Also we do not assume message ordering.

BFT protocols run on each correct node (we do not care their execution on faulty nodes because faulty nodes can behave arbitrarily anyway). We view a BFT protocol as a concurrent object. The client on each correct node can only call the APIs of the object methods to access the functionality. Correct implementation of the protocol should behave as a linearizable object from the client point of view, regardless of the behaviors of the faulty nodes.

Implementation of a protocol consists of two parts, a set of methods and a server. The methods can be called by the client code. On each node, there is only one client thread, and it runs in parallel with the server. The server is a background thread that keeps monitoring the network, receiving incoming messages, and sending protocol messages. The methods (running in the client thread) and the server threads may share variables on the local node. They can also send messages to other nodes through the send and recv primitives.

2.2 The Motivating Example: The Broadcast Protocol

We use the broadcast protocol [Toueg 1984] as an example to explain the effects of faulty nodes and how we reason about them.

The protocol provides a no-duplication broadcast abstraction. It has two methods: broadcast(ts, v) allows the current node t to broadcast the message v with timestamp ts , and deliver(t', ts) allows the current node to deliver the message broadcast by node t' with timestamp ts . The no-duplication property ensures that two correct nodes cannot deliver different messages v_1 and v_2 by calling the same deliver(t', ts), even if t' is a faulty node.

In the concrete implementation of the broadcast protocol in Fig. 2, each correct node t maintains a local MailBox which maps each message (t', ts, v) to a pair (votes, flag). Here votes is a set of

nodes which agree on the message v broadcast by node t' with timestamp ts , and flag is boolean and indicates whether the current node t has already delivered (t', ts, v) , so as to prevent t from delivering the same message more than once. Initially MailBox is empty.

The implementation in Fig. 2 proceeds as follows:

- When a node t calls $\text{broadcast}(ts, v)$, it sends a protocol-level message $(\text{INIT}, (t, ts, v))$ to all the nodes (line 5). Here Nodes denotes the set of all node IDs in the system, and tid denotes the current node ID. They are two built-in constants in every node. Note that, the INIT message is at the protocol level and sent directly over the network (by **send**), while the message v in $\text{broadcast}(ts, v)$ is at the application level and is broadcast using the protocol. The protocol also uses the local variable SentTs recording all the timestamps of sent messages to avoid duplicated timestamps (lines 3 and 4).
- When a node receives an INIT message for the first time, it sends an ECHO message to all the nodes (line 26). The $(\text{ECHO}, (t', ts', v'), t)$ message indicates that the current node t votes for the broadcast of v' with timestamp ts' by t' . Any subsequent $(\text{INIT}, (t', ts', _))$ messages that t receives are discarded, with the help of the local variable InitId (lines 24 and 25). This means a correct node can vote for at most one value for each (t', ts') pair.
- When a node receives an $(\text{ECHO}, \text{cont}, t')$ message, it records cont in MailBox (line 31), and add t' into the set $\text{MailBox}(\text{cont}).\text{votes}$ (line 33), indicating that t' votes for cont.
- When a node t calls $\text{deliver}(t', ts)$, it waits until its MailBox contains sufficiently many votes for (t', ts, v) for a specific v (line 14). Here getEnough is defined as follows:

$$\text{getEnough}(\text{MailBox}, t', ts, \text{TH}) \stackrel{\text{def}}{=} \begin{cases} v & \text{if } \exists N. \text{MailBox}(t', ts, v) = (N, \text{false}) \wedge |N| > \text{TH} \\ \text{None} & \text{otherwise} \end{cases}$$

The threshold TH is a predefined real number, $(|\text{Nodes}| + F)/2$, where F is the total number of faulty nodes in the system. The protocol assumes that $F \leq (|\text{Nodes}| - 1)/3$ holds. Finally, the current node sets the flag bit of $\text{MailBox}(t', ts, v)$ (line 16) to indicate that (t', ts, v) is delivered, and returns v .

Byzantine linearizability. As an extension to linearizability, Byzantine linearizability requires that every method call of *correct* clients should take effect instantaneously at the linearization point (LP) between the method invocation and return. That is, correct clients' method calls should correspond to abstract atomic operations which take place at their LPs. But the sequential execution of these abstract atomic operations may not satisfy the sequential specification of the object, due to the interference from faulty clients. Byzantine linearizability requires that such a sequential execution should be able to be made to satisfy the specification, by inserting additional abstract atomic operations. These additional operations can be viewed as the abstract effects of faulty clients.

Why the broadcast protocol is (Byzantine) linearizable, in the absence of faulty nodes. When there are no faulty nodes, we can show that the broadcast protocol is linearizable by finding the LPs. For the successful call of $\text{broadcast}(ts, v)$, the LP occurs no later than the current node sending the INIT message (line 5). For the unsuccessful call, the LP occurs before the node returns (line 8). For the call of $\text{deliver}(t', ts)$, the LP occurs after the current node receives the first ECHO message. At that time, the $\text{broadcast}(ts, _)$ method of node t' must have been linearized, so it is guaranteed that the message being delivered must be the one being broadcast by t' .

However, the above reasoning does not apply when there are faulty nodes. Specifically, for the call of $\text{deliver}(t', ts)$, if t' is a faulty node, it might never have called broadcast, thus it seems unclear what message is supposed to be delivered. Even if t' is a correct node, we cannot simply locate the LP just after the current node receives the first ECHO message, because this message might be sent by faulty nodes with fabricated content in order to mislead the current node.

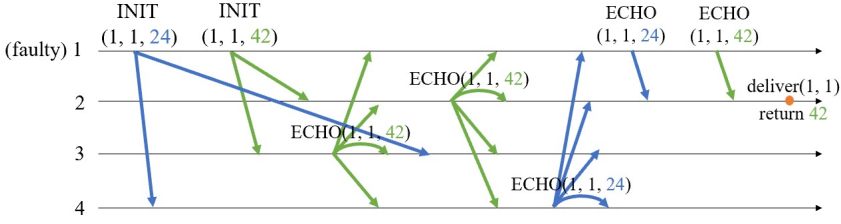


Fig. 3. An execution of the broadcast protocol. Node 2's deliver(1, 1) eventually returns 42.

2.3 What Can Faulty Nodes Do, Concretely and Abstractly?

To understand why the broadcast protocol is Byzantine linearizable in the presence of faulty nodes, we analyze what faulty nodes can do, and how the protocol tolerates their interference.

It might be natural to think that faulty nodes can send arbitrary messages. However, this is often *not* true, because BFT protocols are often accompanied by model assumptions, saying that faulty nodes cannot send messages in the identity of correct nodes. For instance, many protocols assume that messages are authenticated by digital signatures and the signatures cannot be forged, thus faulty nodes cannot send messages with correct nodes' signatures. For the broadcast protocol in Fig. 2, we can simply assume that a faulty node t cannot send an (INIT, $(t', _, _)$) or (ECHO, $_, t'$) message with some correct node ID t' .

Faulty nodes may send messages without complying with the protocol. For instance, they may send ECHO messages without receiving INIT before, and they may send different messages to different nodes. Figure 3 shows an execution of the broadcast protocol, where the system contains 4 nodes, and node 1 is the only faulty node. We draw arrows for messages from sending to receiving, and use different colors to distinguish the values contained in the messages. At first, node 1 sends different (INIT, $(1, 1, _)$) messages to different nodes. Then, the three correct nodes send ECHO messages in accordance with the protocol, among which node 3 echoes 42, the first INIT message it receives. Afterwards, node 1 sends two different ECHO messages to node 2. Finally, when node 2 calls deliver(1, 1), it would get 42, because it receives three ECHO votes for 42, exceeding the threshold $TH = (|Nodes| + F)/2 = (4 + 1)/2 = 2.5$.

The concrete execution in Fig. 3 seems reasonable because it can correspond to the following execution at the *abstract* level: node 1 performs broadcast(1, 42), and then node 2 calls deliver(1, 1) and returns 42. But it is unclear why the faulty node's concrete behaviors can be abstracted as broadcast(1, 42). In particular, since the values that the faulty node sends concretely include both 24 and 42 (and even 24 is sent earlier than 42), it seems hard to determine that the value that the faulty node broadcasts abstractly is 42 rather than 24.

2.4 Our Idea: Helping

To determine the abstract operations of faulty nodes, we take inspiration from the helping mechanism [Liang and Feng 2013; Turon et al. 2013] in linearizability verification, and view that the correct nodes help determine and complete the abstract operations of faulty nodes.

Specifically, faulty nodes' concrete behaviors do not immediately correspond to abstract operations. Only after they are observed by correct nodes, the faulty nodes' behaviors can take effects and correspond to abstract operations. In other words, the correct nodes determine when the faulty nodes perform abstract operations and what abstract operations they perform. For the execution in Fig. 3, node 2 observes three ECHO votes for 42, so it determines that 42 is the value broadcast by node 1 at the abstract level.

To formulate this idea, we introduce an auxiliary command **flin**(t, γ, v) to be instrumented in the protocol code. Its semantics is that, the current node determines that the abstract operation of the faulty node t is $\gamma(v)$, and helps complete the abstract operation. For the broadcast protocol in Fig. 2, we instrument the following code just before the deliver method returns (where Nbad is the set of faulty node IDs):

if ($t' \in \text{Nbad}$) **then** **flin**(t' , broadcast, (ts, res));

Here, Nbad is a global auxiliary variable that records the set of faulty nodes in the system. Note that introducing Nbad here does *not* mean the correct client needs to know the set of faulty nodes, because Nbad is used only in the auxiliary code, and the auxiliary code is actually *parameterized over* Nbad. That is, Nbad is used symbolically, whose value is universally quantified in the soundness theorem of our proof method. The proof does not need to know the concrete value of Nbad. The key point of this segment of instrumented code is that, only after the current node knows what res is, it can determine and help complete the faulty node's abstract operation by executing **flin**.

Comparison with the helping mechanism in linearizability verification. In linearizable algorithms based on helping (e.g. the elimination stack [Hendler et al. 2004]), threads *cooperate*: one thread A records a descriptor of its intended operation in the shared memory, and then another thread B helps complete the operation when seeing A's descriptor. By contrast, in BFT protocols, faulty nodes *interfere* with correct nodes. That is, faulty nodes may intentionally violate the protocol, and even in this case, correct nodes should be able to interpret the faulty nodes' behaviors as some valid abstract operations. As a result, our helping mechanism requires correct nodes to determine the abstract operations of faulty nodes, as well as to help complete the operations.

3 Byzantine Linearizability and Contextual Refinement

Below we first formalize the language settings, and then define Byzantine linearizability. We also define a notion of contextual refinement, which is equivalent to Byzantine linearizability, allowing us to prove Byzantine linearizability by establishing the contextual refinement.

3.1 Language Settings

As shown in Fig. 4, a program P contains a set of *correct* clients, $C_{t_1}, \dots, C_{t_n}$, running on different nodes $t_1, \dots, t_n$. Each correct client can call the methods declared in the object Π . In addition to the method implementations L (e.g. the left column of Fig. 2), Π also contains the server code C_s (e.g. the right column of Fig. 2) and the assumption ζ on faulty nodes' behaviors. In L , each method is defined as a pair (x, C) , where x is the formal argument and C is the method body.

The client code, server code and method code are all defined as C . The **send**(E_1, E_2) command allows the current node to send message E_2 to all the nodes in the set E_1 , and the $x := \text{recv}()$ command receives a message sent to the current node and stores the message in x . The network is unreliable and may lose or duplicate messages. Besides, a node can do local computation (e.g. $x := E$). Clients can use **print**(E) to produce externally observable events, which will be used to define contextual refinement. We assume that none of the object methods and server code produce external events, nor do they contain method calls.

The faulty nodes' assumption ζ is used to specify what messages *cannot* be sent by faulty nodes. It takes the set of faulty nodes and the current message pool as input. For the broadcast protocol in Fig. 2, faulty nodes cannot send an (INIT, (t' , $_$, $_$)) or (ECHO, $_$, t') message with some correct node ID t' , so its ζ can be defined as follows:

$$\zeta \stackrel{\text{def}}{=} \lambda \mathcal{N}_{\text{bad}}, M. \{(\text{INIT}, (t', _, _)) \mid t' \notin \mathcal{N}_{\text{bad}}\} \cup \{(\text{ECHO}, _, t') \mid t' \notin \mathcal{N}_{\text{bad}}\}$$

(Prog)	$P ::= \text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}$	$t \in \text{NodeID}$	(NodeSet) $N ::= \{t_1, \dots, t_n\}$
(ODecl)	$\Pi ::= (L, C_s, \zeta)$		
(MImpl)	$L ::= \{f_1 \rightsquigarrow (x_1, C_1), \dots, f_k \rightsquigarrow (x_k, C_k)\}$	$f \in \text{MName}$	
(Stmt)	$C ::= \text{skip} \mid \text{send}(E, E) \mid x := \text{recv}() \mid x := E \mid \text{print}(E) \mid x := f(E) \mid \text{return } E$ $\mid C; C \mid \text{if } (B) \text{ then } C \text{ else } C \mid \text{while } (B) \text{ do } C$		
(Expr)	$E ::= x \mid v \mid E + E \mid E \cup E \mid E \cap E \mid E \setminus E \mid \dots$	$x, y \in \text{Var}$	$v \in \text{Val}$
(BExpr)	$B ::= \text{true} \mid \text{false} \mid E > E \mid E < E \mid E = E \mid E \in E \mid B \wedge B \mid B \vee B \mid \neg B \mid \dots$		
(Assum)	$\zeta \in \text{NodeSet} \times \text{MsgPool} \rightarrow \mathcal{P}(\text{Val})$		
(MsgPool)	$M ::= \{(\mathcal{N}_1, v_1), \dots, (\mathcal{N}_n, v_n)\}$		

Fig. 4. Syntax of the programming language

(World)	$W ::= (P, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)) \mid ((L, \zeta), DC, M, \mathcal{N}_{\text{bad}})$	(Mem)	$\sigma \in \text{Var} \rightarrow \text{Val}$
(DistConfig)	$DC ::= \{t_1 \rightsquigarrow NC_1, \dots, t_n \rightsquigarrow NC_n\}$	(CallStack)	$\kappa ::= \circ \mid (x, C)$
(NdConfig)	$NC ::= ((C, C_s), (\sigma_c, \sigma_o, \kappa))$	(ExecCtxt)	$\mathbf{E} ::= [] \mid \mathbf{E}; C$
(Event)	$e ::= (t, f, v) \mid (t, \text{ok}, v) \mid (t, \text{out}, v)$	(ETrace)	$T ::= \epsilon \mid e :: T$

Fig. 5. Concrete machine states

We let ζ take the current message pool M as input, in order to model the protocols which allow faulty nodes to forward existing messages. For instance, the following ζ_{ok} says that faulty nodes cannot send an (OK, t') message, where t' is a correct node ID, *unless* the (OK, t') message has already been sent to some nodes $\mathcal{N}$.

$$\zeta_{\text{ok}} \stackrel{\text{def}}{=} \lambda \mathcal{N}_{\text{bad}}. M. \{(\text{OK}, t') \mid t' \notin \mathcal{N}_{\text{bad}} \wedge \neg \exists \mathcal{N}. (\mathcal{N}, (\text{OK}, t')) \in M\}$$

The message pool M (defined at the bottom of Fig. 4) is a set of $(\mathcal{N}, v)$ pairs, where $\mathcal{N}$ specifies the set of nodes that v is intended to sent to. That is, every **send** $(\mathcal{N}, v)$ command adds a new pair $(\mathcal{N}, v)$ to the current M . But this does not mean that every node in such $\mathcal{N}$ will receive v and receive it only once, because, as we mentioned, the network may lose or duplicate messages.

We define the machine states in Fig. 5. The whole distributed system contains both the correct nodes executing the program **let** Π **in** $C_{t_1} \parallel \dots \parallel C_{t_n}$ and a set of faulty nodes $\mathcal{N}_{\text{bad}}$, assuming that $\mathcal{N}_{\text{bad}}$ is disjoint with the set of correct node IDs $\{t_1, \dots, t_n\}$. On each *correct* node t_i , there are *two* concurrent threads: a client thread executing C_i and a server thread executing the server code C_s in Π . Besides the code to be executed, the node configuration NC also contains the node memory $(\sigma_c, \sigma_o, \kappa)$ shared by the two threads. The client thread C_i can only access the client memory σ_c , unless it calls object methods. Both the method implementations and the server code can access the object memory σ_o . We assume that σ_o always contains the constant variables, tid and Nodes , for the current node ID and the set of all node IDs (including both correct and faulty nodes) respectively. The call stack κ is either empty ($\circ$) when the client thread is not executing a method, or a pair (x, C) recording the caller's variable x to receive the return value and the caller's remaining code C to be executed after the method returns.

We show the operational semantic rules in Fig. 6, Fig. 7 and Fig. 8. We have three kinds of transitions: $\longrightarrow$ for the top-level world transitions, $\longrightarrow_t$ for the transitions of node t , and $\longrightarrow_{\rightarrow_t}$ for the steps of a single thread on node t . We label the transitions with events in order to define Byzantine linearizability and contextual refinement. As defined at the bottom of Fig. 5, an event e is either a method invocation (t, f, v) , a method return (t, ok, v) , or an output (t, out, v) produced by **print** (E) . An event trace T is a finite sequence of events.

$$\begin{array}{c}
\frac{\Pi = (L, C_s, \zeta) \quad \mathcal{N}_{\text{ok}} = \text{dom}(DC) = \{t_1, \dots, t_n\} \quad \forall t_i \in \mathcal{N}_{\text{ok}}, DC(t_i) = ((C_{t_i}, C_s), (\sigma_c, \sigma_o \{\text{tid} \rightsquigarrow i, \text{Nodes} \rightsquigarrow \mathcal{N}_{\text{ok}} \uplus \mathcal{N}_{\text{bad}}\}, \circ))}{(\text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)) \# \rightarrow ((L, \zeta), DC, \emptyset, \mathcal{N}_{\text{bad}})} \text{INIT} \\
\\
\frac{DC(t) = NC \quad L \vdash (NC, M) \xrightarrow{e}_t (NC', M')}{((L, \zeta), DC, M, \mathcal{N}_{\text{bad}}) \xrightarrow{e} ((L, \zeta), DC\{t \rightsquigarrow NC'\}, M', \mathcal{N}_{\text{bad}})} \text{CORRECT-STEP} \\
\\
\frac{\mathcal{N}_{\text{bad}} \neq \emptyset \quad v \notin \zeta(\mathcal{N}_{\text{bad}}, M) \quad M' = M \cup \{(\mathcal{N}, v)\}}{((L, \zeta), DC, M, \mathcal{N}_{\text{bad}}) \mapsto ((L, \zeta), DC, M', \mathcal{N}_{\text{bad}})} \text{FAULTY-STEP}
\end{array}$$

Fig. 6. The rules of concrete operational semantics (world transitions)

$$\begin{array}{c}
\frac{\llbracket E \rrbracket_{\sigma_c} = v}{L \vdash (((\mathbf{E}[\text{print}(E)], C_s), (\sigma_c, \sigma_o, \circ)), M) \xrightarrow{(t, \text{out}, v)}_t (((\mathbf{E}[\text{skip}], C_s), (\sigma_c, \sigma_o, \circ)), M)} \text{PRINT} \\
\\
\frac{}{L \vdash (((\mathbf{skip}; C, C_s), (\sigma_c, \sigma_o, \circ)), M) \rightarrow_t (((C, C_s), (\sigma_c, \sigma_o, \circ)), M)} \text{CSKIP} \\
\\
\frac{L(f) = (y, C_m) \quad \llbracket E \rrbracket_{\sigma_c} = v \quad \sigma'_o = \sigma_o \{y \rightsquigarrow v\} \quad \kappa' = (x, \mathbf{E}[\text{skip}])}{L \vdash (((\mathbf{E}[x := f(E)], C_s), (\sigma_c, \sigma_o, \circ)), M) \xrightarrow{(t, f, v)}_t (((C_m, C_s), (\sigma_c, \sigma'_o, \kappa')), M)} \text{CALL} \\
\\
\frac{\kappa = (x, C) \quad \llbracket E \rrbracket_{\sigma_o} = v'}{L \vdash (((\mathbf{E}[\text{return } E], C_s), (\sigma_c, \sigma_o, \kappa)), M) \xrightarrow{(t, \text{ok}, v')}_t (((C, C_s), (\sigma_c \{x \rightsquigarrow v'\}, \sigma_o, \circ)), M)} \text{RETURN} \\
\\
\frac{(C_m, (\sigma_o, M)) \rightarrow_t (C'_m, (\sigma'_o, M'))}{L \vdash (((C_m, C_s), (\sigma_c, \sigma_o, (x, C))), M) \rightarrow_t (((C'_m, C_s), (\sigma_c, \sigma'_o, (x, C))), M')} \text{OBJECTMETHOD} \\
\\
\frac{(C_s, (\sigma_o, M)) \rightarrow_t (C'_s, (\sigma'_o, M'))}{L \vdash (((C, C_s), (\sigma_c, \sigma_o, \kappa)), M) \rightarrow_t (((C, C'_s), (\sigma_c, \sigma'_o, \kappa)), M')} \text{SERVER}
\end{array}$$

Fig. 7. The rules of concrete operational semantics (node-local transitions)

The program executes concurrently with the faulty nodes. As the FAULTY-STEP rule in Fig. 6 shows, faulty nodes can put an arbitrary message v into the message pool, as long as v is allowed by the assumption ζ . This message is sent to an arbitrary set of nodes $\mathcal{N}$.

On each correct node, we non-deterministically choose either the client thread or the server thread to execute. The OBJECTMETHOD rule shows the step of the client thread when it executes the method code. Here we omit the rule for executing the client code (when $\kappa = \circ$), which is similar to the OBJECTMETHOD rule but uses the client memory σ_c instead of the object memory σ_o . The SERVER rule shows the step of the server thread.

The RECV rule reflects that the network may lose or duplicate messages. It says, when a thread t performs $x := \text{recv}()$, it non-deterministically picks a pair $(\mathcal{N}, v)$ in the current message pool M such that $t \in \mathcal{N}$ (that is, the message is sent to the nodes $\mathcal{N}$ including t), and assigns v to x . The message pool is unchanged in this step, so a thread can receive the same message multiple times. Also, it is possible that not all the messages sent to t can be received. If M contain no messages sent to t , then the execution of t is blocked (i.e. it gets stuck).

$$\begin{array}{c}
\frac{\llbracket E_N \rrbracket_{\sigma_o} = \mathcal{N} \quad \llbracket E \rrbracket_{\sigma_o} = v}{(\mathbf{E}[\mathbf{send}(E_N, E)], (\sigma_o, M)) \longrightarrow_t (\mathbf{E}[\mathbf{skip}], (\sigma_o, M \cup \{(N, v)\}))} \text{SEND} \\
\\
\frac{(\mathcal{N}, v) \in M \quad t \in \mathcal{N} \quad \sigma'_o = \sigma_o \{x \rightsquigarrow v\}}{(\mathbf{E}[x := \mathbf{recv}()], (\sigma_o, M)) \longrightarrow_t (\mathbf{E}[\mathbf{skip}], (\sigma'_o, M))} \text{RECV} \\
\\
\frac{\llbracket E \rrbracket_{\sigma_o} = v}{(\mathbf{E}[x := E], (\sigma_o, M)) \longrightarrow_t (\mathbf{E}[\mathbf{skip}], (\sigma_o \{x \rightsquigarrow v\}, M))} \text{ASSN} \\
\\
\frac{\llbracket B \rrbracket_{\sigma_o} = \mathbf{true}}{(\mathbf{E}[\mathbf{if} (B) \mathbf{then} C_1 \mathbf{else} C_2], (\sigma_o, M)) \longrightarrow_t (\mathbf{E}[C_1], (\sigma_o, M))} \text{IF-TRUE} \\
\\
\frac{\llbracket B \rrbracket_{\sigma_o} = \mathbf{false}}{(\mathbf{E}[\mathbf{if} (B) \mathbf{then} C_1 \mathbf{else} C_2], (\sigma_o, M)) \longrightarrow_t (\mathbf{E}[C_2], (\sigma_o, M))} \text{IF-FALSE} \\
\\
\frac{\llbracket B \rrbracket_{\sigma_o} = \mathbf{true}}{(\mathbf{E}[\mathbf{while} (B) \mathbf{do} C], (\sigma_o, M)) \longrightarrow_t (\mathbf{E}[C; \mathbf{while} (B) \mathbf{do} C], (\sigma_o, M))} \text{WHILE-CONT} \\
\\
\frac{\llbracket B \rrbracket_{\sigma_o} = \mathbf{false}}{(\mathbf{E}[\mathbf{while} (B) \mathbf{do} C], (\sigma_o, M)) \longrightarrow_t (\mathbf{E}[\mathbf{skip}], (\sigma_o, M))} \text{WHILE-END} \\
\\
\frac{}{(\mathbf{skip}; C, (\sigma_o, M)) \longrightarrow_t (C, (\sigma_o, M))} \text{SKIP}
\end{array}$$

Fig. 8. The rules of concrete operational semantics (thread-local transitions)

3.2 Byzantine Linearizability

We write $\Pi \preceq_{\varphi, \eta} \Gamma$ to mean that the object Π is Byzantine linearizable w.r.t. the specification Γ . It is defined using the notion of histories H , which are event traces consisting of only method invocation and return events produced by correct nodes. It requires that each concrete history of the object Π should be able to correspond to some abstract sequential history of the specification Γ , after inserting some additional atomic operation that reflects the effects of faulty nodes.

Below we first define the specification Γ and the two parameters φ and η , then explain Byzantine linearizability of histories (i.e. the notion of history correspondence), and finally define $\Pi \preceq_{\varphi, \eta} \Gamma$.

Specification Γ . As shown in Fig. 9, Γ maps method names to their abstract operations γ . Given a specific node ID, γ atomically transforms an argument value and an initial abstract object to a return value with a resulting abstract object. The abstract object θ is shared among the nodes, and maps program variables to abstract values.

For instance, for the broadcast protocol in Fig. 2, we can define the abstract object as a mapping from a shared variable POOL to a finite set of abstract messages $\{(t_1, ts_1) \rightsquigarrow (v_1, \mathcal{N}_1), \dots, (t_n, ts_n) \rightsquigarrow (v_n, \mathcal{N}_1)\}$. Here for each (t_i, ts_i) , v_i is the broadcast message, and $\mathcal{N}_i$ is the set of nodes where the message has already been delivered. Then the specification Γ_B can be defined as follows:

$$\begin{aligned}
(OSpec) \quad \Gamma &::= \{f_1 \rightsquigarrow \gamma_1, \dots, f_n \rightsquigarrow \gamma_n\} \\
(AbsProg) \quad \mathbb{P} &::= \textbf{with } \Gamma \textbf{ do } \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_n} & (AbsStmnt) \quad \mathbb{C} &::= C \mid \textbf{fexec}(f, v) \mid \textbf{fret}(v) \\
(MSpec) \quad \gamma &\in NodeID \rightarrow Val \times AbsObj \rightarrow Val \times AbsObj & (AbsObj) \quad \theta &\in Var \rightarrow Val
\end{aligned}$$

Fig. 9. Object specifications

$$\begin{aligned}
\Gamma_B(\text{broadcast}) &\stackrel{\text{def}}{=} \lambda t. \lambda((ts, v), \theta). \\
&\quad \begin{cases} (\text{bad}, \theta) & \text{if } (t, ts) \in \text{dom}(\theta(\text{POOL})) \\ (\text{ok}, \theta\{\text{POOL} \rightsquigarrow \theta(\text{POOL})\{(t, ts) \rightsquigarrow (v, \emptyset)\}\}) & \text{otherwise} \end{cases} \\
\Gamma_B(\text{deliver}) &\stackrel{\text{def}}{=} \lambda t. \lambda((t', ts), \theta). \\
&\quad \begin{cases} (v, \theta\{\text{POOL} \rightsquigarrow \theta(\text{POOL})\{(t', ts) \rightsquigarrow (v, \mathcal{N} \uplus \{t\})\}\}) & \text{if } \theta(\text{POOL})(t', ts) = (v, \mathcal{N}) \wedge t \notin \mathcal{N} \\ \text{undefined} & \text{otherwise} \end{cases}
\end{aligned}$$

In detail, when the node t does the abstract operation of $\text{broadcast}(ts, v)$, it adds a new mapping $(t, ts) \rightsquigarrow (v, \emptyset)$ into POOL and returns ok. To make sure that time-stamps are unique, if (t, ts) already exists in POOL, the operation simply returns bad without updating POOL. When the node t does the abstract operation of $\text{deliver}(t', ts)$, it queries POOL, returns the message recorded at (t', ts) , and updates POOL to record that the message is delivered at t . If (t', ts) is not in POOL, or if the message has previously been delivered at t , then the operation is blocked. This example illustrates why γ needs to take the current node ID as a parameter.

As shown in Fig. 9, we write $\mathbb{P}$ for the abstract version of a program, where the correct clients in $\mathbb{P}$ interact with the abstract object specification Γ instead of its implementation Π . The abstract machine state and semantics is shown in Fig. 10 and Fig. 11 respectively. The abstract program still executes concurrently with the faulty nodes. The semantics is almost the same as the concrete language shown in Sec. 3.1, except the following two differences.

- The abstract operation γ is executed when the method is called, which now operates over the abstract object θ instead of over the concrete memory σ_o and message pool M . The abstract operation generates a pair of invocation and return events atomically.
- The only thing that a faulty node t can do is to call a method f (which is chosen non-deterministically from Γ) with its current node ID t and a non-deterministic argument value. It executes $\Gamma(f)$ and generates a pair of invocation and return events as if the node was correct, though the return value is useless and discarded.

Refinement mapping φ . We introduce a refinement mapping $\varphi \in Mem \rightarrow AbsObj$ to relate the initial concrete object to the abstract one. For instance, for the broadcast protocol, we can define φ_B as follows: $\varphi_B(\sigma_o) = \theta$ iff $\sigma_o = \{\text{MailBox} \rightsquigarrow \emptyset, \text{SentTs} \rightsquigarrow \emptyset, \text{Initd} \rightsquigarrow \emptyset\} \wedge \theta = \{\text{POOL} \rightsquigarrow \emptyset\}$.

Node proportional relation η . Usually the correctness of a BFT protocol assumes an upper bound of the ratio of the number of faulty nodes to the total number of nodes. For instance, the broadcast protocol assumes that the number of faulty nodes $F \leq (|Nodes| - 1)/3$. To specify this assumption, we introduce a relation $\eta \subseteq Nat \times Nat$ between the number of correct nodes and the number of faulty nodes. For the broadcast protocol, we can define η_B as follows: $\eta_B \stackrel{\text{def}}{=} \{(n, n') \mid n > 2n'\}$.

Byzantine linearizability of histories. Byzantine linearizability $\Pi \preceq_{\varphi, \eta} \Gamma$ is defined based on the concept of Byzantine linearizability of histories. As we said, histories H are event traces consisting of only method invocation and return events. For Π , only correct nodes can generate invocation and return events. By contrast, for Γ , the semantics we just explained lets both correct nodes and faulty nodes generate pairs of invocation and return events atomically. We say a history H is *sequential* iff the first event of H is an invocation, and each invocation, except possibly the last, is immediately

$$\begin{aligned}
(AbsStk) \text{ } ak &::= \circ \mid (x, \mathbb{C}) \\
(DistLocState) \varrho &::= \{t_1 \rightsquigarrow (\sigma_1, ak_1), \dots, t_n \rightsquigarrow (\sigma_n, ak_n)\} \\
(AbsState) \mathbb{S} &::= (\varrho, \theta) \\
(AbsWorld) \mathbb{W} &::= (\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta_0)) \mid (\mathbb{P}, \mathcal{N}_{\text{bad}}, \mathbb{S})
\end{aligned}$$

Fig. 10. Abstract machine states

$$\begin{aligned}
&\text{ABS-INIT} \\
&\frac{\{t_1, \dots, t_n\} \cap \mathcal{N}_{\text{bad}} = \emptyset \quad \varrho = \{t_1 \rightsquigarrow (\sigma_c, \circ), \dots, t_n \rightsquigarrow (\sigma_c, \circ)\} \quad \mathbb{S} = (\varrho, \theta_0)}{(\text{with } \Gamma \text{ do } \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta_0)) \mapsto (\text{with } \Gamma \text{ do } \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, \mathbb{S})} \\
&\text{ABS-BENIGN-STEP} \\
&\frac{\varrho(t_i) = (\sigma_c, ak) \quad \Gamma \vdash (\mathbb{C}_{t_i}, (\sigma_c, \theta, ak)) \xrightarrow{T} t_i (\mathbb{C}'_{t_i}, (\sigma'_c, \theta', ak')) \quad \varrho' = \varrho \{t_i \rightsquigarrow (\sigma'_c, ak')\}}{(\text{with } \Gamma \text{ do } \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_i} \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\varrho, \theta)) \xrightarrow{T} (\text{with } \Gamma \text{ do } \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}'_{t_i} \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\varrho', \theta'))} \\
&\text{ABS-FAULTY-STEP} \\
&\frac{f \in \text{dom}(\Gamma) \quad t \in \mathcal{N}_{\text{bad}} \quad v \in \text{Val} \quad (v', \theta') = \Gamma(f)(t)(v, \theta)}{(\text{with } \Gamma \text{ do } \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\varrho, \theta)) \xrightarrow{(t, f, v) :: (t, \text{ok}, v') :: \epsilon} (\text{with } \Gamma \text{ do } \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\varrho, \theta'))} \\
&\frac{\llbracket E \rrbracket_{\sigma_c} = v}{\Gamma \vdash (\mathbf{E}[\text{print}(E)], (\sigma_c, \theta, \circ)) \xrightarrow{(t, \text{out}, v)}_t (\mathbf{E}[\text{skip}], (\sigma_c, \theta, \circ))} \text{ABS-PRINT} \\
&\frac{}{\Gamma \vdash (\text{skip}; C, (\sigma_c, \theta, \circ)) \xrightarrow{\epsilon}_t (C, (\sigma_c, \theta, \circ))} \text{ABS-SKIP} \\
&\frac{f \in \text{dom}(\Gamma) \quad \llbracket E \rrbracket_{\sigma_c} = v}{\Gamma \vdash (\mathbf{E}[x := f(E)], (\sigma_c, \theta, \circ)) \xrightarrow{\epsilon}_t (\text{fexec}(f, v), (\sigma_c, \theta, (x, \mathbf{E}[\text{skip}])))} \text{ABS-CALL} \\
&\frac{(v', \theta') = \Gamma(f)(t)(v, \theta)}{\Gamma \vdash (\text{fexec}(f, v), (\sigma_c, \theta, ak)) \xrightarrow{(t, f, v) :: (t, \text{ok}, v') :: \epsilon}_t (\text{fret}(v'), (\sigma_c, \theta', ak))} \text{ABS-EXEC} \\
&\frac{\sigma'_c = \sigma_c \{x \rightsquigarrow v'\} \quad ak = (x, C)}{\Gamma \vdash (\text{fret}(v'), (\sigma_c, \theta, ak)) \xrightarrow{\epsilon}_t (C, (\sigma'_c, \theta, \circ))} \text{ABS-FRET}
\end{aligned}$$

Fig. 11. The rules of abstract operational semantics

followed by a matching return event. Thus, the histories generated by Γ are all sequential, and may contain events of faulty nodes. We write $\mathcal{H}[\![P, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)]\!]$ for the set of histories produced by a concrete program P that uses Π , and $\mathcal{H}[\![\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta)]\!]$ for the abstract counterpart.

In Def. 3.8, we define Byzantine linearizability of histories, $H \preceq^{\mathcal{N}_{\text{ok}}} H'$, where H is a concurrent history produced by a program using Π , H' is an abstract sequential history produced by Γ , and $\mathcal{N}_{\text{ok}}$ is the set of correct nodes in the program. As we said, H contains events of only correct nodes, while H' contains events of faulty nodes as well as those of correct nodes.

Definition 3.1. $T|_t$ is inductively defined as follows:

$$\epsilon|_t \stackrel{\text{def}}{=} \epsilon$$

$$\begin{aligned}
\text{get-id}(e) &\stackrel{\text{def}}{=} \begin{cases} t & \text{if } e = (t, \mathbf{out}, _) \vee e = (t, _, _) \vee e = (t, \mathbf{ok}, _) \\ \text{undefined} & \text{otherwise} \end{cases} \\
\text{is-inv}(e) &\stackrel{\text{def}}{=} \exists t, f, v. e = (t, f, v) \\
\text{is-res}(e) &\stackrel{\text{def}}{=} \exists t, v. e = (t, \mathbf{ok}, v) \\
\text{is-out}(e) &\stackrel{\text{def}}{=} \exists t, v. e = (t, \mathbf{out}, v)
\end{aligned}$$

Fig. 12. Auxiliary definitions for event

$$\begin{aligned}
\mathcal{T}[\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)] &\stackrel{\text{def}}{=} \{T \mid \exists W, W'. (\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)) \mapsto W \wedge W \xrightarrow{T}^* W'\} \\
\mathcal{H}[\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)] &\stackrel{\text{def}}{=} \{\text{get-hist}(T) \mid T \in \mathcal{T}[\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)]\} \\
\mathcal{T}[\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta)] &\stackrel{\text{def}}{=} \{T \mid \exists \mathbb{W}, \mathbb{W}'. (\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta)) \mapsto \mathbb{W} \wedge \mathbb{W} \xrightarrow{T}^* \mathbb{W}'\} \\
\mathcal{H}[\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta)] &\stackrel{\text{def}}{=} \{\text{get-hist}(T) \mid T \in \mathcal{T}[\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta)]\}
\end{aligned}$$

Fig. 13. Trace set

$$(e :: T')|_t \stackrel{\text{def}}{=} \begin{cases} e :: (T')|_t & \text{if } \text{get-id}(e) = t \\ T'|_t & \text{otherwise} \end{cases}$$

Definition 3.2. $\text{get-hist}(T)$ is inductively defined as follows:

$$\begin{aligned}
\text{get-hist}(\epsilon) &\stackrel{\text{def}}{=} \epsilon \\
\text{get-hist}(e :: T') &\stackrel{\text{def}}{=} \begin{cases} e :: \text{get-hist}(T') & \text{if } \text{is-inv}(e) \vee \text{is-res}(e) \\ \text{get-hist}(T') & \text{otherwise} \end{cases}
\end{aligned}$$

Definition 3.3. $\text{get-obsv}(T)$ is inductively defined as follows:

$$\begin{aligned}
\text{get-obsv}(\epsilon) &\stackrel{\text{def}}{=} \epsilon \\
\text{get-obsv}(e :: T') &\stackrel{\text{def}}{=} \begin{cases} e :: \text{get-obsv}(T') & \text{if } \text{is-out}(e) \\ \text{get-obsv}(T') & \text{otherwise} \end{cases}
\end{aligned}$$

Definition 3.4. $W \xrightarrow{T}^k W$ is defined as follows:

$$\frac{}{W \xrightarrow{\epsilon}^0 W} \quad \frac{W \xrightarrow{e} W'' \quad W'' \xrightarrow{T}^k W'}{W \xrightarrow{e::T}^{k+1} W'}$$

Definition 3.5. $\mathbb{W} \xrightarrow{T}^k \mathbb{W}$ is defined as follows:

$$\frac{}{\mathbb{W} \xrightarrow{\epsilon}^0 \mathbb{W}} \quad \frac{\mathbb{W} \xrightarrow{e} \mathbb{W}'' \quad \mathbb{W}'' \xrightarrow{T}^k \mathbb{W}'}{\mathbb{W} \xrightarrow{e::T}^{k+1} \mathbb{W}'}$$

Definition 3.6. $W \xrightarrow{T}^* W' \stackrel{\text{def}}{=} \exists k. W \xrightarrow{T}^k W'$

Definition 3.7. $\mathbb{W} \xrightarrow{T}^* \mathbb{W}' \stackrel{\text{def}}{=} \exists k. \mathbb{W} \xrightarrow{T}^k \mathbb{W}'$

Definition 3.8 (Byzantine linearizability of histories). $H \preceq^{\mathcal{N}_{\text{ok}}} H'$ iff

- (1) $\forall t \in \mathcal{N}_{\text{ok}}. H|_t = H'|_t$;
- (2) there exists an injective function π such that $\forall i. H(i) = H'(\pi(i))$ and $\forall i, j. i < j \wedge \text{is-ret}(H(i)) \wedge \text{is-inv}(H(j)) \implies \pi(i) < \pi(j)$.

Here we write $H(i)$ for the i -th event of H , and $H|_t$ for the sub-history consisting of all the events whose node ID is t . The predicates $\text{is-inv}(e)$ and $\text{is-ret}(e)$ mean that the event e is a method invocation and a return respectively. Then, Def. 3.8 says, when projecting to the correct nodes, H' is a permutation of H , preserving the order of events in the same nodes and the order of non-overlapping method calls.

Byzantine linearizability of objects. Before we define Byzantine linearizability of object, we give definitions for complete history. H is complete iff it is well-formed and every invocation has a matching response (Def. 3.14). We handle the pending invocations in an incomplete history by appending zero or more response event, and drop the remaining pending invocations. The dropping is achieved by truncate function (Def. 3.13).

Definition 3.9 (Sequential history). History H is sequential, denoted $\text{seq}(H)$, is inductively defined as follows:

$$\text{seq}(H) \stackrel{\text{def}}{=} (H = \epsilon) \vee (\exists t, f, v. H = (t, f, v) :: \epsilon) \vee (\exists t, f, v, v', H'. H = (t, f, v) :: (t, \text{ok}, v') :: H' \wedge \text{seq}(H'))$$

Definition 3.10 (Well-formed history). H is *well-formed* iff all sub-history $H|_t$ is sequential:

$$\text{well-formed}(H) \stackrel{\text{def}}{=} \forall t. \text{seq}(H|_t)$$

Definition 3.11 (Extensions of a history).

$$\text{extensions}(H) \stackrel{\text{def}}{=} \{H' \mid \text{well-formed}(H') \wedge \exists H_{ok}. H' = H :: H_{ok} \wedge \forall i. \text{is-res}(H_{ok}(i))\}$$

Definition 3.12 (Match). A response e_2 *matches* an invocation e_1 iff they have the same ID:

$$\text{match}(e_1, e_2) \stackrel{\text{def}}{=} \text{is-inv}(e_1) \wedge \text{is-res}(e_2) \wedge \text{get-id}(e_1) = \text{get-id}(e_2)$$

Definition 3.13 (History truncation).

$$\begin{aligned} \text{truncate}(\epsilon) &\stackrel{\text{def}}{=} \epsilon \\ \text{truncate}(e :: H') &\stackrel{\text{def}}{=} \begin{cases} e :: \text{truncate}(H') & \text{if } (\text{is-res}(e) \vee \exists i. \text{match}(e, H'(i))) \\ \text{truncate}(H') & \text{otherwise} \end{cases} \end{aligned}$$

Definition 3.14 (Completions of history).

$$\text{completions}(H) \stackrel{\text{def}}{=} \{\text{truncate}(H') \mid H' \in \text{extensions}(H)\}$$

Finally we define $\Pi \preceq_{\varphi, \eta} \Gamma$ in Def. 3.15. It requires that all the completed concurrent histories of Π should be Byzantine linearizable w.r.t. some abstract sequential histories produced by Γ . Here we *complete* the pending invocations in a history H following the standard linearizability definition [Herlihy and Wing 1990]: we non-deterministically append zero or more return events to H , and drop the remaining pending invocations. Then we get a set of complete histories, which is denoted by $\text{completions}(H)$.

Definition 3.15 (Byzantine linearizability of objects). $\Pi \preceq_{\varphi, \eta} \Gamma$ iff, for all $n, t_1, \dots, t_n, \mathcal{N}_{ok}, \mathcal{N}_{bad}, C_1, \dots, C_n, \sigma_c, \sigma_o, \theta$, if $\mathcal{N}_{ok} = \{t_1, \dots, t_n\}$, $(n, |\mathcal{N}_{bad}|) \in \eta$ and $\mathcal{N}_{ok} \cap \mathcal{N}_{bad} = \emptyset$, then

$$\begin{aligned} &\forall H. H \in \mathcal{H}[\text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{bad}, (\sigma_c, \sigma_o)] \wedge \varphi(\sigma_o) = \theta \\ &\implies \exists H_c, H'. H_c \in \text{completions}(H) \wedge (H_c \preceq^{\mathcal{N}_{ok}} H') \\ &\quad \wedge \exists C'_1, \dots, C'_n, \sigma'_c. H' \in \mathcal{H}[\text{with } \Gamma \text{ do } C'_{t_1} \parallel \dots \parallel C'_{t_n}, \mathcal{N}_{bad}, (\sigma'_c, \theta)] \end{aligned}$$

3.3 Contextual Refinement and the Equivalence Theorem

We prove that Byzantine linearizability is equivalent to the contextual refinement defined below. Informally, the contextual refinement says, for any correct clients, the program P has no more observable behaviors than the corresponding abstract program $\mathbb{P}$, when executing concurrently

with the same set of faulty nodes $\mathcal{N}_{\text{bad}}$. We write $O[P, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)]$ for the set of observable event traces produced by P . It is defined similarly as $\mathcal{H}[P, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)]$, but now the traces consist of observable events (i.e. output events) only. We define $O[\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta)]$ similarly.

Definition 3.16 (Contextual refinement). $\Pi \sqsubseteq_{\varphi, \eta} \Gamma$ iff, for all $n, t_1, \dots, t_n, \mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}, C_1, \dots, C_n, \sigma_c, \sigma_o, \theta$, if $\mathcal{N}_{\text{ok}} = \{t_1, \dots, t_n\}$, $(n, |\mathcal{N}_{\text{bad}}|) \in \eta$, $\mathcal{N}_{\text{ok}} \cap \mathcal{N}_{\text{bad}} = \emptyset$ and $\varphi(\sigma_o) = \theta$, then

$$O[\text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)] \subseteq O[\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta)].$$

Theorem 1 (Equivalence). $\forall \Pi, \Gamma, \varphi, \eta. \Pi \preceq_{\varphi, \eta} \Gamma \iff \Pi \sqsubseteq_{\varphi, \eta} \Gamma$.

In all proofs in this document, $(p_)$ refers to a certain premise of lemma or theorem being currently proved, where $_$ is a placeholder to specify which premise. For example, for following lemma, $(p2)$ in proof of this lemma refers property2(x).

Lemma 1 (Example lemma). for all x , if

1. property1(x),
2. property2(x),

then property3(x).

3.3.1 Byzantine Linearizability implies Contextual Refinement. To prove this direction we need to prove that for all context, if a concrete execution using Π generates trace T , whose corresponding observable behavior is O and history is H , and H is Byzantine linearizable w.r.t. a legal sequential history H' , then the same context can also generate another trace T' , whose corresponding history happens to be H' , and the observable trace happens to be O . We construct the abstract execution of that context according to the concrete execution and the byzantine linearizable relation between H and H' in following way:

- When a benign node takes a step in the concrete execution:
 - If the step is a client step, we make the same step in abstract side. This could be managed because the effect of a client instruction depends only on the client state in our language model, and the client states are always identical on the concrete and abstract sides.
 - If the step is that client invokes a method, we let the abstract client invoke the method as well.
 - If the step is a method step (except return) or is a server step, we let the abstract side go zero step.
 - If the step is a return step which produces a return event e_r in H , we locate e_r in H' , and for every unprocessed return events (note that the unprocessed event's node ID can be faulty clients' ID) before e_r in H' , execute the corresponding atomic method calls on the abstract side, following the order of H' (until e_r is also produced by the abstract client). And then return the current method at the abstract level.
- When a faulty node takes a step in concrete execution, we let the abstract side go zero step.

Definition 3.17. $\Gamma \triangleright (\theta, H)$ means H is a legal sequential history that satisfies Γ :

$$\frac{}{\Gamma \triangleright (\theta, \epsilon)} \quad \frac{(v', \theta') = \Gamma(f)(t)(v, \theta) \quad \Gamma \triangleright (\theta', H)}{\Gamma \triangleright (\theta, (t, f, v) :: (t, \text{ok}, v') :: H)}$$

Definition 3.18. $\Gamma \blacktriangleright (\theta, H, \theta', H')$ means begin with θ , θ' is the intermediate abstract object state between H and H' , where both H and H' satisfies Γ .

$$\frac{\Gamma \triangleright (\theta, H)}{\Gamma \blacktriangleright (\theta, \epsilon, \theta, H)} \quad \frac{(v', \theta') = \Gamma(f)(t)(v, \theta) \quad \Gamma \blacktriangleright (\theta', H', \theta'', H'')}{\Gamma \blacktriangleright (\theta, (t, f, v) :: (t, \text{ok}, v') :: H', \theta'', H')}$$

$$\begin{array}{c}
\frac{L(f) = (y, C_m) \quad \llbracket E \rrbracket_{\sigma_c} = v \quad \sigma'_o = \sigma_o \{y \rightsquigarrow v\} \quad \kappa' = (x, \mathbf{E}[\mathbf{skip}])}{L \vdash (((\mathbf{E}[x := f(E)], C_s), (\sigma_c, \sigma_o, \circ)), M) \xrightarrow{(t, \mathbf{start}, f, v) :: (t, f, v)}_t (((C_m, C_s), (\sigma_c, \sigma'_o, \kappa')), M)} \text{CALL} \\
\\
\frac{\kappa = (x, C) \quad \llbracket E \rrbracket_{\sigma_o} = v'}{L \vdash (((\mathbf{E}[\mathbf{return} E], C_s), (\sigma_c, \sigma_o, \kappa)), M) \xrightarrow{(t, \mathbf{ok}, v') :: (t, \mathbf{end}, v')}_t (((C, C_s), (\sigma_c \{x \rightsquigarrow v'\}, \sigma_o, \circ)), M)} \text{RETURN} \\
\\
\frac{f \in \text{dom}(\Gamma) \quad \llbracket E \rrbracket_{\sigma_c} = v}{\Gamma \vdash (\mathbf{E}[x := f(E)], (\sigma_c, \theta, \circ)) \xrightarrow{(t, \mathbf{start}, f, v)}_t (\mathbf{fexec}(f, v), (\sigma_c, \theta, (x, \mathbf{E}[\mathbf{skip}])))} \text{ABS-CALL} \\
\\
\frac{\sigma'_c = \sigma_c \{x \rightsquigarrow v'\} \quad ak = (x, C)}{\Gamma \vdash (\mathbf{fret}(v'), (\sigma_c, \theta, ak)) \xrightarrow{(t, \mathbf{end}, v')}_t (C, (\sigma'_c, \theta, \circ))} \text{ABS-FRET}
\end{array}$$

Fig. 14. Rules of new operational semantics with tag events generated

Definition 3.19. $\Gamma \blacktriangleright (H, \theta, H') \stackrel{\text{def}}{=} \exists \theta_o. \Gamma \blacktriangleright (\theta_o, H, \theta, H')$

We overload part of our concrete and abstract semantics in Fig. 14. We introduce two observable events $(t, \mathbf{start}, f, v)$ and $(t, \mathbf{end}, v)$.

Definition 3.20. $\text{last}(T) \stackrel{\text{def}}{=} \begin{cases} e & \text{if } T = e :: \epsilon \\ \text{last}(T') & \text{if } T = e :: T' \wedge T' \neq \epsilon \end{cases}$

We define a relation $\mathfrak{R}$ between concrete world and abstract world, which is determined by ext-linearization relation between histories. It takes five parameters: the past abstract trace T'_1 , the entire abstract trace H' , the past concrete trace T_1 , the future concrete trace T_2 , and object specification Γ .

Definition 3.21. $W \mathfrak{R}_{T_1, T_2, \Gamma}^{T'_1, H'} \mathbb{W}$ holds iff there exists θ s.t. (we assume $\mathcal{N}_{\text{ok}}$ occur in this definition is a set that equals to benign client ID set implicit by W)

1. World configuration relation under T'_1 :
 - If there exists $n, t_1, \dots, t_n, C_{t_1}, \dots, C_{t_n}, \mathcal{N}_{\text{bad}}, \sigma_c, \sigma_o$ s.t. $W = (\mathbf{let} \ \Pi \ \mathbf{in} \ C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o))$, then $\mathbb{W} = (\mathbf{with} \ \Gamma \ \mathbf{do} \ C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta))$
 - If there exists $L, \zeta, DC, M, \mathcal{N}_{\text{bad}}$ s.t. $W = ((L, \zeta), DC, M, \mathcal{N}_{\text{bad}})$, let $\mathcal{N}_{\text{ok}} = \{t_1, \dots, t_n\}$, then there exists $\mathbb{C}_{t_1}, \dots, \mathbb{C}_{t_n}, \varrho, \theta$ s.t. $\mathbb{W} = (\mathbf{with} \ \Gamma \ \mathbf{do} \ \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\varrho, \theta))$, $\text{dom}(\varrho) = \mathcal{N}_{\text{ok}}$ and for all $t \in \mathcal{N}_{\text{ok}}$:
 - * If $DC(i). \kappa = \circ$, then $\mathbb{C}_t = DC(t).C$ and $\varrho(t).ak = \circ$
 - * If there exists x, C s.t. $DC(t). \kappa = (_, x, C)$ and $\text{last}(T'_1|_t) = (t, \mathbf{start}, f, v)$, then $\mathbb{C}_t = \mathbf{fexec}(f, v)$ and $\varrho(t).ak = (x, C)$
 - * If there exists x, C s.t. $DC(t). \kappa = (_, x, C)$ and $\text{last}(T'_1|_t) = (t, \mathbf{ok}, v')$, then $\mathbb{C}_t = \mathbf{fret}(v')$ and $\varrho(t).ak = (x, C)$
2. Trace relation under T'_1 : introduce H, H_1, H'_1 where $\text{get-hist}(T_1) = H_1$, $\text{get-hist}(T_1 :: T_2) = H$ and $\text{get-hist}(T'_1) = H'_1$, we have:
 - $(H'_1 \text{ is the ext-linearization of } H_1) \exists H_{c1}. H_{c1} = \text{completions}(H_1) \wedge H_{c1} \preceq^{\mathcal{N}_{\text{ok}}} H'_1$
 - $(H' \text{ is the ext-linearization of } H) \exists H_c. H_c = \text{completions}(H) \wedge H_c \preceq^{\mathcal{N}_{\text{ok}}} H'$
 - $(\theta \text{ is the middle object in } H') \exists H'_2. H' = H'_1 :: H'_2 \wedge \Gamma \blacktriangleright (H'_1, \theta, H'_2)$
 - $T_1 \setminus H_1 = T'_1 \setminus H'_1$, well-formed(T'_1)

Lemma 2. $\forall \Gamma, \theta, H. (\exists n, t_1, \dots, t_n, C_{t_1}, \dots, C_{t_n}, \sigma_c. H \in \mathcal{H}[\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta)]) \implies \Gamma \triangleright (\theta, H)$

Definition 3.22. $\text{filterBad}(e, \mathcal{N}_{\text{ok}}) \stackrel{\text{def}}{=} \begin{cases} e :: \epsilon & \text{if } \text{get-tid}(e) \in \mathcal{N}_{\text{ok}} \\ \epsilon & \text{otherwise} \end{cases}$

Lemma 3. For all $\Pi, \Gamma, n, t_1, \dots, t_n, C_{t_1}, \dots, C_{t_n}, \sigma_c, \sigma_o, \theta, \mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}, T, H'$ if

1. $\mathcal{N}_{\text{ok}} = \{t_1, \dots, t_n\}$,
2. there exists H, H_c s.t. $\text{get-hist}(T) = H, H_c = \text{completions}(H)$ and $\Gamma \triangleright (\theta, H'), H_c \preceq^{\mathcal{N}_{\text{ok}}} H'$,

then $(\text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)) \mathfrak{R}_{\epsilon, T, \Gamma}^{\epsilon, H'} (\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta))$.

PROOF. Immediately by Def. 3.21. □

Lemma 4. For all $W, \mathbb{W}, k, T_0, T'_0, T, \Gamma, H'_{\text{full}}$, if

1. there exists $n, t_1, \dots, t_n, C_{t_1}, \dots, C_{t_n}, \mathcal{N}_{\text{bad}}, \sigma_c, \sigma_o, W_0$ s.t. $(\text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)) \# \longrightarrow W_0$ and $W_0 \xrightarrow{T_0}^* W$,
2. there exists W' s.t. $W \xrightarrow{T}^k W'$,
3. $W \mathfrak{R}_{T_0, T, \Gamma}^{T'_0, H'_{\text{full}}} \mathbb{W}$,

then there exists $T', \mathbb{W}'$ s.t. $W \xrightarrow{T'}^* \mathbb{W}'$ and $\text{get-obsv}(T') = \text{get-obsv}(T)$.

PROOF. By (p1) we know there exists $L, \zeta, DC, M, \mathcal{N}_{\text{bad}}$ s.t. $W = ((L, \zeta), DC, M, \mathcal{N}_{\text{bad}})$. By (p3) we know there exists $\mathbb{C}_{t_1}, \dots, \mathbb{C}_{t_n}$ s.t. $\mathbb{W} = (\text{with } \Gamma \text{ do } \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\varrho, \theta))$. Note that in following proofs we use $\mathcal{N}_{\text{ok}}$ to refer benign client set of W .

We prove the goal by induction on k . The base case is trivial by letting $T' = \epsilon$ and $\mathbb{W}' = \mathbb{W}$. The inductive case is, we need to prove the goal when there exists k' s.t. $k = k' + 1$. We know there exists T_1, T_2, W'' s.t. $T = T_1 :: T_2, W'' \xrightarrow{T_2}^{k'} W'$ and

$$W \xrightarrow{T_1} W'' \quad (1)$$

By induction hypothesis we only need to prove there exists $\mathbb{W}'', T'_1$ s.t.

$$W'' \mathfrak{R}_{T'_0 :: T_1, T_2, \Gamma}^{T'_0 :: T'_1, H'_{\text{full}}} \mathbb{W}'' \quad (\text{goal1})$$

$$W \xrightarrow{T'_1}^* \mathbb{W}'' \text{ and } \text{get-obsv}(T'_1) = \text{get-obsv}(T_1) \quad (\text{goal2})$$

Case analysis over (1) we have the following cases:

- Benign node t take a client step(not function call): then we know there exists $C, C', C_s, \sigma_c, \sigma'_c, \sigma_o$ s.t.

$$\begin{aligned} DC(t) &= ((C, C_s), (\sigma_c, \sigma_o, \circ)), \\ W'' &= ((L, \zeta), DC\{t \rightsquigarrow ((C', C_s), (\sigma'_c, \sigma_o, \circ))\}, M, \mathcal{N}_{\text{bad}}). \end{aligned}$$

By (p3) we know $\mathbb{C}_t = C$ and $\varrho(t).ak = \circ$. We make abstract side take correspond client step, let

$$\begin{aligned} \mathbb{W}'' &= (\text{with } \Gamma \text{ do } \mathbb{C}_{t_1} \parallel \dots \parallel C' \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\varrho\{t \rightsquigarrow (\sigma'_c, \circ)\}, \theta)), \\ T'_1 &= T_1. \end{aligned}$$

By definition we prove the goal immediately.

- Benign node t take a function call step: then there exists $E, x, f, E, C_m, C_s, \sigma_c, \sigma_o, y, v$ s.t. $L(f) = (y, C_m), \llbracket E \rrbracket_{\sigma_c} = v$,

$$\begin{aligned}
T_1 &= (t, \mathbf{start}, f, v) :: (t, f, v), \\
DC(t) &= ((\mathbf{E}[x := f(E)], C_s), (\sigma_c, \sigma_o, \circ)), \\
W'' &= ((L, \zeta), DC\{t \rightsquigarrow ((C_m, C_s), (\sigma_c, \sigma_o\{y \rightsquigarrow v\}, (x, C)))\}, M, \mathcal{N}_{\text{bad}}).
\end{aligned}$$

We make abstract side take an invocation step, let

$$\begin{aligned}
\mathbb{W}'' &= (\mathbf{with} \Gamma \mathbf{do} \mathbb{C}_{t_1} \parallel \dots \mathbf{fexec}(f, v) \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\varrho, \theta)), \\
T'_1 &= (t, \mathbf{start}, f, v) :: \epsilon.
\end{aligned}$$

Then by definition we prove the goal.

- Benign node t take a return step: then there exists $E, v', C_s, \sigma_c, \sigma_o, x, C$ s.t.

$$\begin{aligned}
T_1 &= (t, \mathbf{ok}, v') :: (t, \mathbf{end}, v'), \\
DC(t) &= ((\mathbf{E}[\mathbf{return} E], C_s), (\sigma_c, \sigma_o, (x, C))), \\
W'' &= ((L, \zeta), DC\{t \rightsquigarrow ((C, C_s), (\sigma_c\{x \rightsquigarrow v'\}, \sigma_o, \circ))\}, M, \mathcal{N}_{\text{bad}}).
\end{aligned}$$

To determine $\mathbb{W}''$ and T'_1 , we discuss on $\text{last}(T'_0|_t)$. By concrete semantic we know there exists f, v s.t. $\text{last}(T_0|_t) = (t, f, v)$. By (p3) we know there exists H_0, H'_0 s.t. $\text{get-hist}(T_0) = H_0$, $\text{get-hist}(T'_0) = H'_0$, $T_0 \setminus H_0 = T'_0 \setminus H'_0$ and $\text{well-formed}(T'_0)$, thus we know $\text{last}(T'_0|_t)$ can only be one of the following:

1. there exists f, v s.t. $\text{last}(T'_0|_t) = (t, \mathbf{start}, f, v)$;
2. $\text{last}(T'_0|_t) = (t, \mathbf{ok}, v')$.

Discuss on cases:

- Case 1: from definition of W we know $DC(t).k \neq \circ$, then by (p3) we know

$$\begin{aligned}
\mathbb{W}.\varrho(t) &= (\sigma_c, (x, C)), \\
\mathbb{W}.\mathbb{C}_t &= \mathbf{fexec}(f, v).
\end{aligned}$$

Also by (p3) we know there exists $H_{\text{full}}, H_0, H'_0$ s.t.

$$\begin{aligned}
\text{get-hist}(T_0 :: T) &= H_{\text{full}} \\
\exists H_{c0}. H_{c0} &= \text{completions}(H_0) \wedge H_{c0} \preceq^{N_{\text{ok}}} H'_0 & (1) \\
\exists H_c. H_c &= \text{completions}(H_{\text{full}}) \wedge H_c \preceq^{N_{\text{ok}}} H'_{\text{full}} & (2) \\
\exists H'. H'_{\text{full}} &= H'_0 :: H' \wedge \Gamma \blacktriangleright (H'_0, \theta, H') & (3)
\end{aligned}$$

Let $T'_1 = H'_p :: (t, \mathbf{end}, v')$, where H'_p is the prefix of H' and the last event of H'_p is the first response event of t in H' . By (3) we know H' is a legal sequential history, thus there exists invocation event $e_{i1}, \dots, e_{ik}$ and response event $e_{r1}, \dots, e_{rk}$ s.t. $H'_p = e_{i1} :: e_{r1} :: \dots :: e_{ik} :: e_{rk}$ (note that $e_{rk} = (t, \mathbf{ok}, v')$). Let their thread IDs be $t_{x1}, \dots, t_{xk}$ (note that $t_{xk} = t$).

Let $\mathbb{W}'' = (\mathbb{P}'', \mathcal{N}_{\text{bad}}, (\varrho'', \theta''))$, where

$$\begin{aligned}
\forall t' \in N_{\text{ok}}. \mathbb{P}''(t') &= \begin{cases} C & \text{if } t' = t \\ \mathbb{P}(t') & \text{if } t' \notin \{\text{get-tid}(e_{il}) \mid l \in \{1, \dots, k\}\} \wedge t' \neq t \\ \mathbf{fret}(e_{r1}.v') & \text{if } t' = \text{get-tid}(e_{il}) \wedge t' \neq t \end{cases} \\
\forall t' \in N_{\text{ok}}. \varrho''(t') &= \begin{cases} (\sigma_c\{x \rightsquigarrow v'\}, \circ) & \text{if } t' = t \\ \varrho(t') & \text{if } t' \neq t \end{cases} \\
\Gamma \blacktriangleright &(\theta, H'_p, \theta', \epsilon)
\end{aligned}$$

To prove (goal1), the world configuration relation is trivial due to definition, we only prove 4 trace relation goals in following: first we introduce H_1, H'_0, H'_1 s.t.

$$H_1 = \text{get-hist}(T_1), \quad H'_0 = \text{get-hist}(T'_0), \quad H'_1 = \text{get-hist}(T'_1).$$

- * To prove $\exists H_{c01}. H_{c01} = \text{completions}(H_0 :: H_1) \wedge H_{c01} \preceq^{N_{\text{ok}}} H'_0 :: H'_1$, let

$$H_{c01} = H_0 :: \text{filterBad}(e_{r1}, N_{\text{ok}}) :: \dots :: \text{filterBad}(e_{rk}, N_{\text{ok}}).$$

Thus to prove the goal we only need to prove the following two goals.

1. All events in $e_{i1}, \dots, e_{ik}$ whose thread ID belongs to $\mathcal{N}_{ok}$ are in H_0 : suppose e_{ij} is not in H_0 , and its thread ID belongs to $\mathcal{N}_{ok}$, then we know e_{ij} is in H_{full} . By (2) we know e_{ij} is after $(t, \mathbf{ok}, v')$ in H'_{full} , which contradict e_{ij} is before e_{rk} in H' .
 2. None of $e_{r1}, \dots, e_{rk}$ are in H_0 : given arbitrary e_{rj} , if the thread ID of e_{rj} belongs to $\mathcal{N}_{bad}$, then by (p1) we know T_0 does not contain events from faulty clients, thus the goal is proved; if belongs to $\mathcal{N}_{ok}$, then by (1) and e_{rj} is not in H'_0 we know e_{rj} is not in H_0 .
- * We have the second goal immediately by (2).
- * To prove $\exists H'_2. H' = H'_0 :: H'_1 :: H'_2 \wedge \Gamma \blacktriangleright (H'_0 :: H'_1, \theta', H'_2)$: By definition of H'_p we know $H'_1 = H'_p$, introduce H'_2 s.t.

$$H' = H'_p :: H'_2$$

Then by $\Gamma \blacktriangleright_1 (\theta, H'_p, \theta', \epsilon)$ and (3) we prove the goal.

- * To prove $T_0 :: T_1 \setminus H_0 :: H_1 = T'_0 :: T'_1 \setminus H'_0 :: H'_1$ and well-formed($T'_0 :: T'_1$), by definition and (p3) we have the goal.

To prove (goal2), as $\text{get-obsv}(T'_1) = \text{get-obsv}(T_1)$ is trivial due to definition, we only need

to prove $\mathbb{W} \xrightarrow{T'_1}^* \mathbb{W}'$, that is to prove for all $t' \in \{t_{x1}, \dots, t_{xk}\}$ and $t' \in \mathcal{N}_{ok}$, their abstract code in $\mathbb{W}$ is a **fexec** statement.

When $t' = t$, we already prove $\mathbb{W}.C_t = \mathbf{fexec}(f, v)$, thus we prove this case; When $t' \neq t$, by (p3) we only need to prove

$$DC(t').\kappa \neq \circ \quad (\text{goal2.1})$$

$$\exists f, v. \text{last}(T'_0|_{t'}) = (t', \mathbf{start}, f, v) \quad (\text{goal2.2})$$

To prove (goal2.1), we know that start event corresponds to some invocation event in T_0 and is equal to $\text{last}(T_0|_{t'})$, thus we know $DC(t').\kappa \neq \circ$. To prove (goal2.2), suppose $\text{last}(T'_0|_{t'})$ is not a start event, then contradict the construction of H' . Thus we conclude this case. Now we conclude discussion of case 1.

- Case 2: by (p3) we know

$$\mathbb{W}.C_t = \mathbf{fret}(v'),$$

$$\mathbb{W}.\varrho(t).ak = (x, C).$$

Let $T'_1 = (t, \mathbf{end}, v') :: \epsilon$ and let $\mathbb{W}'' = (\mathbb{P}'', \mathcal{N}_{bad}, (\varrho'', \theta))$ where:

$$\forall t' \in \mathcal{N}_{ok}. \mathbb{P}''(t') = \begin{cases} C & \text{if } t' = t \\ \mathbb{P}(t') & \text{otherwise} \end{cases}$$

$$\forall t' \in \mathcal{N}_{ok}. \varrho''(t') = \begin{cases} (\sigma_c\{x \rightsquigarrow v'\}, \circ) & \text{if } t' = t \\ \varrho(t') & \text{otherwise} \end{cases}$$

By definition we prove the goal.

- Benign node t take an object method step(not return) or a server step: let $\mathbb{W}'' = \mathbb{W}$ and $T'_1 = \epsilon$, by definition immediately.
- Faulty node t take a step: let $\mathbb{W}'' = \mathbb{W}$ and $T'_1 = \epsilon$, by definition immediately.

□

Lemma 5. For all $W_{init}, \mathbb{W}_{init}, T, \Gamma, H'_{full}$, if

1. there exists $n, t_1, \dots, t_n, C_{t_1}, \dots, C_{t_n}, \mathcal{N}_{bad}, \sigma_c, \sigma_o$ s.t. $W_{init} = (\mathbf{let} \ \Pi \ \mathbf{in} \ C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{bad}, (\sigma_c, \sigma_o))$,
2. $T \in \mathcal{T}[\llbracket W_{init} \rrbracket]$,
3. $W_{init} \mathfrak{R}_{\epsilon, T, \Gamma}^{\epsilon, H'_{full}} \mathbb{W}_{init}$,

then there exists T' s.t. $T' \in \mathcal{T}[\llbracket \mathbb{W}_{init} \rrbracket]$ and $\text{get-obsv}(T') = \text{get-obsv}(T)$.

PROOF. By (p2) we know there exists W_0, W' s.t.

$$W_{\text{init}} \# \longrightarrow W_0 \quad (1)$$

$$W \xrightarrow{T}^* W' \quad (2)$$

By (p1) and (p3) we know there exists θ s.t. $\mathbb{W}_{\text{init}} = (\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta))$. Let $\mathbb{W}_{\text{init}}$ take a init step, we know there exists $\mathbb{W}_0$ s.t. $\mathbb{W}_{\text{init}} \# \longrightarrow \mathbb{W}_0$. By (1), (p1) and (3) we know $W_0 \mathfrak{R}_{\epsilon, T, \Gamma}^{\epsilon, H'_{\text{full}}} \mathbb{W}_0$.

Then apply Lm. 4 we prove the goal. $\square$

Lemma 6 (Byzantine linearizability implies contextual refinement). For all $\Pi, \Gamma, \varphi, \eta$, if $\Pi \preceq_{\varphi, \eta} \Gamma$, then $\Pi \sqsubseteq_{\varphi, \eta} \Gamma$.

PROOF. By Def. 3.16, given arbitrary $t_1, \dots, t_n, C_{t_1}, \dots, C_{t_n}, \mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}, \sigma_c, \sigma_o, \theta, T$ s.t. $\mathcal{N}_{\text{ok}} = \{t_1, \dots, t_n\}$, $(n, |\mathcal{N}_{\text{bad}}|) \in \eta$, $\mathcal{N}_{\text{ok}} \cap \mathcal{N}_{\text{bad}} = \emptyset$, $\varphi(\sigma_o) = \theta$ and

$$T \in \mathcal{T}[\text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)] \quad (1)$$

we need to find some proper T' and prove

$$T' \in \mathcal{T}[\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta)] \quad (\text{goal1})$$

$$\text{get-obsv}(T) = \text{get-obsv}(T') \quad (\text{goal2})$$

Introduce H and let $H = \text{get-hist}(T) = H$. Apply premise $\Pi \preceq_{\varphi, \eta} \Gamma$ we there exists some H_c, H' s.t.

$$H_c \in \text{completions}(H) \quad (2)$$

$$\exists C'_{t_1}, \dots, C'_{t_n}, \sigma'_c. H' \in \mathcal{H}[\text{with } \Gamma \text{ do } C'_{t_1} \parallel \dots \parallel C'_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma'_c, \theta)] \quad (3)$$

$$H_c \preceq^{\mathcal{N}_{\text{ok}}} H' \quad (4)$$

Apply (2) on Lm. 2 we know

$$\Gamma \triangleright (\theta, H') \quad (5)$$

Then apply (2) (4) (5) on Lm. 3 we know

$$(\text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)) \mathfrak{R}_{\epsilon, T, \Gamma}^{\epsilon, H'} (\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta)) \quad (6)$$

By (1), (6) and Lm. 5 we prove (goal1) and (goal2). $\square$

3.3.2 Contextual Refinement implies Byzantine Linearizability. To prove this direction, for all concurrent trace T generated with Π , whose corresponding history is H , we need to find a specific context $(C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}})$ that can generate an observable trace O when using Π , and O approximates H . By contextual refinement, we know O can also be produced by an abstract execution of $(C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}})$ using Γ instead. Then we show that H' generated by this abstract execution is just the legal sequential counterpart of H , i.e. a completion of H is Byzantine linearizable w.r.t. H' .

Definition 3.23 (Approximately equivalence). Observable trace O approximates history H , denoted by $O \approx H$, defined as follows:

$$\frac{}{\epsilon \approx \epsilon} \quad \frac{e \approx e' \quad O \approx H}{e :: O \approx e' :: H}$$

where $e \approx e'$ is defined as follows:

- $(t, \text{out}, (f, v)) \approx (t, f, v)$

- $(t, \mathbf{out}, v) \approx (t, \mathbf{ok}, v)$

On the concrete side, for all history H generated by context $(C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}})$ using Π , we can find a context $(C'_{t_1} \parallel \dots \parallel C'_{t_n}, \mathcal{N}_{\text{bad}})$ and an observable trace O , so that an execution of the context using Γ generates O and $O \approx H$. The first step to formalize the idea is introducing a transformation function $\mathbf{T}$. We construct $(C'_{t_1} \parallel \dots \parallel C'_{t_n}, \mathcal{N}_{\text{bad}})$ by applying $\mathbf{T}$ on each client code $C_{t_1}, \dots, C_{t_n}$, where each method call $x = f(E)$ in t 's client code is transformed into the following code: (assume y_{arg} and y_{res} are unused in original client code)

```

 $y_{\text{arg}} := E;$ 
 $\mathbf{print}((f, y_{\text{arg}}));$ 
 $y_{\text{res}} := f(y_{\text{arg}});$ 
 $\mathbf{print}(y_{\text{res}});$ 
 $x := y_{\text{res}};$ 

```

Note that the argument of method call is recorded in y_{arg} , and the return value is recorded in y_{res} . Also, to eliminate influence of output behavior except that is introduced through method call transformation, we erase print statements in original client code. The program structure and other statements are left unchanged.

We define toHist to convert an observable trace generated by some transformed client code to its history counterpart.

Definition 3.24 (Convert to history).

$$\text{toHist}(T) \stackrel{\text{def}}{=} \begin{cases} (t, f, v) :: \text{toHist}(T) & \text{if } T = (t, \mathbf{out}, (f, v)) :: T' \\ (t, \mathbf{end}, v) :: \text{toHist}(T) & \text{if } T = (t, \mathbf{out}, v) :: T' \\ \epsilon & \text{if } T = \epsilon \end{cases}$$

Lemma 7. $\forall T_m, T. \text{get-obsv}(T_m) \approx \text{get-hist}(T) \implies \text{toHist}(\text{get-obsv}(T_m)) = \text{get-hist}(T)$

Then we formalize the idea that all history generated by a context could have an observable counterpart generated by the transformed context by Lm. 8.

Lemma 8. For all $P, \mathcal{N}_{\text{bad}}, \sigma_c, \sigma_o, T$, if $T \in \mathcal{T}[\![P, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)]\!]$, then there exists σ'_c, T_m s.t.

1. $T_m \in \mathcal{T}[\![\mathbf{T}(P), \mathcal{N}_{\text{bad}}, (\sigma'_c, \sigma_o)]\!]$,
2. $\text{get-obsv}(T_m) \approx \text{get-hist}(T)$.

PROOF. From premise we know there exists some W, k, W' s.t.

$$\begin{aligned} (P, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)) &\# \longrightarrow W \quad (1) \\ W &\xrightarrow{T}^k W' \end{aligned}$$

Induction on k :

- If $k = 0$, then $T = \epsilon$, trivial.
- If $k = k' + 1$, then by (1) we know there exists L, ζ, DC s.t. $W = ((L, \zeta), DC, \emptyset, \mathcal{N}_{\text{bad}})$. From Lm. 9 we know there exists $DC_{\text{trans}} \in \mathbf{T}(DC), W_{\text{trans}}, T_m$ s.t.

$$\begin{aligned} \text{get-obsv}(T_m) &\approx \text{get-hist}(T), \\ ((L, \zeta), DC_{\text{trans}}, \emptyset, \mathcal{N}_{\text{bad}}) &\xrightarrow{T_m}^* W_{\text{trans}}. \end{aligned}$$

Then by definition we prove two goals.

□

Lemma 9. For all $L, \zeta, DC, M, \mathcal{N}_{\text{bad}}, T, k, W', DC_{\text{trans}}$, if

$\mathbf{T}(P) \in \text{Prog}$ is inductively defined as follows:

$$\mathbf{T}(\text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}) \stackrel{\text{def}}{=} \text{let } \Pi \text{ in } \mathbf{T}(C_{t_1}) \parallel \dots \parallel \mathbf{T}(C_{t_n})$$

$\mathbf{T}(C) \in \text{Stmt}$ is defined as follows:

$$\begin{aligned} \mathbf{T}(\text{skip}) &\stackrel{\text{def}}{=} \text{skip} \\ \mathbf{T}(\text{send}(E_1, E_2)) &\stackrel{\text{def}}{=} \text{send}(E_1, E_2) \\ \mathbf{T}(x := \text{recv}()) &\stackrel{\text{def}}{=} x := \text{recv}() \\ \mathbf{T}(x := E) &\stackrel{\text{def}}{=} x := E \\ \mathbf{T}(\text{print}(E)) &\stackrel{\text{def}}{=} \text{skip} \\ \mathbf{T}(x := f(E)) &\stackrel{\text{def}}{=} y_{\text{arg}} := E; \text{print}((f, y_{\text{arg}})); y_{\text{res}} := f(y_{\text{arg}}); \text{print}(y_{\text{res}}); x := y_{\text{res}} \\ \mathbf{T}(\text{return } E) &\stackrel{\text{def}}{=} \text{return } E \\ \mathbf{T}(C_1; C_2) &\stackrel{\text{def}}{=} \mathbf{T}(C_1); \mathbf{T}(C_2) \\ \mathbf{T}(\text{if } (B) \text{ then } C_1 \text{ else } C_2) &\stackrel{\text{def}}{=} \text{if } (B) \text{ then } \mathbf{T}(C_1) \text{ else } \mathbf{T}(C_2) \\ \mathbf{T}(\text{while } (B) \text{ do } C) &\stackrel{\text{def}}{=} \text{while } (B) \text{ do } \mathbf{T}(C) \end{aligned}$$

$\mathbf{T}(E) \in \text{ExecCtxt}$ is inductively defined as follows:

$$\begin{aligned} \mathbf{T}([\]) &\stackrel{\text{def}}{=} [\] \\ \mathbf{T}(E; C) &\stackrel{\text{def}}{=} \mathbf{T}(E); \mathbf{T}(C) \end{aligned}$$

$\mathbf{T}(DC) \in \mathcal{P}(\text{DistConfig})$ is defined as follows:

$$\mathbf{T}(DC) \stackrel{\text{def}}{=} \{DC' \mid DC' = \biguplus_{t \in \text{dom}(DC)} \{t \rightsquigarrow N' \mid N' \in \mathbf{T}(N) \wedge DC(t) = N\} \}$$

$\mathbf{T}(N) \in \mathcal{P}(\text{Node})$ is defined as follows:

$$\mathbf{T}((C, C_s), (\sigma_c, \sigma_o, \kappa)) \stackrel{\text{def}}{=} \{((\mathbf{T}(C), C_s), (\sigma'_c, \sigma_o, \mathbf{T}(\kappa))) \mid \sigma'_c \in \mathbf{T}(\sigma_c)\}$$

$\mathbf{T}(\sigma) \in \text{Mem}$ is defined as follows:

$$\mathbf{T}(\sigma) \stackrel{\text{def}}{=} \{\sigma \uplus \{y_{\text{arg}} \rightsquigarrow v_1, y_{\text{res}} \rightsquigarrow v_2\} \mid v_1, v_2 \in \text{Val}\}$$

$\mathbf{T}(\kappa) \in \text{CallStack}$ is defined as follows:

$$\mathbf{T}(\kappa) \stackrel{\text{def}}{=} \begin{cases} \circ & \text{if } \kappa = \circ \\ (y_{\text{res}}, \mathbf{T}(E)[\text{print}(y_{\text{res}}); x := y_{\text{res}}]) & \text{if } \kappa = (x, E[\text{skip}]) \end{cases}$$

Fig. 15. Correspondence for code and states

1. $((L, \zeta), DC, M, \mathcal{N}_{\text{bad}}) \xrightarrow{T}^k W'$,
2. $DC_{\text{trans}} \in \mathbf{T}(DC)$,

then there exists W'_{trans}, T_m s.t.

1. $\text{get-obsv}(T_m) \approx \text{get-hist}(T)$,
2. $((L, \zeta), DC_{\text{trans}}, M, \mathcal{N}_{\text{bad}}) \xrightarrow{T_m}^* W'_{\text{trans}}$.

PROOF. We prove by induction on k , base case is trivial. Now we discuss on inductive case $k = k' + 1$, then there exists DC'', M'', e, T' s.t.

$$((L, \zeta), DC, M, \mathcal{N}_{\text{bad}}) \xrightarrow{e} ((L, \zeta), DC'', M'', \mathcal{N}_{\text{bad}}) \quad (1)$$

$$((L, \zeta), DC'', M'', \mathcal{N}_{\text{bad}}) \xrightarrow{T'}^* W' \quad (2)$$

We prove these goals by further case analysis on (1):

- Benign node t take a client step(not function call): Then we know there exists $C, C_s, \sigma_c, \sigma_o, C'', \sigma_c''$ s.t.

$$\begin{aligned} DC(t) &= ((C, C_s), (\sigma_c, \sigma_o, \circ)), \\ DC'' &= DC\{t \rightsquigarrow ((C'', C_s), (\sigma_c'', \sigma_o, \circ))\}. \\ M'' &= M \end{aligned}$$

By (p2) we know there exists σ_{trans} s.t. $\sigma_{trans} \in \mathbf{T}(\sigma_c)$ and $DC_{trans} = ((\mathbf{T}(C), C_s), (\sigma_{trans}, \sigma_o, \circ))$. Thus we know there exists v, v' s.t. $\sigma_{trans} = \sigma_c \uplus \{y_{arg} \rightsquigarrow v, y_{res} \rightsquigarrow v'\}$.

Let $DC''_{trans} = DC_{trans}\{t \rightsquigarrow ((\mathbf{T}(C''), C_s), (\sigma_c'' \uplus \{y_{arg} \rightsquigarrow v, y_{res} \rightsquigarrow v'\}, \sigma_o, \kappa))\}$. By definition we know $DC''_{trans} \in \mathbf{T}(DC'')$, thus apply induction hypothesis we know there exists W'_{trans}, T' s.t.

$$\begin{aligned} \text{get-obsv}(T'_m) &\approx \text{get-hist}(T'), \\ ((L, \zeta), DC''_{trans}, M'', \mathcal{N}_{bad}) &\xrightarrow{T'_m}^* W'_{trans}. \end{aligned}$$

By definition of approximately equal, we only need to prove there exists T_e s.t.

$$\begin{aligned} \text{get-obsv}(T_e) &\approx \text{get-hist}(e), \\ ((L, \zeta), DC_{trans}, M, \mathcal{N}_{bad}) &\xrightarrow{T_e}^* ((L, \zeta), DC''_{trans}, M'', \mathcal{N}_{bad}). \end{aligned}$$

By semantics and definition trivially.

- Benign node t take a function call step: Then there exists $\mathbf{E}, x, f, E, C_m, C_s, \sigma_c, \sigma_o, y, v_E$ s.t. $L(f) = (y, C_m), \llbracket E \rrbracket_{\sigma_c} = v_E$,

$$\begin{aligned} e &= (t, f, v_E), \\ DC(t) &= ((\mathbf{E}[x := f(E)], C_s), (\sigma_c, \sigma_o, \circ)), \\ DC'' &= DC\{t \rightsquigarrow ((C_m, C_s), (\sigma_c, \sigma_o\{y \rightsquigarrow v_E\}, (x, C)))\}, \\ M'' &= M. \end{aligned}$$

By (p2) we know there exists σ_{trans}, v, v' s.t. $\sigma_{trans} = \sigma_c \uplus \{y_{arg} \rightsquigarrow v, y_{res} \rightsquigarrow v'\}$ and $DC_{trans} = ((\mathbf{T}(\mathbf{E}[x := f(E)]), C_s), (\sigma_{trans}, \sigma_o, \circ))$. Let $DC''_{trans} = DC_{trans}\{t \rightsquigarrow ((C_m, C_s), (\sigma_c \uplus \{y_{arg} \rightsquigarrow v_E, y_{res} \rightsquigarrow v'\}, \sigma_o\{y \rightsquigarrow v_E\}, (y_{res}, \mathbf{T}(\mathbf{E}[\mathbf{print}(y_{res}); x := y_{res}])))\}$.

We know $DC''_{trans} \in \mathbf{T}(DC'')$, then apply induction hypothesis we only need to prove there exists T_e s.t.

$$\begin{aligned} \text{get-obsv}(T_e) &\approx \text{get-hist}(e), \\ ((L, \zeta), DC_{trans}, M, \mathcal{N}_{bad}) &\xrightarrow{T_e}^* ((L, \zeta), DC''_{trans}, M'', \mathcal{N}_{bad}). \end{aligned}$$

By definition we know $\mathbf{T}(\mathbf{E}[x := f(E)])$ is

$$\mathbf{T}(\mathbf{E})[y_{arg} := E; \mathbf{print}((f, y_{arg})); y_{res} := f(y_{arg}); \mathbf{print}(y_{res}); x := y_{res}].$$

We let the transformed world go three step, and let $T_e = (t, \mathbf{out}, (f, v_E)) :: (t, f, v_E)$. Thus by definition we prove two goals.

- Benign node t take a return step: Then there exists $E, v_E, C_s, \sigma_c, \sigma_o, x, \mathbf{E}$ s.t.

$$\begin{aligned} e &= (t, \mathbf{ok}, v_E), \\ DC(t) &= ((\mathbf{E}[\mathbf{return} E], C_s), (\sigma_c, \sigma_o, (x, \mathbf{E}[\mathbf{skip}]))), \\ DC'' &= DC\{t \rightsquigarrow ((\mathbf{E}[\mathbf{skip}], C_s), (\sigma_c\{x \rightsquigarrow v_E\}, \sigma_o, \circ))\}, \\ M'' &= M. \end{aligned}$$

By (p2) we know there exists σ_{trans}, v, v' s.t. $\sigma_{trans} = \sigma_c \uplus \{y_{arg} \rightsquigarrow v, y_{res} \rightsquigarrow v'\}$ and $DC_{trans} = ((\mathbf{T}(\mathbf{E}[\mathbf{return} E]), C_s), (\sigma_{trans}, \sigma_o, (y_{res}, \mathbf{T}(\mathbf{E})[\mathbf{print}(y_{res}); x := y_{res}])))$. Let $DC''_{trans} = DC_{trans}\{t \rightsquigarrow ((\mathbf{T}(\mathbf{E}[\mathbf{skip}]), C_s), ((\sigma_c\{x \rightsquigarrow v_E\}) \uplus \{y_{arg} \rightsquigarrow v, y_{res} \rightsquigarrow v_E\}, \sigma_o, \circ))\}$.

We know $DC''_{trans} \in \mathbf{T}(DC'')$, then apply induction hypothesis we only need to prove there exists T_e s.t.

$$\begin{aligned} & \text{get-obsv}(T_e) \approx \text{get-hist}(e), \\ & ((L, \zeta), DC_{trans}, M, \mathcal{N}_{bad}) \xrightarrow{T_e}^* ((L, \zeta), DC''_{trans}, M'', \mathcal{N}_{bad}). \end{aligned}$$

We let the transformed world go three step, and let $T_e = (t, \mathbf{ok}, v_E) :: (t, \mathbf{out}, v_E)$, thus by definition we prove two goals.

- Benign node t take an object method step (not return) or a server step: Then we know object method code does not contain function call or print statement, thus $\mathbf{T}(DC(t).C) = DC(t).C$, we also know the target object state and call stack are the same as source side, thus we know there is a corresponding target step.
- Faulty node t take a step: Similar to the last case.

□

Lemma 10. For all $t_1, \dots, t_n, C_{t_1}, \dots, C_{t_n}T, \mathcal{N}_{bad}, \sigma_c, \theta, H, H_o, O$, if

1. $T \in \mathcal{T}[\![\mathbf{with} \Gamma \mathbf{do} \mathbf{T}(C_{t_1}) \parallel \dots \parallel \mathbf{T}(C_{t_n}), \mathcal{N}_{bad}, (\sigma_c, \theta)]\!]$,
2. $H = \text{get-hist}(T), O = \text{get-obsv}(T)$,
3. $O \approx H_o$,

then there exists H_c s.t.

1. $H_c \in \text{completions}(H_o)$,
2. $H_c \preceq_{\{t_1, \dots, t_n\}} H$.

PROOF. We first introduce $\mathcal{N}_{ok}$ and let $\mathcal{N}_{ok} = \{t_1, \dots, t_n\}$. From (p1) we know there exists $\mathbb{W}$ s.t.

$$\begin{aligned} & (\mathbf{with} \Gamma \mathbf{do} \mathbf{T}(C_{t_1}) \parallel \dots \parallel \mathbf{T}(C_{t_n}), \mathcal{N}_{bad}, (\sigma_c, \theta)) \twoheadrightarrow \mathbb{W}, \\ & \mathbb{W} \xrightarrow{T}^* _ . \end{aligned}$$

The goal is to find some H_c and prove

$$\begin{aligned} H_c & \in \text{completions}(H_o) & (\text{goal1}) \\ H_c & \preceq_{\{t_1, \dots, t_n\}} H & (\text{goal2}) \end{aligned}$$

By abstract operational semantics we know for all $t \in \mathcal{N}_{ok}$, $O|_t$ and $H|_t$ must satisfy one of the following:

1. $O|_t \approx H|_t$;
2. $\exists v. O|_t :: (t, \mathbf{out}, v) \approx H|_t$;
3. $\exists f, v. O|_t \approx H|_t :: (t, f, v)$.

Since $O \approx H_o$, we know for all $t \in \mathcal{N}_{ok}$, H_o and H must satisfy one of the following:

1. $H_o|_t = H|_t$;
2. $\exists v. H_o|_t :: (t, \mathbf{ok}, v) = H|_t$;
3. $\exists f, v. H_o|_t = H|_t :: (t, f, v)$.

Now we construct H_c . First we construct H_e : for all $t \in \mathcal{N}_{ok}$, if it's the above case 2, then append the corresponding return event to H_o , thus we get result H_e by doing these. We know for all $t \in \mathcal{N}_{ok}$, H_e and H must satisfy one of the following:

1. $H_e|_t = H|_t$;
2. $\exists f, v. H_e|_t = H|_t :: (t, f, v)$.

Let $H_c = \text{truncate}(H_e)$, by definition we have $H_c \in \text{completions}(H_o)$. Since $\forall t \in \mathcal{N}_{\text{ok}}, \text{is-res}(\text{last}(H|_t)) \wedge \text{seq}(H|_t)$, we know for all $t \in \mathcal{N}_{\text{ok}}, H_c|_t = H|_t$.

Now we have proved (*goal1*) and part of (*goal2*), by definition we only need to prove that there exists some bijection π s.t.

$$(\forall i. H_c(i) = H(\pi(i))) \wedge (\forall i, j. (i < j) \wedge (\text{is-res}(H_c(i)) \wedge \text{is-inv}(H_c(j)))) \implies \pi(i) < \pi(j))$$

We informally show that the bijection π implicit in $\forall t \in \mathcal{N}_{\text{ok}}, H_c|_t = H|_t$ preserves the response invocation order.

Let $H_c(i)$ be a response event and $H_c(j)$ be a invocation event. suppose $i < j$, we need to prove $\pi(i) < \pi(j)$. By the construction of H_c from H_o , we know the same response and invocation events are in H_o , and the response happens before the invocation. Let i' and j' be the indices of these events in H_o , and we know $i' < j'$. Since $O \approx H_o$, we know i' and j' are exactly the indices of the corresponding observable events in H_o , and $O(i')$ is a receive event and $O(j')$ is the send event. By abstract operational semantics and the definition of $\mathbf{T}$, we know in H the history response event is before the history invocation event. Thus the goal is proved. $\square$

Now that we have proved trace relation in concrete side (Lm. 8) and in abstract side (Lm. 10), we combine these relations and deduce conclusion in Lm. 11.

Lemma 11 (Contextual refinement implies Byzantine linearizability). For all Π, Γ, φ , if $\Pi \sqsubseteq_{\varphi, \eta} \Gamma$, then $\Pi \preceq_{\varphi, \eta} \Gamma$.

PROOF. By Def. 3.15, given arbitrary $n, t_1, \dots, t_n, C_{t_1}, \dots, C_{t_n}, \mathcal{N}_{\text{bad}}, \sigma_c, \sigma_o, \theta, T, H$, let $P = \mathbf{let} \ \Pi \ \mathbf{in} \ C_{t_1} \parallel \dots \parallel C_{t_n}$, if

$$(n, |\mathcal{N}_{\text{bad}}|) \in \eta \wedge \{t_1, \dots, t_n\} \cap \mathcal{N}_{\text{bad}} = \emptyset \quad (p1)$$

$$T \in \mathcal{T} \llbracket P, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o) \rrbracket \quad (p2)$$

$$\varphi(\sigma_o) = \theta \quad (p3)$$

$$\text{get-hist}(T) = H \quad (p4)$$

then we need to find some $H_c, H', C'_{t_1}, \dots, C'_{t_n}, \sigma'_c$ and prove

$$H_c \in \text{completions}(H) \quad (\text{goal1})$$

$$H' \in \mathcal{H} \llbracket \mathbf{with} \ \Gamma \ \mathbf{do} \ C'_{t_1} \parallel \dots \parallel C'_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma'_c, \theta) \rrbracket \quad (\text{goal2})$$

$$H_c \preceq_{\{t_1, \dots, t_n\}} H' \quad (\text{goal3})$$

Apply Lm. 8 we know there exists σ'_c, T_m s.t.

$$\begin{aligned} T_m &\in \mathcal{T} \llbracket \mathbf{T}(P), \mathcal{N}_{\text{bad}}, (\sigma'_c, \sigma_o) \rrbracket \\ \text{get-obsv}(T_m) &\approx \text{get-hist}(T) \end{aligned} \quad (1)$$

Let $\mathbb{P} = \mathbf{with} \ \Gamma \ \mathbf{do} \ C_{t_1} \parallel \dots \parallel C_{t_n}$, then by Def. 3.16 we know there exists T'_m s.t.

$$T'_m \in \mathcal{T} \llbracket \mathbf{T}(\mathbb{P}), \mathcal{N}_{\text{bad}}, (\sigma'_c, \theta) \rrbracket \quad (2)$$

$$\text{get-obsv}(T_m) = \text{get-obsv}(T'_m) \quad (3)$$

Apply Lm. 10 we know there exists H_c s.t.

$$H_c \in \text{completions}(\text{toHist}(\text{get-obsv}(T'_m))) \quad (4)$$

$$H_c \preceq_{\{t_1, \dots, t_n\}} \text{get-hist}(T'_m) \quad (5)$$

Let $H' = \text{get-hist}(T'_m)$. To prove (*goal1*), apply Lm. 7 on (1) we know $\text{toHist}(\text{get-obsv}(T_m)) = \text{get-hist}(T)$, then by (3)(4) we prove the goal. We have (*goal2*) by (2) immediately. We have (*goal3*) by (5) immediately, thus we conclude all goals. $\square$

$$\begin{array}{ll}
(\text{InsProg}) & \tilde{P} ::= \text{let } (\tilde{\Pi}, \Gamma) \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n} \\
(\text{InsODecl}) & \tilde{\Pi} ::= (\tilde{L}, \tilde{C}_s, \zeta) \quad (\text{InsMImpl}) \tilde{L} ::= \{f_1 \rightsquigarrow (x_1, \tilde{C}_1), \dots, f_n \rightsquigarrow (x_n, \tilde{C}_n)\} \\
(\text{InsStmt}) & \tilde{C} ::= \dot{C} \mid \text{skip} \mid \text{send}(E, E) \mid x := \text{recv}() \mid x := E \mid \text{return } E \mid \langle \tilde{C} \rangle \\
& \mid \tilde{C}; \tilde{C} \mid \text{if } (B) \text{ then } \tilde{C} \text{ else } \tilde{C} \mid \text{while } (B) \text{ do } \tilde{C} \\
(\text{AuxStmt}) & \dot{C} ::= \text{skip} \mid x :=_a E \mid \text{lin}(E) \mid \text{trylin}(E) \mid \text{commit}(p) \mid \text{flin}(E, \gamma, E) \\
& \mid \dot{C}; \dot{C} \mid \text{if } (B) \text{ then } \dot{C} \text{ else } \dot{C} \mid \text{foreach } x \text{ in } E \text{ do } \dot{C}
\end{array}$$

Fig. 16. Syntax of the instrumented code

$$\begin{array}{ll}
(\text{InsWorld}) & \tilde{W} ::= (\tilde{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o, \sigma_a, \theta)) \mid ((\tilde{L}, \zeta, \Gamma), \tilde{DC}, (M, \sigma_a, \Delta), \mathcal{N}_{\text{bad}}) \\
(\text{InsDistConfig}) & \tilde{DC} ::= \{t_1 \rightsquigarrow \tilde{NC}_1, \dots, t_n \rightsquigarrow \tilde{NC}_n\} \\
(\text{InsNdConfig}) & \tilde{NC} ::= ((C, \tilde{C}_s), (\sigma_c, \sigma_o, \circ)) \mid ((\tilde{C}, \tilde{C}_s), (\sigma_c, \sigma_o, (x, C))) \\
(\text{PendState}) & \Delta ::= \{(U_1, \theta_1), \dots, (U_1, \theta_n)\} \\
(\text{PendSet}) & U ::= \{t_1 \rightsquigarrow \Upsilon_1, \dots, t_n \rightsquigarrow \Upsilon_n\} \\
(\text{AbsOp}) & \Upsilon ::= (\gamma, v) \mid (\text{end}, v)
\end{array}$$

Fig. 17. Instrumented machine states

4 Instrumented Code for Verification

As we explained in Sec. 2, our key idea for verifying Byzantine linearizability is to let the correct nodes help determine and complete the abstract operations of faulty nodes. To formulate the idea, we instrument the protocol code with auxiliary statements to specify the helping mechanism. In this section, we explain the syntax and semantics of the instrumented code.

Syntax. As shown in Fig. 16, for a concrete program, we instrument the method implementations and the server code with auxiliary statements, and hence obtain $\tilde{P}$. We also record the object specification Γ in $\tilde{P}$ to help formulate the semantics of the auxiliary statements.

In the auxiliary statements $\dot{C}$, we introduce $\text{lin}(t)$ for completing the abstract operation of the *correct* node t (which would be helping t if t is not the current node), and $\text{flin}(t, \gamma, v)$ for helping complete the operation γ with argument value v for the *faulty* node t . To help faulty nodes, programmers must explicitly specify what abstract operation to do, and the information can be known from the context, as we explained in Sec. 2.4 for the broadcast example. We also introduce two other auxiliary statements to support LPs that depend on future executions.

Note that we allow the instrumented statements $\tilde{C}$ to contain atomic blocks $\langle \tilde{C}_1 \rangle$. This means, the auxiliary statements $\dot{C}$ can be inserted into the same atomic block with the concrete statement. For instance, for the broadcast protocol, since the LP for the successful call of $\text{broadcast}(ts, v)$ occurs no later than line 5, we can instrument line 4 with $\text{lin}(\text{tid})$ using an atomic block: $\langle \text{SentTs} := \text{SentTs} \cup \{ts\}; \text{lin}(\text{tid}); \rangle$. Using atomic blocks can often simplify our verification proof.

Instrumented states and semantics. Before explaining the semantics of the instrumented code, we first show the state model in Fig. 17. Similar to the concrete world defined in Fig. 5, $\tilde{P}$ is executed concurrently with the set of faulty nodes $\mathcal{N}_{\text{bad}}$ and with the initial client memory σ_c and object memory σ_o , but now the initial world also contains the abstract object θ and some auxiliary shared state σ_a for executing the instrumented code. We assume that σ_a always contains two auxiliary variables Nok and Nbad , with constant values, that record the sets of correct nodes and faulty nodes respectively. They are introduced solely for verification purposes. The distributed configuration $\tilde{DC}$ contains all the correct nodes' configurations $\tilde{NC}$, and each $\tilde{NC}$ is defined similarly as the concrete NC in Fig. 5, but using the instrumented object code instead of concrete code.

To support the helping mechanism, we introduce Δ in the world state, which is a nonempty set of (U, θ) pairs shared among the correct nodes, each representing a speculative branch at abstract level. Here, speculative branch in U maps a correct node ID to its remaining abstract operation Υ , which is either (γ, v) (i.e. the operation γ needs to be executed with argument v), or (end, v) (i.e. the operation has been finished with the return value v). At the beginning of the execution of $\tilde{P}$, U is set to $\{(\emptyset, \emptyset)\}$. When a correct node t starts a method call of f with argument v , we update all the current U in Δ to $U\{t \rightsquigarrow (\Gamma(f), v)\}$. The auxiliary statements **lin**(E) and **flin**(E, γ, E) execute the abstract operations in Δ . As shown in Fig. 22, **lin**(E) executes the node E 's pending operation in Δ , and **flin**(E_t, γ, E_v) executes γ for the faulty node E_t . Here we use the auxiliary definitions $\Delta \rightarrow_t \Delta'$ (Def. 4.1) and $\Delta \xrightarrow{(t, \gamma, v)} \Delta'$ (Def. 4.2) to specify the transitions. The former transitions t 's pending operation in all speculative branches to the end state, while the latter does not modify speculation set U since U only tracks the abstract operations of correct nodes.

We assume that Δ is always *domain-exact* (defined as follows), which means all speculations in Δ should describe the same set of node IDs and same domain of abstract object. Any Δ containing a single speculation is domain-exact. This property can be preserved under the step of any command in our instrumented language, thus it is reasonable to assume that this property always holds.

$$\text{DomExact}(\Delta) \stackrel{\text{def}}{=} \forall U, \theta, U', \theta'. (U, \theta) \in \Delta \wedge (U', \theta') \in \Delta \implies \text{dom}(U) = \text{dom}(U') \wedge \text{dom}(\theta) = \text{dom}(\theta')$$

Definition 4.1 (Transition of pending operations).

$$\begin{array}{c} \frac{U(t) = (\text{end}, _)}{(U, \theta) \twoheadrightarrow_t (U, \theta)} \quad \frac{U(t) = (\gamma, v) \quad (v', \theta') = \gamma(t)(v, \theta)}{(U, \theta) \twoheadrightarrow_t (U' \{t \rightsquigarrow (\text{end}, v')\}, \theta')} \\[10pt] \frac{}{\emptyset \rightarrow_t \emptyset} \quad \frac{(U, \theta) \twoheadrightarrow_t (U', \theta') \quad \Delta \rightarrow_t \Delta'}{\{(U, \theta)\} \uplus \Delta \rightarrow_t \{(U', \theta')\} \uplus \Delta'} \end{array}$$

Definition 4.2 (Transition of helping faulty nodes).

$$\frac{}{\emptyset \xrightarrow{(t, \gamma, v)} \emptyset} \quad \frac{\gamma(t)(v, \theta) = (_, \theta') \quad \Delta \xrightarrow{(t, \gamma, v)} \Delta'}{\{(U, \theta)\} \uplus \Delta \xrightarrow{(t, \gamma, v)} \{(U, \theta')\} \uplus \Delta'}$$

We show operational semantic rules in Fig. 18, Fig. 19, Fig. 21, Fig. 22, Fig. 23. We have five kinds of transitions: $\rightsquigarrow$ for top-level world initialization transitions, $\rightsquigarrow$ for top-level world transitions, $\twoheadrightarrow_t$ for transitions of node t , $\hookrightarrow_t$ for the steps of a single thread on node t , $\hookrightarrow$ for the steps of auxiliary commands. Below we introduce the semantics of newly introduced auxiliary commands $\dot{C}$ (also called “dot commands”) and atomic blocks.

The auxiliary command $x :=_a E$ assign evaluation of E to variable x in global auxiliary state. The program aborts if x is a variable in local object state.

lin(E) execute every unfinished abstract operation of ID E in every U in Δ and update the abstract object θ correspondingly. It is executed when we know for sure that a step is the LP. **trylin**(E) introduce uncertainty by adding the executed speculations back into original Δ . We insert **trylin**(E) when we know a step could or could not be the LP. By doing this, both speculative branches are considered. **commit**(p) keeps only the subset of speculations consistent with p and drop the rest. It is executed when we are sure that only speculations satisfying p is possible. **flin**(E, γ, E') execute operation (γ, E') of faulty node E on every θ of U in Δ , then update the abstract object to the result object. The **flin** statement is inserted when we need to adjust the abstract object so that following linearization get reasonable result.

$$\begin{array}{c}
\frac{
\begin{array}{l}
N_{\text{ok}} = \text{dom}(\widetilde{DC}) = \{t_1, \dots, t_n\} \quad \Pi = (\widetilde{L}, \widetilde{C}_s, \zeta) \\
\forall t \in N_{\text{ok}}, \widetilde{DC}(t) = ((C_t, \widetilde{C}_s), (\sigma_c, \sigma_o \{\text{tid} \rightsquigarrow t, \text{Nodes} \rightsquigarrow N_{\text{ok}} \uplus N_{\text{bad}}\}, \circ)) \\
\sigma'_a = \sigma_a \{\text{Nok} \rightsquigarrow N_{\text{ok}}, \text{Nbad} \rightsquigarrow N_{\text{bad}}\} \quad \Delta = \{(\emptyset, \theta)\}
\end{array}
}{
(\text{let } (\widetilde{\Pi}, \Gamma) \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, N_{\text{bad}}, (\sigma_c, \sigma_o, \sigma_a, \theta)) \rightsquigarrow ((\widetilde{L}, \zeta, \Gamma), \widetilde{DC}, (\emptyset, \sigma'_a, \Delta), N_{\text{bad}})
} \text{INS-INIT} \\
\\
\frac{
(\widetilde{L}, \Gamma) \vdash (\widetilde{DC}(t), (M, \sigma_a, \Delta)) \xrightarrow{e}_t (\widetilde{NC}', (M', \sigma'_a, \Delta'))
}{
((\widetilde{L}, \zeta, \Gamma), \widetilde{DC}, (M, \sigma_a, \Delta), N_{\text{bad}}) \xrightarrow{e} ((\widetilde{L}, \zeta, \Gamma), \widetilde{DC}\{t \rightsquigarrow \widetilde{NC}'\}, (M', \sigma'_a, \Delta'), N_{\text{bad}})
} \text{INS-BENIGN-STEP} \\
\\
\frac{
(\widetilde{L}, \Gamma) \vdash (\widetilde{DC}(t), (M, \sigma_a, \Delta)) \xrightarrow{t} \text{abort}
}{
((\widetilde{L}, \zeta, \Gamma), \widetilde{DC}, (M, \sigma_a, \Delta), N_{\text{bad}}) \xrightarrow{} \text{abort}
} \text{INS-BENIGN-ABORT} \\
\\
\frac{
N_{\text{bad}} \neq \emptyset \quad v \notin \zeta(N_{\text{bad}}, M) \quad M' = M \cup \{(N, v)\}
}{
((\widetilde{L}, \zeta, \Gamma), \widetilde{DC}, (M, \sigma_a, \Delta), N_{\text{bad}}) \xrightarrow{} ((\widetilde{L}, \zeta, \Gamma), \widetilde{DC}, (M', \sigma_a, \Delta), N_{\text{bad}})
} \text{INS-FAULTY-STEP}
\end{array}$$

Fig. 18. The rules of instrumented operational semantics (world transitions)

The semantics of **foreach** x in E **do** $\hat{C}$ is that, for each element in set E , independent of the processing order, assign it to x and run $\hat{C}$ exactly once per element. To ensure termination, the execution of $\hat{C}$ must not modify any variables occurring free in E . In this case E remains the same evaluation in each round, thus the **foreach** statement eventually stop. For convenience of designing **foreach** statement's sequential rule, we also forbid $\hat{C}$ from modifying x .

The atomic block $\langle \widetilde{C} \rangle$ finishes $\widetilde{C}$ in a single step. Since the concrete code does not include atomic blocks, the instrumentation must ensure that each atomic block contains at most one concrete operation.

The newly introduced auxiliary commands do not change concrete state σ_o and M , nor do they affect program control flow. Thus the instrumented code does not change program behaviors, unless they are inserted into the wrong places or executed with wrong arguments, leading the instrumented program to abort, but this will be prevented by our program logic.

$$\begin{array}{c}
\frac{\llbracket E \rrbracket_{\sigma_c} = v \quad e = (t, \mathbf{out}, v)}{(\tilde{L}, \Gamma) \vdash (((\mathbf{E}[\mathbf{print}(E)], \tilde{C}_s), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \xrightarrow{e}_t (((\mathbf{E}[\mathbf{skip}], \tilde{C}_s), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta))} \text{INS-PRINT} \\
\\
\frac{}{(\tilde{L}, \Gamma) \vdash (((\mathbf{skip}; C, \tilde{C}_s), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \xrightarrow{}_t (((C, \tilde{C}_s), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta))} \text{INS-CLT-SKIP} \\
\\
\frac{\tilde{L}(f) = (y, \tilde{C}_m) \quad \llbracket E \rrbracket_{\sigma_c} = v \quad \kappa = (x, \mathbf{E}[\mathbf{skip}]) \quad \sigma'_o = \sigma_o \{y \rightsquigarrow v\} \quad \Delta' = \Delta \{t \rightsquigarrow (\Gamma(f), v)\}}{(\tilde{L}, \Gamma) \vdash (((\mathbf{E}[x := f(E)], \tilde{C}_s), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \xrightarrow{(t, f, v)}_t (((\tilde{C}_m, \tilde{C}_s), (\sigma_c, \sigma'_o, \kappa)), (M, \sigma_a, \Delta'))} \text{INS-CALL} \\
\\
\frac{f \in \text{dom}(L) \quad f \notin \text{dom}(\Gamma)}{(\tilde{L}, \Gamma) \vdash (((\mathbf{E}[x := f(E)], \tilde{C}_s), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \xrightarrow{}_t \mathbf{abort}} \text{INS-CALL-ABORT} \\
\\
\frac{(\mathbf{E}_i[\mathbf{return} E], ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{}_t (\mathbf{skip}, ((\sigma_o, M, \sigma_a), \Delta)) \quad v = \llbracket E \rrbracket_{\sigma_o} \quad \kappa = (x, C) \quad \sigma'_c = \sigma_c \{x \rightsquigarrow v\}}{(\tilde{L}, \Gamma) \vdash (((\mathbf{E}_i[\mathbf{return} E], \tilde{C}_s), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) \xrightarrow{(t, \mathbf{ok}, v)}_t (((C, \tilde{C}_s), (\sigma'_c, \sigma_o, \circ)), (M, \sigma_a, \Delta))} \text{INS-ND-RETURN} \\
\\
\frac{\tilde{C} \neq \mathbf{E}_i[\mathbf{return} _] \quad \kappa \neq \circ \quad (\tilde{C}, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{}_t (\tilde{C}', ((\sigma'_o, M', \sigma'_a), \Delta'))}{(\tilde{L}, \Gamma) \vdash (((\tilde{C}, \tilde{C}_s), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) \xrightarrow{}_t (((\tilde{C}', \tilde{C}_s), (\sigma_c, \sigma'_o, \kappa)), (M', \sigma'_a, \Delta'))} \text{INS-OBJECTMETHOD} \\
\\
\frac{(\tilde{C}_s, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{}_t (\tilde{C}'_s, ((\sigma'_o, M', \sigma'_a), \Delta'))}{(\tilde{L}, \Gamma) \vdash (((\tilde{C}, \tilde{C}_s), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) \xrightarrow{}_t (((\tilde{C}, \tilde{C}'_s), (\sigma_c, \sigma'_o, \kappa)), (M', \sigma'_a, \Delta'))} \text{INS-SERVER} \\
\\
\frac{(\tilde{C}_s, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{}_t \mathbf{abort} \quad \text{or} \quad (\kappa \neq \circ \text{ and } (\tilde{C}, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{}_t \mathbf{abort})}{(\tilde{L}, \Gamma) \vdash (((\tilde{C}, \tilde{C}_s), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) \xrightarrow{}_t \mathbf{abort}} \text{INS-ND-ABORT}
\end{array}$$

Fig. 19. The Rules of instrumented operational semantics (node-local transitions)

$$\begin{array}{lcl}
\Delta \{t \rightsquigarrow \Upsilon\} & \stackrel{\text{def}}{=} & \begin{cases} \{ \{U \{t \rightsquigarrow \Upsilon\}, \theta\} \} \cup \Delta' \{t \rightsquigarrow \Upsilon\} & \text{if } \Delta = \{ \{U, \theta\} \} \uplus \Delta' \\ \emptyset & \text{if } \Delta = \emptyset \end{cases} \\
\text{noModifyVars}(X, \tilde{C}) & \stackrel{\text{def}}{=} & \forall x, k, t, \tilde{C}', \Sigma, \Sigma'. x \in X \wedge (\tilde{C}, \Sigma) \xrightarrow{t}_k (\tilde{C}', \Sigma') \implies (x \in \text{dom}(\Sigma.\sigma_o \uplus \Sigma.\sigma_a) \implies \llbracket x \rrbracket_\Sigma = \llbracket x \rrbracket_{\Sigma'} \wedge (x \in \text{dom}(\Sigma.\sigma_o \uplus \Sigma.\sigma_a) \iff x \in \text{dom}(\Sigma'.\sigma_o \uplus \Sigma'.\sigma_a)) \\
\text{specExact}(p) & \stackrel{\text{def}}{=} & \forall s, \Delta, \Delta'. (s, \Delta) \models p \wedge (s, \Delta') \models p \implies \Delta = \Delta' \\
(s, \Delta)|_p = \Delta' & \stackrel{\text{def}}{=} & \exists \Delta'', \Delta_p. (\Delta = \Delta' \uplus \Delta'') \wedge (s, \Delta_p) \models p \wedge \Delta'|_{\text{dom}(\Delta_p)} = \Delta_p \wedge \Delta''|_{\text{dom}(\Delta_p)} \cap \Delta_p = \emptyset \\
\Delta|_D & \stackrel{\text{def}}{=} & \{ (U, \theta) \mid \text{dom}(\{ (U, \theta) \}) = D \wedge \exists U', \theta'. (U \uplus U', \theta \uplus \theta') \in \Delta \} \\
\text{dom}(\Delta) & \stackrel{\text{def}}{=} & (\text{dom}(U), \text{dom}(\theta)) \quad \text{where } (U, \theta) \in \Delta \\
\sigma \perp \sigma' & \stackrel{\text{def}}{=} & \text{dom}(\sigma) \cap \text{dom}(\sigma') = \emptyset
\end{array}$$

Fig. 20. Auxiliary definitions for instrumented operational semantics

$$\begin{array}{c}
\frac{\sigma'_o \perp \sigma_a \quad (C, (\sigma_o, M)) \longrightarrow_t (C', (\sigma'_o, M'))}{(\mathbf{E_i}[C], ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t (\mathbf{E_i}[C'], ((\sigma'_o, M', \sigma_a), \Delta))} \text{INS-CONCRETE} \\
\\
\frac{\neg \sigma'_o \perp \sigma_a \quad (C, (\sigma_o, M)) \longrightarrow_t (C', (\sigma'_o, M'))}{(\mathbf{E_i}[C], ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t \mathbf{abort}} \text{INS-CONCRETE-ABORT} \\
\\
\frac{(\sigma_o, M) \vdash (\dot{C}, (\sigma_a, \Delta)) \hookrightarrow (\dot{C}', (\sigma'_a, \Delta'))}{(\mathbf{E_i}[\dot{C}], ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t (\mathbf{E_i}[\dot{C}'], ((\sigma_o, M, \sigma'_a), \Delta'))} \text{INS-DOT-COMM} \\
\\
\frac{(\sigma_o, M) \vdash (\dot{C}, (\sigma_a, \Delta)) \hookrightarrow \mathbf{abort}}{(\mathbf{E_i}[\dot{C}], ((\sigma_o, M, \sigma'_a), \Delta)) \longmapsto_t \mathbf{abort}} \text{INS-DOT-COMM-ABORT} \\
\\
\frac{\llbracket E \rrbracket_{\sigma_o} = v \quad \Delta = \Delta' * \{(\{t \rightsquigarrow (\text{end}, v)\}, \emptyset)\}}{(\mathbf{E_i}[\mathbf{return} E], ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t (\mathbf{skip}, ((\sigma_o, M, \sigma_a), \Delta))} \text{INS-RETURN} \\
\\
\frac{\llbracket E \rrbracket_{\sigma_o} = v \quad \nexists \Delta'. \Delta = \Delta' * \{(\{t \rightsquigarrow (\text{end}, v)\}, \emptyset)\}}{(\mathbf{E_i}[\mathbf{return} E], ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t \mathbf{abort}} \text{INS-RETURN-ABORT} \\
\\
\frac{(\widetilde{C}, ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t^* (\mathbf{skip}, ((\sigma'_o, M', \sigma'_a), \Delta'))}{(\mathbf{E_i}[\langle \widetilde{C} \rangle], ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t (\mathbf{E_i}[\mathbf{skip}], ((\sigma'_o, M', \sigma'_a), \Delta'))} \text{INS-ATOM} \\
\\
\frac{(\widetilde{C}, ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t^* \mathbf{abort}}{(\mathbf{E_i}[\langle \widetilde{C} \rangle], ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t \mathbf{abort}} \text{INS-ATOM-ABORT} \quad \frac{}{(\mathbf{skip}; \widetilde{C}, \Sigma) \longmapsto_t (\widetilde{C}, \Sigma)} \text{INS-SKIP}
\end{array}$$

Fig. 21. The rules of instrumented operational semantics (thread-local transitions)

Definition 4.3 (Multistep of thread-local instrumented statement transition).

$$\frac{}{(\widetilde{C}, \Sigma) \longmapsto_t^0 (\widetilde{C}, \Sigma)} \quad \frac{(\widetilde{C}, \Sigma) \longmapsto_t (\widetilde{C}'', \Sigma'') \quad (\widetilde{C}'', \Sigma'') \longmapsto_t^k (\widetilde{C}', \Sigma')}{(\widetilde{C}, \Sigma) \longmapsto_t^{k+1} (\widetilde{C}', \Sigma')}$$

Definition 4.4. $(\widetilde{C}, \Sigma) \longmapsto_t^* (\widetilde{C}', \Sigma') \stackrel{\text{def}}{=} \exists k. (\widetilde{C}, \Sigma) \longmapsto_t^k (\widetilde{C}', \Sigma')$

$$\begin{array}{c}
\frac{x \notin \text{dom}(\sigma_o) \quad \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} = v}{(\sigma_o, M) \vdash (\mathbf{E_d}[x :=_a E], (\sigma_a, \Delta)) \hookrightarrow (\mathbf{E_d}[\mathbf{skip}], (\sigma_a \{x \rightsquigarrow v\}, \Delta))} \text{DOT-AUX-ASSN} \\
\\
\frac{\llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} = t \quad \Delta \rightarrow_t \Delta'}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{lin}(E)], (\sigma_a, \Delta)) \hookrightarrow (\mathbf{E_d}[\mathbf{skip}], (\sigma_a, \Delta'))} \text{LIN} \\
\\
\frac{\llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} = t \quad \Delta \rightarrow_t \Delta'}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{trylin}(E)], (\sigma_a, \Delta)) \hookrightarrow (\mathbf{E_d}[\mathbf{skip}], (\sigma_a, \Delta \cup \Delta'))} \text{TRYLIN} \\
\\
\frac{\text{specExact}(p) \quad ((\sigma_o, M, \sigma_a), \Delta)|_p = \Delta'}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{commit}(p)], (\sigma_a, \Delta)) \hookrightarrow (\mathbf{E_d}[\mathbf{skip}], (\sigma_a, \Delta'))} \text{COMMIT} \\
\\
\frac{\llbracket (E_t, E_v) \rrbracket_{\sigma_o \uplus \sigma_a} = (t, v) \quad t \in \llbracket \text{Nbad} \rrbracket_{\sigma_a} \quad \Delta \xrightarrow{(t, \gamma, v)} \Delta'}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{flin}(E_t, \gamma, E_v)], (\sigma_a, \Delta)) \hookrightarrow (\mathbf{E_d}[\mathbf{skip}], (\sigma_a, \Delta'))} \text{FLIN} \\
\\
\frac{\llbracket B \rrbracket_{\sigma_o \uplus \sigma_a} = \mathbf{true}}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{if}(B) \mathbf{then} \dot{C}_1 \mathbf{else} \dot{C}_2], (\sigma_a, \Delta)) \hookrightarrow (\mathbf{E_d}[\dot{C}_1], (\sigma_a, \Delta))} \text{DOT-IF-TRUE} \\
\\
\frac{\llbracket B \rrbracket_{\sigma_o \uplus \sigma_a} = \mathbf{false}}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{if}(B) \mathbf{then} \dot{C}_1 \mathbf{else} \dot{C}_2], (\sigma_a, \Delta)) \hookrightarrow (\mathbf{E_d}[\dot{C}_2], (\sigma_a, \Delta))} \text{DOT-IF-FALSE} \\
\\
\frac{v \in \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} \quad x \notin \text{dom}(\sigma_o) \quad x \notin \text{FV}(E) \quad \text{noModifyVars}(\text{FV}(E) \cup \{x\}, \dot{C}) \quad \dot{C}' = \dot{C}; \mathbf{foreach} \ x \ \mathbf{in} \ E \setminus \{v\} \ \mathbf{do} \ \dot{C} \quad \sigma'_a = \sigma_a \{x \rightsquigarrow v\}}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{foreach} \ x \ \mathbf{in} \ E \ \mathbf{do} \ \dot{C}], (\sigma_a, \Delta)) \hookrightarrow (\mathbf{E_d}[\dot{C}'], (\sigma'_a, \Delta))} \text{DOT-FOR-CONT} \\
\\
\frac{\llbracket E \rrbracket_{\sigma_a \uplus \sigma_o} = \emptyset}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{foreach} \ x \ \mathbf{in} \ E \ \mathbf{do} \ \dot{C}], (\sigma_a, \Delta)) \hookrightarrow (\mathbf{E_d}[\mathbf{skip}], (\sigma_a, \Delta))} \text{DOT-FOR-END} \\
\\
\frac{}{(\sigma_o, M) \vdash (\mathbf{skip}; \dot{C}, (\sigma_a, \Delta)) \hookrightarrow (\dot{C}, (\sigma_a, \Delta))} \text{DOT-SKIP}
\end{array}$$

Fig. 22. The rules of dot commands' operational semantics

$$(\text{DotCommExecContext}) \ \mathbf{E_d} ::= [\] \mid \mathbf{E_d}; \dot{C}$$

$$\begin{array}{c}
\frac{\llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} \text{undefined} \quad \text{or} \quad x \in \text{dom}(\sigma_o)}{(\sigma_o, M) \vdash (\mathbf{E_d}[x :=_a E], (\sigma_a, \Delta)) \hookrightarrow \mathbf{abort}} \text{DOT-AUX-ASSN-ABORT} \\
\\
\frac{\begin{array}{c} \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} \text{undefined} \quad \text{or} \quad \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} \text{is not a nat} \quad \text{or} \\ (\llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} = t \quad \text{and} \quad \nexists \Delta'. \Delta \rightarrow_t \Delta') \end{array}}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{lin}(E)], (\sigma_a, \Delta)) \hookrightarrow \mathbf{abort}} \text{DOT-LIN-ABORT} \\
\\
\frac{\begin{array}{c} \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} \text{undefined} \quad \text{or} \quad \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} \text{is not a nat} \quad \text{or} \\ (\llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} = t \quad \text{and} \quad \nexists \Delta'. \Delta \rightarrow_t \Delta') \end{array}}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{trylin}(E)], (\sigma_a, \Delta)) \hookrightarrow \mathbf{abort}} \text{DOT-TRYLIN-ABORT} \\
\\
\frac{\neg \text{specExact}(p) \quad \text{or} \quad \nexists \Delta'. ((\sigma_o, M, \sigma_a), \Delta)|_p = \Delta'}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{commit}(p)], (\sigma_a, \Delta)) \hookrightarrow \mathbf{abort}} \text{DOT-COMMIT-ABORT} \\
\\
\frac{\begin{array}{c} \llbracket (E_t, E_v) \rrbracket_{\sigma_o \uplus \sigma_a} \text{undefined} \quad \text{or} \quad \llbracket (E_t, E_v) \rrbracket_{\sigma_o \uplus \sigma_a} \text{is not a nat-val pair} \quad \text{or} \\ (\llbracket (E_t, E_v) \rrbracket_{\sigma_o \uplus \sigma_a} = (t, v) \quad \text{and} \quad (t \notin \llbracket \text{Nbad} \rrbracket_{\sigma_o \uplus \sigma_a} \quad \text{or} \quad \nexists \Delta'. \Delta \xrightarrow{(t, \gamma, v)} \Delta')) \end{array}}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{flin}(E_t, \gamma, E_v)], (\sigma_a, \Delta)) \hookrightarrow \mathbf{abort}} \text{DOT-FLIN-ABORT} \\
\\
\frac{\llbracket B \rrbracket_{\sigma_o \uplus \sigma_a} \text{undefined} \quad \text{or} \quad \llbracket B \rrbracket_{\sigma_o \uplus \sigma_a} \text{is not a bool}}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{if} (B) \mathbf{then} \dot{C}_1 \mathbf{else} \dot{C}_2], (\sigma_a, \Delta)) \hookrightarrow \mathbf{abort}} \text{DOT-IF-ABORT} \\
\\
\frac{\begin{array}{c} \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} \text{undefined} \quad \text{or} \quad \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} \text{is not a set} \quad \text{or} \\ (v \in \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} \quad \text{and} \quad (\neg \text{noModifyVars}(\text{FV}(E) \cup \{x\}, \dot{C}) \quad \text{or} \quad x \in \text{dom}(\sigma_o))) \end{array}}{(\sigma_o, M) \vdash (\mathbf{E_d}[\mathbf{foreach} x \mathbf{in} E \mathbf{do} \dot{C}], (\sigma_a, \Delta)) \hookrightarrow \mathbf{abort}} \text{DOT-FOR-END}
\end{array}$$

Fig. 23. The rules of dot commands' operational semantics (*continued*)

$$\begin{aligned}
(\text{Assn}) \quad p, q &::= \text{true} \mid \text{false} \mid B \mid \text{objVar}(x) \mid \text{auxVar}(x) \mid p/x \rightarrow E \mid p/_a x \rightarrow E \\
&\mid \text{isDelta}(\Delta) \mid \text{emp} \mid x \models E \mid E \mapsto (\gamma, E) \mid E \mapsto (\text{end}, E) \mid \text{msgPool}(E) \\
&\mid p \wedge q \mid p * q \mid p \oplus q \mid p \Rightarrow q \mid \exists x. p \mid \forall x. p \mid \neg p \mid \bigwedge_{x \in E} p(x) \\
&\mid \bigoplus_{x \in E} p(x) \mid \bigotimes_{x \in E} p(x) \\
(\text{Action}) \quad R, G &::= p \ltimes q \mid [p] \mid \text{IdVExcept}(X) \mid \text{IdSpec} \mid \text{nonLocal}(R) \mid R \wedge R \mid R \vee R \\
&\mid R \Rightarrow R \mid \exists x. R \mid \forall x. R \\
(\text{RelState}) \quad \Sigma &::= (s, \Delta) \quad s ::= (\sigma_o, M, \sigma_a)
\end{aligned}$$

Fig. 24. Syntax of assertions

5 Program Logic

We develop a program logic to derive the judgment $\vdash \{p, \eta\}(\Pi, \Gamma)$, which establishes the contextual refinement $\Pi \sqsubseteq_{\varphi, \eta} \Gamma$ defined in Def. 3.16. Then, by the equivalence theorem (Thm. 1), we know Byzantine linearizability $\Pi \preceq_{\varphi, \eta} \Gamma$ holds.

Our program logic is based on Rely-Guarantee reasoning [Jones 1983]. We reason about the instrumented code, and interpret the assertions over relational states (i.e. pairs of concrete and abstract states). Below we first explain the assertion language and then the key logic rules.

5.1 Assertions

We show the syntax of assertions in Fig. 24. The state assertions p and q specify the relational states Σ , and the rely and guarantee conditions R and G specify the relational state transitions (Σ, Σ') . Here Σ has two parts: one part contains the current node's concrete object memory σ_o , the message pool M shared among all the nodes, and the auxiliary state σ_a introduced for verification; and the other part is Δ specifying the remaining abstract operations U of correct nodes and the abstract object θ .

We give the semantics of the state assertion in Fig. 25. Note that in this figure, we use $\llbracket E \rrbracket_\Sigma$ as a sugar for $\llbracket E \rrbracket_{\Sigma.s.\sigma_o \uplus \Sigma.s.\sigma_a}$, and we use $\llbracket E \rrbracket_s$ as a sugar for $\llbracket E \rrbracket_{s.\sigma_o \uplus s.\sigma_a}$.

When describing state σ_o and σ_a , distinguishing the variables from concrete state and auxiliary state is necessary. Thus, we introduce assertion $\text{objVar}(x)$ and $\text{auxVar}(x)$, where the former indicates that x is a local (object-level) variable, while the latter denotes that x is a globally shared auxiliary variable.

The assertion substitution is achieved by defining evaluation of p in an updated state, thus we introduce $p/x \rightarrow E$ and $p/_a x \rightarrow E$ in assertion language. The former denotes substituting every free occurrence of the object variable x in p with expression E , while the latter denotes substituting global auxiliary variable x occurring free in p with E .

We view Δ as resource and describe it by introducing state assertions in the style of separation logic. The assertion emp says that there are no abstract operations in Δ and the abstract object is empty. To describe non-empty abstract objects, we use the assertion $x \models E$, which says that the abstract object contains only x with value E . Like emp , $x \models E$ also requires that there are no abstract operations. We describe the abstract operations using $E_1 \mapsto (\gamma, E_2)$ and $E_1 \mapsto (\text{end}, E_2)$. The separating conjunction $p * q$ says that p and q hold over disjoint parts of Δ . Here we define $\Sigma * \Sigma', \Delta_1 * \Delta_2$ and $\Delta_1 \# \Delta_2$ as follows:

$$\begin{aligned}
\Sigma * \Sigma' &\stackrel{\text{def}}{=} \begin{cases} (s, \Delta * \Delta') & \text{if } \Sigma = (s, \Delta) \wedge \Sigma' = (s, \Delta') \wedge \Delta \# \Delta' \\ \text{undefined} & \text{otherwise} \end{cases} \\
\Delta_1 * \Delta_2 &\stackrel{\text{def}}{=} \{(U_1 \uplus U_2, \theta_1 \uplus \theta_2) \mid (U_1, \theta_1) \in \Delta_1 \wedge (U_2, \theta_2) \in \Delta_2\} \\
\Delta_1 \# \Delta_2 &\stackrel{\text{def}}{=} \exists U_1, \theta_1, U_2, \theta_2. (U_1, \theta_1) \in \Delta_1 \wedge (U_2, \theta_2) \in \Delta_2 \wedge U_1 \perp U_2 \wedge \theta_1 \perp \theta_2
\end{aligned}$$

For instance, we can write the assertion $(\text{tid} \mapsto (\text{broadcast}, (ts, v))) * (\text{POOL} \Rightarrow \emptyset)$ to mean that tid needs to finish its abstract operation broadcast at an empty abstract POOL .

We also introduce assertion $\text{isDelta}(\Delta)$ to specify exact current speculation set. However, this assertion is not meant to be part of these pre-/post-conditions in RG judgment because it is difficult to ensure its stability under environmental interference. $\text{isDelta}(\Delta)$ is introduced to provide local guarantees, ensuring that, for example, during a single execution step, the speculation set remains unchanged.

We can also introduce separation logic assertions for concrete states σ . We do not do so, but instead specify the concrete states using boolean expressions B directly, just because we want to keep the assertions simple.

$p \oplus q$ asserts that p and q holds for two sets of speculative branches respectively. This assertion is introduced to represent uncertainty of abstract side. For instance, assertion $x \mapsto (\text{end}, 1) \oplus x \mapsto (\text{end}, 2)$ means that the abstract operation of node x is finished, but the speculation set record two possible LPs, one yielding a return value 1, and the other yielding 2.

We introduce the assertion $\text{msgPool}(E)$ to say that the current message pool M is the same as E . Then we define the shorthand notation $\text{sent}(E_1, E_2)$ to mean that the message E_2 has been sent to *at least* the set of nodes E_1 .

We define the semantics of rely and guarantee assertions that specify transitions (Σ, Σ') in Fig. 26. We use $p \bowtie q$ to mean that the initial state of the transition satisfies p and the resulting state satisfies q , and $[p]$ describes an identity transition at the state satisfying p . Then, we write True and Id to represent arbitrary transitions and arbitrary identity transitions respectively.

Sometimes we want to specify transitions that keep some parts of the states unchanged but allow the other parts to change. To this end, we introduce IdSpec , requiring that the Δ part in Σ is unchanged. We also define $\text{IdV}(X)$ to mean that the values of the variables contained in X (which is a set of variable names) are unchanged. The meaning of $\text{IdVExcept}(X)$ is similar, but requires that the variables except those in X are unchanged. We write IdV short for $\text{IdVExcept}(\emptyset)$, i.e. all the variables are unchanged. Besides, IdM specifies transitions where the message pool is unchanged. We also define $\text{SendAct}(E_1, E_2)$ to represent the transitions of the **send** (E_1, E_2) command.

Finally, we explain the semantics of $\text{nonLocal}(R)$. It is used to transform the actions R of other nodes (i.e. the guarantee of other nodes) to the interference on the current node (i.e. the rely of the current node). More specifically, as we said in Sec. 3.1, in the concrete world, each correct node has its own local object memory, and all the nodes (whether correct or faulty) share the global message pool. Every program variable in the protocol code has a copy in each correct node's object memory. From the perspective of the current node, the executions of other nodes (whether correct or faulty) do not affect the current node's concrete object memory σ_o . Thus, to reason about the protocol code in a node-local way, we interpret assertions on $\Sigma = ((\sigma_o, M, \sigma_a), \Delta)$, where σ_o is the node-local concrete object memory, and M, σ_a and Δ are shared. For the actions R made by the other nodes, we originally interpret R on the other node's concrete object memory, and then use $\text{nonLocal}(R)$ to translate R to the effects on the current node's view: the changes on M, σ_a and Δ are kept, but the current node's σ_o must be unaffected. The idea is formulated as follows:

$$(\Sigma, \Sigma') \models \text{nonLocal}(R) \text{ iff } \Sigma.\sigma_o = \Sigma'.\sigma_o \wedge \exists \sigma, \sigma'. (\Sigma|_{\sigma_o \mapsto \sigma}, \Sigma'|_{\sigma_o \mapsto \sigma'}) \models R.$$

It says, $\text{nonLocal}(R)$ holds if the current node's σ_o is unchanged, and there are some other nodes' transitions satisfying R . Here we write $\Sigma|_{\sigma_o \mapsto \sigma}$ to replace the concrete object memory in Σ by σ , which is defined as follows:

$$\Sigma|_{\sigma_o \mapsto \sigma} \stackrel{\text{def}}{=} ((\sigma, M, \sigma_a), \Delta), \text{ if } \Sigma = ((_, M, \sigma_a), \Delta).$$

$\Sigma \models \text{true}$	always holds
$\Sigma \models \text{false}$	never holds
$((\sigma_o, M, \sigma_a), \Delta) \models \text{objVar}(x)$	iff $x \in \text{dom}(\sigma_o)$
$((\sigma_o, M, \sigma_a), \Delta) \models \text{auxVar}(x)$	iff $x \in \text{dom}(\sigma_a)$
$((\sigma_o, M, \sigma_a), \Delta) \models p/x \rightarrow E$	iff $((\sigma_o\{x \rightsquigarrow \llbracket E \rrbracket_{\sigma_o}\}, M, \sigma_a), \Delta) \models p$
$((\sigma_o, M, \sigma_a), \Delta) \models p/a \rightarrow E$	iff $((\sigma_o, M, \sigma_a\{x \rightsquigarrow \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a}\}), \Delta) \models p$
$((\sigma_o, M, \sigma_a), \Delta) \models B$	iff $\llbracket B \rrbracket_{\sigma_o \uplus \sigma_a} = \text{true}$
$((\sigma_o, M, \sigma_a), \Delta) \models \text{isDelta}(\Delta')$	iff $\Delta = \Delta'$
$((\sigma_o, M, \sigma_a), \Delta) \models \text{emp}$	iff $\Delta = \{(\emptyset, \emptyset)\}$
$((\sigma_o, M, \sigma_a), \Delta) \models x \mapsto E$	iff $\exists v. \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} = v \wedge \Delta = \{(\emptyset, \{x \rightsquigarrow v\})\}$
$((\sigma_o, M, \sigma_a), \Delta) \models E_1 \mapsto (\gamma, E_2)$	iff $\exists t, v. \llbracket (E_1, E_2) \rrbracket_{\sigma_o \uplus \sigma_a} = (t, v) \wedge \Delta = \{(\{t \rightsquigarrow (\gamma, v)\}, \emptyset)\}$
$((\sigma_o, M, \sigma_a), \Delta) \models E_1 \mapsto (\text{end}, E_2)$	iff $\exists t, v. \llbracket (E_1, E_2) \rrbracket_{\sigma_o \uplus \sigma_a} = (t, v) \wedge \Delta = \{(\{t \rightsquigarrow (\text{end}, v)\}, \emptyset)\}$
$((\sigma_o, M, \sigma_a), \Delta) \models \text{msgPool}(E)$	iff $M = \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a}$
$\Sigma \models \neg p$	iff $\neg(\Sigma \models p)$
$\Sigma \models p \vee q$	iff $\Sigma \models p \vee \Sigma \models q$
$\Sigma \models p \wedge q$	iff $\Sigma \models p \wedge \Sigma \models q$
$\Sigma \models p \Rightarrow q$	iff $(\Sigma \models p) \Rightarrow (\Sigma \models q)$
$\Sigma \models p * q$	iff $\exists \Sigma_1, \Sigma_2. \Sigma = \Sigma_1 * \Sigma_2 \wedge \Sigma_1 \models p \wedge \Sigma_2 \models q$
$((\sigma_o, M, \sigma_a), \Delta) \models p \oplus q$	iff $\exists \Delta_1, \Delta_2. \Delta = \Delta_1 \cup \Delta_2 \wedge ((\sigma_o, M, \sigma_a), \Delta_1) \models p \wedge ((\sigma_o, M, \sigma_a), \Delta_2) \models q$
$((\sigma_o, M, \sigma_a), \Delta) \models \exists x. p$	iff $x \notin \text{dom}(\sigma_a) \wedge \exists v. ((\sigma_o \uplus \{x \rightsquigarrow v\}, M, \sigma_a), \Delta) \models p$
$((\sigma_o, M, \sigma_a), \Delta) \models \forall x. p$	iff $x \notin \text{dom}(\sigma_a) \wedge \forall v. ((\sigma_o \uplus \{x \rightsquigarrow v\}, M, \sigma_a), \Delta) \models p$
$\Sigma \models \bigwedge_{x \in E} p(x)$	iff $\exists n, v_1, \dots, v_n. \llbracket E \rrbracket_\Sigma = \{v_1, \dots, v_n\} \wedge \forall i \in [1..n]. \Sigma \models p(v_i)$
$(s, \Delta) \models \bigoplus_{x \in E} p(x)$	iff $\exists n, v_1, \dots, v_n, \Delta_1, \dots, \Delta_n. \llbracket E \rrbracket_s = \{v_1, \dots, v_n\} \wedge \Delta = \bigoplus_{i \in [1..n]} \Delta_i$ $\wedge \forall i \in [1..n]. (s, \Delta_i) \models p(v_i)$
$(s, \Delta) \models \otimes_{x \in E} p(x)$	iff $\exists n, v_1, \dots, v_n, \Delta_1, \dots, \Delta_n. \llbracket E \rrbracket_s = \{v_1, \dots, v_n\} \wedge \Delta = \otimes_{i \in [1..n]} \Delta_i$ $\wedge \forall i \in [1..n]. (s, \Delta_i) \models p(v_i)$

$$\bigoplus_{i \in N} \Delta_i \stackrel{\text{def}}{=} \begin{cases} \emptyset & \text{if } N = \emptyset \\ \Delta_{i'} \oplus \bigoplus_{i \in N'} \Delta_i & \text{if } N = N' \cup \{i'\} \end{cases} \quad \otimes_{i \in N} \Delta_i \stackrel{\text{def}}{=} \begin{cases} \{(\emptyset, \emptyset)\} & \text{if } N = \emptyset \\ \Delta_{i'} * \otimes_{i \in N'} \Delta_i & \text{if } N = N' \cup \{i'\} \end{cases}$$

$$\text{sent}(E_1, E_2) \stackrel{\text{def}}{=} \exists x, y. \text{msgPool}(x) \wedge (y, E_2) \in x \wedge E_1 \subseteq y$$

Fig. 25. Semantics of state assertions

As a simple example, let R_1 be $x = 1 \bowtie x = 2$ and R_2 be $x = 2 \bowtie x = 3$ (assuming that x is a variable in the protocol code), then $\text{nonLocal}(R_1)$ is the same as $\text{nonLocal}(R_2)$, both specifying transitions where x is unchanged.

5.2 Inference Rules

We show the key logic rules in Fig. 27, Fig. 28 and Fig. 29. In addition to the top judgment $\vdash \{p, \eta\}(\Pi, \Gamma)$, we also introduce two judgments, $R, G \vdash \{p\} \widetilde{C} \{q\}$ and $\vdash \{p\} \widetilde{C} \{q\}$, for reasoning about the instrumented object code (i.e. the instrumented method implementations and the instrumented server code).

Informally, the rely-guarantee judgment $R, G \vdash \{p\} \widetilde{C} \{q\}$ requires that, starting from initial states satisfying p and under the environment interference satisfying R , the transitions of $\widetilde{C}$ should satisfy G , and the final states should satisfy q when $\widetilde{C}$ terminates. Here the environment of $\widetilde{C}$ includes both the other nodes (whether correct or faulty) and the other thread within the same node.

$(\Sigma, \Sigma') \models p \ltimes q$	iff $\Sigma \models p \wedge \Sigma' \models q$
$(\Sigma, \Sigma') \models [p]$	iff $\Sigma \models p \wedge \Sigma' = \Sigma$
$(\Sigma, \Sigma') \models \text{IdSpec}$	iff $\Sigma.\Delta = \Sigma'.\Delta$
$(\Sigma, \Sigma') \models \text{nonLocal}(R)$	iff $\Sigma.\sigma_o = \Sigma'.\sigma_o \wedge \exists \sigma, \sigma'. (\Sigma _{\sigma_o \mapsto \sigma}, \Sigma' _{\sigma_o \mapsto \sigma'}) \models R$
$(\Sigma, \Sigma') \models R_1 \wedge R_2$	iff $(\Sigma, \Sigma') \models R_1 \wedge (\Sigma, \Sigma') \models R_2$
$(\Sigma, \Sigma') \models R_1 \vee R_2$	iff $(\Sigma, \Sigma') \models R_1 \vee (\Sigma, \Sigma') \models R_2$
$(\Sigma, \Sigma') \models R_1 * R_2$	iff $\exists \Sigma_1, \Sigma'_1, \Sigma_2, \Sigma'_2. \Sigma = \Sigma_1 * \Sigma_2 \wedge \Sigma' = \Sigma'_1 * \Sigma'_2 \wedge (\Sigma_1, \Sigma'_1) \models R_1 \wedge (\Sigma_2, \Sigma'_2) \models R_2$
$(\Sigma, \Sigma') \models R_1 \Rightarrow R_2$	iff $(\Sigma, \Sigma') \models R_1 \implies (\Sigma, \Sigma') \models R_2$
$(\Sigma, \Sigma') \models \exists x. R$	iff $x \notin \text{dom}(\Sigma.\sigma_a \cup \Sigma'.\sigma_a) \wedge \exists v. (\Sigma _{\sigma_o \mapsto \sigma_o \uplus \{x \mapsto v\}}, \Sigma' _{\sigma_o \mapsto \sigma_o \uplus \{x \mapsto v\}}) \models R$
$(\Sigma, \Sigma') \models \forall x. R$	iff $x \notin \text{dom}(\Sigma.\sigma_a \cup \Sigma'.\sigma_a) \wedge \forall v. (\Sigma _{\sigma_o \mapsto \sigma_o \uplus \{x \mapsto v\}}, \Sigma' _{\sigma_o \mapsto \sigma_o \uplus \{x \mapsto v\}}) \models R$

$$\begin{aligned}
\text{True} &\stackrel{\text{def}}{=} \text{true} \ltimes \text{true} & \text{Id} &\stackrel{\text{def}}{=} [\text{true}] & \text{IdM} &\stackrel{\text{def}}{=} \exists x. \mathbf{msgPool}(x) \ltimes \mathbf{msgPool}(x) \\
\text{SendAct}(E_1, E_2) &\stackrel{\text{def}}{=} \exists x. \mathbf{msgPool}(x) \ltimes \mathbf{msgPool}(x \cup \{(E_1, E_2)\}) & \text{IdV} &\stackrel{\text{def}}{=} \text{IdVExcept}(\emptyset) \\
\text{IdV}(X) &\stackrel{\text{def}}{=} \forall x \in X. \exists y. x = y \ltimes x = y & \text{IdVExcept}(X) &\stackrel{\text{def}}{=} \forall x \notin X. \exists y. x = y \ltimes x = y
\end{aligned}$$

Fig. 26. Semantics of rely and guarantee assertions

$$\begin{aligned}
&\text{Er}(\tilde{L}, \tilde{C}_s, \zeta) = \Pi \quad \text{checkFlin}_{\Gamma}(\tilde{L}, \tilde{C}_s, \zeta) \quad G^f = \text{GF}(\zeta) \\
&\text{for all } t : \quad p_t \Rightarrow \text{tid} = t \wedge \text{readOnlyVarProp}(\eta) \quad G_t^m \vee G_t^s \Rightarrow \text{IdV}(\text{readOnlyVarSet}) \\
&\quad \bigvee_{t' \neq t} (\text{InvokeAct}(\Pi.L)(t') \vee G_{t'}^m \vee G_{t'}^s) \vee G^f \Rightarrow R_t \quad \text{noBadBeh}(G^s, R)(t) \\
&\quad (\text{InvokeAct}(\Pi.L)(t) \vee G_t^m \vee \text{nonLocal}(R_t)), G_t^s \vdash \{p_t\} \tilde{C}_s \{\mathbf{true}\} \\
&\quad \text{for all } f : \quad \tilde{L}(f) = (x, \tilde{C}) \\
&\frac{(G_t^s \vee \text{nonLocal}(R_t)), G_t^m \vdash \{p_t \wedge (\text{tid} \mapsto (\Gamma(f), x) * \mathbf{true})\} \tilde{C} \{p_t \wedge (\exists v. \text{tid} \mapsto (\text{end}, v) * \mathbf{true})\}}{\vdash \{p, \eta\}(\Pi, \Gamma)} \text{OBJ}
\end{aligned}$$

Fig. 27. Object rule

The sequential judgment $\vdash \{p\} \tilde{C} \{q\}$ is used to reason about atomic statements without interference with the environment.

The OBJ rule. As shown in Fig. 27, the OBJ rule has several premises, and we will explain them one by one. First, to verify $\vdash \{p, \eta\}(\Pi, \Gamma)$, we instrument the protocol code Π and obtain $\tilde{\Pi}$, so that (1) erasing the auxiliary statements of $\tilde{\Pi}$ gives us exactly Π (i.e. $\text{Er}(\tilde{\Pi}) = \Pi$ holds); and (2) for every **flin**(E, γ, E') statement in $\tilde{\Pi}$, γ is a valid method specification in Γ (i.e. $\text{checkFlin}_{\Gamma}(\tilde{\Pi})$ holds).

Then we prepare three kinds of guarantee conditions: G^m for the method code, G^s for the server code, and G^f for the faulty nodes. We generate G^f from the faulty nodes' assumption ζ declared in Π , using the generator GF which is defined as follows:

$$\begin{aligned}
\text{GF}(\zeta) &\stackrel{\text{def}}{=} \text{Id} \vee (\text{IdV} \wedge \text{IdSpec} \wedge \exists M, \text{msg}. \\
&\quad \mathbf{msgPool}(M) \wedge \text{msg} \notin \zeta(\text{Nbad}, M) \wedge \text{Nbad} \neq \emptyset \ltimes \mathbf{msgPool}(M \cup \{(_, \text{msg})\}))
\end{aligned}$$

It says that faulty nodes can do identity transitions (Id), or send a message allowed by the assumption ζ . We can see that $\text{GF}(\zeta)$ precisely captures the behaviors of faulty nodes in the concrete semantics (see the FAULTY-STEP rule in Fig. 6).

As the method code and server code are replicated on every node, we assume that the method guarantee G^m , server guarantee G^s and state assertions p are implicitly parametrized over node IDs. We require p_t to state that the constant variable tid is set to t , and that $\text{readOnlyVarProp}(\eta)$ holds. The latter says that, the predefined constant variables tid , Nok , Nbad and Nodes have consistent values while satisfying the node proportional relation η :

$$\text{readOnlyVarProp}(\eta) \stackrel{\text{def}}{=} (\text{tid} \in \text{Nok}) \wedge (\text{Nodes} = \text{Nok} \uplus \text{Nbad}) \wedge ((|\text{Nok}|, |\text{Nbad}|) \in \eta)$$

Since p encodes the $\text{readOnlyVarProp}(\eta)$ condition, when reasoning about the protocol code, we are able to make use of the assumption of the node proportional relation η that the correctness of the protocol relies on.

For G^m and G^s , we require that they both ensure $\text{IdV}(\text{readOnlyVarSet})$. Here readOnlyVarSet is the set of the predefined constant variables tid , Nok , Nbad and Nodes . Their values should be unchanged in executions of the protocol code, so we require the transitions of G^m and G^s to satisfy $\text{IdV}(\text{readOnlyVarSet})$.

Next we define the rely conditions that specify the assumptions on the environment behaviors. As we said, when verifying a thread $\tilde{C}$ on node t , the environment includes both the other correct and faulty nodes and the other thread within the same node t . Thus, the rely condition for $\tilde{C}$ must be in the form of $G_t^1 \vee \text{nonLocal}(R_t)$, where G_t^1 specifies the transitions of the other thread within the current node t , and R_t specifies the transitions of the other nodes. Note that we use nonLocal to translate R_t to the effects on the current node's view. Below we first explain R_t and then G_t^1 .

Whether $\tilde{C}$ is the method code or the server code, we can define R_t in the same way. We require R_t to include the faulty nodes' guarantee condition G^f , as well as G^m and G^s of every other node t' . R_t should also include the actions of method invocations of the other nodes, represented by $\text{InvokeAct}(\Pi.L)(t')$. We define $\text{InvokeAct}(L)$ as follows:

$$\begin{aligned} \text{InvokeAct}(L) &\stackrel{\text{def}}{=} \lambda t. ((\text{emp} \vee t \mapsto (\text{end}, _)) \times (t \mapsto _)) * \text{IdSpec} \\ &\quad \wedge \text{IdVExcept}(\text{parameterSet}(L)) \wedge \text{IdM} \\ \text{parameterSet}(L) &\stackrel{\text{def}}{=} \{x \mid \exists f. L(f) = (x, _)\} \end{aligned}$$

It conservatively says that, the step of invoking a method by the node t can arbitrarily change the remaining abstract operation and assign new values to the method parameter variables (denoted by $\text{parameterSet}(L)$), but must keep the other parts of the relational state Σ unchanged.

For method thread, its environment must not contain any transition that introduce new unfinished operation for current thread. For example, the environment of node t 's methods thread must not transition a state that satisfies $t \mapsto (\text{end}, _)$ to a state satisfies $t \mapsto (y, v)$. This is formalized by $\text{noBadBeh}(G^s, R)(t)$, where G^s and R is the main part for constructing environment of method thread, and t specifies the current node ID, defined as follows: (note that in definition of $\text{finish}(t, \Delta)$, we write $t \notin \text{dom}(\Delta.U)$ as shorthand for $\forall U. (U, _) \in \Delta \implies t \notin \text{dom}(U)$)

$$\begin{aligned} \text{noBadBeh}(G^s, R)(t) &\stackrel{\text{def}}{=} (G_t^s \implies \neg \text{badBeh}(t)) \wedge (R_t \implies \bigwedge_{t' \neq t} (\neg \text{badBeh}(t'))) \\ \text{badBeh}(t) &\stackrel{\text{def}}{=} \exists \Delta, \Delta'. \text{isDelta}(\Delta) \wedge \text{finish}(t, \Delta) \times \text{isDelta}(\Delta') \wedge \neg \text{finish}(t, \Delta) \\ \text{finish}(t, \Delta) &\stackrel{\text{def}}{=} t \notin \text{dom}(\Delta.U) \vee \exists v. \forall U. (U, _) \in \Delta \implies U(t) = (\text{end}, v) \end{aligned}$$

Since G_t^1 specifies the transitions of the other thread within the current node t , it should be defined differently depending on whether $\tilde{C}$ is the method code or the server code. When $\tilde{C}$ is the method code, G_t^1 is simply G_t^s , i.e. the guarantee condition of the server code on the node t . When $\tilde{C}$ is the server code, G_t^1 includes G_t^m , i.e. the guarantee condition of the method code on the node t , as well as the method invocations $\text{InvokeAct}(\Pi.L)(t)$.

Finally, and most importantly, we reason about the instrumented server code $\tilde{C}_s$ and every instrumented method's code $\tilde{L}(f)$, using the rely and guarantee conditions we have just explained. The precondition p is an object invariant that should be preserved by every method call, so we use it in both the precondition and the postcondition of the method code. In addition, the precondition of the method code $\tilde{L}(f)$ also requires that the remaining abstract operation of the current node should be $\Gamma(f)$, and the postcondition requires that this abstract operation should have been finished. For the server code $\tilde{C}_s$, the precondition is p , and the postcondition is **true** because we do not care

about its final states. In fact, the server code often does not terminate, as in the broadcast example in Fig. 2. In this work, we do not care whether the code terminates or not, since our goal is to verify Byzantine linearizability, which is a partial correctness property.

We define c as the set of concrete statements in the method body or the server, each executable within a single step in thread-local transition:

$$c ::= \text{skip} \mid x := E \mid \text{send}(E_N, E) \mid x := \text{recv}() \mid \text{return } E$$

Definition 5.1 (Erasure).

$$\begin{aligned} \text{Er}((\tilde{L}, \tilde{C}_s, \zeta)) &\stackrel{\text{def}}{=} (\text{Er}(\tilde{L}), \text{Er}(\tilde{C}_s), \zeta) \\ \text{Er}(\{f_1 \rightsquigarrow (x_1, \tilde{C}_1), \dots, f_n \rightsquigarrow (x_n, \tilde{C}_n)\}) &\stackrel{\text{def}}{=} \{f_1 \rightsquigarrow (x_1, \text{Er}(\tilde{C}_1)), \dots, f_n \rightsquigarrow (x_n, \text{Er}(\tilde{C}_n))\} \\ \text{Er}(\tilde{C}) \in \text{Stmt} &\text{ is inductively defined as follows:} \\ \text{Er}(c) &\stackrel{\text{def}}{=} c \\ \text{Er}(\dot{C}) &\stackrel{\text{def}}{=} \text{skip} \\ \text{Er}(\langle \tilde{C} \rangle) &\stackrel{\text{def}}{=} \text{ErAtom}(\tilde{C}) \\ \text{Er}(\tilde{C}_1; \tilde{C}_2) &\stackrel{\text{def}}{=} \text{Er}(\tilde{C}_1); \text{Er}(\tilde{C}_2) \\ \text{Er}(\text{if } (B) \text{ then } \tilde{C}_1 \text{ else } \tilde{C}_2) &\stackrel{\text{def}}{=} \text{if } (B) \text{ then } \text{Er}(\tilde{C}_1) \text{ else } \text{Er}(\tilde{C}_2) \\ \text{Er}(\text{while } (B) \text{ do } \tilde{C}) &\stackrel{\text{def}}{=} \text{while } (B) \text{ do } \text{Er}(\tilde{C}) \end{aligned}$$

Definition 5.2. $\text{ErAtom}(\tilde{C}) \in \text{Stmt}$ is defined as follows:

$$\begin{aligned} \text{ErAtom}(c) &\stackrel{\text{def}}{=} c \\ \text{ErAtom}(\dot{C}) &\stackrel{\text{def}}{=} \text{skip} \\ \text{ErAtom}(\dot{C}; c) &\stackrel{\text{def}}{=} c \\ \text{ErAtom}(c; \dot{C}) &\stackrel{\text{def}}{=} c \\ \text{ErAtom}(\dot{C}_1; c; \dot{C}_2) &\stackrel{\text{def}}{=} c \end{aligned}$$

Definition 5.3 (Checking **flin** command).

$$\begin{aligned} \text{checkFlin}_\Gamma((\tilde{L}, \tilde{C}_s, \zeta)) &\stackrel{\text{def}}{=} \text{checkFlin}_\Gamma(\tilde{L}) \wedge \text{checkFlin}_\Gamma(\tilde{C}_s) \\ \text{checkFlin}_\Gamma(\{f_1 \rightsquigarrow (x_1, \tilde{C}_1), \dots, f_n \rightsquigarrow (x_n, \tilde{C}_n)\}) &\stackrel{\text{def}}{=} \forall i \in [1..n]. \text{checkFlin}_\Gamma(\tilde{C}_i) \\ \text{checkFlin}_\Gamma(\tilde{C}) &\stackrel{\text{def}}{=} (\exists c. \tilde{C} = c) \vee (\exists \dot{C}. \tilde{C} = \dot{C} \wedge \text{checkFlin}_\Gamma(\dot{C})) \\ &\quad \vee (\exists \tilde{C}'. \tilde{C} = \langle \tilde{C}' \rangle \wedge \text{checkFlin}_\Gamma(\tilde{C}')) \\ &\quad \vee (\exists \tilde{C}_1, \tilde{C}_2. \tilde{C} = \tilde{C}_1; \tilde{C}_2 \wedge \text{checkFlin}_\Gamma(\tilde{C}_1) \wedge \text{checkFlin}_\Gamma(\tilde{C}_2)) \\ &\quad \vee (\exists B, \tilde{C}_1, \tilde{C}_2. \tilde{C} = \text{if } (B) \text{ then } \tilde{C}_1 \text{ else } \tilde{C}_2 \wedge \text{checkFlin}_\Gamma(\tilde{C}_1) \wedge \text{checkFlin}_\Gamma(\tilde{C}_2)) \\ &\quad \vee (\exists B, \tilde{C}'. \tilde{C} = \text{while } (B) \text{ do } \tilde{C}' \wedge \text{checkFlin}_\Gamma(\tilde{C}')) \\ \text{checkFlin}_\Gamma(\dot{C}) &\stackrel{\text{def}}{=} \dot{C} = \text{skip} \vee (\exists x, E. \dot{C} = x :=_a E) \vee (\exists E. \dot{C} = \text{lin}(E)) \vee (\exists E. \dot{C} = \text{trylin}(E)) \\ &\quad \vee (\exists p. \dot{C} = \text{commit}(p)) \vee (\exists E, E', \gamma. \dot{C} = \text{flin}(E, \gamma, E') \wedge \gamma \in \text{range}(\Gamma)) \\ &\quad \vee (\exists \dot{C}_1, \dot{C}_2. \dot{C} = \dot{C}_1; \dot{C}_2 \wedge \text{checkFlin}_\Gamma(\dot{C}_1) \wedge \text{checkFlin}_\Gamma(\dot{C}_2)) \\ &\quad \vee (\exists B, \dot{C}_1, \dot{C}_2. \dot{C} = \text{if } (B) \text{ then } \dot{C}_1 \text{ else } \dot{C}_2 \wedge \text{checkFlin}_\Gamma(\dot{C}_1) \wedge \text{checkFlin}_\Gamma(\dot{C}_2)) \\ &\quad \vee (\exists x, E, \dot{C}'. \dot{C} = \text{foreach } x \text{ in } E \text{ do } \dot{C}' \wedge \text{checkFlin}_\Gamma(\dot{C}')) \end{aligned}$$

Rules for atomic statements. Following RGSep [Vafeiadis 2008], we have two rules for verifying atomic blocks. As shown in Fig. 28, the **ATOM** rule allows us to reduce the rely-guarantee reasoning to sequential reasoning, as long as the overall transitions satisfy the guarantee condition G , and the environment consists of identity transitions Id only. To allow a more general environment R , we can apply the **ATOM-R** rule later, which requires that both the precondition and the postcondition be stable with respect to R . The stability is defined in the standard way:

$$\begin{array}{c}
\frac{\vdash \{p\} \widetilde{C}\{q\} \quad p \times q \Rightarrow G}{\text{Id}, G \vdash \{p\} \langle \widetilde{C} \rangle \{q\}} \text{ATOM} \quad \frac{\text{Id}, G \vdash \{p\} \langle \widetilde{C} \rangle \{q\} \quad \text{stable}(\{p, q\}, R)}{R, G \vdash \{p\} \langle \widetilde{C} \rangle \{q\}} \text{ATOM-R} \\
\\
\frac{R, G \vdash \{p\} \widetilde{C}_1\{q\} \quad R, G \vdash \{q\} \widetilde{C}_2\{r\}}{R, G \vdash \{p\} \widetilde{C}_1; \widetilde{C}_2\{r\}} \text{P-SEQ} \quad \frac{R, G \vdash \{p \wedge B\} \widetilde{C}\{p\} \quad \text{stable}(\neg B, R)}{R, G \vdash \{p\} \mathbf{while} (B) \mathbf{do} \widetilde{C}\{p \wedge \neg B\}} \text{P-WHILE} \\
\\
\frac{R, G \vdash \{p \wedge B\} \widetilde{C}_1\{q\} \quad R, G \vdash \{p \wedge \neg B\} \widetilde{C}_2\{q\} \quad \text{stable}(p, R)}{R, G \vdash \{p\} \mathbf{if} (B) \mathbf{then} \widetilde{C}_1 \mathbf{else} \widetilde{C}_2\{q\}} \text{P-IF} \\
\\
\frac{R, G \vdash \{p\} \widetilde{C}\{q\} \quad x \notin \text{FV}(R) \cup \text{FV}(G) \cup \text{FV}(\widetilde{C})}{R, G \vdash \{\exists x. p\} \widetilde{C}\{\exists x. q\}} \text{P-EX} \\
\\
\frac{R' \Rightarrow R \quad p' \Rightarrow p \quad R, G \vdash \{p\} \widetilde{C}\{q\} \quad G \Rightarrow G' \quad q \Rightarrow q'}{R', G' \vdash \{p'\} \widetilde{C}\{q'\}} \text{P-CONSEQ}
\end{array}$$

Fig. 28. Logic rules for rely-guarantee reasoning

$$\text{stable}(p, R) \quad \text{iff} \quad \forall \Sigma, \Sigma'. \Sigma \models p \wedge (\Sigma, \Sigma') \models R \implies \Sigma' \models p$$

When verifying the primitive commands, such as the concrete commands **send**(E_1, E_2) and $x := \mathbf{recv}()$, and the auxiliary commands **lin**(E) and **flin**(E, γ, E_2), we always assume that they are wrapped in atomic blocks, and apply the **ATOM** and **ATOM-R** rules to reduce the concurrency verification to sequential reasoning. The other concurrency rules (such as the consequence rule **P-CONSEQ** in Fig. 28) are mostly standard.

*Sequential rules for the **lin** and **flin** statements.* As shown in Fig. 29, the two rules **LIN** and **LIN-END** allow us to reason about **lin**(E). If the abstract operation γ of the node E has not been done, then **lin**(E) will finish it (see the **LIN** rule); otherwise, **lin**(E) has no effects (see the **LIN-END** rule). The condition $[p]\gamma(E, E_1)[q, E_2]$ in the **LIN** rule describes the effect of the abstract operation γ executed by the node E and with the argument E_1 . It transforms the abstract objects satisfying p to new ones satisfying q , and returns E_2 . We show the formal definition below.

$$\begin{aligned}
[p]\gamma(E, E_1)[q, E_2] \quad \text{iff} \quad & \forall \sigma_o, M, \sigma_a, \Delta, t. ((\sigma_o, M, \sigma_a), \Delta) \models E \mapsto (\gamma, E_1) * p \wedge E = t \\
& \implies \exists \Delta'. \Delta \rightarrow_t \Delta' \wedge ((\sigma_o, M, \sigma_a), \Delta') \models E \mapsto (\text{end}, E_2) * q
\end{aligned}$$

Here $\Delta \rightarrow_t \Delta'$ was previously used for defining the operational semantics of **lin**(E) (see Fig. 22). Thus $[p]\gamma(E, E_1)[q, E_2]$ precisely captures the semantics of **lin**(E).

The **FLIN** rule allows us to reason about **flin**(E, γ, E_1). It requires E to be a faulty node (see the first premise $p \Rightarrow (E \in \text{Nbad})$). The second premise $[p]\gamma(E, E_1)[q]$ is defined below.

$$\begin{aligned}
[p]\gamma(E, E_1)[q] \quad \text{iff} \quad & \forall t, v, \sigma_o, M, \sigma_a, \Delta, v. ((\sigma_o, M, \sigma_a), \Delta) \models p \wedge E = t \wedge E_1 = v \\
& \implies \exists \Delta'. \Delta \xrightarrow{(t, \gamma, v)} \Delta' \wedge ((\sigma_o, M, \sigma_a), \Delta') \models q
\end{aligned}$$

The definition is similar to $[p]\gamma(E, E_1)[q, E_2]$, except that here we rely on the auxiliary transition $\Delta \xrightarrow{(t, \gamma, v)} \Delta'$ which was previously used in the operational semantics of **flin**(E, γ, E_1) (see Fig. 22). The abstract operation γ is not recorded in Δ , reflecting that executing γ is to *help a faulty node*.

*Sequential rules for the **send** and **recv** statements.* As shown in Fig. 29, the **SEND** rule captures the atomic effect of **send**(E_N, E). It puts the pair (E_N, E) into the message pool E_M . Sometimes we need to prove that some assertion p holds both before and after a send operation. However, since **send** statement modifies the message pool, this cannot be established by **INVARIANCE** rule. Thus,

$$\begin{array}{c}
\frac{[p]\gamma(E, E_1)[q, E_2]}{\vdash \{E \mapsto (\gamma, E_1) * p\} \mathbf{lin}(E) \{E \mapsto (\text{end}, E_2) * q\}} \text{LIN} \qquad \frac{p \Rightarrow (E \in \text{Nbad}) \quad [p]\gamma(E, E_1)[q]}{\vdash \{p\} \mathbf{flin}(E, \gamma, E_1) \{q\}} \text{FLIN} \\
\\
\frac{}{\vdash \{E \mapsto (\text{end}, E')\} \mathbf{lin}(E) \{E \mapsto (\text{end}, E')\}} \text{LIN-END} \\
\\
\frac{[p]\gamma(E, E_1)[q, E_2]}{\vdash \{E \mapsto (\gamma, E_1) * p\} \mathbf{trylin}(E) \{(E \mapsto (\gamma, E_1) * p) \oplus (E \mapsto (\text{end}, E_2) * q)\}} \text{TRYLIN} \\
\\
\frac{}{\vdash \{E \mapsto (\text{end}, E')\} \mathbf{trylin}(E) \{E \mapsto (\text{end}, E')\}} \text{TRYLIN-END} \qquad \frac{\text{specExact}(p) \quad p' \Rightarrow p}{\vdash \{p' \oplus \text{true}\} \mathbf{commit}(p) \{p'\}} \text{COMMIT} \\
\\
\frac{}{\vdash \{\mathbf{msgPool}(E_M)\} \mathbf{send}(E_N, E) \{\mathbf{msgPool}(E_M \cup \{(E_N, E)\})\}} \text{SEND} \qquad \frac{\text{sendPres}(p, E_N, E)}{\vdash \{p\} \mathbf{send}(E_N, E) \{p\}} \text{SEND} \\
\\
\frac{x \notin \text{FV}(E_M)}{\vdash \{\mathbf{msgPool}(E_M)\} x := \mathbf{recv}() \{\mathbf{msgPool}(E_M) \wedge \exists x_N. (x_N, x) \in E_M \wedge \text{tid} \in x_N\}} \text{RECV} \\
\\
\frac{}{\vdash \{(p/x \rightarrow E) \wedge \neg \text{auxVar}(x)\} x := E \{p\}} \text{ASSN} \qquad \frac{}{\vdash \{(p/_a x \rightarrow E) \wedge \neg \text{objVar}(x)\} x :=_a E \{p\}} \text{AUX-ASSN} \\
\\
\frac{}{\vdash \{\text{tid} \mapsto (\text{end}, E)\} \mathbf{E}_i[\mathbf{return} E] \{\text{tid} \mapsto (\text{end}, E)\}} \text{RET} \\
\\
\frac{p \Rightarrow \text{legalForPara}(x, E) \quad x \notin \text{FV}(E) \cup \text{FV}(q) \quad \text{noModifyVars}(\text{FV}(E) \cup \{x\}, \dot{C})}{\frac{\text{pure}(q) \quad \text{side-cond}(q, \dot{C}) \quad \vdash \{p \wedge x \in E\} \dot{C} \{p \wedge q(x)\}}{\vdash \{p\} \mathbf{foreach} x \text{ in } E \text{ do } \dot{C} \{p \wedge \bigwedge_{y \in E} q(y)\}} \text{FOR} \\
\\
\frac{\vdash \{p\} \tilde{C}\{q\} \quad \vdash \{p'\} \tilde{C}\{q'\}}{\vdash \{p \oplus p'\} \tilde{C}\{q \oplus q'\}} \text{SPEC-CONJ} \qquad \frac{\vdash \{p\} \tilde{C}\{q\} \quad \vdash \{p'\} \tilde{C}\{q'\}}{\vdash \{p \wedge p'\} \tilde{C}\{q \wedge q'\}} \text{CONJ} \\
\\
\frac{\vdash \{p\} \tilde{C}_1\{q\} \quad \vdash \{q\} \tilde{C}_2\{r\}}{\vdash \{p\} \tilde{C}_1; \tilde{C}_2\{r\}} \text{SEQ} \\
\\
\frac{\vdash \{p\} \tilde{C}\{q\} \quad \text{side-cond}(r, \tilde{C})}{\vdash \{p * r\} \tilde{C}\{q * r\}} \text{FRAME} \qquad \frac{\vdash \{p\} \tilde{C}\{q\} \quad \text{pure}(r) \quad \text{side-cond}(r, \tilde{C})}{\vdash \{p \wedge r\} \tilde{C}\{q \wedge r\}} \text{INVARIANCE}
\end{array}$$

Fig. 29. Selected logic rules for sequential reasoning

we introduce SEND-PRES rule to reason about cases where p must hold in both pre- and post-states. This rule is directly based on operational semantics, where $\text{sendPres}(p, E_N, E)$ is defined as follows:

$$\text{sendPres}(p, E_N, E) \stackrel{\text{def}}{=} \forall \sigma_o, M, \sigma_a, \Delta. ((\sigma_o, M, \sigma_a), \Delta) \models p \implies ((\sigma_o, M \cup \{\llbracket (E_N, E) \rrbracket_{\sigma_o}\}, \sigma_a), \Delta) \models p$$

The RECV rule says that, when the $x := \mathbf{recv}()$ command completes, x is assigned the message sent to tid in the message pool E_M . Recall that the execution of $x := \mathbf{recv}()$ would get stuck if the message pool does not contain a message sent to tid . This is fine because we care about partial correctness only. We know that if $x := \mathbf{recv}()$ terminates, the postcondition must hold.


```

{true}
S :=a ∅;
{S ⊆ {1, 2, 3}}
foreach x in {1, 2, 3} do S :=a S ∪ {x}
{S ⊆ {1, 2, 3} ∧ ∧x∈{1,2,3} x ∈ S}
{S = {1, 2, 3}}

```

Fig. 30. An example demonstrating the application of the FOR rule.

Sequential rules for local reasoning. The FRAME rule allows us to first reason about $\tilde{C}$ in a local context that $\tilde{C}$ actually accesses, i.e., $\vdash \{p\}\tilde{C}\{q\}$, and then extend the proof to the larger context with the frame r . It is a natural adaption of the frame rule in separation logic to our setting. Here $\text{side-cond}(r, \tilde{C})$ requires that the free variables in r are not modified by $\tilde{C}$, and if r specifies the message pool, then the message pool is not modified by $\tilde{C}$ as well, defined as follows: (here $\text{FV}(r)$ stands for the set of free variables in r)

$$\begin{aligned}
\text{side-cond}(r, \tilde{C}) &\stackrel{\text{def}}{=} \text{noModifyVars}(\text{FV}(r), \tilde{C}) \wedge \text{noModifyRestrictedMP}(r, \tilde{C}) \\
\text{restrictMP}(r) &\stackrel{\text{def}}{=} \exists M, M', \sigma_o, \sigma_a, \Delta. ((\sigma_o, M, \sigma_a), \Delta) \models r \wedge (\neg((\sigma_o, M', \sigma_a), \Delta) \models r) \\
\text{noModifyRestrictedMP}(r, \tilde{C}) &\stackrel{\text{def}}{=} \forall x, k, t, \tilde{C}', \Sigma, \Sigma'. \text{restrictMP}(r) \wedge (\tilde{C}, \Sigma) \xrightarrow{t}^k (\tilde{C}', \Sigma') \\
&\implies \Sigma.M = \Sigma'.M
\end{aligned}$$

Since we only support separation logic assertions for the abstract resource Δ , the separating conjunction $*$ can be degenerated to the conjunction $\wedge$ when the frame r is a pure assertion, as shown in the INVARIANCE rule. We define $\text{pure}(p)$ as follows:

$$\text{pure}(p) \quad \text{iff} \quad \forall \sigma_o, M, \sigma_a, \Delta, \Delta'. ((\sigma_o, M, \sigma_a), \Delta) \models p \iff ((\sigma_o, M, \sigma_a), \Delta') \models p$$

Informally, it requires p to not talk about the Δ resource.

*Sequential rules for **foreach** statement.* The FOR rule defines how to reason about **foreach** loop using invariant p and result assertion q . Above the horizontal line, $\vdash \{p \wedge x \in E\} \dot{C} \{p \wedge q(x)\}$ means that if invariant p holds before executing $\dot{C}$, and set E is non-empty with element x , then p still hold after $\dot{C}$, and $q(x)$ holds for the post-state. q is an assertion taking a variable, or a value as argument. For example $q = \lambda x. x \in S$. $\text{pure}(q)$ is sugar for $\forall v. \text{pure}(q(v))$. $\text{noModifyRestrictedMP}(q, \dot{C})$ is sugar for $\forall v. \text{noModifyRestrictedMP}(q(v), \dot{C})$. We also require that x is not one of the free variables in q . With these requirements, the result $q(v)$ preserves in each subsequent round, where v is the value being process in current round. When the loop terminates, q holds for the entire set E . Other requirements are directly from the semantics of **foreach** command, where $\text{legalForPara}(x, E)$ is defined as follows:

$$\text{legalForPara}(x, E) \stackrel{\text{def}}{=} E = \emptyset \vee (\exists v_1, \dots, v_n. E = \{v_1, \dots, v_n\} \wedge \neg \text{objVar}(x))$$

Fig. 30 present an example for using FOR rule. Note that each pre- post-condition in this example omit condition $\neg \text{objVar}(x) \wedge \neg \text{objVar}(S)$.

5.3 Logic Soundness

In this section we prove that the logic ensures Byzantine linearizability. Fig. 31 shows how the proof is constructed. As we have proved the equivalence between Byzantine linearizability and contextual refinement in Sec. 3.3.1 (Thm. 1), the proof in this section focus on how to derive contextual refinement from object judgment.

The soundness of the logic is shown in Thm. 2. $\text{refMap}_\eta(\varphi, p)$ relates refinement mapping φ with object invariant p w.r.t. node proportional relation η . This relation concerns η since φ only

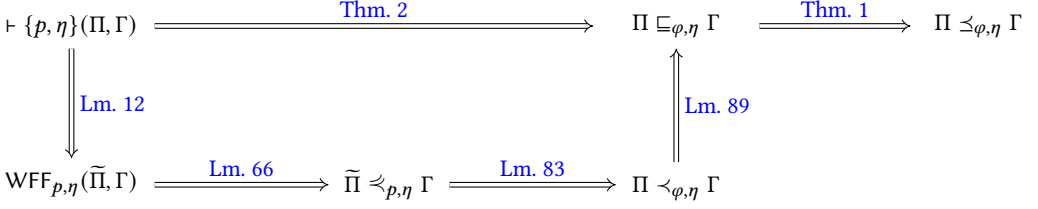


Fig. 31. Proof path

specifies relation between initial concrete object and abstract one, while object invariant p also involves specific benign and faulty node set.

First we perform a semantic interpretation on the object rule by replacing RG judgments with corresponding semantics in Lm. 12. Then we define a forward-backward simulation between instrumented machine and abstract machine, and prove our logic establish such simulation (Lm. 66). After that we prove this simulation implies another forward-backward simulation between concrete and abstract machine (Lm. 83), i.e. proving a forward simulation between concrete and instrumented side. The instrumentation is eliminated in this process. At last, we prove the latter simulation implies contextual refinement (Lm. 89). The proofs of these sub-lemmas are distributed among the subsections. Below we introduce each sub-lemma in detail.

Definition 5.4. $(\sigma_o, \sigma_a, \theta, \mathcal{N}_{ok}, \mathcal{N}_{bad}) \models_{\text{init}} p \stackrel{\text{def}}{=} \forall t \in \mathcal{N}_{ok}. ((\sigma_o\{\text{tid} \rightsquigarrow t, \text{Nodes} \rightsquigarrow \mathcal{N}_{ok} \uplus \mathcal{N}_{bad}\}, \emptyset, \sigma_a\{\text{Nok} \rightsquigarrow \mathcal{N}_{ok}, \text{Nbad} \rightsquigarrow \mathcal{N}_{bad}\}), \{(\emptyset, \theta)\}) \models p_t$

Definition 5.5. $\text{refMap}_{\eta}(\varphi, p) \stackrel{\text{def}}{=} \forall \sigma_o, \theta. \varphi(\sigma_o) = \theta \implies (\exists \sigma_a. \forall \mathcal{N}_{ok}, \mathcal{N}_{bad}. \mathcal{N}_{ok} \cap \mathcal{N}_{bad} = \emptyset \wedge (|\mathcal{N}_{ok}|, |\mathcal{N}_{bad}|) \in \eta \implies (\sigma_o, \sigma_a, \theta, \mathcal{N}_{ok}, \mathcal{N}_{bad}) \models_{\text{init}} p)$

Theorem 2 (Logic Soundness). $\forall p, \eta, \varphi, \Pi, \Gamma. \text{refMap}_{\eta}(\varphi, p) \wedge \vdash \{p, \eta\}(\Pi, \Gamma) \implies \Pi \sqsubseteq_{\varphi, \eta} \Gamma$

PROOF. by Lm. 12, Lm. 66, Lm. 83 and Lm. 89 immediately. $\square$

Proof of Lm. 12 primarily consists of establishing the soundness of the RG conditions. Following Liang and Feng [2013], the semantics of RG judgment is given as $R, G \models_t \{p\} \widetilde{C} \{q\}$, formally in Def. 5.16. It encodes partial correctness of $\widetilde{C}$ w.r.t. pre- post-condition p and q . We show that $R, G \vdash \{p\} \widetilde{C} \{q\}$ implies $R, G \models_t \{p \wedge \text{tid} = t\} \widetilde{C} \{q \wedge \text{tid} = t\}$ in Lm. 14.

Definition 5.6. $\text{WFF}_{p, \eta}(\tilde{\Pi}, \Gamma)$ holds iff there exists p, G^m, G^s, G^f, R s.t. for all t , we have

1. $\text{checkFlin}_{\Gamma}(\tilde{\Pi})$;
2. $G^f = \text{GF}(\tilde{\Pi}, \zeta)$;
3. $p_t \implies \text{tid} = t$;
4. $\bigvee_{t' \neq t} (\text{InvokeAct}(\tilde{\Pi}, \tilde{L})(t') \vee G_{t'}^m \vee G_{t'}^s) \vee G^f \implies R_t$;
5. $\text{noBadBeh}(G^s, R)(t)$
6. $\text{stable}(p_t, \text{InvokeAct}(\tilde{\Pi}, \tilde{L})(t) \vee G_t^m \vee G_t^s \vee \text{nonLocal}(R_t))$;
7. $(\text{InvokeAct}(\tilde{\Pi}, \tilde{L})(t) \vee G_t^m \vee \text{nonLocal}(R_t)), G_t^s \models_t \{p_t\} \tilde{\Pi}. \tilde{C}_s \{\text{true}\}$;
8. for all f , if $\tilde{\Pi}. \tilde{L}(f) = (x, \tilde{C})$, then $(G_t^s \vee \text{nonLocal}(R_t)), G_t^m \models_t \{p_t \wedge (\text{tid} \rightsquigarrow (\Gamma(f), x) * \text{true})\} \tilde{C} \{p_t \wedge (\exists v. \text{tid} \rightsquigarrow (\text{end}, v) * \text{true})\}$.

In Lm. 66, the conclusion $\tilde{\Pi} \preceq_{p, \eta} \Gamma$ says that any context using $\tilde{\Pi}$ is simulated by the same context using Γ . The simulation holds if the speculative executions in both sides can be related.

On the left side, each speculative branch in Δ specifies abstract operations for each node and the current abstract object. On the right side, there is a collection of abstract worlds (denoted as Ω), each specifies essentially the same information.

We define the transition $\Omega \Rightarrow \Omega'$ in Def. 5.7, which indicates that any abstract world in the resulting set Ω' must be reachable from some world in Ω . This transition allows three possibilities:

1. executing an abstract operation of some benign node t ;
2. dropping some abstract worlds in Ω ;
3. executing an abstract operation γ with argument v on behalf of some faulty node t' .

The first corresponds to a step of **lin**(t) or **trylin**(t), the second can be interpreted as a **commit** step, and the third resembles a step of **flin**(t', γ, v).

In the proof of Lm. 66, we first lift the RG judgment semantics for method to client thread's guarantee $\text{benignNodeGuar}_{\bar{L}, \Gamma}(t, p_t, G_t^s \vee \text{nonLocal}(R_t))(C, \text{skip})(\text{Id} \vee \text{InvokeAct}(\bar{L})(t) \vee G_t^m)$ in Lm. 67, where C here represents arbitrary client code. $\text{benignNodeGuar}_{\bar{L}, \Gamma}(t, p, R)(C, \tilde{C}_s)(G)$ (formally defined in Def. 5.20) asserts that, starting from a state satisfying p and under environment R , a node t executing client code C and instrumented server code $\tilde{C}_s$ concurrently guarantees G at every step. Next, we compose the client thread guarantee with the RG judgment semantics for server to establish the full benign node's guarantee in Lm. 68. Finally, we compose the benign nodes' guarantees and the faulty nodes' guarantee to derive the contextual instrumented world simulation in Lm. 80.

Definition 5.7 (Speculative transition of abstract world).

$$\Omega \Rightarrow \Omega' \stackrel{\text{def}}{=} \forall \mathbb{W}' \in \Omega'. \exists \mathbb{W} \in \Omega. \mathbb{W} \xrightarrow{T}^* \mathbb{W}'$$

Definition 5.8. $\text{benignCltSet}(\text{let } (\tilde{\Pi}, \Gamma) \text{ in } \tilde{C}_{t_1} \parallel \dots \parallel \tilde{C}_{t_n}) \stackrel{\text{def}}{=} \{t_1, \dots, t_n\}$

Definition 5.9 (Instrumented world simulation).

$$\tilde{P} \preceq_p^{\mathcal{N}_{\text{bad}}} \mathbb{P} \stackrel{\text{def}}{=} \forall \sigma_c, \sigma_o, \sigma_a, \theta. (\sigma_o, \sigma_a, \theta, \text{benignCltSet}(\tilde{P}), \mathcal{N}_{\text{bad}}) \models_{\text{init}} p \implies (\tilde{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o, \sigma_a, \theta)) \preceq \{(\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta))\}$$

Whenever $\tilde{W} \preceq \Omega$, we have the following:

- $\tilde{W} \not\hookrightarrow \text{abort}$;
- if $\tilde{W} \hookrightarrow \tilde{W}'$, then there exists Ω' s.t. for all $\mathbb{W}'$ in Ω' , there exists $\mathbb{W}$ in Ω s.t. $\mathbb{W} \xrightarrow{*} \mathbb{W}'$, and $\tilde{W}' \preceq \Omega'$;
- if $\tilde{W} \xrightarrow{e} \tilde{W}'$, then there exists $\Omega', T, \Omega \Rightarrow \Omega', \tilde{W}' \preceq \Omega'$ and $\text{get-obsv}(e) = \text{get-obsv}(T)$.

Definition 5.10 (Contextual instrumented world simulation).

$$\tilde{\Pi} \preceq_{p, \eta} \Gamma \stackrel{\text{def}}{=} \forall n, \mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}, t_1, \dots, t_n, C_{t_1}, \dots, C_{t_n}. \mathcal{N}_{\text{ok}} = \{t_1, \dots, t_n\} \wedge (n, |\mathcal{N}_{\text{bad}}|) \in \eta \wedge \mathcal{N}_{\text{ok}} \cap \mathcal{N}_{\text{bad}} = \emptyset \implies \text{let } (\tilde{\Pi}, \Gamma) \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n} \preceq_p^{\mathcal{N}_{\text{bad}}} \text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}$$

In Lm. 83, the way of defining conclusion $\tilde{\Pi} \preceq_{p, \eta} \Gamma$ is similar to the definition of $\tilde{\Pi} \preceq_{p, \eta} \Gamma$, but the left side is the concrete machine. To prove this lemma, we only need to prove that a step of concrete side corresponds to zero or multiple steps in the instrumented side.

Definition 5.11 (World simulation).

$$P \preceq_{\varphi}^{\mathcal{N}_{\text{bad}}} \mathbb{P} \stackrel{\text{def}}{=} \forall \sigma_c, \sigma_o, \theta. \varphi(\sigma_o) = \theta \implies (P, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)) \prec \{(\mathbb{P}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta))\}$$

Whenever $W \prec \Omega$, we have the following:

- if $W \xrightarrow{*} W'$, then there exists Ω' s.t. for all $\mathbb{W}'$ in Ω' , there exists $\mathbb{W}$ in Ω s.t. $\mathbb{W} \xrightarrow{*} \mathbb{W}'$, and $W' \prec \Omega'$;
- if $W \xrightarrow{e} W'$, then there exists $\Omega', T, \Omega \Rightarrow \Omega', W' \prec \Omega'$ and $\text{get-obsv}(e) = \text{get-obsv}(T)$.

Definition 5.12 (Contextual world simulation).

$\Pi \prec_{\varphi, \eta} \Gamma \stackrel{\text{def}}{=} \forall n, N_{\text{ok}}, N_{\text{bad}}, t_1, \dots, t_n, C_{t_1}, \dots, C_{t_n}. N_{\text{ok}} = \{t_1, \dots, t_n\} \wedge (n, |N_{\text{bad}}|) \in \eta \wedge N_{\text{ok}} \cap N_{\text{bad}} = \emptyset \implies \text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n} \prec_{\varphi}^{N_{\text{bad}}} \text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}$

5.3.1 Proof of Lm. 12 (Logic Judgment to Semantics).

Lemma 12. $\forall p, \eta, \Pi, \Gamma. \vdash \{p, \eta\}(\Pi, \Gamma) \implies \exists \tilde{\Pi}. \text{Er}(\tilde{\Pi}) = \Pi \wedge \text{WFF}_{p, \eta}(\tilde{\Pi}, \Gamma)$

PROOF. By definition of OBJ rule, Lm. 13 and Lm. 14 immediately. $\square$

Lemma 13. $\forall R, G, p, q, \tilde{C}. R, G \vdash \{p\}\tilde{C}\{q\} \implies \text{stable}(\{p, q\}, R)$

Definition 5.13. $(\tilde{C}, \Sigma) \hookrightarrow_t^k \text{abort}$ is defined as follows:

$$\frac{}{(\tilde{C}, \Sigma) \hookrightarrow_t \text{abort}} \quad \frac{(\tilde{C}, \Sigma) \hookrightarrow_t (\tilde{C}', \Sigma') \quad (\tilde{C}', \Sigma') \hookrightarrow_t^k \text{abort}}{(\tilde{C}, \Sigma) \hookrightarrow_t^{k+1} \text{abort}}$$

Definition 5.14. $(\tilde{C}, \Sigma) \hookrightarrow_t^* \text{abort} \stackrel{\text{def}}{=} \exists k. (\tilde{C}, \Sigma) \hookrightarrow_t^k \text{abort}$

Note that $(\tilde{C}, \Sigma) \hookrightarrow_t^* \text{abort}$ is introduced as a sugar for $\nexists k. (\tilde{C}, \Sigma) \hookrightarrow_t^k \text{abort}$.

Definition 5.15 (Sequential judgment semantics). $\models_t \{p\}\tilde{C}\{q\}$ iff for all Σ , if $\Sigma \models p$, then

1. $(\tilde{C}, \Sigma) \hookrightarrow_t^* \text{abort}$;
2. for all Σ' , if $(\tilde{C}, \Sigma) \hookrightarrow_t^* (\text{skip}, \Sigma')$, then $\Sigma' \models q$.

Definition 5.16 (Rely-guarantee-style judgment semantics). $R, G \models_t \{p\}\tilde{C}\{q\}$ iff for all Σ , if $\Sigma \models p$, the following holds

1. for all Σ' , if $(\tilde{C}, \Sigma) \xrightarrow{R}_t^* (\text{skip}, \Sigma')$, then $\Sigma' \models q$;
2. for all n , $(\tilde{C}, \Sigma, R) \text{ guar}_t^n G$.

where $(\tilde{C}, \Sigma, R) \text{ guar}_t^n G$ is inductively defined as follows:

1. $(\tilde{C}, \Sigma, R) \text{ guar}_t^0 G$ always holds;
2. $(\tilde{C}, \Sigma, R) \text{ guar}_t^{k+1} G$ iff
 - a. $(\tilde{C}, \Sigma) \hookrightarrow_t^* \text{abort}$;
 - b. for all $\tilde{C}', \Sigma'$, if $(\tilde{C}, \Sigma) \hookrightarrow_t (\tilde{C}', \Sigma')$, then $(\Sigma, \Sigma') \models G$ and $(\tilde{C}', \Sigma', R) \text{ guar}_t^k G$;
 - c. for all Σ' , if $(\Sigma, \Sigma') \models R$ then $(\tilde{C}, \Sigma', R) \text{ guar}_t^k G$.

where

$$\frac{(\tilde{C}, \Sigma) \hookrightarrow_t (\tilde{C}', \Sigma')}{(\tilde{C}, \Sigma) \xrightarrow{R}_t (\tilde{C}', \Sigma')} \text{LOCAL} \quad \frac{(\Sigma, \Sigma') \models R}{(\tilde{C}, \Sigma) \xrightarrow{R}_t (\tilde{C}, \Sigma')} \text{ENVIRONMENT}$$

In the following rely-guarantee judgment soundness proof, all R implicitly ensures $R \implies (\exists t. \text{tid} = t \times \text{tid} = t)$.

Lemma 14 (Logic soundness as rely-guarantee reasoning).

$\forall R, G, p, t, \tilde{C}, q. R, G \vdash \{p\}\tilde{C}\{q\} \implies R, G \models_t \{p \wedge \text{tid} = t\}\tilde{C}\{q \wedge \text{tid} = t\}$

PROOF. Induction over the derivation of the judgment $R, G \vdash \{p\}\tilde{C}\{q\}$, then by Lm. 29, Lm. 16, Lm. 18, Lm. 20, Lm. 21, Lm. 22, Lm. 26, Lm. 28 we the goal is proved. $\square$

Lemma 15. $\forall \Sigma, t, n, G. (\text{skip}, \Sigma, \text{Id}) \text{ guar}_t^n G$

PROOF. By induction on n immediately. $\square$

Lemma 16 (ATOM sound). $\forall G, p, \tilde{C}, q, t. \models_t \{p \wedge \text{tid} = t\} \tilde{C} \{q \wedge \text{tid} = t\} \wedge p \times q \Rightarrow G \implies \text{Id}, G \models_t \{p \wedge \text{tid} = t\} \langle \tilde{C} \rangle \{q \wedge \text{tid} = t\}$

PROOF. By definition, given arbitrary $\Sigma \models p \wedge \text{tid} = t$, we need to prove

$$\forall \Sigma'. (\langle \tilde{C} \rangle, \Sigma) \xrightarrow{\text{Id}}^* (\mathbf{skip}, \Sigma') \implies \Sigma' \models q \wedge \text{tid} = t \quad (\text{goal1})$$

$$\forall n. (\langle \tilde{C} \rangle, \Sigma, \text{Id}) \text{ guar}_t^n G \quad (\text{goal2})$$

To prove (goal1), that is to take arbitrary Σ', k s.t.

$$(\langle \tilde{C} \rangle, \Sigma) \xrightarrow{\text{Id}}^k (\mathbf{skip}, \Sigma') \quad (1)$$

Then prove $\Sigma' \models q \wedge \text{tid} = t$. We prove by induction on k . The base case is trivial by contradiction as $\langle \tilde{C} \rangle \neq \mathbf{skip}$; the inductive case is that we need to prove the goal when there exists k' s.t. $k = k' + 1$. We know there exists $\tilde{C}'', \Sigma''$ s.t.

$$(\langle \tilde{C} \rangle, \Sigma) \xrightarrow{\text{Id}}_t (\tilde{C}'', \Sigma'') \quad (1)$$

$$(\tilde{C}'', \Sigma'') \xrightarrow{\text{Id}}^{k'} (\mathbf{skip}, \Sigma') \quad (2)$$

By further case analysis on (1), we have the following cases:

- LOCAL: Then $\tilde{C}'' = \mathbf{skip}$ and $(\langle \tilde{C} \rangle, \Sigma) \xrightarrow{*}_t (\mathbf{skip}, \Sigma')$. Apply premise $\models_t \{p \wedge \text{tid} = t\} \tilde{C} \{q \wedge \text{tid} = t\}$ we prove $\Sigma'' \models q \wedge \text{tid} = t$, then by (2) we know $\Sigma' = \Sigma''$, thus we prove $\Sigma' \models q \wedge \text{tid} = t$.
- ENVIRONMENT: Then $\tilde{C}'' = \langle \tilde{C} \rangle$ and $(\Sigma, \Sigma'') \models \text{Id}$, thus we know $\Sigma'' \models p \wedge \text{tid} = t$, by (2) and induction hypothesis we prove the goal.

We prove (goal2) by induction on n , the base case trivially holds by definition; the inductive case is, when there exists n' s.t. $n = n' + 1$, we need to prove

$$(\langle \tilde{C} \rangle, \Sigma) \xrightarrow{*}_t \mathbf{abort} \quad (\text{goal2.1})$$

$$\forall \tilde{C}', \Sigma'. (\langle \tilde{C} \rangle, \Sigma) \xrightarrow{*}_t (\tilde{C}', \Sigma') \implies (\Sigma, \Sigma') \models G \wedge (\tilde{C}', \Sigma', \text{Id}) \text{ guar}_t^{n'} G \quad (\text{goal2.2})$$

$$\forall \Sigma'. (\Sigma, \Sigma') \models \text{Id} \implies (\langle \tilde{C} \rangle, \Sigma', \text{Id}) \text{ guar}_t^{n'} G \quad (\text{goal2.3})$$

By premise we know $(\tilde{C}, \Sigma) \xrightarrow{*}_t \mathbf{abort}$, thus we prove (goal2.1). To prove (goal2.2), given arbitrary $\tilde{C}', \Sigma'$ s.t. $(\langle \tilde{C} \rangle, \Sigma) \xrightarrow{*}_t (\tilde{C}', \Sigma')$, by INS-ATOM we know $\tilde{C}' = \mathbf{skip}$, thus by premise $\models_t \{p \wedge \text{tid} = t\} \tilde{C} \{q \wedge \text{tid} = t\}$ we know $\Sigma' \models q \wedge \text{tid} = t$, thus we know $(\Sigma, \Sigma') \models p \wedge \text{tid} = t \times q \wedge \text{tid} = t$, then by $p \times q \Rightarrow G$ we prove $(\Sigma, \Sigma') \models G$. By Lm. 15 we prove $(\mathbf{skip}, \Sigma', \text{Id}) \text{ guar}_t^{n'} G$

To prove (goal2.3), we only need to prove $(\langle \tilde{C} \rangle, \Sigma, \text{Id}) \text{ guar}_t^{n'} G$, which by induction hypothesis immediately. $\square$

Lemma 17. $\forall \Sigma, t, n, G, p. \text{stable}(p, R) \wedge \Sigma \models p \implies (\mathbf{skip}, \Sigma, R) \text{ guar}_t^n G$

PROOF. By induction on n . $\square$

Lemma 18 (ATOM-R sound). $\forall R, G, p, q, \tilde{C}, t. \text{Id}, G \models_t \{p \wedge \text{tid} = t\} \langle \tilde{C} \rangle \{q \wedge \text{tid} = t\} \wedge \text{stable}(\{p, q\}, R) \implies R, G \models_t \{p \wedge \text{tid} = t\} \langle \tilde{C} \rangle \{q \wedge \text{tid} = t\}$

PROOF. By definition, given arbitrary $\Sigma \models p \wedge \text{tid} = t$, we need to prove

$$\forall \Sigma'. (\langle \tilde{C} \rangle, \Sigma) \xrightarrow{R}^* (\mathbf{skip}, \Sigma') \implies \Sigma' \models q \wedge \text{tid} = t \quad (\text{goal1})$$

$$\forall n. (\langle \tilde{C} \rangle, \Sigma, R) \text{ guar}_t^n G \quad (\text{goal2})$$

To prove (goal1), that is to take arbitrary Σ', k s.t.

$$(\langle \widetilde{C} \rangle, \Sigma) \xrightarrow{R}_t^k (\mathbf{skip}, \Sigma')$$

We prove $\Sigma' \models q \wedge \text{tid} = t$ by induction on k . The base case is trivial by contradiction as $\langle \widetilde{C} \rangle \neq \mathbf{skip}$; the inductive case is to prove the goal when there exists k' s.t. $k = k' + 1$. Then we know there exists $\widetilde{C}'', \Sigma''$ s.t.

$$(\langle \widetilde{C} \rangle, \Sigma) \xrightarrow{R}_t (\widetilde{C}'', \Sigma'') \quad (1)$$

$$(\widetilde{C}'', \Sigma'') \xrightarrow{R}_{t'}^{k'} (\mathbf{skip}, \Sigma') \quad (2)$$

Further case analysis on (1):

- **LOCAL:** Then $\widetilde{C}'' = \mathbf{skip}$ and $(\langle \widetilde{C} \rangle, \Sigma) \xrightarrow{R}_t (\mathbf{skip}, \Sigma')$. Apply premise $\text{Id}, G \models_t \{p \wedge \text{tid} = t\} \langle \widetilde{C} \rangle \{q \wedge \text{tid} = t\}$ we prove $\Sigma'' \models q \wedge \text{tid} = t$. Then by premise we know $\text{stable}(q \wedge \text{tid} = t, R)$, thus along with (2) we prove $\Sigma' \models q \wedge \text{tid} = t$.
- **ENVIRONMENT:** Then $\widetilde{C}'' = \langle \widetilde{C} \rangle$ and $(\Sigma, \Sigma'') \models R$, thus we know $\Sigma'' \models p \wedge \text{tid} = t$, by (2) and induction hypothesis we prove the goal.

We prove *(goal2)* by induction on n . The base case trivially holds by definition; the inductive case is, when there exists n' s.t. $n = n' + 1$, we need to prove

$$(\langle \widetilde{C} \rangle, \Sigma) \xrightarrow{R}_t \mathbf{abort} \quad (\text{goal2.1})$$

$$\forall \widetilde{C}', \Sigma'. (\langle \widetilde{C} \rangle, \Sigma) \xrightarrow{R}_t (\widetilde{C}', \Sigma') \implies (\Sigma, \Sigma') \models G \wedge (\widetilde{C}', \Sigma', R) \text{ guar}_{t'}^{n'} G \quad (\text{goal2.2})$$

$$\forall \Sigma'. (\Sigma, \Sigma') \models R \implies (\langle \widetilde{C} \rangle, \Sigma', R) \text{ guar}_{t'}^{n'} G \quad (\text{goal2.3})$$

By premise $\text{Id}, G \models_t \{p \wedge \text{tid} = t\} \langle \widetilde{C} \rangle \{q \wedge \text{tid} = t\}$ and $\Sigma \models p \wedge \text{tid} = t$ we know

$$\forall n. (\langle \widetilde{C} \rangle, \Sigma, \text{Id}) \text{ guar}_t^n G \quad (3)$$

By (3) we know $(\langle \widetilde{C} \rangle, \Sigma) \xrightarrow{R}_t \mathbf{abort}$, thus we prove *(goal2.1)*. To prove *(goal2.2)*, given arbitrary $\widetilde{C}', \Sigma'$ s.t. $(\langle \widetilde{C} \rangle, \Sigma) \xrightarrow{R}_t (\widetilde{C}', \Sigma')$, by **INS-ATOM** we know $\widetilde{C}' = \mathbf{skip}$, thus apply $(\langle \widetilde{C} \rangle, \Sigma) \xrightarrow{R}_t (\widetilde{C}', \Sigma')$ on (3) we prove $(\Sigma, \Sigma') \models G$. By premise we also know $\Sigma' \models q \wedge \text{tid} = t$, thus by $\text{stable}(q \wedge \text{tid} = t, R)$ and **Lm. 17** we prove $(\mathbf{skip}, \Sigma', R) \text{ guar}_{t'}^{n'} G$.

To prove *(goal2.3)*, given arbitrary Σ' s.t. $(\Sigma, \Sigma') \models R$, we need to prove $(\langle \widetilde{C} \rangle, \Sigma', R) \text{ guar}_{t'}^{n'} G$. By premise we know $\text{stable}(p \wedge \text{tid} = t, R)$, we know $\Sigma' \models p \wedge \text{tid} = t$, thus by induction hypothesis we prove the goal. $\square$

Lemma 19. $\forall \widetilde{C}_1, \widetilde{C}_2, \Sigma, \Sigma', R, k, t. (\widetilde{C}_1; \widetilde{C}_2, \Sigma) \xrightarrow{R}_t^k (\mathbf{skip}, \Sigma') \implies \exists \Sigma''. (\widetilde{C}_1, \Sigma) \xrightarrow{R}_t^* (\mathbf{skip}, \Sigma'') \wedge (\widetilde{C}_2, \Sigma'') \xrightarrow{R}_t^* (\mathbf{skip}, \Sigma')$

PROOF. By induction on k . $\square$

Lemma 20 (**P-SEQ** sound).

$\forall R, G, p, q, r, \widetilde{C}_1, \widetilde{C}_2, t. R, G \models_t \{p \wedge \text{tid} = t\} \widetilde{C}_1 \{q \wedge \text{tid} = t\} \wedge R, G \models_t \{q \wedge \text{tid} = t\} \widetilde{C}_2 \{r \wedge \text{tid} = t\} \implies R, G \models_t \{p \wedge \text{tid} = t\} \widetilde{C}_1; \widetilde{C}_2 \{r \wedge \text{tid} = t\}$

PROOF. By definition, given arbitrary $\Sigma \models p \wedge \text{tid} = t$, we need to prove

$$\forall \Sigma'. (\widetilde{C}_1; \widetilde{C}_2, \Sigma) \xrightarrow{R}_t^* (\mathbf{skip}, \Sigma') \implies \Sigma' \models r \wedge \text{tid} = t \quad (\text{goal1})$$

$$\forall n. (\widetilde{C}_1; \widetilde{C}_2, \Sigma, R) \text{ guar}_t^n G \quad (\text{goal2})$$

To prove *(goal1)*, given arbitrary Σ', k s.t.

$$(\tilde{C}_1; \tilde{C}_2, \Sigma) \xrightarrow{R}_t^k (\mathbf{skip}, \Sigma'),$$

apply Lm. 19 we know there exists Σ'' s.t.

$$(\tilde{C}_1, \Sigma) \xrightarrow{R}_t^* (\mathbf{skip}, \Sigma'') \quad (1)$$

$$(\tilde{C}_2, \Sigma'') \xrightarrow{R}_t^* (\mathbf{skip}, \Sigma') \quad (2)$$

Apply (1) on premise $R, G \models_t \{p \wedge \text{tid} = t\} \tilde{C}_1 \{q \wedge \text{tid} = t\}$ we know $\Sigma'' \models q \wedge \text{tid} = t$, then by (2) and premise $R, G \models_t \{q \wedge \text{tid} = t\} \tilde{C}_2 \{r \wedge \text{tid} = t\}$ we prove $\Sigma' \models r \wedge \text{tid} = t$, thus we prove (goal1).

To prove (goal2), first by premise we know

$$\forall k. (\tilde{C}_1, \Sigma, R) \text{ guar}_t^k G \quad (3)$$

$$\forall \Sigma''. \Sigma'' \models q \wedge \text{tid} = t \implies \forall k. (\tilde{C}_2, \Sigma'', R) \text{ guar}_t^k G \quad (4)$$

We prove (goal2) by induction on n . The base case trivially holds by definition; the inductive case is, when there exists n' s.t. $n = n' + 1$, we need to prove

$$(\tilde{C}_1; \tilde{C}_2, \Sigma) \xrightarrow{R}_t \mathbf{abort} \quad (\text{goal2.1})$$

$$\forall \tilde{C}', \Sigma'. (\tilde{C}_1; \tilde{C}_2, \Sigma) \xrightarrow{R}_t (\tilde{C}', \Sigma') \implies (\Sigma, \Sigma') \models G \wedge (\tilde{C}', \Sigma', R) \text{ guar}_t^{n'} G \quad (\text{goal2.2})$$

$$\forall \Sigma'. (\Sigma, \Sigma') \models R \implies (\tilde{C}_1; \tilde{C}_2, \Sigma', R) \text{ guar}_t^{n'} G \quad (\text{goal2.3})$$

We prove (goal2.1) by contradiction. Assume $(\tilde{C}_1; \tilde{C}_2, \Sigma) \xrightarrow{R}_t \mathbf{abort}$, then we know $(\tilde{C}_1, \Sigma) \xrightarrow{R}_t \mathbf{abort}$, which contradict (3).

To prove (goal2.2), given arbitrary $\tilde{C}', \Sigma'$ s.t. $(\tilde{C}_1; \tilde{C}_2, \Sigma) \xrightarrow{R}_t (\tilde{C}', \Sigma')$, case analysis on execution:

- If $\tilde{C}_1 = \mathbf{skip}$ and $(\mathbf{skip}; \tilde{C}_2, \Sigma) \xrightarrow{R}_t (\tilde{C}_2, \Sigma)$, then by premise we know $\Sigma \models q \wedge \text{tid} = t$, thus apply (4) we prove $(\tilde{C}_2, \Sigma, R) \text{ guar}_t^{n'} G$.
- If there exists $\tilde{C}'_1$ s.t. $(\tilde{C}_1, \Sigma) \xrightarrow{R}_t (\tilde{C}'_1, \Sigma')$ and $(\tilde{C}_1; \tilde{C}_2, \Sigma) \xrightarrow{R}_t (\tilde{C}'_1; \tilde{C}_2, \Sigma')$, then apply (3) we prove $(\Sigma, \Sigma') \models G$ and $\forall k. (\tilde{C}'_1, \Sigma', R) \text{ guar}_t^k G$. Apply induction hypothesis we know $(\tilde{C}'_1; \tilde{C}_2, \Sigma', R) \text{ guar}_t^{n'} G$, thus we prove the goal.

To prove (goal2.3), for arbitrary Σ' s.t. $(\Sigma, \Sigma') \models R$, by (3) we know $\forall n. (\tilde{C}_1, \Sigma', R) \text{ guar}_t^n G$. Then apply induction hypothesis we prove $(\tilde{C}_1; \tilde{C}_2, \Sigma', R) \text{ guar}_t^{n'} G$.

□

Lemma 21 (P-WHILE sound).

$$\forall R, G, p, q, \tilde{C}, B, t. R, G \models_t \{p \wedge B \wedge \text{tid} = t\} \tilde{C} \{p \wedge \text{tid} = t\} \wedge \text{stable}(\neg B, R) \wedge \text{stable}(p, R) \implies R, G \models_t \{p \wedge \text{tid} = t\} \mathbf{while} (B) \mathbf{do} \tilde{C} \{p \wedge \neg B \wedge \text{tid} = t\}$$

PROOF. By definition, given arbitrary $\Sigma \models p \wedge \text{tid} = t$, we need to prove

$$\forall \Sigma'. (\mathbf{while} (B) \mathbf{do} \tilde{C}, \Sigma) \xrightarrow{R}_t^* (\mathbf{skip}, \Sigma') \implies \Sigma' \models p \wedge \neg B \wedge \text{tid} = t \quad (\text{goal1})$$

$$\forall n. (\mathbf{while} (B) \mathbf{do} \tilde{C}, \Sigma, R) \text{ guar}_t^n G \quad (\text{goal2})$$

To prove (goal1), given arbitrary k, Σ' s.t. $(\mathbf{while} (B) \mathbf{do} \tilde{C}, \Sigma) \xrightarrow{R}_t^k (\mathbf{skip}, \Sigma')$, induction on k . The base case is trivial as $\mathbf{while} (B) \mathbf{do} \tilde{C} \neq \mathbf{skip}$; in inductive case we need to prove the goal when there exists k' s.t. $k = k' + 1$. We know there exists $\tilde{C}''$ and Σ'' s.t.

$$(\mathbf{while} (B) \mathbf{do} \tilde{C}, \Sigma) \xrightarrow{R}_t (\tilde{C}'', \Sigma'') \quad (1)$$

$$(\tilde{C}'', \Sigma'') \xrightarrow{R}_t^{k'} (\mathbf{skip}, \Sigma') \quad (2)$$

By case analysis on (1), we have the following 3 cases:

- If environment take a step, formally $(\Sigma, \Sigma'') \models R$ and $\widetilde{C}'' = \mathbf{while} (B) \mathbf{do} \widetilde{C}$, then by (2) we know $(\mathbf{while} (B) \mathbf{do} \widetilde{C}, \Sigma'') \xrightarrow{R}_t^* (\mathbf{skip}, \Sigma')$. By $\text{stable}(p \wedge \text{tid} = t, R)$ we know $\Sigma'' \models p \wedge \text{tid} = t$, thus apply induction hypothesis we know $\Sigma' \models p \wedge \neg B \wedge \text{tid} = t$.
- If it is a local step and the while loop continues, formally $\llbracket B \rrbracket_{\Sigma, \sigma_o} = \mathbf{true}$, $\widetilde{C}'' = \widetilde{C}; \mathbf{while} (B) \mathbf{do} \widetilde{C}$ and $\Sigma'' = \Sigma$, then we know $\Sigma'' \models p \wedge B \wedge \text{tid} = t$. By (2) we know there exists Σ''' s.t.

$$(\widetilde{C}, \Sigma'') \xrightarrow{R}_t^* (\mathbf{skip}, \Sigma''') \quad (3)$$

$$(\mathbf{while} (B) \mathbf{do} \widetilde{C}, \Sigma'') \xrightarrow{R}_t^* (\mathbf{skip}, \Sigma') \quad (4)$$

Apply (3) on premise we know $\Sigma''' \models p \wedge \text{tid} = t$, along with (4) and induction hypothesis we prove $\Sigma' \models p \wedge \neg B \wedge \text{tid} = t$.

- If local step and the while loop ends, formally $\llbracket B \rrbracket_{\Sigma, \sigma_o} = \mathbf{false}$, $\widetilde{C}'' = \mathbf{skip}$ and $\Sigma'' = \Sigma$, then we know $\Sigma'' \models p \wedge \neg B \wedge \text{tid} = t$. Along with (2), $\text{stable}(\neg B, R)$ and $\text{stable}(p \wedge \text{tid} = t, R)$ we conclude $\Sigma' \models p \wedge \neg B \wedge \text{tid} = t$.

We prove (goal2) by induction on n . The base case is trivial due to definition; the inductive case is there exists n' s.t. $n = n' + 1$, we need to prove

$$(\mathbf{while} (B) \mathbf{do} \widetilde{C}, \Sigma) \xrightarrow{t} \mathbf{abort} \quad (\text{goal2.1})$$

$$\forall \widetilde{C}', \Sigma'. (\mathbf{while} (B) \mathbf{do} \widetilde{C}, \Sigma) \xrightarrow{t} (\widetilde{C}', \Sigma') \implies (\Sigma, \Sigma') \models G$$

$$\wedge (\widetilde{C}', \Sigma', R) \text{ guar}_{t'}^{n'} G \quad (\text{goal2.2})$$

$$\forall \Sigma'. (\Sigma, \Sigma') \models R \implies (\mathbf{while} (B) \mathbf{do} \widetilde{C}, \Sigma', R) \text{ guar}_{t'}^{n'} G \quad (\text{goal2.3})$$

To prove (goal2.1), we only need to prove $\forall k. \neg((\mathbf{while} (B) \mathbf{do} \widetilde{C}, \Sigma) \xrightarrow{t}^k \mathbf{abort})$. Induction on k , the base case trivially holds as

$$(\mathbf{while} (B) \mathbf{do} \widetilde{C}, \Sigma) \xrightarrow{t}^0 (\mathbf{while} (B) \mathbf{do} \widetilde{C}, \Sigma).$$

The inductive case is there exists k' s.t. $k = k' + 1$, then we need to prove there exists $\widetilde{C}'', \Sigma''$ s.t.

$$(\mathbf{while} (B) \mathbf{do} \widetilde{C}, \Sigma) \xrightarrow{t} (\widetilde{C}'', \Sigma''),$$

$$\neg((\widetilde{C}'', \Sigma'') \xrightarrow{t}^{k'} \mathbf{abort}).$$

The first sub-goal is trivial as **while** statement never goes abort in one step. We prove the second sub-goal by case analysis on the first sub-goal (we already proved it):

- If the **while** statement continues, formally $\widetilde{C}'' = \widetilde{C}; \mathbf{while} (B) \mathbf{do} \widetilde{C}$, $\llbracket B \rrbracket_{\Sigma, \sigma_o}$ and $\Sigma'' = \Sigma$, then apply premise we know there exists Σ''' s.t. $(\widetilde{C}, \Sigma) \xrightarrow{t}^* (\mathbf{skip}, \Sigma''')$ and $\Sigma''' \models p \wedge \text{tid} = t$. By induction hypothesis we know $\neg((\mathbf{while} (B) \mathbf{do} \widetilde{C}, \Sigma'') \xrightarrow{t}^* \mathbf{abort})$, thus by definition we conclude the second sub-goal.
- If the **while** statement ends, formally $\widetilde{C}'' = \mathbf{skip}$, $\llbracket B \rrbracket_{\Sigma, \sigma_o} = \mathbf{false}$ and $\Sigma'' = \Sigma$, then by definition we prove $\neg((\mathbf{skip}, \Sigma) \xrightarrow{t}^{k'} \mathbf{abort})$, thus we conclude the second sub-goal.

To prove (goal2.2), given arbitrary $\widetilde{C}', \Sigma'$ s.t. $(\mathbf{while} (B) \mathbf{do} \widetilde{C}, \Sigma) \xrightarrow{t} (\widetilde{C}', \Sigma')$, by case analysis we have the following case:

- If $\llbracket B \rrbracket_{\Sigma, \sigma_o} = \mathbf{true}$ and $\widetilde{C}' = \widetilde{C}; \mathbf{while} (B) \mathbf{do} \widetilde{C}$, then we only need to prove:

$$\forall k < n. (\forall n'. (\widetilde{C}, \Sigma, R) \text{ guar}_{t'}^{n'} G) \implies (\widetilde{C}; \mathbf{while} (B) \mathbf{do} \widetilde{C}, \Sigma, R) \text{ guar}_{t'}^n G,$$

which by further induction on k immediately.

- If $\llbracket B \rrbracket_{\Sigma, \sigma_o} = \mathbf{false}$ and $\widetilde{C}' = \mathbf{skip}$, then apply Lm. 17 we conclude the goal immediately.

To prove (*goal2.3*), given arbitrary Σ' s.t. $(\Sigma, \Sigma') \models R$. By premise we know $\Sigma' \models p \wedge \text{tid} = t$, thus apply induction hypothesis we prove **(while** (B) **do** $\widetilde{C}, \Sigma', R$) $\text{guar}_t^{n'}$ G . Therefore, we have concluded (*goal2.3*). $\square$

Lemma 22 (P-IF sound).

$\forall R, G, p, q, B, \widetilde{C}_1, \widetilde{C}_2, t. R, G \models_t \{p \wedge B \wedge \text{tid} = t\} \widetilde{C}_1 \{q \wedge \text{tid} = t\} \wedge R, G \models_t \{p \wedge \neg B \wedge \text{tid} = t\} \widetilde{C}_2 \{q \wedge \text{tid} = t\} \wedge \text{stable}(p, R) \implies R, G \models_t \{p \wedge \text{tid} = t\} \text{if } (B) \text{ then } \widetilde{C}_1 \text{ else } \widetilde{C}_2 \{q \wedge \text{tid} = t\}$

PROOF. Similar to the proof of Lm. 21. $\square$

Lemma 23. For all $x, v, R, \widetilde{C}, \widetilde{C}', \sigma_o, \sigma'_o, M, M', \sigma_a, \sigma'_a, \Delta, \Delta', t, k$, if

1. $x \notin \text{dom}(\sigma_a)$,
2. $x \notin \text{FV}(R) \cup \text{FV}(\widetilde{C})$,
3. $(\widetilde{C}, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{R}_t^k (\widetilde{C}', ((\sigma'_o, M', \sigma'_a), \Delta'))$,

then $(\widetilde{C}, ((\sigma_o \uplus \{x \rightsquigarrow v\}, M, \sigma_a), \Delta)) \xrightarrow{R}_t^k (\widetilde{C}', ((\sigma'_o \uplus \{x \rightsquigarrow v\}, M', \sigma'_a), \Delta'))$.

Lemma 24. For all $\widetilde{C}, \sigma_o, M, \sigma_a, \Delta, x, v$, if

1. $x \notin \text{dom}(\sigma_a)$,
2. $x \notin \text{FV}(\widetilde{C})$,
3. $(\widetilde{C}, ((\sigma_o \uplus \{x \rightsquigarrow v\}, M, \sigma_a), \Delta)) \not\hookrightarrow_t \text{abort}$,

then $(\widetilde{C}, ((\sigma_o, M, \sigma_a), \Delta)) \not\hookrightarrow_t \text{abort}$.

Lemma 25. $\forall x, G. x \notin \text{FV}(G) \implies ((\exists x. G) \Leftrightarrow G)$

Definition 5.17. $\Sigma|_{\sigma_o \mapsto \sigma_o \uplus \sigma} \stackrel{\text{def}}{=} ((\Sigma. \sigma_o \uplus \sigma, \Sigma. M, \Sigma. \sigma_a), \Sigma. \Delta)$

Lemma 26 (P-EX sound). $\forall R, G, p, q, \widetilde{C}, t. R, G \models_t \{p \wedge \text{tid} = t\} \widetilde{C} \{q \wedge \text{tid} = t\} \wedge x \notin \text{FV}(R) \cup \text{FV}(G) \cup \text{FV}(\widetilde{C}) \implies R, G \models_t \{(\exists x. p) \wedge \text{tid} = t\} \widetilde{C} \{(\exists x. q) \wedge \text{tid} = t\}$

PROOF. By definition, given arbitrary $\Sigma \models (\exists x. p) \wedge \text{tid} = t$, we need to prove

$$\begin{aligned} \forall \Sigma'. (\widetilde{C}, \Sigma) \xrightarrow{R}_t^* (\text{skip}, \Sigma') &\implies \Sigma' \models (\exists x. q) \wedge \text{tid} = t & (\text{goal1}) \\ \forall n. (\widetilde{C}, \Sigma, R) \text{ guar}_t^n G & & (\text{goal2}) \end{aligned}$$

Let $((\sigma_o, M, \sigma_a), \Delta) = \Sigma$, by definition we know $x \notin \text{dom}(\sigma_a)$, $x \neq \text{tid}$ and there exists v s.t.

$$((\sigma_o \uplus \{x \rightsquigarrow v\}, M, \sigma_a), \Delta) \models p \quad (1)$$

Apply premise we know

$$\begin{aligned} \forall \Sigma'. (\widetilde{C}, ((\sigma_o \uplus \{x \rightsquigarrow v\}, M, \sigma_a), \Delta)) \xrightarrow{R}_t^* (\text{skip}, \Sigma') &\implies \Sigma' \models q \wedge \text{tid} = t & (2) \\ \forall n. (\widetilde{C}, ((\sigma_o \uplus \{x \rightsquigarrow v\}, M, \sigma_a), \Delta), R) \text{ guar}_t^n G & & (3) \end{aligned}$$

To prove (*goal1*), given arbitrary Σ' s.t. $(\widetilde{C}, \Sigma) \xrightarrow{R}_t^* (\text{skip}, \Sigma')$, apply Lm. 23 we know $(\widetilde{C}, ((\sigma_o \uplus \{x \rightsquigarrow v\}, M, \sigma_a), \Delta)) \xrightarrow{R}_t^* (\widetilde{C}', \Sigma'|_{\sigma_o \mapsto \sigma_o \uplus \{x \rightsquigarrow v\}})$. Apply (2) we know $\Sigma'|_{\sigma_o \mapsto \sigma_o \uplus \{x \rightsquigarrow v\}} \models q \wedge \text{tid} = t$, thus by definition we prove $\Sigma' \models (\exists x. q) \wedge \text{tid} = t$.

We prove (*goal2*) by induction on n . The base case trivially holds by definition; the inductive case is, when there exists n' s.t. $n = n' + 1$, we need to prove:

$$(\tilde{C}, \Sigma) \hookrightarrow_t \mathbf{abort} \quad (\text{goal2.1})$$

$$\forall \tilde{C}', \Sigma'. (\tilde{C}, \Sigma) \hookrightarrow_t (\tilde{C}', \Sigma') \implies (\Sigma, \Sigma') \models G \wedge (\tilde{C}', \Sigma', R) \text{ guar}_t^{n'} G \quad (\text{goal2.2})$$

$$\forall \Sigma'. (\Sigma, \Sigma') \models R \implies (\tilde{C}, \Sigma', R) \text{ guar}_t^{n'} G \quad (\text{goal2.3})$$

To prove (goal2.1), by (3) we know $(\tilde{C}, ((\sigma_o \uplus \{x \rightsquigarrow v\}, M, \sigma_a), \Delta)) \hookrightarrow_t \mathbf{abort}$, thus apply Lm. 24 we prove the goal.

To prove (goal2.2), given arbitrary $\tilde{C}', \Sigma'$ s.t. $(\tilde{C}, \Sigma) \hookrightarrow_t (\tilde{C}', \Sigma')$, apply Lm. 23 we know

$$(\tilde{C}, \Sigma|_{\sigma_o \mapsto \sigma_o \uplus \{x \rightsquigarrow v\}}) \hookrightarrow_t (\tilde{C}', \Sigma'|_{\sigma_o \mapsto \sigma_o \uplus \{x \rightsquigarrow v\}}).$$

Thus apply (3) we know

$$(\Sigma|_{\sigma_o \mapsto \sigma_o \uplus \{x \rightsquigarrow v\}}, \Sigma'|_{\sigma_o \mapsto \sigma_o \uplus \{x \rightsquigarrow v\}}) \models G \quad (4)$$

$$\forall n. (\tilde{C}', \Sigma'|_{\sigma_o \mapsto \sigma_o \uplus \{x \rightsquigarrow v\}}, R) \text{ guar}_t^n G \quad (5)$$

Then apply (4) on Lm. 25 we prove $(\Sigma, \Sigma') \models G$. Apply (5) on induction hypothesis we prove $(\tilde{C}', \Sigma', R) \text{ guar}_t^{n'} G$, thus we conclude (goal2.1).

To prove (goal2.3), for arbitrary Σ' s.t. $(\Sigma, \Sigma') \models R$, apply Lm. 25 we know

$$(\Sigma|_{\sigma_o \mapsto \sigma_o \uplus \{x \rightsquigarrow v\}}, \Sigma'|_{\sigma_o \mapsto \sigma_o \uplus \{x \rightsquigarrow v\}}) \models R.$$

Then apply (3) we know $\forall n. (\tilde{C}, \Sigma'|_{\sigma_o \mapsto \sigma_o \uplus \{x \rightsquigarrow v\}}, R) \text{ guar}_t^n G$. Apply induction hypothesis we prove $(\tilde{C}, \Sigma', R) \text{ guar}_t^{n'} G$, thus we conclude (goal2.3). $\square$

Lemma 27. $\forall \tilde{C}, \Sigma, \tilde{C}', \Sigma', R, R', t, k. R' \implies R \wedge (\tilde{C}, \Sigma) \xrightarrow{R'}_t^k (\tilde{C}', \Sigma') \implies (\tilde{C}, \Sigma) \xrightarrow{R}_t^k (\tilde{C}', \Sigma')$

Lemma 28 (P-CONSEQ sound).

$$\forall R, R', G, G', p, p', q, q', \tilde{C}, t. R' \implies R \wedge G \implies G' \wedge p' \implies p \wedge q \implies q' \wedge R, G \models_t \{p \wedge \text{tid} = t\} \tilde{C} \{q \wedge \text{tid} = t\} \implies R', G' \models_t \{p' \wedge \text{tid} = t\} \tilde{C} \{q' \wedge \text{tid} = t\}$$

PROOF. By definition, given arbitrary $\Sigma \models p' \wedge \text{tid} = t$, we need to prove

$$\forall \Sigma'. (\tilde{C}, \Sigma) \xrightarrow{R'}_t^* (\mathbf{skip}, \Sigma') \implies \Sigma' \models q' \wedge \text{tid} = t \quad (\text{goal1})$$

$$\forall n. (\tilde{C}, \Sigma, R') \text{ guar}_t^n G' \quad (\text{goal2})$$

By premise $p' \implies p$ we know $\Sigma \models p \wedge \text{tid} = t$. Apply premise $R, G \models_t \{p \wedge \text{tid} = t\} \tilde{C} \{q \wedge \text{tid} = t\}$ we know:

$$\forall \Sigma'. (\tilde{C}, \Sigma) \xrightarrow{R}_t^* (\mathbf{skip}, \Sigma') \implies \Sigma' \models q \wedge \text{tid} = t \quad (1)$$

$$\forall n. (\tilde{C}, \Sigma, R) \text{ guar}_t^n G \quad (2)$$

To prove (goal1), given arbitrary Σ' s.t. $(\tilde{C}, \Sigma) \xrightarrow{R'}_t^* (\mathbf{skip}, \Sigma')$, apply Lm. 27 we know:

$$(\tilde{C}, \Sigma) \xrightarrow{R}_t^* (\mathbf{skip}, \Sigma').$$

Apply (1) we know $\Sigma' \models q \wedge \text{tid} = t$, thus by premise we prove $\Sigma' \models q' \wedge \text{tid} = t$.

To prove (goal2), induction on n . Base case is trivial due to definition, the inductive case is, when there exists n' s.t. $n = n' + 1$, we need to prove

$$(\tilde{C}, \Sigma) \hookrightarrow_t \mathbf{abort} \quad (\text{goal2.1})$$

$$\forall \tilde{C}', \Sigma'. (\tilde{C}, \Sigma) \hookrightarrow_t (\tilde{C}', \Sigma') \implies (\Sigma, \Sigma') \models G' \wedge (\tilde{C}', \Sigma', R') \text{ guar}_t^{n'} G' \quad (\text{goal2.2})$$

$$\forall \Sigma'. (\Sigma, \Sigma') \models R' \implies (\tilde{C}, \Sigma', R') \text{ guar}_t^{n'} G' \quad (\text{goal2.3})$$

By (2) we conclude (*goal2.1*) immediately. To prove (*goal2.2*), given arbitrary $\widetilde{C}', \Sigma'$ s.t. $(\widetilde{C}, \Sigma) \xrightarrow{t} (\widetilde{C}', \Sigma')$, apply (2) we know $(\Sigma, \Sigma') \models G$ and $\forall n. (\widetilde{C}', \Sigma', R) \text{ guar}_t^n G$. Apply premise $G \Rightarrow G'$ we prove $(\Sigma, \Sigma') \models G'$; apply induction hypothesis we prove $(\widetilde{C}', \Sigma', R') \text{ guar}_t^{n'} G'$. Thus we conclude (*goal2.2*).

To prove (*goal2.3*), given arbitrary Σ' s.t. $(\Sigma, \Sigma') \models R'$, by premise $R' \Rightarrow R$ we know $(\Sigma, \Sigma') \models R$, then apply (2) we know $\forall n. (\widetilde{C}, \Sigma', R) \text{ guar}_t^n G$. By induction hypothesis we prove $(\widetilde{C}, \Sigma', R') \text{ guar}_t^{n'} G'$, thus we conclude (*goal2.3*). $\square$

Lemma 29 (Sequential rule soundness). $\forall p, \widetilde{C}, q, t. \vdash \{p\}\widetilde{C}\{q\} \Rightarrow \models_t \{p \wedge \text{tid} = t\}\widetilde{C}\{q \wedge \text{tid} = t\}$

PROOF. Induction over the derivation of judgment $\vdash \{p\}\widetilde{C}\{q\}$, then by Lm. 30, Lm. 31, Lm. 32, Lm. 33, Lm. 36, Lm. 37, Lm. 38, Lm. 39, Lm. 40, Lm. 41, Lm. 42, Lm. 46, Lm. 47, Lm. 52, Lm. 65 we prove the goal. $\square$

The following sub-lemma proofs for sequential rule soundness implicitly rely on condition: all Σ holds for the precondition have $\Sigma.\sigma_o \perp \Sigma.\sigma_a$.

Lemma 30 (LIN sound). $\forall p, q, E, E_1, E_2, t. [p]\gamma(E, E_1)[q, E_2] \Rightarrow \models_t \{((E \rightarrow (\gamma, E_1) * p) \wedge \text{tid} = t) \wedge \text{tid} = t\} \text{lin}(E)\{E \rightarrow (\text{end}, E_2) * q\} \wedge \text{tid} = t\}$

PROOF. By sequential judgment semantics, given arbitrary Σ s.t. $\Sigma \models (E \rightarrow (\gamma, E_1) * p) \wedge \text{tid} = t$, we need to prove

$$\begin{aligned} & (\text{lin}(E), \Sigma) \xrightarrow{*} \text{abort} & (\text{goal1}) \\ \forall \Sigma'. (\text{lin}(E), \Sigma) \xrightarrow{*}_t (\text{skip}, \Sigma') & \Rightarrow \Sigma' \models (E \rightarrow (\text{end}, E_2) * q) \wedge \text{tid} = t & (\text{goal2}) \end{aligned}$$

By premise we know

$$\Sigma \models E \rightarrow (\gamma, E_1) * p \quad (1)$$

By (1) we know there exists Σ_1, Σ_2 s.t.

$$\Sigma_1 * \Sigma_2 = \Sigma \quad (2)$$

$$\Sigma_1 \models E \rightarrow (\gamma, E_1) \quad (3)$$

$$\Sigma_2 \models p$$

To prove (*goal1*), by definition, we only need to prove there exists t' and Δ' s.t. $\llbracket E \rrbracket_\Sigma = t'$ and $\Sigma.\Delta \rightarrow_{t'} \Delta'$. By (2) and (3) we know there exists t' s.t. $\llbracket E \rrbracket_\Sigma = t'$; apply premise $[p]\gamma(E, E_1)[q, E_2]$ we prove there exists Δ' s.t. $\Sigma.\Delta \rightarrow_{t'} \Delta'$.

To prove (*goal2*), by (1) and premise $[p]\gamma(E, E_1)[q, E_2]$ we know there exists t', Δ' s.t.

$$\begin{aligned} & \llbracket E \rrbracket_\Sigma = t' \\ & \Sigma.\Delta \rightarrow_{t'} \Delta' \\ & \Sigma|_{\Delta \mapsto \Delta'} \models E \rightarrow (\text{end}, E_2) * q \quad (4) \end{aligned}$$

Then by DOT-LIN rule and INS-DOT-COMM rule we know $(\text{lin}(E), \Sigma) \xrightarrow{*}_t (\text{skip}, \Sigma|_{\Delta \mapsto \Delta'})$. By (4) and $\Sigma \models \text{tid} = t$ we prove (*goal2*). $\square$

Lemma 31 (LIN-END sound). $\forall t, E, E'. \models_t \{E \rightarrow (\text{end}, E') \wedge \text{tid} = t\} \text{lin}(E)\{E \rightarrow (\text{end}, E') \wedge \text{tid} = t\}$

PROOF. Given arbitrary Σ s.t. $\Sigma \models E \rightarrow (\text{end}, E') \wedge \text{tid} = t$, we need to prove

$$\begin{aligned} & (\mathbf{lin}(E), \Sigma) \hookrightarrow_t^* \mathbf{abort} & (goal1) \\ \forall \Sigma'. (\mathbf{lin}(E), \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma') \implies \Sigma' \models E \multimap (\mathbf{end}, E') \wedge \mathbf{tid} = t & (goal2) \end{aligned}$$

By $\Sigma \models E \multimap (\mathbf{end}, E')$ we know there exists t', v s.t. $\llbracket (E, E') \rrbracket_\Sigma = (t', v)$ and $\Sigma.\Delta = \{(\{t' \leadsto (\mathbf{end}, v)\}, \emptyset)\}$. By definition we know

$$\Sigma.\Delta \rightarrow_{t'} \Sigma.\Delta \quad (1)$$

Thus we prove $(goal1)$, by (1) and DOT-LIN rule and INS-DOT-COMM rule we know $(\mathbf{lin}(E), \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma)$, along with $\Sigma \models E \multimap (\mathbf{end}, E') \wedge \mathbf{tid} = t$ we prove $(goal2)$. $\square$

Lemma 32 (TRYLIN sound). $\forall p, q, E, E_1, E_2, t. [p]_\gamma(E, E_1)[q, E_2] \implies \models_t \{(E \multimap (\gamma, E_1) * p) \wedge \mathbf{tid} = t\} \mathbf{trylin}(E) \{((E \multimap (\gamma, E_1) * p) \oplus (E \multimap (\mathbf{end}, E_2) * q)) \wedge \mathbf{tid} = t\}$

PROOF. Similar to the proof of Lm. 30. $\square$

Lemma 33 (TRYLIN-END sound). $\forall E, E', t. \models_t \{E \multimap (\mathbf{end}, E') \wedge \mathbf{tid} = t\} \mathbf{trylin}(E) \{E \multimap (\mathbf{end}, E') \wedge \mathbf{tid} = t\}$

PROOF. Similar to the proof of Lm. 31. $\square$

$$(Spec) \delta ::= (U, \theta)$$

Definition 5.18. $(U, \theta) \uplus (U', \theta') \stackrel{\text{def}}{=} (U \uplus U', \theta \uplus \theta')$

Lemma 34.

1. For all Δ, Δ', D , if $\Delta|_D = \Delta'$, then for all $\delta \in \Delta$, there exists δ', δ'' s.t. $\delta = \delta' \uplus \delta''$ and $\delta' \in \Delta'$.
2. For all Δ, Δ' , if $\Delta|_{\text{dom}(\Delta')} \cap \Delta' = \emptyset$, then there does not exists δ, δ' s.t. $\delta \uplus \delta' \in \Delta$ and $\delta \in \Delta'$.

PROOF. The following proof use $\Delta' \neq \emptyset$ implicitly.

Proof of 1: By definition we know $\text{dom}(\Delta') = D$ and $\forall \delta \in \Delta'. \exists \delta'. \delta' \uplus \delta \in \Delta$, then we prove $D \subseteq \text{dom}(\Delta)$. Thus for all $\delta \in \Delta$, there exists δ', δ'' s.t. $\delta = \delta' \uplus \delta''$ and $\text{dom}(\delta') = D$. Thus we prove $\delta' \in \Delta|_D$, we know $\delta' \in \Delta'$.

Proof of 2: By definition we know for all δ, δ' , if $\delta \uplus \delta' \in \Delta$, then $\delta \notin \Delta'$. Thus we know $\nexists \delta, \delta'. \delta \uplus \delta' \in \Delta \wedge \delta \in \Delta'$. $\square$

Lemma 35. $\forall s, \Delta, \Delta_1, \Delta_2. \text{specExact}(p) \wedge (s, \Delta)|_p = \Delta_1 \wedge (s, \Delta)|_p = \Delta_2 \implies \Delta_1 = \Delta_2$

PROOF. By definition and premise we know there exists $\Delta'_1, \Delta'_2, \Delta_p$ s.t.

$$\begin{aligned} (s, \Delta_p) & \models p, \\ \Delta &= \Delta_1 \uplus \Delta'_1 & \Delta_1|_{\text{dom}(\Delta_p)} &= \Delta_p & \Delta'_1|_{\text{dom}(\Delta_p)} \cap \Delta_p &= \emptyset, \\ \Delta &= \Delta_2 \uplus \Delta'_2 & \Delta_2|_{\text{dom}(\Delta_p)} &= \Delta_p & \Delta'_2|_{\text{dom}(\Delta_p)} \cap \Delta_p &= \emptyset. \end{aligned}$$

By Lm. 34, $\Delta_1|_{\text{dom}(\Delta_p)} = \Delta_p$ and $\Delta'_2|_{\text{dom}(\Delta_p)} \cap \Delta_p = \emptyset$ we know $\Delta_1 \cap \Delta'_2 = \emptyset$, similarly we know $\Delta'_1 \cap \Delta_2 = \emptyset$. Since $\Delta_1 \uplus \Delta'_1 = \Delta_2 \uplus \Delta'_2$, we prove $\Delta_1 = \Delta_2$. $\square$

Lemma 36 (COMMIT sound). $\forall p, p'. \text{specExact}(p) \wedge p' \Rightarrow p \implies \models_t \{(p' \oplus \mathbf{true}) \wedge \mathbf{tid} = t\} \mathbf{commit}(p) \{p' \wedge \mathbf{tid} = t\}$

PROOF. Given arbitrary $(s, \Delta) \models (p' \oplus \mathbf{true}) \wedge \mathbf{tid} = t'$, we need to prove

$$\begin{aligned}
& (\mathbf{commit}(p), (s, \Delta)) \hookrightarrow_t^* \mathbf{abort} & (goal1) \\
& \forall \Sigma'. (\mathbf{commit}(p), (s, \Delta)) \hookrightarrow_t^* (\mathbf{skip}, \Sigma') \implies \Sigma' \models p' \wedge \text{tid} = t & (goal2)
\end{aligned}$$

By premise $\text{specExact}(p)$, to prove $(goal1)$ we only need to prove there exists Δ' s.t. $(s, \Delta)|_p = \Delta'$. That is to prove

$$\exists \Delta', \Delta'', \Delta_p. \Delta = \Delta' \uplus \Delta'' \wedge (s, \Delta_p) \models p \wedge \Delta'|_{\text{dom}(\Delta_p)} = \Delta_p \wedge \Delta''|_{\text{dom}(\Delta_p)} \cap \Delta_p = \emptyset.$$

By $(s, \Delta) \models p' \oplus \text{true}$ we know there exists Δ', Δ'' s.t. $\Delta' \uplus \Delta'' = \Delta$ and $(s, \Delta') \models p'$. By $p' \implies p$ we know $(s, \Delta') \models p$. Let $\Delta_p = \Delta'$, by definition we conclude $(goal1)$ immediately.

To prove $(goal2)$, if $(\mathbf{commit}(p), (s, \Delta)) \hookrightarrow_t^* (\mathbf{skip}, \Sigma')$, we know $\Sigma'.s = s$ and $(s, \Delta)|_p = \Sigma'.\Delta$. By Lm. 35 we know $\Sigma'.\Delta = \Delta'$, thus we prove $\Sigma' \models p'$. As tid is a read-only variable, we know $\Sigma' \models \text{tid} = t$, thus we conclude $(goal2)$. $\square$

Lemma 37 (FLIN sound). $\forall p, q, E, E'. p \implies (E \in \text{Nbad}) \wedge [p]\gamma(E, E')[q] \implies \models_t \{p \wedge \text{tid} = t\} \mathbf{flin}(E, \gamma, E') \{q \wedge \text{tid} = t\}$

PROOF. Given arbitrary Σ s.t. $\Sigma \models p \wedge \text{tid} = t$, we need to prove

$$\begin{aligned}
& (\mathbf{flin}(E, \gamma, E'), \Sigma) \hookrightarrow_t^* \mathbf{abort} & (goal1) \\
& \forall \Sigma'. (\mathbf{flin}(E, \gamma, E'), \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma') \implies \Sigma' \models q \wedge \text{tid} = t & (goal2)
\end{aligned}$$

We prove $(goal1)$ by contradiction. Assume that there exists k s.t. $(\mathbf{flin}(E, \gamma, E'), \Sigma) \hookrightarrow_t^k \mathbf{abort}$. By definition we know $(\mathbf{flin}(E, \gamma, E'), \Sigma) \hookrightarrow_t \mathbf{abort}$, then by semantics we know there exists t', v s.t. $\llbracket (E, E') \rrbracket_\Sigma = (t', v)$ and $\nexists \Delta'. \Sigma.\Delta \xrightarrow{(t', \gamma, v)} \Delta'$. The latter contradict premise $[p]\gamma(E, E')[q]$.

To prove $(goal2)$, given arbitrary Σ' s.t. $(\mathbf{flin}(E, \gamma, E'), \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma')$. By semantics we know there exists t', v s.t. $\llbracket (E, E') \rrbracket_\Sigma = (t', v)$, thus we know $\Sigma \models (E, E') = (t', v)$. Apply $[p]\gamma(E, E')[q]$ we know there exists Δ' s.t.

$$\begin{aligned}
& \Sigma.\Delta \xrightarrow{(t', \gamma, v)} \Delta' & (1) \\
& \Sigma|_{\Delta \mapsto \Delta'} \models q & (2)
\end{aligned}$$

Also by premise $p \implies (E \in \text{Nbad})$ we know $t' \in \llbracket \text{Nbad} \rrbracket_\Sigma$, thus along with (1) we know $\Sigma' = \Sigma|_{\Delta \mapsto \Delta'}$, then by (2) and $\Sigma \models \text{tid} = t$ we prove $(goal2)$. $\square$

Lemma 38 (SEND sound). $\forall E_M, E_N, E, t. \models_t \{\mathbf{msgPool}(E_M) \wedge \text{tid} = t\} \mathbf{send}(E_N, E) \{\mathbf{msgPool}(E_M \cup \{(E_N, E)\}) \wedge \text{tid} = t\}$

PROOF. Given arbitrary Σ s.t. $\Sigma \models \mathbf{msgPool}(E_M) \wedge \text{tid} = t$, we need to prove $(\mathbf{send}(E_N, E), \Sigma) \hookrightarrow_t^* \mathbf{abort}$. This is obvious by our assumption $\Sigma.\sigma_o \perp \Sigma.\sigma_a$. As $\mathbf{send}$ statement execute at most 1 step, we discuss on cases. Zero step case is trivial; if $\mathbf{send}$ statement execute 1 step, then by semantics we have

$$\begin{aligned}
& \Sigma.\sigma_o = \Sigma'.\sigma_o, \\
& \Sigma.\sigma_a = \Sigma'.\sigma_a.
\end{aligned}$$

Thus by semantics we know it does not go abort.

Now we only need to take arbitrary Σ, Σ' s.t. $\Sigma \models \mathbf{msgPool}(E_M) \wedge \text{tid} = t$ and $(\mathbf{send}(E_N, E), \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma')$, then prove

$$\Sigma' \models \mathbf{msgPool}(E_M \cup \{(E_N, E)\}) \wedge \text{tid} = t.$$

By semantics we know there exists $\mathcal{N}, v$ s.t. $\llbracket (E_N, E) \rrbracket_{\Sigma, \sigma_o} = (\mathcal{N}, v)$ and $\Sigma' = ((\Sigma, \sigma_o, \Sigma.M \cup \{(\mathcal{N}, v)\}, \Sigma, \sigma_a), \Sigma, \Delta)$. By $\Sigma \models \mathbf{msgPool}(E_M) \wedge \text{tid} = t$ we know $\Sigma.M = \llbracket E_M \rrbracket_{\Sigma}$, thus by definition of Σ' we prove the goal. $\square$

Lemma 39 (RECV sound). $\forall E_M, x, t. x \notin \text{FV}(E_M) \implies \models_t \{\mathbf{msgPool}(E_M) \wedge \neg \text{auxVar}(x) \wedge \text{tid} = t\} x := \mathbf{recv}() \{\exists x_N. \mathbf{msgPool}(E_M) \wedge \text{tid} \in x_N \wedge (x_N, x) \in E_M \wedge \text{tid} = t\}$

PROOF. First we need to prove $(x := \mathbf{recv}(), \Sigma) \hookrightarrow_t^* \mathbf{abort}$ when $\Sigma \models \neg \text{auxVar}(x)$ and $\Sigma, \sigma_o \perp \Sigma, \sigma_a$, which is obvious by definition and semantics.

Now we only need to take arbitrary Σ, Σ' s.t. $\Sigma \models \mathbf{msgPool}(E_M) \wedge \text{tid} = t$ and $(x := \mathbf{recv}(), \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma')$, then prove

$$\Sigma' \models \mathbf{msgPool}(E_M) \wedge (\{\text{tid}\}, x) \in E_M \wedge \text{tid} = t.$$

By semantics we know there exists $\mathcal{N}, v$ s.t.

$$\begin{aligned} (\mathcal{N}, v) &\in \Sigma.M, \\ t &\in \mathcal{N}, \\ \Sigma' &= ((\Sigma, \sigma_o \{x \rightsquigarrow v\}, \Sigma.M, \Sigma, \sigma_a), \Sigma, \Delta). \end{aligned}$$

As we have assumed that tid is a read-only variable, thus $x \neq \text{tid}$. By $\Sigma \models \mathbf{msgPool}(E_M) \wedge \text{tid} = t$ we know $\Sigma.M = \llbracket E_M \rrbracket_{\Sigma}$ and $\llbracket \text{tid} \rrbracket_{\Sigma} = t$, by premise $x \notin \text{FV}(E_M)$ we know $\llbracket E_M \rrbracket_{\Sigma'} = \Sigma.M$. Thus by definition we prove the goal. $\square$

Lemma 40 (ASSN sound). $\forall t, p, x, E. \models_t \{(p/x \rightarrow E) \wedge \neg \text{auxVar}(x) \wedge \text{tid} = t\} x := E \{p \wedge \text{tid} = t\}$

PROOF. First we need to prove $(x := E, \Sigma) \hookrightarrow_t^* \mathbf{abort}$ when $\Sigma \models \neg \text{auxVar}(x)$ and $\Sigma, \sigma_o \perp \Sigma, \sigma_a$, which is obvious by definition and semantics.

Now we only need to take arbitrary Σ, Σ' s.t. $\Sigma \models (p/x \rightarrow E) \wedge \text{tid} = t$ and $(x := E, \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma')$, then prove

$$\Sigma' \models p \wedge \text{tid} = t.$$

Let $((\sigma_o, M, \sigma_a), \Delta) = \Sigma$. By definition we know $((\sigma_o \{x \rightsquigarrow \llbracket E \rrbracket_{\sigma_o}\}, M, \sigma_a), \Delta) \models p$. By semantics we know $\Sigma' = ((\sigma_o \{x \rightsquigarrow \llbracket E \rrbracket_{\sigma_o}\}, M, \sigma_a), \Delta)$, along with assumption (tid is read-only) we know $x \neq \text{tid}$, thus The goal is proved. $\square$

Lemma 41 (AUX-ASSN sound). $\forall t, p, x, E. \models_t \{(p/_a x \rightarrow E) \wedge \neg \text{objVar}(x) \wedge \text{tid} = t\} x :=_a E \{p \wedge \text{tid} = t\}$

PROOF. Similar to the proof of Lm. 40. $\square$

Lemma 42 (RET sound). $\forall t, E. \models_t \{\text{tid} \rightsquigarrow (\text{end}, E) \wedge \text{tid} = t\} \mathbf{E}_i[\mathbf{return} E] \{\text{tid} \rightsquigarrow (\text{end}, E) \wedge \text{tid} = t\}$

PROOF. Given arbitrary Σ s.t. $\Sigma \models \text{tid} \rightsquigarrow (\text{end}, E) \wedge \text{tid} = t$, we need to prove

$$\begin{aligned} (\mathbf{E}_i[\mathbf{return}(E)], \Sigma) &\hookrightarrow_t^* \mathbf{abort} & (\text{goal1}) \\ \forall \Sigma'. (\mathbf{E}_i[\mathbf{return}(E)], \Sigma) &\hookrightarrow_t^* (\mathbf{skip}, \Sigma') \implies \Sigma' \models \text{tid} \rightsquigarrow (\text{end}, E) \wedge \text{tid} = t & (\text{goal2}) \end{aligned}$$

We prove (*goal1*) by contradiction. Assume there exists k s.t. $(\mathbf{return}(E), \Sigma) \xrightarrow{t}^k \mathbf{abort}$. By definition we know $(\mathbf{return}(E), \Sigma) \xrightarrow{t} \mathbf{abort}$. By semantics we know there exists v s.t. $\llbracket E \rrbracket_\Sigma = v$ and $\nexists \Delta'. \Delta = \Delta' * \{(\{t \rightsquigarrow (\text{end}, v)\}, \emptyset)\}$. By $\Sigma \models \text{tid} \rightsquigarrow (\text{end}, E) \wedge \text{tid} = t$ we know $\Sigma.\Delta = \{(\{t \rightsquigarrow (\text{end}, v)\}, \emptyset)\}$, which leads to contradiction.

To prove (*goal2*), given arbitrary Σ' s.t. $(\mathbf{return}(E), \Sigma) \xrightarrow{t}^* (\mathbf{skip}, \Sigma')$, by semantics we know $\Sigma = \Sigma'$, thus by $\Sigma \models \text{tid} \rightsquigarrow (\text{end}, E) \wedge \text{tid} = t$ the goal is proved. $\square$

Lemma 43. For all $\tilde{C}, s, \Delta, \Delta_1, \Delta_2, t$, if

1. $\Delta = \Delta_1 \cup \Delta_2$,
2. $(\tilde{C}, (s, \Delta_1)) \not\xrightarrow{t} \mathbf{abort}$,
3. $(\tilde{C}, (s, \Delta_2)) \not\xrightarrow{t} \mathbf{abort}$,

then $(\tilde{C}, (s, \Delta)) \not\xrightarrow{t} \mathbf{abort}$.

PROOF. This proof use $\text{dom}(\Delta_1) = \text{dom}(\Delta_2)$ implicitly, this is acceptable as we assume all Δ in this document to be domain-exact.

We prove this lemma by case analysis on $\tilde{C}$:

- If $\tilde{C} = \mathbf{E}_i[x := E]$, or $\tilde{C} = \mathbf{E}_i[\mathbf{send}(E_N, E)]$, or $\tilde{C} = \mathbf{E}_i[x := \mathbf{recv}()]$, then let $\tilde{C} = C$. By (p1) we know there exists s' s.t.

$$\begin{aligned} s'.\sigma_a &= s.\sigma_a, \\ s'.\sigma_o &\perp s.\sigma_a, \\ (C, (s.\sigma_o, s.M)) &\longrightarrow_t (\mathbf{skip}, (s.\sigma'_o, s.M')). \end{aligned}$$

Thus we get the conclusion.

- If $\tilde{C} = \mathbf{E}_i[\mathbf{return} E]$, then there exists v s.t. $\llbracket E \rrbracket_{s.\sigma_o} = v$ and

$$\begin{aligned} \exists \Delta'_1. \Delta_1 &= \Delta'_1 * \{(\{t \rightsquigarrow (\text{end}, v)\}, \emptyset)\}, \\ \exists \Delta'_2. \Delta_2 &= \Delta'_2 * \{(\{t \rightsquigarrow (\text{end}, v)\}, \emptyset)\}. \end{aligned}$$

As $\text{dom}(\Delta_1) = \text{dom}(\Delta_2)$ and $\Delta = \Delta_1 \cup \Delta_2$, we know there exists Δ' s.t. $\Delta = \Delta' * \{(\{t \rightsquigarrow (\text{end}, v)\}, \emptyset)\}$, thus we get the conclusion.

- $\tilde{C} = \mathbf{E}_i[\dot{C}]$: Further analysis on $\dot{C}$:
 - If $\dot{C} = \mathbf{lin}(E)$, then by (p2)(p3) we know there exists t' s.t. $\llbracket E \rrbracket_s = t'$ and there exists Δ'_1, Δ'_2 s.t. $\Delta_1 \rightarrow_{t'} \Delta'_1$ and $\Delta_2 \rightarrow_{t'} \Delta'_2$. Since $\Delta = \Delta_1 \cup \Delta_2$, we know $\Delta \rightarrow_{t'} \Delta'_1 \cup \Delta'_2$. Thus we get the conclusion.
 - If $\dot{C} = \mathbf{flin}(E, \gamma, E')$, then by (p2)(p3) we know there exists t', v s.t. $\llbracket (E, E') \rrbracket_s = (t', v)$, $t' \in \llbracket \text{Nbad} \rrbracket_s$, and there exists Δ'_1, Δ'_2 s.t. $\Delta_1 \xrightarrow{(t', \gamma, v)} \Delta'_1$ and $\Delta_2 \xrightarrow{(t', \gamma, v)} \Delta'_2$. By $\text{dom}(\Delta_1) = \text{dom}(\Delta_2)$ and $\Delta = \Delta_1 \cup \Delta_2$ we know $\Delta \xrightarrow{(t', \gamma, v)} \Delta'_1 \cup \Delta'_2$, thus we get the conclusion.
 - If $\dot{C} = \mathbf{commit}(p)$, then by (p2)(p3) we know $\text{specExact}(p)$ and there exists Δ'_1, Δ'_2 s.t. $(s, \Delta_1)|_p = \Delta'_1$ and $(s, \Delta_2)|_p = \Delta'_2$. By $\Delta = \Delta_1 \cup \Delta_2$ we know $(s, \Delta)|_p = \Delta'_1 \cup \Delta'_2$, thus we get the conclusion.
 - Other cases are similar.
- Other cases are similar.

$\square$

Lemma 44.

1. For all $\Delta, t, \Delta', \Delta_1, \Delta_2$, if $\Delta \rightarrow_t \Delta'$ and $\Delta = \Delta_1 \cup \Delta_2$, then there exists Δ'_1 and Δ'_2 s.t. $\Delta_1 \rightarrow_t \Delta'_1$, $\Delta_2 \rightarrow_t \Delta'_2$ and $\Delta' = \Delta'_1 \cup \Delta'_2$.

2. For all $\Delta, t, \gamma, v, \Delta', \Delta_1, \Delta_2$, if $\Delta \xrightarrow{(t, \gamma, v)} \Delta'$ and $\Delta = \Delta_1 \cup \Delta_2$, then there exists Δ'_1 and Δ'_2 s.t.
 $\Delta_1 \xrightarrow{(t, \gamma, v)} \Delta'_1, \Delta_2 \xrightarrow{(t, \gamma, v)} \Delta'_2$ and $\Delta' = \Delta'_1 \cup \Delta'_2$.

Lemma 45. For all $\Delta, \Delta_1, \Delta_2, \tilde{C}, s, t, s', \Delta'$, if

1. $\Delta = \Delta_1 \cup \Delta_2$,
2. $(\tilde{C}, (s, \Delta)) \xrightarrow{t} (\mathbf{skip}, (s', \Delta'))$,
3. $(\tilde{C}, (s, \Delta_1)) \not\xrightarrow{t} \mathbf{abort}$,
4. $(\tilde{C}, (s, \Delta_2)) \not\xrightarrow{t} \mathbf{abort}$,

then there exists Δ'_1, Δ'_2 s.t.

1. $\Delta' = \Delta'_1 \cup \Delta'_2$,
2. $(\tilde{C}, (s, \Delta_1)) \xrightarrow{t} (\mathbf{skip}, (s', \Delta'_1))$,
3. $(\tilde{C}, (s, \Delta_2)) \xrightarrow{t} (\mathbf{skip}, (s', \Delta'_2))$.

PROOF. We prove this lemma by case analysis on $\tilde{C}$:

- $\tilde{C} = \mathbf{skip}$: Trivial.
- $\tilde{C} = \mathbf{E}_i[x := E]$, or $\tilde{C} = \mathbf{E}_i[\mathbf{send}(E_N, E)]$, or $\tilde{C} = \mathbf{E}_i[x := \mathbf{recv}()]$: By (p2) we know

$$\begin{aligned} s'.\sigma_a &= s.\sigma_a, \\ s'.\sigma_o &\perp s.\sigma_a, \\ (C, (s.\sigma_o, s.M)) &\xrightarrow{t} (\mathbf{skip}, (s.\sigma'_o, s.M')). \end{aligned}$$

Let $\Delta'_1 = \Delta_1, \Delta'_2 = \Delta_2$, by semantics we have all goals.

- $\tilde{C} = \mathbf{E}_i[\mathbf{return} E]$: By (p2) we know

$$\exists \Delta'. \Delta = \Delta' * \{(\{t \rightsquigarrow (\mathbf{end}, v)\}, \emptyset)\}.$$

Let $\Delta'_1 = \Delta_1, \Delta'_2 = \Delta_2$ we prove all goals.

- $\tilde{C} = \mathbf{E}_i[\dot{C}]$: Further case analysis on $\dot{C}$ we have
 - $\dot{C} = \mathbf{lin}(E)$: By (p1) we know there exists t' s.t. $\llbracket E \rrbracket_s = t'$ and $\Delta \rightarrow_{t'} \Delta'$.

Then by Lm. 44 we prove all goals.

- $\dot{C} = \mathbf{flin}(E, \gamma, E')$: By (p1) we know there exists t', v s.t. $\llbracket (E, E') \rrbracket_s = (t', v)$, $t' \in \llbracket \mathbf{Nbad} \rrbracket_s$, and $\Delta \xrightarrow{(t', \gamma, v)} \Delta'$, by Lm. 44 we prove all goals.
- $\dot{C} = \mathbf{commit}(p)$: By (p3)(p4) we know $\mathbf{specExact}(p)$ and there exists Δ'_1, Δ'_2 s.t. $(s, \Delta_1)|_p = \Delta'_1$ and $(s, \Delta_2)|_p = \Delta'_2$. By $\Delta = \Delta_1 \cup \Delta_2$ we know $(s, \Delta)|_p = \Delta'_1 \cup \Delta'_2$, by Lm. 35 we know $\Delta' = \Delta'_1 \cup \Delta'_2$, thus we prove the goals.
- Other cases are similar.
- Other cases are similar.

□

Lemma 46 (SPEC-CONJ sound). $\forall t, p_1, p_2, q_1, q_2, C. \models_t \{p_1 \wedge \mathbf{tid} = t\} \tilde{C} \{q_1 \wedge \mathbf{tid} = t\} \wedge \models_t \{p_2 \wedge \mathbf{tid} = t\} \tilde{C} \{q_2 \wedge \mathbf{tid} = t\} \implies \models_t \{(p_1 \oplus p_2) \wedge \mathbf{tid} = t\} \tilde{C} \{(q_1 \oplus q_2) \wedge \mathbf{tid} = t\}$

PROOF. By definition, given arbitrary $(s, \Delta) \models (p_1 \oplus p_2) \wedge \mathbf{tid} = t$, we need to prove:

$$\begin{aligned} (\tilde{C}, (s, \Delta)) &\xrightarrow{t}^* \mathbf{abort} & (\text{goal1}) \\ \forall s', \Delta'. (\tilde{C}, (s, \Delta)) &\xrightarrow{t}^* (\mathbf{skip}, (s', \Delta')) \implies (s', \Delta') \models (q_1 \oplus q_2) \wedge \mathbf{tid} = t & (\text{goal2}) \end{aligned}$$

By $(s, \Delta) \models (p_1 \oplus p_2) \wedge \mathbf{tid} = t$ we know there exists Δ_1, Δ_2 s.t. $\Delta = \Delta_1 \cup \Delta_2$, $\Delta_1 \models p_1 \wedge \mathbf{tid} = t$ and $\Delta_2 \models p_2 \wedge \mathbf{tid} = t$. Apply premise we know

$$\begin{aligned}
& (\tilde{C}, (s, \Delta_1)) \not\hookrightarrow_t^* \mathbf{abort}, \\
& \forall s', \Delta'_1. (\tilde{C}, (s, \Delta_1)) \hookrightarrow_t^* (\mathbf{skip}, (s', \Delta'_1)) \implies (s', \Delta'_1) \models q_1 \wedge \text{tid} = t, \\
& (\tilde{C}, (s, \Delta_2)) \not\hookrightarrow_t^* \mathbf{abort}, \\
& \forall s', \Delta'_2. (\tilde{C}, (s, \Delta_2)) \hookrightarrow_t^* (\mathbf{skip}, (s', \Delta'_2)) \implies (s', \Delta'_2) \models q_2 \wedge \text{tid} = t.
\end{aligned}$$

Apply Lm. 43 and Lm. 45 we prove (*goal1*). Apply Lm. 45 we know there exists Δ'_1, Δ'_2 s.t.

$$\begin{aligned}
& \Delta'_1 \cup \Delta'_2 = \Delta', \\
& (\tilde{C}, (s, \Delta_1)) \hookrightarrow_t^* (\mathbf{skip}, (s', \Delta'_1)), \\
& (\tilde{C}, (s, \Delta_2)) \hookrightarrow_t^* (\mathbf{skip}, (s', \Delta'_2)).
\end{aligned}$$

Then we know $(s', \Delta'_1) \models q_1 \wedge \text{tid} = t$ and $(s', \Delta'_2) \models q_2 \wedge \text{tid} = t$, thus we conclude (*goal2*) immediately. $\square$

Lemma 47 (CONJ sound). $\forall t, p, p', q, q', \tilde{C}. \models_t \{p \wedge \text{tid} = t\} \tilde{C} \{q \wedge \text{tid} = t\} \wedge \models_t \{p \wedge \text{tid} = t\} \tilde{C} \{q \wedge \text{tid} = t\} \implies \models_t \{p \wedge p' \wedge \text{tid} = t\} \tilde{C} \{q \wedge q' \wedge \text{tid} = t\}$

PROOF. Given arbitrary Σ s.t. $\Sigma \models p \wedge p' \wedge \text{tid} = t$, we need to prove

$$\begin{aligned}
& (\tilde{C}, \Sigma) \not\hookrightarrow_t^* \mathbf{abort} & (\text{goal1}) \\
& \forall \Sigma'. (\tilde{C}, \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma') \implies \Sigma' \models q \wedge q' \wedge \text{tid} = t & (\text{goal2})
\end{aligned}$$

To prove (*goal1*), by $\Sigma \models p \wedge p' \wedge \text{tid} = t$ we know $\Sigma \models p \wedge \text{tid} = t$, then apply premise $\models_t \{p \wedge \text{tid} = t\} \tilde{C} \{q \wedge \text{tid} = t\}$ we prove (*goal1*).

To prove (*goal2*), given arbitrary Σ' s.t. $(\tilde{C}, \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma')$, by premise we know $\Sigma' \models q \wedge \text{tid} = t$ and $\Sigma' \models q' \wedge \text{tid} = t$, thus by definition we prove (*goal2*). $\square$

Lemma 48. $\forall \tilde{C}_1, \tilde{C}_2, \Sigma, \Sigma', k, t. (\tilde{C}_1; \tilde{C}_2, \Sigma) \hookrightarrow_t^k (\mathbf{skip}, \Sigma') \implies \exists \Sigma''. (\tilde{C}_1, \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma'') \wedge (\tilde{C}_2, \Sigma'') \hookrightarrow_t^* (\mathbf{skip}, \Sigma')$

PROOF. By induction on k (discuss $\tilde{C}_1$ in inductive case). $\square$

Lemma 49 (SEQ sound). $\forall t, p, q, r, \tilde{C}_1, \tilde{C}_2. \models_t \{p \wedge \text{tid} = t\} \tilde{C}_1 \{q \wedge \text{tid} = t\} \wedge \models_t \{q \wedge \text{tid} = t\} \tilde{C}_2 \{r \wedge \text{tid} = t\} \implies \models_t \{p \wedge \text{tid} = t\} \tilde{C}_1; \tilde{C}_2 \{r \wedge \text{tid} = t\}$

PROOF. Given arbitrary Σ s.t. $\Sigma \models p \wedge \text{tid} = t$, we need to prove

$$\begin{aligned}
& (\tilde{C}_1; \tilde{C}_2, \Sigma) \not\hookrightarrow_t^* \mathbf{abort} & (\text{goal1}) \\
& \forall \Sigma'. (\tilde{C}_1; \tilde{C}_2, \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma') \implies \Sigma' \models r \wedge \text{tid} = t & (\text{goal2})
\end{aligned}$$

We prove (*goal1*) by contradiction. Assume there exists k s.t. $(\tilde{C}_1; \tilde{C}_2, \Sigma) \hookrightarrow_t^k \mathbf{abort}$, then we have the following two case:

- If $(\tilde{C}_1, \Sigma) \hookrightarrow_t^k \mathbf{abort}$, then this contradicts premise $\models_t \{p \wedge \text{tid} = t\} \tilde{C}_1 \{q \wedge \text{tid} = t\}$.
- If there exists k_1, k_2, Σ' s.t. $k_1 + k_2 + 1 = k$ and

$$\begin{aligned}
& (\tilde{C}_1, \Sigma) \hookrightarrow_t^{k_1} (\mathbf{skip}, \Sigma') & (1) \\
& (\tilde{C}_2, \Sigma') \hookrightarrow_t^{k_2} \mathbf{abort} & (2)
\end{aligned}$$

then by (1) and premise we know $\Sigma' \models q \wedge \text{tid} = t$, thus we know (2) contradicts premise $\models_t \{q \wedge \text{tid} = t\} \tilde{C}_2 \{r \wedge \text{tid} = t\}$.

Therefore, we conclude (*goal1*).

To prove (*goal2*), given arbitrary Σ' s.t. $(\tilde{C}_1; \tilde{C}_2, \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma')$, apply Lm. 48 we know there exists Σ'' s.t.

$$(\tilde{C}_1, \Sigma) \xrightarrow{*}_t (\mathbf{skip}, \Sigma'') \quad (1)$$

$$(\tilde{C}_2, \Sigma'') \xrightarrow{*}_t (\mathbf{skip}, \Sigma') \quad (2)$$

Apply (1) on premise $\models_t \{p \wedge \text{tid} = t\} \tilde{C}_1 \{q \wedge \text{tid} = t\}$ we know $\Sigma'' \models q \wedge \text{tid} = t$, then by (2) and premise $\models_t \{q \wedge \text{tid} = t\} \tilde{C}_2 \{r \wedge \text{tid} = t\}$ we prove $\Sigma' \models r \wedge \text{tid} = t$. $\square$

Lemma 50. $\forall x, p, \Sigma. x \notin \text{FV}(p) \wedge x \in \text{dom}(\Sigma.\sigma_o) \implies (\forall v. \Sigma \models p \Leftrightarrow \Sigma|_{\sigma_o \mapsto \sigma_o\{x \rightsquigarrow v\}} \models p)$

PROOF. By induction on p . $\square$

Lemma 51. For all $r, \tilde{C}, \tilde{C}', \Sigma, \Sigma', n, t$, if

1. $\text{noModifyVars}(\text{FV}(r), \tilde{C})$,
2. $\text{noModifyRestrictedMP}(r, \tilde{C})$,
3. $\text{pure}(r)$,
4. $\Sigma \models r$,
5. $(\tilde{C}, \Sigma) \xrightarrow{n}_t (\tilde{C}', \Sigma')$,

then $\Sigma' \models r$.

PROOF. By induction on n . The base case trivially holds; the inductive case is: if there exists k s.t. $n = k + 1$, we need to prove $\Sigma' \models r$. By (p5) we know there exists $\tilde{C}'', \Sigma''$ s.t.

$$(\tilde{C}, \Sigma) \xrightarrow{1}_t (\tilde{C}'', \Sigma'') \quad (1)$$

$$(\tilde{C}'', \Sigma'') \xrightarrow{k}_t (\tilde{C}', \Sigma') \quad (2)$$

By (2) and induction hypothesis we only need to prove $\text{noModifyVars}(\text{FV}(r), \tilde{C}'')$, $\text{noModifyRestrictedMP}(r, \tilde{C}'')$ and $\Sigma'' \models r$. Apply (1) on $\text{noModifyVars}(\text{FV}(r), \tilde{C})$ we prove $\text{noModifyVars}(\text{FV}(r), \tilde{C}'')$. Apply (1) on $\text{noModifyRestrictedMP}(r, \tilde{C})$ we prove $\tilde{C}''$. Now we only need to prove $\Sigma'' \models r$, we prove this by further induction on $\tilde{C}$:

- $\tilde{C} = \mathbf{skip}$: Contradict (1).
- $\tilde{C} = x := E$: By (1) and INS-CONCRETE and ASSN we know there exists v s.t.

$$\begin{aligned} \llbracket E \rrbracket_{\Sigma.\sigma_o} &= v, \\ \tilde{C}'' &= \mathbf{skip}, \\ \Sigma'' &= \Sigma|_{\sigma_o \mapsto \sigma_o\{x \rightsquigarrow v\}}. \end{aligned}$$

Discuss on case:

- If $x \in \text{FV}(r)$, then apply $\text{noModifyVars}(\text{FV}(r), \tilde{C})$ we know $\llbracket x \rrbracket_{\Sigma''.\sigma_o \uplus \Sigma''.\sigma_a} = \llbracket x \rrbracket_{\Sigma.\sigma_o \uplus \Sigma.\sigma_a}$, thus we know $\Sigma.\sigma_o \perp \Sigma.\sigma_a$ and $\Sigma''.\sigma_o \perp \Sigma''.\sigma_a$. We have $\llbracket x \rrbracket_{\Sigma.\sigma_o} = \llbracket x \rrbracket_{\Sigma''.\sigma_o}$. thus $\Sigma'' = \Sigma$, we prove $\Sigma'' \models r$.
- If $x \notin \text{FV}(r)$, then by Lm. 50 we prove $\Sigma'' \models r$.
- $\tilde{C} = \mathbf{send}(E, E')$: Then there exists v, v' s.t. $\Sigma'' = \Sigma|_{M \mapsto M \cup \{(v, v')\}}$ and $\tilde{C}'' = \mathbf{skip}$. We discuss on case: if $\text{restrictMP}(r)$, then by $\text{noModifyRestrictedMP}(r, \tilde{C})$ we know $\Sigma.M = \Sigma''.M$, thus $\Sigma = \Sigma''$, we prove $\Sigma' \models r$; if $\neg \text{restrictMP}(r)$, then we know

$$\forall M, M', \sigma_o, \sigma_a, \Delta. ((\sigma_o, M, \sigma_a), \Delta) \models r \Leftrightarrow ((\sigma_o, M', \sigma_a), \Delta) \models r$$

Thus we prove $\Sigma' \models r$. We conclude this case.

- $\tilde{C} = x := \mathbf{recv}()$ or $\tilde{C} = \mathbf{return} E$: Then by (1) we know $\Sigma'' = \Sigma$, we prove the goal trivially.
- $\tilde{C} = \tilde{C}'$: then further induction on $\tilde{C}$ we have
 - $\tilde{C} = x :=_a E$: The case is similar to $\tilde{C} = x := E$ case.

- $\tilde{C} = \mathbf{lin}(E)$, or $\tilde{C} = \mathbf{trylin}(E)$, or $\tilde{C} = \mathbf{commit}(p)$, or $\tilde{C} = \mathbf{flin}(E, \gamma, E')$: By semantics we know there exists Δ'' s.t. $\Sigma'' = \Sigma|_{\Delta \mapsto \Delta''}$, then by $\text{pure}(r)$ we prove $\Sigma' \models r$.
- $\tilde{C} = \dot{C}_1; \dot{C}_2$: We discuss on case as follows
 - * $\tilde{C}'' = \dot{C}_2$: Then we know $\dot{C}_1 = \mathbf{skip}$, then $\Sigma'' = \Sigma$, we prove $\Sigma'' \models r$
 - * There exists $\dot{C}'_1$ s.t. $\tilde{C}'' = \dot{C}'_1; \dot{C}_2$, $(\sigma_o, M) \vdash (\dot{C}_1, (\sigma_a, \Delta)) \hookrightarrow_t (\dot{C}'_1, (\sigma'_a, \Delta'))$ and $\Sigma = ((\sigma_o, M, \sigma_a), \Delta)$, $\Sigma' = ((\sigma_o, M, \sigma'_a), \Delta')$: Apply induction hypothesis we prove $\Sigma'' \models r$.
- $\tilde{C} = \mathbf{if} (B) \mathbf{then} \dot{C}_1 \mathbf{else} \dot{C}_2$ or $\tilde{C} = \mathbf{foreach} x \mathbf{in} E \mathbf{do} \dot{C}_1$: By semantics we know $\Sigma'' = \Sigma$, thus we prove $\Sigma'' \models r$
- $\tilde{C} = \langle \tilde{C}_1 \rangle$: This does not happen as we require that $\tilde{C}$ does not contain atomic statement when proving sequential judgment.
- $\tilde{C} = \tilde{C}_1; \tilde{C}_2$: We discuss on case as follows:
 - $\tilde{C}'' = \tilde{C}_2$: Then we know $\tilde{C}_1 = \mathbf{skip}$, then $\Sigma'' = \Sigma$, we prove $\Sigma'' \models r$.
 - There exists $\tilde{C}'_1$ s.t. $\tilde{C}'' = \tilde{C}'_1; \tilde{C}_2$ and $(\tilde{C}_1, \Sigma) \hookrightarrow_t (\tilde{C}'_1, \Sigma')$: Then apply induction hypothesis we prove $\Sigma'' \models r$.
- $\tilde{C} = \mathbf{if} (B) \mathbf{then} \tilde{C}_1 \mathbf{else} \tilde{C}_2$ or $\tilde{C} = \mathbf{while} (B) \mathbf{do} \tilde{C}_1$: We know $\Sigma'' = \Sigma$, thus we prove $\Sigma'' \models r$.

□

Lemma 52 (INVARIANCE sound). For all $t, p, q, r, \tilde{C}$, if

1. $\text{noModifyVars}(\text{FV}(r), \tilde{C})$,
2. $\text{noModifyRestrictedMP}(r, \tilde{C})$,
3. $\text{pure}(r)$,
4. $\models_t \{p \wedge \text{tid} = t\} \tilde{C} \{q \wedge \text{tid} = t\}$,

then $\models_t \{p \wedge r \wedge \text{tid} = t\} \tilde{C} \{q \wedge r \wedge \text{tid} = t\}$.

PROOF. By definition given arbitrary Σ, Σ' s.t. $\Sigma \models p \wedge r$, we need to prove $(\tilde{C}, \Sigma) \hookrightarrow_t^* \mathbf{abort}$, which by apply $\Sigma \models p$ on (p4) immediately, and prove if $(\tilde{C}, \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma')$, then $\Sigma' \models q \wedge r$. apply (p4) we know $\Sigma' \models q \wedge \text{tid} = t$, now we only need to prove $\Sigma' \models r$, which by apply Lm. 51 immediately. □

Lemma 53 (FOR sound). For all $t, x, E, \dot{C}, p, q$, if

1. $\text{noModifyVars}(\text{FV}(q) \cup \text{FV}(E) \cup \{x\}, \dot{C})$,
2. $\text{noModifyRestrictedMP}(q, \dot{C})$,
3. $\text{pure}(q)$,
4. $p \Rightarrow \text{legalForPara}(x, E)$,
5. $x \notin \text{FV}(E) \cup \text{FV}(q)$,
6. $\models \{p \wedge x \in E \wedge \text{tid} = t\} \dot{C} \{p \wedge q(x) \wedge \text{tid} = t\}$,

then $\models \{p \wedge \text{tid} = t\} \mathbf{foreach} x \mathbf{in} E \mathbf{do} \dot{C} \{p \wedge \bigwedge_{y \in E} q(y) \wedge \text{tid} = t\}$.

PROOF. Given arbitrary $\Sigma \models p \wedge \text{tid} = t$, we need to prove the following:

$$(\mathbf{foreach} x \mathbf{in} E \mathbf{do} \dot{C}, \Sigma) \hookrightarrow_t^* \mathbf{abort} \quad (\text{goal1})$$

$$\begin{aligned} \forall \Sigma'. (\mathbf{foreach} x \mathbf{in} E \mathbf{do} \dot{C}, \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma') \\ \implies \Sigma' \models p \wedge \bigwedge_{y \in E} q(y) \wedge \text{tid} = t \end{aligned} \quad (\text{goal2})$$

By (p4) we know $\llbracket E \rrbracket_\Sigma$ is a set. Induction on $\llbracket E \rrbracket_\Sigma$, if $\llbracket E \rrbracket_\Sigma = \emptyset$, then by semantics we know

$$(\mathbf{foreach} x \mathbf{in} E \mathbf{do} \dot{C}, \Sigma) \hookrightarrow_t^* (\mathbf{skip}, \Sigma).$$

Thus we conclude (goal1)(goal2) in this case. If there exists $v_1, \dots, v_n$ s.t. $\llbracket E \rrbracket_\Sigma = \{v_1, \dots, v_n\}$, then by (p4) we know $\Sigma \models \neg \text{objVar}(x)$. By semantics we know there exists v s.t.

$$v \in \llbracket E \rrbracket_\Sigma \quad (1)$$

$$(\text{foreach } x \text{ in } E \text{ do } \dot{C}, \Sigma) \xrightarrow{t} (\dot{C}; \text{foreach } x \text{ in } E \setminus \{v\} \text{ do } \dot{C}, \Sigma|_{\sigma_a \mapsto \sigma_a\{x \rightsquigarrow v\}}) \quad (2)$$

By (1), (2) and $\Sigma \models p \wedge \text{tid} = t$ we know $\Sigma|_{\sigma_a \mapsto \sigma_a\{x \rightsquigarrow v\}} \models p \wedge \text{tid} = t \wedge x \in E$. Thus apply (6) we know

$$(\dot{C}, \Sigma|_{\sigma_a \mapsto \sigma_a\{x \rightsquigarrow v\}}) \xrightarrow{t}^* \text{abort} \quad (3)$$

$$\forall \Sigma''. (\dot{C}, \Sigma|_{\sigma_a \mapsto \sigma_a\{x \rightsquigarrow v\}}) \xrightarrow{t}^* (\text{skip}, \Sigma'') \implies \Sigma'' \models p \wedge q(x) \wedge \text{tid} = t \quad (4)$$

To prove (goal1), by (2)(3) we only need to prove if $\dot{C}$ terminates with state being Σ'' , then

$$(\text{foreach } x \text{ in } E \setminus \{v\} \text{ do } \dot{C}, \Sigma'') \xrightarrow{t}^* \text{abort}.$$

By (4) we know $\Sigma'' \models p \wedge q(x) \wedge \text{tid} = t$, thus $\Sigma'' \models p \wedge \text{tid} = t$. From (p5) and (p1) we know $\llbracket E \rrbracket_\Sigma = \llbracket E \rrbracket_{\Sigma''}$, thus apply induction hypothesis we prove the goal.

To prove (goal2), by (2), if $\dot{C}$ terminates with state being Σ'' , we only need to prove for all Σ' , if $(\text{foreach } x \text{ in } E \setminus \{v\} \text{ do } \dot{C}, \Sigma'') \xrightarrow{t}^* (\text{skip}, \Sigma')$, then $\Sigma' \models p \wedge \bigwedge_{y \in E} q(y) \wedge \text{tid} = t$. By (4) we know $\Sigma'' \models p \wedge q(x) \wedge \text{tid} = t$, by (p1) we know $\text{noModifyVars}(\{x\}, \dot{C})$, thus $\llbracket x \rrbracket_\Sigma = \llbracket x \rrbracket_{\Sigma''}$. Thus we know $\Sigma'' \models x = v$, we have $\Sigma'' \models p \wedge q(v) \wedge \text{tid} = t$.

From (p5) and (p1) we know $\llbracket E \rrbracket_\Sigma = \llbracket E \rrbracket_{\Sigma''}$, thus apply induction hypothesis we know $\Sigma' \models p \wedge \bigwedge_{y \in E \setminus \{v\}} q(y) \wedge \text{tid} = t$. Now we only need to prove $\Sigma' \models q(v)$. By (p1) and (p5) we know

$$\text{noModifyVars}(\text{FV}(q(v)), \text{foreach } x \text{ in } E \setminus \{v\} \text{ do } \dot{C}).$$

By (p2) and definition we know

$$\text{noModifyRestrictedMP}(q(v), \text{foreach } x \text{ in } E \setminus \{v\} \text{ do } \dot{C}).$$

Then apply above and (p3) on Lm. 51 we prove the goal. □

Definition 5.19 (Locality). $\text{locality}(\tilde{C})$ iff for all $s, s', \Delta_1, \Delta_2, \Delta, t, \tilde{C}'$, if

1. $(\tilde{C}, (s, \Delta_1)) \xrightarrow{t}^* \text{abort}$,
2. $\Delta = \Delta_1 * \Delta_2, \Delta_1 \# \Delta_2, \Delta_2 \neq \emptyset$,

then

1. $(\tilde{C}, (s, \Delta)) \xrightarrow{t}^* \text{abort}$,
2. if $(\tilde{C}, (s, \Delta)) \xrightarrow{t}^* (\tilde{C}', (s', \Delta'))$, then there exists Δ'_1 s.t. $(\tilde{C}, (s, \Delta_1)) \xrightarrow{t}^* (\tilde{C}', (s', \Delta'_1))$, $\Delta = \Delta'_1 * \Delta_2$ and $\Delta'_1 \# \Delta_2$.

Lemma 54. $\forall t, \Delta_1, \Delta'_1, \Delta_2, \Delta'_2. \Delta_1 \rightarrow_t \Delta'_1 \wedge \Delta_2 \rightarrow_t \Delta'_2 \wedge \Delta_1 \cap \Delta_2 = \emptyset \implies \Delta_1 \uplus \Delta_2 \rightarrow_t \Delta'_1 \uplus \Delta'_2$

PROOF. Induction on $\Delta_1 \rightarrow \Delta'_1$:

- $\Delta_1 = \Delta'_1 = \emptyset$: then by $\Delta_2 \rightarrow_t \Delta'_2$ we prove the goal immediately.
- There exists $\Delta_o, \Delta'_o, U, U, \theta, \theta'$ s.t. $\Delta_1 = \Delta_o \uplus \{(U, \theta)\}$, $\Delta'_1 = \Delta'_o \uplus \{(U', \theta')\}$, $(U, \theta) \rightarrow_t (U', \theta')$ and $\Delta_1 \rightarrow_t \Delta'_1$: then by definition we know $\Delta_o \cap \Delta_2 = \emptyset$, thus apply induction hypothesis we know $\Delta_o \uplus \Delta_2 \rightarrow_t \Delta'_o \uplus \Delta'_2$. Then by $(U, \theta) \rightarrow_t (U', \theta')$ we know

$$\{(U, \theta)\} \uplus \Delta_o \uplus \Delta_2 \rightarrow_t \{(U', \theta')\} \uplus \Delta'_o \uplus \Delta'_2.$$

Thus we prove $\Delta_1 \uplus \Delta_2 \rightarrow_t \Delta'_1 \uplus \Delta'_2$.

□

Lemma 55. $\forall t, \Delta, \Delta', \Delta_o. t \in \text{dom}(\Delta.U) \wedge \Delta \# \Delta_o \wedge \Delta \rightarrow_t \Delta' \implies \Delta * \Delta_o \rightarrow_t \Delta' * \Delta_o$

PROOF. Induction on $\Delta \rightarrow_t \Delta'$:

- $\Delta = \emptyset$ and $\Delta' = \emptyset$: Contradict $t \in \text{dom}(\Delta.U)$.
- There exists $\Delta_1, \Delta'_1, U, U', \theta, \theta'$ s.t. $\Delta = \Delta_1 \uplus \{(U, \theta)\}$, $\Delta' = \Delta'_1 \uplus \{(U', \theta')\}$, $(U, \theta) \twoheadrightarrow_t (U', \theta')$ and $\Delta_o \rightarrow_t \Delta'_o$: By definition we know $\Delta_1 \# \Delta_o$ and $t \in \text{dom}(\Delta_1.U)$, thus apply induction hypothesis we know

$$\Delta_1 * \Delta_o \rightarrow_t \Delta'_1 * \Delta_o \quad (1)$$

Similarly we know

$$\{(U, \theta)\} * \Delta_o \rightarrow_t \{(U', \theta')\} * \Delta_o \quad (2)$$

Apply (1)(2) on Lm. 54 we know

$$(\Delta_1 * \Delta_o) \uplus (\{(U, \theta)\} * \Delta_o) \rightarrow_t (\Delta'_1 * \Delta_o) \uplus (\{(U', \theta')\} * \Delta_o).$$

Thus by definition we prove $\Delta * \Delta_o \rightarrow_t \Delta' * \Delta_o$.

□

Lemma 56. $\forall t, \gamma, v, \Delta, \Delta', \Delta_o. \Delta \# \Delta_o \wedge \Delta \xrightarrow{(t, \gamma, v)} \Delta' \implies \Delta * \Delta_o \xrightarrow{(t, \gamma, v)} \Delta' * \Delta_o$

PROOF. Similar to the proof of Lm. 55.

□

Lemma 57. $\forall \Delta, \Delta', \Delta_o, D. \Delta|_D = \Delta' \wedge \Delta \# \Delta_o \implies \Delta * \Delta_o|_D = \Delta'$

PROOF. The proof of this lemma use $\Delta' \neq \emptyset$ and $\Delta_o \neq \emptyset$ implicitly.

By definition we know $D \subseteq \text{dom}(\Delta)$, along with $\Delta \# \Delta_o$ we know $\text{dom}(\Delta_o) \cap D = (\emptyset, \emptyset)$, thus we have the goal immediately through definition.

□

Lemma 58. $\forall s, p, \Delta_1, \Delta'_1, \Delta_2. \text{specExact}(p) \wedge (s, \Delta_1)|_p = \Delta'_1 \wedge \Delta_1 \# \Delta_2 \implies (s, \Delta_1 * \Delta_2)|_p = \Delta'_1 * \Delta_2$

PROOF. The following proof use $\Delta'_1 \neq \emptyset$ and $\Delta_2 \neq \emptyset$ implicitly.

By premise we know there exists Δ_p, Δ''_1 s.t.

$$\Delta_1 = \Delta'_1 \uplus \Delta''_1, \quad (s, \Delta_p) \models p, \quad \Delta'_1|_{\text{dom}(\Delta_p)} = \Delta_p, \quad \Delta''_1|_{\text{dom}(\Delta_p)} \cap \Delta_p = \emptyset.$$

By $\text{specExact}(p)$, to prove the goal we only need to prove $\Delta'_1 * \Delta_2|_{\text{dom}(\Delta_p)} = \Delta_p$ and $\Delta''_1 * \Delta_2|_{\text{dom}(\Delta_p)} \cap \Delta_p = \emptyset$. By Lm. 57 we know $\Delta'_1 * \Delta_2|_{\text{dom}(\Delta_p)} = \Delta'_1|_{\text{dom}(\Delta_p)} = \Delta_p$, thus we conclude the first goal. The second goal is similar to achieve through condition $\Delta''_1|_{\text{dom}(\Delta_p)} \cap \Delta_p = \emptyset$.

□

Lemma 59. $\forall p. \text{locality}(\text{commit}(p))$

PROOF. First we prove safety. From premise $(\text{commit}(p), (s, \Delta_1)) \hookrightarrow_t^* \text{abort}$ we know there exists Δ'_1 s.t. $\text{specExact}(p)$ and

$$(s, \Delta_1)|_p = \Delta'_1 \quad (1)$$

Apply (1) on Lm. 58 we know $(s, \Delta_1 * \Delta_2)|_p = \Delta'_1 * \Delta_2$, thus we prove $(\text{commit}(p), (s, \Delta)) \hookrightarrow_t^* \text{abort}$.

Now we prove frame property. By premise there exists k s.t. $(\text{commit}(p), (s, \Delta)) \hookrightarrow_t^k (\tilde{C}', (s', \Delta'))$, we discuss on k . By semantics we know $k \in \{0, 1\}$. If $k = 0$, then let $\Delta'_1 = \Delta_1$ we prove the goal trivially. If $k = 1$, then $\tilde{C}' = \text{skip}$, $s' = s$, $\text{specExact}(p)$ and $(s, \Delta)|_p = \Delta'$. By definition we know $(s, \Delta_p) \models p$ and

$$\Delta = \Delta' \uplus \Delta'', \quad \Delta'|_{\text{dom}(\Delta_p)} = \Delta_p, \quad \Delta''|_{\text{dom}(\Delta_p)} \cap \Delta_p = \emptyset.$$

From (1) and $\text{specExact}(p)$ we know

$$\Delta_1 = \Delta'_1 \uplus \Delta''_1, \quad \Delta'_1|_{\text{dom}(\Delta_p)} = \Delta_p, \quad \Delta''_1|_{\text{dom}(\Delta_p)} \cap \Delta_p = \emptyset.$$

Now we only need to prove $\Delta'_1 \# \Delta_2$ and $\Delta'_1 * \Delta_2 = \Delta'$, we prove these goals in following:

- $\Delta'_1 \# \Delta_2$: By definition we know $\Delta'_1 \subseteq \Delta_1$, thus $\text{dom}(\Delta'_1) = \text{dom}(\Delta_1)$, from $\Delta_1 \# \Delta_2$ we prove $\Delta'_1 \# \Delta_2$, thus we conclude this sub-goal.
- $\Delta'_1 * \Delta_2 \subseteq \Delta'$: That is to take arbitrary δ'_1, δ_2 s.t. $\delta'_1 \in \Delta'_1$ and $\delta_2 \in \Delta_2$, we need to prove $\delta'_1 \uplus \delta_2 \in \Delta'$. As $\Delta_1 = \Delta'_1 \uplus \Delta''_1$ and $\Delta = \Delta_1 * \Delta_2$, we know $\delta'_1 \uplus \delta_2 \in \Delta$. From $\Delta = \Delta' \uplus \Delta''$ we know we only need to prove $\delta'_1 \uplus \delta_2 \notin \Delta''$.

Apply $\Delta'_1|_{\text{dom}(\Delta_p)} = \Delta_p$ on Lm. 34.1 we know there exists δ_p and δ_r s.t. $\delta'_1 = \delta_p \uplus \delta_r$ and $\delta_p \in \Delta_p$. Apply $\Delta''_1|_{\text{dom}(\Delta_p)} \cap \Delta_p = \emptyset$ on Lm. 34.2 we know $\nexists \delta, \delta'. \delta \uplus \delta' \in \Delta'' \wedge \delta \in \Delta_p$. Thus $\delta'_1 \uplus \delta_2 \in \Delta''$ leads to contradiction, we prove $\delta'_1 \uplus \delta_2 \notin \Delta''$. Thus we conclude this sub-goal.

- $\Delta' \subseteq \Delta'_1 * \Delta_2$: That is to take arbitrary δ' , if $\delta' \in \Delta'$, we need to prove there exists $\delta'_1 \in \Delta'_1$, $\delta_2 \in \Delta_2$ and $\delta'_1 \uplus \delta_2 = \delta'$.

By $\Delta'_1|_{\text{dom}(\Delta_p)} = \Delta_p$ and Lm. 34.1 we know there exists δ_p, δ_r s.t. $\delta_p \uplus \delta_r = \delta'$ and $\delta_p \in \Delta_p$. By $\Delta' \subseteq \Delta$ and $\Delta = \Delta_1 * \Delta_2$ we know there exists $\delta_1 \in \Delta_1, \delta_2 \in \Delta_2$ s.t. $\delta_1 \uplus \delta_2 = \delta'$, thus we know

$$\delta_1 \uplus \delta_2 = \delta_p \uplus \delta_r \quad (2)$$

By $\Delta'_1|_{\text{dom}(\Delta_p)} = \Delta_p$ and $\Delta_p \neq \emptyset$ we know $\text{dom}(\Delta_p) \subseteq \text{dom}(\Delta'_1)$. Then by $\Delta'_1 \# \Delta_2$ we know $\Delta_p \# \Delta_2$, thus we know $\delta_2 \cap \delta_p = (\emptyset, \emptyset)$. Then by (2) we know there exists δ'_r s.t. $\delta_r = \delta_2 \uplus \delta'_r$ and $\delta_1 = \delta_p \uplus \delta'_r$.

Now we only need to prove $\delta_1 \in \Delta'_1$, which can be proved by $\delta_1 \notin \Delta''_1$. Apply $\Delta''_1|_{\text{dom}(\Delta_p)} \cap \Delta_p = \emptyset$ on Lm. 34.2 we prove $\delta_1 \notin \Delta''_1$ immediately. Thus we conclude this sub-goal. $\square$

Lemma 60. $\forall \dot{C}. \text{locality}(\dot{C})$

PROOF. Induction on $\dot{C}$:

- $\dot{C} = \text{skip}$: Trivial.
- $\dot{C} = x :=_a E$: Trivial.
- $\dot{C} = \text{lin}(E)$: First we prove safety. By premise $(\dot{C}, (s, \Delta_1)) \not\hookrightarrow_t^* \text{abort}$ we know there exists t', Δ'_1 s.t. $\llbracket E \rrbracket_s = t', \Delta_1 \rightarrow_{t'} \Delta'_1$ and $t' \in \text{dom}(\Delta_1.U)$. Apply Lm. 55 we know

$$\Delta_1 * \Delta_2 \rightarrow_{t'} \Delta'_1 * \Delta_2 \quad (1)$$

As $\Delta_2 \neq \emptyset$ and $t' \in \text{dom}(\Delta.U)$ we know $t' \in \text{dom}((\Delta_1 * \Delta_2).U)$, thus we prove $(\dot{C}, (s, \Delta)) \not\hookrightarrow_t^* \text{abort}$.

Now we prove frame property. From premise there exists k s.t. $(\widetilde{C}, (s, \Delta)) \hookrightarrow_t^k (\widetilde{C}', (s', \Delta'))$, we discuss on k . By semantics we know $k \in \{0, 1\}$. If $k = 0$, then let $\Delta'_1 = \Delta_1$ we prove the goal trivially. If $k = 1$, then $\widetilde{C}' = \text{skip}, s' = s$ and $\Delta \rightarrow_{t'} \Delta'$. From (1) and we only need to prove $(\widetilde{C}, (s, \Delta_1)) \hookrightarrow_t (\text{skip}, (s, \Delta'_1))$, which by $\llbracket E \rrbracket_s = t', \Delta_1 \rightarrow_{t'} \Delta'_1$ and $t' \in \text{dom}(\Delta_1.U)$ immediately. Thus we conclude the second goal and this case.

- $\dot{C} = \text{trylin}(E)$: Similar to the proof of $\dot{C} = \text{lin}(E)$ case.
- $\dot{C} = \text{commit}(p)$: By Lm. 59.
- $\dot{C} = \text{flin}(E_t, \gamma, E_v)$: First we prove safety. By premise $(\dot{C}, (s, \Delta_1)) \not\hookrightarrow_t^* \text{abort}$ we know there exists t', v s.t. $\llbracket (E_t, E_v) \rrbracket_s = (t', v), t' \in \llbracket \text{Nbad} \rrbracket_s$ and $\Delta_1 \xrightarrow{(t', \gamma, v)} \Delta'_1$. Apply Lm. 56 we know

$$\Delta_1 * \Delta_2 \xrightarrow{(t', \gamma, v)} \Delta'_1 * \Delta_2 \quad (2)$$

Thus we prove $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^* \mathbf{abort}$.

Now we prove frame property. By premise there exists k s.t. $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^k (\tilde{C}', (s', \Delta'))$, we discuss on k . By semantics we know $k \in \{0, 1\}$. If $k = 0$, then let $\Delta'_1 = \Delta_1$ we prove the goal trivially. If $k = 1$, then $s' = s$ and $\Delta \xrightarrow{(t', \gamma, v)} \Delta'$. By (2) we only need to prove $(\tilde{C}, (s, \Delta_1)) \hookrightarrow_t (\mathbf{skip}, (s, \Delta'_1))$, which by $\llbracket (E_t, E_v) \rrbracket_s = (t', v)$, $t' \in \llbracket \mathbf{Nbad} \rrbracket_s$ and $\Delta_1 \xrightarrow{(t', \gamma, v)} \Delta'_1$ immediately.

- $\dot{C} = \dot{C}_1; \dot{C}_2$: Trivial.
- $\dot{C} = \mathbf{if} (B) \mathbf{then} \dot{C}_1 \mathbf{else} \dot{C}_2$: Trivial.
- $\dot{C} = \mathbf{foreach} x \mathbf{in} E \mathbf{do} \dot{C}_1$: Trivial.

□

Lemma 61. $\forall \tilde{C}$. locality($\tilde{C}$)

PROOF. Induction on $\tilde{C}$:

- $\tilde{C} = \mathbf{skip}$: First we prove safety. By semantics we know $\tilde{C}$ never goes abort. Now we prove frame property. From premise $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^* (\tilde{C}', (s', \Delta'))$ and semantics we know $s' = s$ and $\Delta' = \Delta$. Let $\Delta'_1 = \Delta_1$ we prove the goal.
- $\tilde{C} = x := E$, or $\tilde{C} = x := \mathbf{recv}()$, or $\tilde{C} = \mathbf{send}(E_N, E)$: First we prove safety. By premise $(\tilde{C}, (s, \Delta_1)) \hookrightarrow_t^* \mathbf{abort}$ and semantics we prove $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^* \mathbf{abort}$ trivially. Now we prove frame property. From premise $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^* (\tilde{C}', (s', \Delta'))$, by semantics we know $\Delta' = \Delta$ and $(\tilde{C}, (s, \Delta_1)) \hookrightarrow_t^* (\tilde{C}', (s', \Delta_1))$. Let $\Delta'_1 = \Delta_1$ we prove the goal.
- $\tilde{C} = \mathbf{return} E$: First we prove safety. Let $v = \llbracket E \rrbracket_s$, by premise $(\tilde{C}, (s, \Delta_1)) \hookrightarrow_t^* \mathbf{abort}$ and semantics we know there exists Δ_o s.t. $\Delta_1 = \Delta_o * \{(\{t \rightsquigarrow (\mathbf{end}, v)\}, \emptyset)\}$. Then by semantics, premise $\Delta = \Delta_1 * \Delta_2$ and $\Delta_1 \# \Delta_2$ we know $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^* \mathbf{abort}$. Now we prove frame property. From premise $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^* (\tilde{C}', (s', \Delta'))$, by semantics we know $s' = s$, $\Delta' = \Delta$. Thus we know $(\tilde{C}, (s, \Delta_1)) \hookrightarrow_t^* (\tilde{C}', (s, \Delta_1))$. Let $\Delta'_1 = \Delta_1$ we prove the goal.
- $\tilde{C} = \dot{C}$: By Lm. 60 immediately.
- $\tilde{C} = \langle \tilde{C}_1 \rangle$: First we prove safety. From premise $(\tilde{C}, (s, \Delta_1)) \hookrightarrow_t^* \mathbf{abort}$ we know $(\tilde{C}_1, (s, \Delta_1)) \hookrightarrow_t^* \mathbf{abort}$, apply induction hypothesis we know $(\tilde{C}_1, (s, \Delta)) \hookrightarrow_t^* \mathbf{abort}$. Thus by semantics we prove $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^* \mathbf{abort}$. Now we prove frame property. From premise we know there exists k s.t. $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^k (\tilde{C}', (s', \Delta'))$, by semantics we know $k \in \{0, 1\}$. If $k = 0$, then let $\Delta'_1 = \Delta_1$ the goal trivially holds; If $k = 1$, then by semantics we know $\tilde{C}' = \mathbf{skip}$ and $(\tilde{C}_1, (s, \Delta)) \hookrightarrow_t^* (\mathbf{skip}, (s', \Delta'))$. Then apply induction hypothesis we know there exists Δ_1 s.t.

$$(\tilde{C}_1, (s, \Delta_1)) \hookrightarrow_t^* (\mathbf{skip}, (s', \Delta'_1)) \quad (1)$$

$$\Delta' = \Delta'_1 * \Delta_2 \quad (2)$$

$$(s', \Delta_2) \models r \quad (3)$$

Apply (1) we know $(\langle \tilde{C}_1 \rangle, (s, \Delta_1)) \hookrightarrow_t^k (\mathbf{skip}, (s', \Delta'_1))$, along with (2)(3) we prove the goal.

- $\tilde{C} = \tilde{C}_1; \tilde{C}_2$: First we prove safety. From premise $(\tilde{C}, (s, \Delta_1)) \hookrightarrow_t^* \mathbf{abort}$ we know $(\tilde{C}_1, (s, \Delta_1)) \hookrightarrow_t^* \mathbf{abort}$ and $(\tilde{C}_2, (s, \Delta_1)) \hookrightarrow_t^* \mathbf{abort}$, apply induction hypothesis we know $(\tilde{C}_1, (s, \Delta)) \hookrightarrow_t^* \mathbf{abort}$ and $(\tilde{C}_2, (s, \Delta)) \hookrightarrow_t^* \mathbf{abort}$, thus we prove $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^* \mathbf{abort}$. Now we prove frame property. From premise $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^* (\tilde{C}', (s', \Delta'))$, by semantics we have the following two case:

- There exists $\tilde{C}_1$ s.t. $(\tilde{C}_1, (s, \Delta)) \hookrightarrow_t^k (\tilde{C}_1', (s', \Delta'))$, then apply induction hypothesis we prove the goal immediately.
- There exists $s'', \Delta'', \tilde{C}_2$ s.t.

$$(\tilde{C}_1, (s, \Delta)) \hookrightarrow_t^k (\mathbf{skip}, (s'', \Delta'')) \quad (4)$$

$$(\tilde{C}_2, (s'', \Delta'')) \hookrightarrow_t^k (\tilde{C}_2', (s', \Delta')) \quad (5)$$

Apply (4) on induction hypothesis we know there exists Δ_1'' s.t. $(\tilde{C}_1, (s, \Delta_1)) \hookrightarrow_t^* (\mathbf{skip}, (s'', \Delta_1''))$, $\Delta'' = \Delta_1'' * \Delta_2$ and $\Delta_1'' \# \Delta_2$. Then apply (5) on induction hypothesis we know there exists Δ_1' s.t. $(\tilde{C}_2, (s'', \Delta_1'')) \hookrightarrow_t^* (\mathbf{skip}, (s', \Delta_1'))$, $\Delta' = \Delta_1' * \Delta_2$ and $\Delta_1' \# \Delta_2$. Thus we prove the goal.

- $\tilde{C} = \mathbf{if} (B) \mathbf{then} \tilde{C}_1 \mathbf{else} \tilde{C}_2$: First we prove safety. From premise $(\tilde{C}, (s, \Delta_1)) \not\hookrightarrow_t^* \mathbf{abort}$ we know $(\tilde{C}_1, (s, \Delta_1)) \not\hookrightarrow_t^* \mathbf{abort}$ and $(\tilde{C}_2, (s, \Delta_1)) \not\hookrightarrow_t^* \mathbf{abort}$. Apply induction hypothesis we know $(\tilde{C}_1, (s, \Delta)) \not\hookrightarrow_t^* \mathbf{abort}$ and $(\tilde{C}_2, (s, \Delta)) \not\hookrightarrow_t^* \mathbf{abort}$, thus we prove $(\tilde{C}, (s, \Delta)) \not\hookrightarrow_t^* \mathbf{abort}$.

Now we prove frame property. From premise $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^* (\tilde{C}', (s', \Delta'))$, by semantics we have the following two case:

- If $(\mathbf{if} (B) \mathbf{then} \tilde{C}_1 \mathbf{else} \tilde{C}_2, (s, \Delta)) \hookrightarrow_t^* (\mathbf{if} (B) \mathbf{then} \tilde{C}_1 \mathbf{else} \tilde{C}_2, (s', \Delta'))$, trivial.
- If $(\mathbf{if} (B) \mathbf{then} \tilde{C}_1 \mathbf{else} \tilde{C}_2, (s, \Delta)) \hookrightarrow_t^* (\tilde{C}_1, (s, \Delta))$ and $(\tilde{C}_1, (s, \Delta)) \hookrightarrow_t^* (\tilde{C}', (s', \Delta'))$, then apply induction hypothesis we know there exists Δ_1' s.t. $(\tilde{C}_1, (s, \Delta_1)) \hookrightarrow_t^* (\tilde{C}', (s', \Delta_1'))$, $\Delta' = \Delta_1' * \Delta_2$ and $\Delta_1' \# \Delta_2$. Thus we know $(\mathbf{if} (B) \mathbf{then} \tilde{C}_1 \mathbf{else} \tilde{C}_2, (s, \Delta_1)) \hookrightarrow_t^* (\tilde{C}_1, (s', \Delta_1'))$, we prove the goal.
- If $(\mathbf{if} (B) \mathbf{then} \tilde{C}_1 \mathbf{else} \tilde{C}_2, (s, \Delta)) \hookrightarrow_t^* (\tilde{C}_2, (s, \Delta))$ and $(\tilde{C}_1, (s, \Delta)) \hookrightarrow_t^* (\tilde{C}', (s', \Delta'))$, then this case is similar to the second case in this discussion session.
- $\tilde{C} = \mathbf{while} (B) \mathbf{do} \tilde{C}_1$: This case is similar to $\tilde{C} = \mathbf{if} (B) \mathbf{then} \tilde{C}_1 \mathbf{else} \tilde{C}_2$ case.

□

Lemma 62. $\forall E, \Sigma, \Sigma'. (\forall x \in \text{FV}(E). \llbracket x \rrbracket_\Sigma = \llbracket x \rrbracket_{\Sigma'}) \implies \llbracket E \rrbracket_\Sigma = \llbracket E \rrbracket_{\Sigma'}$

PROOF. By induction on E immediately. □

Lemma 63. $\forall B, \Sigma, \Sigma'. (\forall x \in \text{FV}(B). \llbracket x \rrbracket_\Sigma = \llbracket x \rrbracket_{\Sigma'}) \implies \llbracket B \rrbracket_\Sigma = \llbracket B \rrbracket_{\Sigma'}$

PROOF. By induction on B immediately. □

Lemma 64. $\forall r, \sigma_o, \sigma_a, \sigma'_o, \sigma'_a. (\forall x \in \text{FV}(r). \llbracket x \rrbracket_{\sigma_o \uplus \sigma_a} = \llbracket x \rrbracket_{\sigma'_o \uplus \sigma'_a} \wedge x \in \text{dom}(\sigma_o) \Leftrightarrow x \in \text{dom}(\sigma'_o)) \implies (\forall M, \Delta. ((\sigma_o, M, \sigma_a), \Delta) \models r \Leftrightarrow ((\sigma'_o, M, \sigma'_a), \Delta) \models r)$

PROOF. By induction on r immediately (use Lm. 62 and Lm. 63 in base case discussion). □

Lemma 65 (FRAME sound). for all $t, p, q, r, \tilde{C}$, if

1. $\text{noModifyVars}(\text{FV}(r), \tilde{C})$,
2. $\text{noModifyRestrictedMP}(r, \tilde{C})$,
3. $\models_t \{p \wedge \text{tid} = t\} \tilde{C} \{q \wedge \text{tid} = t\}$,

then $\models_t \{(p * r) \wedge \text{tid} = t\} \tilde{C} \{(q * r) \wedge \text{tid} = t\}$.

PROOF. By definition, given arbitrary $t, s, s', \Delta, \Delta'$, if $\Sigma \models (p * r) \wedge \text{tid} = t$ and $(\tilde{C}, (s, \Delta)) \hookrightarrow_t^* (\mathbf{skip}, (s', \Delta'))$, we need to prove:

$$\begin{aligned} (\tilde{C}, (s, \Delta)) &\not\hookrightarrow_t^* \mathbf{abort} && (\text{goal1}) \\ (s', \Delta') &\models (q * r) \wedge \text{tid} = t && (\text{goal2}) \end{aligned}$$

By $(s, \Delta) \models (p * r) \wedge \text{tid} = t$ we know there exists Δ_1, Δ_2 s.t. $(s, \Delta_1) \models p \wedge \text{tid} = t$, $(s, \Delta_2) \models r \wedge \text{tid} = t$ and $\Delta_1 \# \Delta_2$. Apply (p3) we know $(\widetilde{C}, (s, \Delta_1)) \hookrightarrow_t^* \mathbf{abort}$. Then apply Lm. 61 we prove (goal1).

To prove (goal2), apply Lm. 61 we know there exists Δ'_1 s.t.

$$\begin{aligned} (\widetilde{C}, (s, \Delta_1)) &\hookrightarrow_t^* (\mathbf{skip}, (s', \Delta'_1)), \\ \Delta &= \Delta'_1 * \Delta_2, \\ \Delta_1 \# \Delta_2. \end{aligned}$$

Apply (p3) we know $(s', \Delta'_1) \models q \wedge \text{tid} = t$. Now we only need to prove $(s', \Delta_2) \models r \wedge \text{tid} = t$. First by (p1), Lm. 64, assumption that tid is a read-only variable, and $(s, \Delta_2) \models r \wedge \text{tid} = t$ we know $((s'.\sigma_o, s.M, s'.\sigma_a), \Delta_2) \models r$. Then we discuss on restrictMP(r), if restrictMP(r), then by (p2) we know $s.M = s'.M$, thus we prove $(s', \Delta_2) \models r$; if $\neg \text{restrictMP}(r)$, then by definition we know

$$\forall \sigma_o, \sigma_a, M, M', \Delta. ((\sigma_o, M, \sigma_a), \Delta) \models r \iff ((\sigma_o, M', \sigma_a), \Delta) \models r.$$

Thus by $((s'.\sigma_o, s.M, s'.\sigma_a), \Delta_2) \models r$ we also prove $(s', \Delta_2) \models r$. Therefore, we conclude the discussion. $\square$

5.3.2 Proof of Lm. 66 (From Judgment Semantics to World Simulation).

Lemma 66. $\forall p, \eta. \text{WFF}_{p, \eta}(\widetilde{\Pi}, \Gamma) \implies \widetilde{\Pi} \preceq_{p, \eta} \Gamma$

PROOF. By definition of WFF, Lm. 67, Lm. 68 and Lm. 80 immediately. $\square$

Definition 5.20 (Benign node's guarantee). Starting with a state that satisfies p and under environment R , node t that execute client code C and instrumented server code $\widetilde{C}_s$ concurrently guarantees G at every step, formally defined as follows:

$$\text{benignNodeGuar}_{\widetilde{L}, \Gamma}(t, p, R)(C, \widetilde{C}_s)(G) \stackrel{\text{def}}{=} \forall \sigma_o, M, \sigma_a, \Delta, \sigma_c, k. ((\sigma_o, M, \sigma_a), \Delta) \models p \wedge \text{finish}(t, \Delta) \implies (((\widetilde{C}, \widetilde{C}_s), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{\widetilde{L}, \Gamma}^k G$$

where $((\widetilde{C}, \widetilde{C}_s), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta), R) \text{bGuar}_{\widetilde{L}, \Gamma}^n G$ is inductively defined as follows:

1. $((\widetilde{C}, \widetilde{C}_s), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta), R) \text{bGuar}_{\widetilde{L}, \Gamma}^0 G$ always holds;
2. $((\widetilde{C}, \widetilde{C}_s), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta), R) \text{bGuar}_{\widetilde{L}, \Gamma}^{k+1} G$ holds iff
 - a. $\kappa = \circ \implies \text{finish}(t, \Delta)$
 - b. $(\widetilde{L}, \Gamma) \vdash (((\widetilde{C}, \widetilde{C}_s), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) \hookrightarrow_t^* \mathbf{abort}$;
 - c. if $(\widetilde{L}, \Gamma) \vdash (((\widetilde{C}, \widetilde{C}_s), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) \xrightarrow{e}_t ((\widetilde{C}', \widetilde{C}'_s), (\sigma'_c, \sigma'_o, \kappa'), (M', \sigma'_a, \Delta'))$, then
 - i. $((\sigma_o, M, \sigma_a), \Delta), ((\sigma'_o, M', \sigma'_a), \Delta')) \models G$
 - ii. and $((\widetilde{C}', \widetilde{C}'_s), (\sigma'_c, \sigma'_o, \kappa')), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{\widetilde{L}, \Gamma}^k G$;
 - d. if $((\sigma_o, M, \sigma_a), \Delta), ((\sigma'_o, M', \sigma'_a), \Delta')) \models R$, then $((\widetilde{C}, \widetilde{C}_s), (\sigma_c, \sigma'_o, \kappa)), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{\widetilde{L}, \Gamma}^k G$.

Lemma 67. For all $\widetilde{L}, \Gamma, R, G^m, G^s, p, t, C, \widetilde{C}_s$, let $G^c = \text{InvokeAct}(\widetilde{L})(t) \vee G^m \vee \text{Id}$, if

1. $p \implies \text{tid} = t$,
2. $\text{stable}(p, G^c \vee G^s \vee R)$,
3. $R \implies \neg \text{badBeh}(t)$
4. $\forall f, x, \widetilde{C}. \widetilde{L}(f) = (x, \widetilde{C}) \implies (G^s \vee R), G^m \models_t \{p \wedge (\text{tid} \mapsto (\Gamma(f), x) * \text{true})\} \widetilde{C}\{p \wedge (\text{tid} \mapsto (\text{end}, _) * \text{true})\}$,

then $\text{benignNodeGuar}_{\widetilde{L}, \Gamma}(t, p, G^s \vee R)(C, \mathbf{skip})(G^c)$.

PROOF. Given arbitrary $\sigma_o, M, \sigma_a, \Delta, \sigma_c$, if $((\sigma_o, M, \sigma_a), \Delta) \models p$, then we need to prove

$$\forall n. (((C, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), G^s \vee R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n G^c.$$

We prove the goal by induction on C .

- $C = \mathbf{skip}$: Apply Lm. 70 we prove the goal.
- $C = x := E$ or $C = \mathbf{print}(E)$: Apply Lm. 71 we prove the goal.
- $C = \mathbf{send}(E, E')$, or $C = x := \mathbf{recv}()$, or $C = \mathbf{return} E$: By our assumption that the client code cannot contain **send**, **recv** or **return** statement we have contradiction.
- $C = y := f(E)$: By (p3) and Lm. 72 we prove the goal.
- $C = C_1; C_2$: By induction hypothesis and Lm. 75 we prove the goal.
- $C = \mathbf{if} (B) \mathbf{then} C_1 \mathbf{else} C_2$: By induction hypothesis and Lm. 77 we prove the goal.
- $C = \mathbf{while} (B) \mathbf{do} C'$: By induction hypothesis and Lm. 79 we prove the goal.

□

Lemma 68 (Client server composition). For all $\tilde{L}, C, \tilde{C}_s, p, R, G^s, G^c, t, \Gamma$, if

1. $\text{stable}(p, G^c \vee G^s \vee R)$,
2. $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, G^s \vee R)(C, \mathbf{skip})(G^c)$,
3. $(G^c \vee R), G^s \models_t \{p\} \tilde{C}_s \{\text{true}\}$,

then $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(C, \tilde{C}_s)(G^c \vee G^s)$.

PROOF. By definition, given arbitrary $\sigma_o, M, \sigma_a, \Delta, \sigma_c$ s.t. $((\sigma_o, M, \sigma_a), \Delta) \models p$ and $\text{finish}(t, \Delta)$, we need to prove

$$\forall n. (((C, \tilde{C}_s), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n G^c \vee G^s.$$

Apply $((\sigma_o, M, \sigma_a), \Delta) \models p$ and $\text{finish}(t, \Delta)$ on (p2) we know

$$\forall k. (((C, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), G^s \vee R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G^c.$$

Apply $((\sigma_o, M, \sigma_a), \Delta) \models p$ on (p3) we know

$$\forall k. (\tilde{C}_s, \Sigma, G^c \vee R) \text{guar}_t^k G^s.$$

Then by Lm. 69 we prove the goal.

□

Lemma 69. For all $\tilde{L}, C, \tilde{C}_s, p$, if

1. $\text{stable}(p, G^c \vee G^s \vee R)$,
2. $\kappa = \circ \implies \exists C. \tilde{C} = C$,
3. $((\sigma_o, M, \sigma_a), \Delta) \models p$,
4. $\forall k. (((\tilde{C}, \mathbf{skip}), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta), G^s \vee R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G^c$,
5. $\forall k. (\tilde{C}_s, ((\sigma_o, M, \sigma_a), \Delta), G^c \vee R) \text{guar}_t^k G^s$,

then $\forall n. (((\tilde{C}, \tilde{C}_s), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n G^c \vee G^s$.

PROOF. Induction on n . The base case is trivial; the inductive case is we need to prove the goal when $n = n' + 1$, by definition we prove sub-goals as follows:

- We need to prove $\kappa = \circ \implies \text{finish}(t, \Delta)$, which by (p4) immediately.

- We need to prove $(\tilde{L}, \Gamma) \vdash (((\tilde{C}, \tilde{C}_s), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) \hookrightarrow_t \mathbf{abort}$. By INS-CALL-ABORT, INS-ND-ABORT rule and (p2) we only need to prove

$$\begin{aligned} (\tilde{C}_s, ((\sigma_o, M, \sigma_a), \Delta)) &\hookrightarrow_t \mathbf{abort} & (\text{goal1.1}) \\ \kappa = \circ \vee (\tilde{C}, ((\sigma_o, M, \sigma_a), \Delta)) &\hookrightarrow_t \mathbf{abort} & (\text{goal1.2}) \end{aligned}$$

By (p5) we prove (goal1.1). By (p4) we know

$$(\tilde{L}, \Gamma) \vdash (((\tilde{C}, \mathbf{skip}), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) \hookrightarrow_t \mathbf{abort},$$

then by INS-ND-ABORT rule we prove (goal1.2). Thus we conclude this sub-goal.

- For all $\tilde{C}', \tilde{C}_s', \sigma'_c, \sigma'_o, \kappa', M', \sigma'_a, \Delta'$, if

$$(\tilde{L}, \Gamma) \vdash (((\tilde{C}, \tilde{C}_s), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) \xrightarrow{e}_t (((\tilde{C}', \tilde{C}_s'), (\sigma'_c, \sigma'_o, \kappa')), (M', \sigma'_a, \Delta'))$$

then we need to prove

$$(((\sigma_o, M, \sigma_a), \Delta), ((\sigma'_o, M', \sigma'_a), \Delta')) \models G^c \vee G^s \quad (\text{goal2.1})$$

$$(((\tilde{C}', \tilde{C}_s'), (\sigma'_c, \sigma'_o, \kappa')), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^{n'} G^c \vee G^s \quad (\text{goal2.2})$$

Discuss on the node execution:

- INS-PRINT, or INS-CLT-SKIP, or INS-CALL, or INS-ND-RETURN, or INS-OBJECTMETHOD: By semantics we know

$$\begin{aligned} (\tilde{L}, \Gamma) \vdash (((\tilde{C}, \mathbf{skip}), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) &\xrightarrow{e}_t \\ &(((\tilde{C}', \mathbf{skip}), (\sigma'_c, \sigma'_o, \kappa')), (M', \sigma'_a, \Delta')). \end{aligned}$$

By semantics and (p2) we know

$$\kappa' = \circ \implies \exists C. \tilde{C}' = C \quad (1)$$

By (p4) we know

$$(((\sigma_o, M, \sigma), \Delta), ((\sigma'_o, M', \sigma'_a), \Delta')) \models G^c \quad (2)$$

$$\forall k. (((\tilde{C}', \mathbf{skip}), (\sigma'_c, \sigma'_o, \kappa')), (M', \sigma'_a, \Delta'), G^s \vee R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G^c \quad (3)$$

By (2) we prove (goal2.1). By (2), (p3) and (p1) we know

$$((\sigma'_o, M', \sigma'_a), \Delta') \models p \quad (4)$$

Apply (2) on (p5) we know

$$\forall k. (\tilde{C}_s, ((\sigma'_o, M', \sigma'_a), \Delta'), G^c \vee R) \text{guar}_t^k G^s \quad (5)$$

Apply (1)(3)(4)(5) on induction hypothesis we prove (goal2.2). Thus we conclude this case.

- INS-SERVER: By semantics we know $\tilde{C}' = \tilde{C}$, $\sigma'_c = \sigma_c$, $\kappa' = \kappa$ and

$$(\tilde{C}_s, ((\sigma_o, M, \sigma_a), \Delta)) \hookrightarrow_t (\tilde{C}_s', ((\sigma'_o, M', \sigma'_a), \Delta')) \quad (6)$$

Apply (6) on (p5) we prove

$$(((\sigma_o, M, \sigma), \Delta), ((\sigma'_o, M', \sigma'_a), \Delta')) \models G^s \quad (7)$$

$$\forall k. (\tilde{C}_s', ((\sigma'_o, M', \sigma'_a), \Delta'), G^c \vee R) \text{guar}_t^k G^s \quad (8)$$

By (7) we prove (goal2.1). By (7) and (p1) we know

$$((\sigma'_o, M', \sigma'_a), \Delta') \models p \quad (9)$$

Apply (7) on (p4) we know

$$\forall k. (((\tilde{C}, \mathbf{skip}), (\sigma_c, \sigma_o, \kappa)), (M', \sigma'_a, \Delta'), G^s \vee R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G^c \quad (10)$$

Apply (8)(9)(10) on induction hypothesis we prove (goal2.2) Thus we conclude this case. Thus we conclude this sub-goal.

- For all $\sigma'_o, M', \sigma'_a, \Delta'$, if $(((\sigma_o, M, \sigma_a), \Delta), ((\sigma'_o, M', \sigma'_a), \Delta')) \models R$, then we need to prove

$$(((\tilde{C}, \tilde{C}_s), (\sigma_c, \sigma'_o, \kappa)), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^{n'} G^c \vee G^s.$$

By (p1) and (p3) we know $((\sigma'_o, M', \sigma'_a), \Delta') \models p$. From (p4) we know

$$\forall k. (((\tilde{C}, \mathbf{skip}), (\sigma_c, \sigma'_o, \kappa)), (M', \sigma'_a, \Delta'), G^s \vee R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G^c.$$

By (p5) we know

$$\forall k. (\tilde{C}_s, ((\sigma'_o, M', \sigma'_a), \Delta'), G^c \vee R) \text{guar}_t^k G^s.$$

Then apply induction hypothesis we conclude this sub-goal.

□

Lemma 70 (Node guarantee holds on skip).

For all $\tilde{L}, \Gamma, R, G, p, t$, if $\text{stable}(p, R)$ and $R \Rightarrow \neg \text{badBeh}(t)$, then $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(\mathbf{skip}, \mathbf{skip})(G)$.

PROOF. By definition, given arbitrary $\sigma_o, M, \sigma_a, \Delta$ s.t. $((\sigma_o, M, \sigma_a), \Delta) \models p$ and $\text{finish}(t, \Delta)$, we need to prove

$$\forall n, \sigma_c. (((\mathbf{skip}, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n G.$$

Induction on n . The base case is trivial; the inductive case is when $n = k + 1$, we need to prove the following sub-goals:

- We need to prove $\kappa = \circ \implies \text{finish}(t, \Delta)$, which by $\text{finish}(t, \Delta)$ immediately.
- We need to prove $(\tilde{L}, \Gamma) \vdash (((\mathbf{skip}, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \hookrightarrow_t \mathbf{abort}$, which by semantics immediately.
- For all $\sigma'_o, M, \sigma'_a, \Delta'$ s.t. $((\sigma_o, M, \sigma_a), \Delta), ((\sigma'_o, M', \sigma'_a), \Delta')) \models R$, we need to prove

$$(((\mathbf{skip}, \mathbf{skip}), (\sigma_c, \sigma'_o, \circ)), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G.$$

By premise $\text{stable}(p, R)$ we know $((\sigma'_o, M', \sigma'_a), \Delta') \models p$. By $R \Rightarrow \neg \text{badBeh}(t)$ we know $\text{finish}(t, \Delta')$. Then apply the induction hypothesis we prove the goal.

□

Lemma 71 (Node guarantee holds on assignment and print command).

For all $t, t', C, R, G, \tilde{L}, \Gamma, p, t, E, x$, if

1. $\text{Id} \Rightarrow G$,
2. $R \Rightarrow \neg \text{badBeh}(t)$
3. $\text{stable}(p, R \vee G)$,
4. $\exists x, E. C = x := E \vee C = \mathbf{print}(E)$,

then $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(C, \mathbf{skip})(G)$.

PROOF. Similar to Lm. 70.

□

Lemma 72 (Node guarantee holds on method call). For all $y, f, E, R, G^m, t, \tilde{L}, \Gamma$, if

1. $\tilde{L}(f) = (x, \tilde{C})$,
2. $p \Rightarrow \text{tid} = t$,
3. $\text{stable}(p, \text{InvokeAct}(\tilde{L})(t) \vee G^m \vee R)$, $R \Rightarrow \neg \text{badBeh}(t)$,
4. $R, G^m \models \{p \wedge (\text{tid} \mapsto (\Gamma(f), x) * \text{true})\} \tilde{C} \{p \wedge (\text{tid} \mapsto (\text{end}, _) * \text{true})\}$,

then $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(y := f(E), \mathbf{skip})(\text{InvokeAct}(\tilde{L})(t) \vee G^m \vee \text{Id})$.

PROOF. By definition and Lm. 73 immediately.

□

Lemma 73. For all $y, f, E, \sigma_c, \sigma_o, M, \sigma_a, \Delta, R, G^m, t, \tilde{L}, \Gamma$, let $\Sigma = ((\sigma_o, M, \sigma_a), \Delta)$, if

1. $\tilde{L}(f) = (x, \tilde{C})$,
2. $\Sigma \models p, p \Rightarrow \text{tid} = t, \text{finish}(t, \Delta)$,
3. $\text{stable}(p, \text{InvokeAct}(\tilde{L})(t) \vee G^m \vee R), R \Rightarrow \neg \text{badBeh}(t)$,
4. $R, G^m \models \{p \wedge (\text{tid} \mapsto (\Gamma(f), x) * \text{true})\} \tilde{C} \{p \wedge (\text{tid} \mapsto (\text{end}, _) * \text{true})\}$,

then $\forall n. (((y := f(E), \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n (\text{InvokeAct}(\tilde{L})(t) \vee G^m \vee \text{Id})$.

PROOF. Induction on n . The base case can be proved by definition trivially; the inductive case is we need to prove the goal when $n = k + 1$, that is to prove the following sub-goals:

- We need to prove $\kappa = \circ \Rightarrow \text{finish}(t, \Delta)$, which by (p2) immediately.
- We need to prove $(\tilde{L}, \Gamma) \vdash (((y := f(E), \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \not\hookrightarrow_t \mathbf{abort}$, which by (p1) and semantics immediately (note that we always assume the domain of $\tilde{L}$ and Γ is equal).
- For all $\tilde{C}_m, \sigma'_o, \Delta', v, e$, if $\tilde{L}(f) = (x, \tilde{C}_m), v = \llbracket E \rrbracket_{\sigma_c}, \sigma'_o = \sigma_o \{x \rightsquigarrow v\}, \Delta' = \Delta \{t \rightsquigarrow (\Gamma(f), v)\}, \kappa = (y, \mathbf{skip})$ and

$$(\tilde{L}, \Gamma) \vdash (((y := f(E), \tilde{C}_s), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \xrightarrow{e}_t \\ (((\tilde{C}_m, \tilde{C}_s), (\sigma_c, \sigma'_o, \kappa)), (M, \sigma_a, \Delta')),$$

then we need to prove:

$$(((\sigma_o, M, \sigma_a), \Delta), ((\sigma'_o, M, \sigma_a), \Delta')) \models \text{InvokeAct}(\tilde{L})(t) \vee G^m \vee \text{Id} \quad (\text{goal1})$$

$$(((\tilde{C}_m, \tilde{C}_s), (\sigma_c, \sigma'_o, \kappa)), (M, \sigma_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G^m \quad (\text{goal2})$$

By $\text{finish}(t, \Delta)$ we know $t \notin \text{dom}(\Delta.U)$ or $\exists v'. \forall U. (U, _) \in \Delta \Rightarrow U(t) = (\text{end}, v')$. Then as $\Delta' = \Delta \{t \rightsquigarrow (\Gamma(f), v)\}$, by definition of InvokeAct we prove (goal1). To prove (goal2), by (p3) we know $((\sigma'_o, M, \sigma_a), \Delta') \models p$. From $p \Rightarrow \text{tid} = t$ we know $\llbracket \text{tid} \rrbracket_{\sigma_o \uplus \sigma_a} = t$, also by definition of Δ' and κ' we know

$$((\sigma'_o, M, \sigma_a), \Delta') \models p \wedge (\text{tid} \mapsto (\Gamma(f), v) * \text{true}).$$

Then apply (p4) we know

$$\forall \Sigma'. (\tilde{C}, ((\sigma'_o, M, \sigma_a), \Delta')) \xrightarrow{R}_* (\mathbf{skip}, \Sigma') \Rightarrow \Sigma' \models p \wedge (t \mapsto (\text{end}, _) * \text{true}), \\ \forall k. (\tilde{C}, ((\sigma'_o, M, \sigma_a), \Delta'), R) \text{guar}_t^k G^m.$$

At last apply Lm. 74 we prove the goal.

- For all $\Sigma', \sigma'_o, M', \sigma'_a, \Delta', \Sigma'$, if $\Sigma' = ((\sigma'_o, M', \sigma'_a), \Delta')$ and $(\Sigma, \Sigma') \models R$, then we need to prove:

$$(((y := f(E), \tilde{C}_s), (\sigma_c, \sigma'_o, \circ)), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k \text{InvokeAct}(\tilde{L})(t) \vee G^m \vee \text{Id}.$$

By (p3) we know $\Sigma' \models p$ and $\text{finish}(t, \Delta')$, then apply induction hypothesis we prove the goal.

□

Lemma 74. For all $\tilde{C}, \sigma_c, \sigma_o, \kappa, y, M, \sigma_a, \Delta, R, G, t, \tilde{L}, \Gamma$, let $\Sigma = ((\sigma_o, M, \sigma_a), \Delta)$, if

1. $\text{stable}(p, G \vee R), \text{Id} \Rightarrow G$,
2. $\Sigma \models p$,
3. $\forall k. (\tilde{C}, \Sigma, R) \text{guar}_t^k G$,
4. $\kappa = (y, \mathbf{skip})$,

then $\forall n. (((\tilde{C}, \mathbf{skip}), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n G$.

PROOF. Induction on n . The base case can be proved by definition trivially; The inductive case is when there exists k s.t. $n = k + 1$, we have following sub-goals:

- We need to prove $\kappa = \circ \implies \text{finish}(t, \Delta)$, which by (p4) immediately.
- We need to prove $(\tilde{L}, \Gamma) \vdash (((\tilde{C}, \mathbf{skip}), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) \hookrightarrow_t \mathbf{abort}$. We prove this by contradiction, if $(\tilde{L}, \Gamma) \vdash (((\tilde{C}, \mathbf{skip}), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) \not\hookrightarrow_t \mathbf{abort}$, then we know $(\tilde{C}, \Sigma) \hookrightarrow_t \mathbf{abort}$, which contradict (p3).
- If $(\tilde{L}, \Gamma) \vdash (((\tilde{C}, \mathbf{skip}), (\sigma_c, \sigma_o, \kappa)), (M, \sigma_a, \Delta)) \xrightarrow{T} ((\tilde{C}', \mathbf{skip}), (\sigma'_c, \sigma'_o, \kappa')), (M', \sigma'_a, \Delta'))$, then we need to prove

$$(((\sigma_o, M, \sigma_a), \Delta), ((\sigma'_o, M', \sigma'_a), \Delta')) \models G \quad (\text{goal1})$$

$$(((\tilde{C}', \mathbf{skip}), (\sigma'_c, \sigma'_o, \kappa')), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G \quad (\text{goal2})$$

By definition of instrumented machine node-local transition, we have the following cases: (by our assumption, **print** statement does not appear in method body code, and there is no nested function call, we don't need to discuss on INS-PRINT and INS-CALL cases):

- INS-ND-RETURN: then there exists v s.t. $\tilde{C}' = \mathbf{skip}$, $\sigma'_o = \sigma_o$, $\kappa' = \circ$, $M' = M$, $\sigma'_a = \sigma_a$, $\Delta' = \Delta$, $\sigma'_c = \sigma_c \{x \rightsquigarrow v\}$ and

$$\exists \Delta''. \Delta = \Delta'' * \{(t \rightsquigarrow (\text{end}, v), \emptyset)\} \quad (1)$$

As $\text{Id} \Rightarrow G$, we prove (goal1). Apply (p1) on Lm. 70 we know

$$\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(\mathbf{skip}, \mathbf{skip})(G)$$

By (1) we know $\text{finish}(t, \Delta')$, and by definition we know $((\sigma'_o, M', \sigma'_a), \Delta') \models p$, then apply these on above we prove (goal2).

- INS-OBJECTMETHOD: then $\sigma'_c = \sigma_c$, $\tilde{C}'_s = \tilde{C}$, $\kappa' = \kappa$, $\kappa \neq \circ$ and (let $\Sigma' = ((\sigma'_o, M', \sigma'_a), \Delta')$)

$$(\tilde{C}, \Sigma) \hookrightarrow_t (\tilde{C}', \Sigma') \quad (2)$$

Apply (2) on (p3) we prove (goal1). To prove (goal2), first by (p1) we know $\text{stable}(p, G)$, thus $\Sigma' \models p$; by (p3) and (2) we know $\forall k. (\tilde{C}', \Sigma', R) \text{guar}_t^k G$; Apply above on induction hypothesis we prove (goal2).

- For all Σ' , if $(\Sigma, \Sigma') \models R$, then we need to prove $(((\tilde{C}, \tilde{C}_s), (\sigma_c, \sigma'_o, \kappa)), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G$. First by (p1) we know $\text{stable}(p, R)$, then by $(\Sigma, \Sigma') \models R$ we know $\Sigma' \models p$; apply (p3) we know $\forall k. (\tilde{C}, \Sigma', R) \text{guar}_t^k G$; Apply above on induction hypothesis we prove the goal.

□

Lemma 75 (Node guarantee holds on sequential statement). For all $C_1, C_2, p, R, G, \tilde{L}, \Gamma, t$, if

1. $\text{Id} \Rightarrow G$,
2. $\text{stable}(p, R \vee G)$,
3. $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(C_1, \mathbf{skip})(G)$,
4. $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(C_2, \mathbf{skip})(G)$,

then $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(C_1; C_2, \mathbf{skip})(G)$.

PROOF. By definition, given arbitrary $\sigma_c, \sigma_o, M, \sigma_a, \Delta$, if $((\sigma_o, M, \sigma_a), \Delta) \models p$ and $\text{finish}(t, \Delta)$, then prove

$$\forall n. (((C_1; C_2, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n G.$$

Apply (p3) we know $\forall k. (((C_1, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G$, then apply Lm. 76 we prove the goal.

□

Lemma 76. For all $C_1, C_2, \sigma_c, \sigma_o, M, \sigma_a, \Delta, R, G, p, \tilde{L}, \Gamma, t$, if

1. $\text{Id} \Rightarrow G$,
2. $((\sigma_o, M, \sigma_a), \Delta) \models p$,
3. $\text{stable}(p, R \vee G)$,
4. $\forall k. (((C_1, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G$,
5. $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(C_2, \mathbf{skip})(G)$,

then $\forall n. (((C_1; C_2, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n G$.

PROOF. Induction on n . The base case is trivial; The inductive case is when $n = k + 1$, we have following sub-goals (let $\Sigma = ((\sigma_o, M, \sigma_a), \Delta)$):

- We need to prove $\kappa = \circ \implies \text{finish}(t, \Delta)$, which by (p4) immediately.
- We need to prove $(\tilde{L}, \Gamma) \vdash (((C_1; C_2, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \hookrightarrow_t \mathbf{abort}$. We prove this by contradiction. Assume $(\tilde{L}, \Gamma) \vdash (((C_1; C_2, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \not\hookrightarrow_t \mathbf{abort}$, then we know $(\tilde{L}, \Gamma) \vdash (((C_1, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \hookrightarrow_t \mathbf{abort}$, which contradict (p4).
- For all $\tilde{C}', \sigma'_c, \sigma'_o, \kappa', M', \sigma'_a, \Delta', e$, if

$$(\tilde{L}, \Gamma) \vdash (((C_1; C_2, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \xrightarrow{e}_t ((\tilde{C}', \mathbf{skip}), (\sigma'_c, \sigma'_o, \kappa')), (M', \sigma'_a, \Delta')) \quad (1)$$

then we need to prove

$$(\Sigma, ((\sigma'_o, M', \sigma'_a), \Delta')) \models G \quad (\text{goal1})$$

$$(((\tilde{C}', \mathbf{skip}), (\sigma'_c, \sigma'_o, \kappa')), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G \quad (\text{goal2})$$

We prove the goals by further case analysis on (1):

- If the client take a step and $C_1 = \mathbf{skip}$, that is $\tilde{C}' = C_2, (\sigma'_c, \sigma'_o, \kappa') = (\sigma_c, \sigma_o, \kappa)$ and $(M', \sigma'_a, \Delta') = (M, \sigma_a, \Delta)$, then by (p1) we prove (goal1). by (p2) and (p5) we have (goal2).
- If the client take a function call step, that is there exists $x, f, E, v, y, \tilde{C}_m$ s.t. $C_1 = x := f(E), v = \llbracket E \rrbracket_{\sigma_c}, \tilde{L}(f) = (y, \tilde{C}_m)$ and

$$\begin{aligned} \tilde{C}' &= \tilde{C}_m, \\ (\sigma'_c, \sigma'_o, \kappa') &= (\sigma_c, \sigma_o \{y \rightsquigarrow v\}, (x, \mathbf{skip}; C_2)), \\ (M', \sigma'_a, \Delta') &= (M, \sigma_a, \Delta \{t \rightsquigarrow (\Gamma(f), v)\}), \end{aligned}$$

then by INS-CALL we know

$$(\tilde{L}, \Gamma) \vdash (((C_1, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \xrightarrow{(t, f, v)}_t ((\tilde{C}_m, \mathbf{skip}), (\sigma_c, \sigma'_o, (x, \mathbf{skip}))), (M, \sigma_a, \Delta')).$$

Apply (p4) we have (goal1) and

$$\forall k. (((\tilde{C}_m, \mathbf{skip}), (\sigma_c, \sigma'_o, (x, \mathbf{skip}))), (M, \sigma_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G \quad (2)$$

We prove (goal2) by prove the following (goal2.1):

For all $\sigma_c, \sigma_o, M, \sigma_a, \Delta, t, \tilde{L}, \Gamma, R, G$, if

- $\forall k. (((\tilde{C}_m, \mathbf{skip}), (\sigma_c, \sigma_o, (x, \mathbf{skip}))), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G$,
- $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(C_2, \mathbf{skip})(G)$,
- $((\sigma_o, M, \sigma_a), \Delta) \models p$,
- $\text{stable}(p, R \vee G)$,

then for all $k < n$, $(((\tilde{C}_m, \mathbf{skip}), (\sigma_c, \sigma_o, (x, \mathbf{skip}; C_2))), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G$.

We prove (goal2.1) by further induction on k . The base case trivially holds; the inductive case is when $k = k' + 1$, then

- * If there exists $E, \mathbf{E}_i, v$ s.t. $\tilde{C}_m = \mathbf{E}_i[\mathbf{return} E]$ and

$$(\tilde{L}, \Gamma) \vdash (((\mathbf{E}_i[\mathbf{return} E], \mathbf{skip}), (\sigma_c, \sigma_o, (x, \mathbf{skip}; C_2))), (M, \sigma_a, \Delta)) \xrightarrow{(t, \mathbf{ok}, v)}_t (((\mathbf{skip}; C_2, \mathbf{skip}), (\sigma_c \{x \rightsquigarrow v\}, \sigma_o, \circ)), (M, \sigma_a, \Delta)),$$

then by INS-ND-RETURN rule we know

$$(\tilde{L}, \Gamma) \vdash (((\mathbf{E}_i[\mathbf{return} E], \mathbf{skip}), (\sigma_c, \sigma_o, (x, \mathbf{skip}))), (M, \sigma_a, \Delta)) \xrightarrow{(t, \mathbf{ok}, v)}_t (((\mathbf{skip}, \mathbf{skip}), (\sigma_c \{x \rightsquigarrow v\}, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \quad (3)$$

Apply (3) on the first premise of (*goal2.1*) we prove

$$(((\sigma_o, M, \sigma_a), \Delta), ((\sigma_o, M, \sigma_a), \Delta)) \models G.$$

and we have $\forall k. (((\mathbf{skip}, \mathbf{skip}), (\sigma_c \{x \rightsquigarrow v\}, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G$. Then apply the first induction hypothesis we prove

$$(((\mathbf{skip}; C_2, \mathbf{skip}), (\sigma_c \{x \rightsquigarrow v\}, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^{k'} G.$$

* For all $\tilde{C}_m'', \sigma_o'', M'', \sigma_a'', \Delta''$ s.t.

$$(\tilde{L}, \Gamma) \vdash (((\tilde{C}_m'', \mathbf{skip}), (\sigma_c, \sigma_o, (x, \mathbf{skip}; C_2))), (M, \sigma_a, \Delta)) \xrightarrow{\quad}_t (((\tilde{C}_m'', \mathbf{skip}), (\sigma_c, \sigma_o'', (x, \mathbf{skip}; C_2))), (M'', \sigma_a'', \Delta'')),$$

by semantics we know

$$(\tilde{L}, \Gamma) \vdash (((\tilde{C}_m'', \mathbf{skip}), (\sigma_c, \sigma_o, (x, \mathbf{skip}))), (M, \sigma_a, \Delta)) \xrightarrow{\quad}_t (((\tilde{C}_m'', \mathbf{skip}), (\sigma_c, \sigma_o'', (x, \mathbf{skip}))), (M'', \sigma_a'', \Delta'')) \quad (4)$$

Apply (4) on first premise of (*goal2.1*) we prove

$$(((\sigma_o, M, \sigma_a), \Delta), ((\sigma_o'', M'', \sigma_a''), \Delta'')) \models G.$$

and we know $\forall k. (((\tilde{C}_m'', \mathbf{skip}), (\sigma_c, \sigma_o'', (x, \mathbf{skip}))), (M'', \sigma_a'', \Delta''), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G$. By $\text{stable}(p, G)$ we know $((\sigma_o'', M'', \sigma_a''), \Delta'') \models p$, then apply the second hypothesis we prove

$$(((\tilde{C}_m'', \mathbf{skip}), (\sigma_c, \sigma_o'', (x, \mathbf{skip}; C_2))), (M'', \sigma_a'', \Delta''), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^{k'} G.$$

* For all $\sigma_o'', M'', \sigma_a'', \Delta''$ s.t. $(((\sigma_o, M, \sigma_a), \Delta), ((\sigma_o'', M'', \sigma_a''), \Delta'')) \models R$, apply the first premise of (*goal2.1*) we know

$$\forall k. (((\tilde{C}_m, \mathbf{skip}), (\sigma_c, \sigma_o'', (x, \mathbf{skip}))), (M'', \sigma_a'', \Delta''), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G.$$

By $\text{stable}(p, R)$ we know $((\sigma_o'', M'', \sigma_a''), \Delta'') \models p$, then apply the second hypothesis we prove

$$(((\tilde{C}_m, \mathbf{skip}), (\sigma_c, \sigma_o'', (x, \mathbf{skip}; C_2))), (M'', \sigma_a'', \Delta''), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^{k'} G.$$

Thus we have proved (*goal2.1*). We conclude (*goal2*) immediately.

- If $C_1 \neq \mathbf{skip}$ and $\nexists x, f, E. C_1 = x := f(E)$, then by semantics we know there exists C'_1, σ'_c s.t. $\tilde{C}' = C'_1; C_2, \sigma'_o = \sigma_o, \kappa' = \circ$ and $(M', \sigma'_a, \Delta') = (M, \sigma_a, \Delta)$. By definition we know there exists e s.t.

$$(\tilde{L}, \Gamma) \vdash (((C_1, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \xrightarrow{e}_t (((C'_1, \mathbf{skip}), (\sigma'_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)).$$

By definition and (*p1*) we prove (*goal1*), then apply (*p4*) we know

$$\forall k. (((C'_1, \mathbf{skip}), (\sigma'_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G.$$

At last apply the first hypothesis we prove the goal.

- For all $\sigma'_o, M', \sigma'_a, \Delta'$, if $(\Sigma, ((\sigma'_o, M', \sigma'_a), \Delta')) \models R$, then we need to prove

$$(((C_1; C_2, \tilde{C}_s), (\sigma_c, \sigma'_o, \kappa)), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G.$$

Apply $(\Sigma, ((\sigma'_o, M', \sigma'_a), \Delta')) \models R$ on (p3) we know $((\sigma'_o, M', \sigma'_a), \Delta') \models p$; apply (p4) we know

$$\forall k. (((C_1, \mathbf{skip}), (\sigma_c, \sigma'_o, \circ)), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G,$$

then apply the first hypothesis immediately. □

Lemma 77 (Node guarantee holds on if statement). For all $B, C_1, C_2, p, R, G, \tilde{L}, \Gamma, t$, if

1. $\text{Id} \Rightarrow G$,
2. $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(C_1, \mathbf{skip})(G)$,
3. $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(C_2, \mathbf{skip})(G)$,

then $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(\text{if } (B) \text{ then } C_1 \text{ else } C_2, \mathbf{skip})(G)$.

PROOF. Given arbitrary $\sigma_c, \sigma_o, M, \sigma_a, \Delta$ s.t. $((\sigma_o, M, \sigma_a), \Delta) \models p$ and $\text{finish}(t, \Delta)$, we need to prove

$$\forall n. (((\text{if } (B) \text{ then } C_1 \text{ else } C_2, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n G.$$

Apply (p2)(p3) we know

$$\begin{aligned} &\forall k. (((C_1, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G, \\ &\forall k. (((C_2, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G, \end{aligned}$$

then apply Lm. 78 we prove the goal. □

Lemma 78. For all $B, C_1, C_2, p, R, G, \tilde{L}, \Gamma, t$, if

1. $\text{Id} \Rightarrow G$,
2. $\forall k. (((C_1, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G$,
3. $\forall k. (((C_2, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G$,

then $\forall n. (((\text{if } (B) \text{ then } C_1 \text{ else } C_2, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n G$.

PROOF. We prove the goal by induction on n . The base case is trivial; the inductive case is when $n = k + 1$, we have the following sub-goals (let $\Sigma = ((\sigma_o, M, \sigma_a), \Delta)$):

- We need to prove $\kappa = \circ \Rightarrow \text{finish}(t, \Delta)$, which by (1) immediately.
- We need to prove $(\tilde{L}, \Gamma) \vdash (((\text{if } (B) \text{ then } C_1 \text{ else } C_2, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \hookrightarrow_t \mathbf{abort}$.
By semantics we know if step never goes abort.
- If the client takes a step and $\llbracket B \rrbracket_{\sigma_c} = \mathbf{true}$, then we know

$$\begin{aligned} &(\tilde{L}, \Gamma) \vdash (((\text{if } (B) \text{ then } C_1 \text{ else } C_2, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \hookrightarrow_t \\ &(((C_1, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)). \end{aligned}$$

by (1) we prove $(\Sigma, \Sigma) \models G$, then apply (p2) we prove the goal.

- If the client takes a step and $\llbracket B \rrbracket_{\sigma_c} = \mathbf{false}$, the proof is similar to the $\llbracket B \rrbracket_{\sigma_c} = \mathbf{true}$ case.
- For all $\sigma'_o, M', \sigma'_a, \Delta'$, if $(\Sigma, ((\sigma'_o, M', \sigma'_a), \Delta')) \models R$, then we need to prove

$$(((\text{if } (B) \text{ then } C_1 \text{ else } C_2, \mathbf{skip}), (\sigma_c, \sigma'_o, \circ)), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G.$$

Apply $(\Sigma, ((\sigma'_o, M', \sigma'_a), \Delta')) \models R$ on (p2) and (p3) we know

$$\begin{aligned} &\forall k. (((C_1, \mathbf{skip}), (\sigma_c, \sigma'_o, \circ)), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G, \\ &\forall k. (((C_2, \mathbf{skip}), (\sigma_c, \sigma'_o, \circ)), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G, \end{aligned}$$

then apply the hypothesis we prove the goal. $\square$

Lemma 79 (Node guarantee holds on while statement). For all $C_1, C_2, p, R, G, \tilde{L}, \Gamma, t$, if

1. $\text{Id} \Rightarrow G$,
2. $\text{stable}(p, R \vee G)$,
3. $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(C, \mathbf{skip})(G)$,

then $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(\mathbf{while} (B) \mathbf{do} C, \mathbf{skip})(G)$.

PROOF. Given arbitrary $\sigma_c, \sigma_o, M, \sigma_a, \Delta, n$ s.t. $((\sigma_o, M, \sigma_a), \Delta) \models p$ and $\text{finish}(t, \Delta)$, we need to prove

$$(((\mathbf{while} (B) \mathbf{do} C, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n G.$$

Apply (p3) we know

$$\forall k. (((C, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G \quad (1)$$

Induction on n . The base case is trivial; the inductive case is when $n = k + 1$, we have the following sub-goals:

- We need to prove $\kappa = \circ \Rightarrow \text{finish}(t, \Delta)$, which by (p3) immediately.
- We need to prove $((\mathbf{while} (B) \mathbf{do} C, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta) \not\leftrightarrow_t \mathbf{abort}$: by semantics we know while step never goes abort.
- If the client takes a step and $\llbracket B \rrbracket_{\sigma_c} = \mathbf{true}$, then we need to prove $(\Sigma, \Sigma) \models G$, which by (p1) immediately, and prove

$$(((C; \mathbf{while} (B) \mathbf{do} C, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n G.$$

Apply Lm. 75 we know $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p, R)(C; \mathbf{while} (B) \mathbf{do} C, \mathbf{skip})(G)$, Apply $((\sigma_o, M, \sigma), \Delta) \models p$ on it we prove the goal.

- If the client takes a step and $\llbracket B \rrbracket_{\sigma_c} = \mathbf{false}$, then we need to prove $(\Sigma, \Sigma) \models G$, which by (p1) immediately, and prove

$$(((\mathbf{skip}, \mathbf{skip}), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^n G,$$

which by Lm. 70 immediately.

- For all $\sigma'_o, M', \sigma'_a, \Delta'$ s.t. $(\Sigma, ((\sigma'_o, M', \sigma'_a), \Delta')) \models R$, we need to prove

$$(((\mathbf{while} (B) \mathbf{do} C, \mathbf{skip}), (\sigma_c, \sigma'_o, \circ)), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G.$$

Apply $(\Sigma, ((\sigma'_o, M', \sigma'_a), \Delta')) \models R$ on (1) we know

$$\forall k. (((C, \mathbf{skip}), (\sigma_c, \sigma'_o, \circ)), (M', \sigma'_a, \Delta'), R) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G,$$

then apply the induction hypothesis we prove the goal. $\square$

Lemma 80. For all $\mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}, t_1, \dots, t_n, C_{t_1}, \dots, C_{t_n}, \tilde{L}, \tilde{C}_s, \zeta, \Gamma, p, R, G, t \in \{t_1, \dots, t_n\}$, if

1. $\text{checkFlin}_{\Gamma}((\tilde{L}, \tilde{C}_s, \zeta))$,
2. $\mathcal{N}_{\text{ok}} = \{t_1, \dots, t_n\}$, $\mathcal{N}_{\text{ok}} \cap \mathcal{N}_{\text{bad}} = \emptyset$,
3. $\bigvee_{t' \neq t} G_{t'} \vee \text{GF}(\zeta) \Rightarrow R_t$,
4. $\text{stable}(p_t, G_t \vee \text{nonLocal}(R_t))$,
5. $\text{benignNodeGuar}_{\tilde{L}, \Gamma}(t, p_t, \text{nonLocal}(R_t))(C_t, \tilde{C}_s)(G_t)$,

then **let** $((\tilde{L}, \tilde{C}_s, \zeta), \Gamma)$ **in** $C_{t_1} \parallel \dots \parallel C_{t_n} \preceq_p^{\mathcal{N}_{\text{bad}}}$ **with** Γ **do** $C_{t_1} \parallel \dots \parallel C_{t_n}$.

PROOF. Given arbitrary $\sigma_c, \sigma_a, \sigma_o, \theta$, if $(\sigma_o, \sigma_a, \theta, \mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}) \models_{\text{init}} p$, then we need to prove

$$(\text{let } ((\tilde{L}, \tilde{C}_s, \zeta), \Gamma) \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o, \sigma_a)) \preceq \{(\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta))\}.$$

By co-induction, given arbitrary $\tilde{W}'$, if

$$(\text{let } (\tilde{L}, \tilde{C}_s, \zeta) \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o, \sigma_a)) \# \rightarrow \tilde{W}' \quad (1)$$

then we need to find a Ω' and prove

$$\begin{aligned} \forall \mathbb{W}' \in \Omega'. \exists \mathbb{W} \in \Omega. \mathbb{W} \# \rightarrow \mathbb{W}' & \quad (\text{goal1}) \\ \mathbb{W}' \preceq \Omega' & \quad (\text{goal2}) \end{aligned}$$

Let

$$\begin{aligned} \sigma'_o(t) &= \sigma_o \uplus \{\text{tid} \rightsquigarrow t, \text{Nodes} \rightsquigarrow \mathcal{N}_{\text{ok}} \uplus \mathcal{N}_{\text{bad}}\} \\ \sigma'_a &= \sigma_a \{\text{Nok} \rightsquigarrow \mathcal{N}_{\text{ok}}, \text{Nbad} \rightsquigarrow \mathcal{N}_{\text{bad}}\} \end{aligned}$$

By (1) and INS-INIT we know there exists $\widetilde{DC}$ s.t.

$$\begin{aligned} \tilde{W}' &= ((\tilde{L}, \zeta), \widetilde{DC}, (\emptyset, \sigma'_a, \{(\emptyset, \theta)\}), \mathcal{N}_{\text{bad}}), \\ \forall t' \in \mathcal{N}_{\text{ok}}. \widetilde{DC}(t') &= ((C_{t'}, \tilde{C}_s), (\sigma_c, \sigma'_o(t'), \circ)). \end{aligned}$$

Let $\Omega' = \{(\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\{t_1 \rightsquigarrow (\sigma_c, \circ), \dots, t_n \rightsquigarrow (\sigma_c, \circ)\}, \theta))\}$. By definition and ABS-INIT rule we have (goal1) immediately.

By $(\sigma_o, \sigma_a, \theta, \mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}) \models_{\text{init}} p$ we know

$$\forall t' \in \mathcal{N}_{\text{ok}}. ((\sigma'_o(t'), \emptyset, \sigma'_a), \{(\emptyset, \theta)\}) \models p_{t'}.$$

Then apply (p5) we know

$$\forall t' \in \mathcal{N}_{\text{ok}}. \forall k. (\widetilde{DC}(t'), (\emptyset, \sigma'_a, \{(\emptyset, \theta)\}), \text{nonLocal}(R_{t'})) \text{bGuar}_{t', \tilde{L}, \Gamma}^k G_{t'}.$$

Apply Lm. 81 we have (goal2). □

Definition 5.21.

$$\begin{aligned} \text{checkFlin}_{\Gamma}(\{t_1 \rightsquigarrow \widetilde{NC}_{t_1}, \dots, t_n \rightsquigarrow \widetilde{NC}_{t_n}\}) &\stackrel{\text{def}}{=} \bigwedge_{t \in \{t_1, \dots, t_n\}} \text{checkFlin}_{\Gamma}(\widetilde{NC}_t) \\ \text{checkFlin}_{\Gamma}(\widetilde{NC}) &\stackrel{\text{def}}{=} \begin{cases} \text{checkFlin}_{\Gamma}(\tilde{C}_s) & \text{if } \widetilde{NC} = ((C, \tilde{C}_s), (\sigma_c, \sigma_o, \circ)) \\ \text{checkFlin}_{\Gamma}(\tilde{C}) \wedge \text{checkFlin}_{\Gamma}(\tilde{C}_s) & \text{if } \widetilde{NC} = ((\tilde{C}, \tilde{C}_s), (\sigma_c, \sigma_o, (x, C))) \end{cases} \end{aligned}$$

Lemma 81. For all $p, R, G, \mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}, t_1, \dots, t_n, \mathbb{C}_{t_1}, \dots, \mathbb{C}_{t_n}, \tilde{L}, \Gamma, \zeta, \Gamma, \widetilde{DC}, M, \sigma_a, \Delta, t \in \{t_1, \dots, t_n\}$, if

1. $\mathcal{N}_{\text{ok}} \cap \mathcal{N}_{\text{bad}} = \emptyset, \text{dom}(\widetilde{DC}) = \mathcal{N}_{\text{ok}}, \text{dom}(\Delta.U) \subseteq \mathcal{N}_{\text{ok}}, \sigma_a(\text{Nbad}) = \mathcal{N}_{\text{bad}},$
2. $\text{checkFlin}_{\Gamma}(\widetilde{DC}),$
3. $\bigvee_{t' \neq t} G_{t'} \vee \text{GF}(\zeta) \Rightarrow R_t,$
4. $\text{stable}(p_t, G_t \vee \text{nonLocal}(R_t)),$
5. $((\widetilde{DC}(t). \sigma_o, M, \sigma_a), \Delta) \models p_t,$
6. $\forall k. (\widetilde{DC}(t), (M, \sigma_a, \Delta), \text{nonLocal}(R_t)) \text{bGuar}_{t, \tilde{L}, \Gamma}^k G_t,$

then $((\tilde{L}, \zeta), \widetilde{DC}, (M, \sigma_a, \Delta), \mathcal{N}_{\text{bad}}) \preceq [(\Gamma, \widetilde{DC}, \mathcal{N}_{\text{bad}}, \Delta)]$,

where

$$\begin{aligned} [(\Gamma, \widetilde{DC}, \mathcal{N}_{\text{bad}}, \Delta)] &\stackrel{\text{def}}{=} \{(\mathbf{with} \ \Gamma \ \mathbf{do} \ \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\text{toAbsLocState}(\widetilde{DC}), \theta)) \mid (U, \theta) \in \\ &\Delta \wedge \forall t \in \text{dom}(\widetilde{DC}). \mathbb{C}_t = \text{toAbsStmnt}_{\Gamma}(\widetilde{DC}(t).k, \widetilde{DC}(t).C, U(t))\} \\ \text{toAbsLocState}(\widetilde{DC}) &\stackrel{\text{def}}{=} \begin{cases} \{t \rightsquigarrow (\widetilde{NC}. \sigma_c, \widetilde{NC}.k)\} \uplus \text{toAbsLocState}(\widetilde{DC}') & \text{if } \widetilde{DC} = \widetilde{DC}' \uplus \{t \rightsquigarrow \widetilde{NC}\} \\ \emptyset & \text{otherwise} \end{cases} \\ \text{toAbsStmnt}_{\Gamma}(k, C, \Upsilon) &\stackrel{\text{def}}{=} \begin{cases} C & \text{if } k = \circ \\ \mathbf{fexec}(f, v) & \text{if } k \neq \circ \wedge \Upsilon = (\Gamma(f), v) \\ \mathbf{fret}(v') & \text{if } k \neq \circ \wedge \Upsilon = (\text{end}, v') \end{cases} \end{aligned}$$

PROOF. Let the left side of proof goal be $\widetilde{W}$, the right side be Ω , by definition and co-induction, given arbitrary $\widetilde{DC}'$, M' , σ'_a , Δ' , e , if $\widetilde{W} \xleftarrow{e} ((\tilde{L}, \zeta), \widetilde{DC}', (M', \sigma'_a, \Delta'), \mathcal{N}_{\text{bad}})$ (Let $\widetilde{W}' = ((\tilde{L}, \zeta), \widetilde{DC}', (M', \sigma'_a, \Delta'), \mathcal{N}_{\text{bad}})$), by induction hypothesis we need to prove there exists T s.t.

$$\forall t \in \mathcal{N}_{\text{ok}}. ((\widetilde{DC}'(t). \sigma_o, M', \sigma'_a), \Delta') \models p_t \quad (\text{goal1})$$

$$\forall t \in \mathcal{N}_{\text{ok}}, k. (\widetilde{DC}'(t), (M', \sigma'_a, \Delta'), \text{nonLocal}(R_t)) \text{ bGuar}_{t, \tilde{L}, \Gamma}^k G_t \quad (\text{goal2})$$

$$\Omega \xrightarrow{T} [(\Gamma, \widetilde{DC}', \mathcal{N}_{\text{bad}}, \Delta')] \quad (\text{goal3})$$

$$\text{get-obsv}(e) = \text{get-obsv}(T) \quad (\text{goal4})$$

By case analysis on $\widetilde{W} \xleftarrow{e} \widetilde{W}'$, we have the following two case:

- **INS-BENIGN-STEP:** Then there exists t' , $\widetilde{NC}'$ s.t.

$$\widetilde{DC}' = \widetilde{DC}\{t' \rightsquigarrow \widetilde{NC}'\} \quad (1)$$

$$(\tilde{L}, \Gamma) \vdash (\widetilde{DC}(t), (M, \sigma_a, \Delta)) \xrightarrow{e}_{t'} (\widetilde{NC}', (M', \sigma'_a, \Delta')) \quad (2)$$

Apply (2) on (p6) we know

$$(((\widetilde{DC}(t'). \sigma_o, M, \sigma_a), \Delta), (\widetilde{DC}'(t'). \sigma_o, M', \sigma'_a), \Delta') \models G_{t'} \quad (3)$$

To prove (goal1), if $t = t'$, then by (p4) and (3) we prove the case; if $t \neq t'$, then by (3) and (p3) we know the action satisfies R_t , also by (1) we know $\widetilde{DC}(t). \sigma_o = \widetilde{DC}'(t). \sigma_o$, then by definition of nonLocal we know

$$(((\widetilde{DC}(t). \sigma_o, M, \sigma_a), \Delta), (\widetilde{DC}'(t). \sigma_o, M', \sigma'_a), \Delta') \models \text{nonLocal}(R_t) \quad (4)$$

then by (p5) and (p4) we prove the goal.

To prove (goal2), if $t = t'$, then apply (2) on (p6) we prove the goal immediately; if $t \neq t'$, then apply (4) on (p6) we conclude the goal.

Let the right side of (goal3) be Ω' , to prove (goal3) and (goal4), we need to take arbitrary $\mathbb{W}'$ from Ω' , then pick a $\mathbb{W}$ from Ω , and prove $\mathbb{W} \xrightarrow{T}^* \mathbb{W}'$ and $\text{get-obsv}(e) = \text{get-obsv}(T)$. By definition of Ω' we know there exists $\mathbb{C}_{t_1}, \dots, \mathbb{C}_{t_n}, U, \theta$ and

$$\begin{aligned} \mathbb{W}' &= (\mathbf{with} \ \Gamma \ \mathbf{do} \ \mathbb{C}'_{t_1} \parallel \dots \parallel \mathbb{C}'_{t_n}, \mathcal{N}_{\text{bad}}, (\text{toAbsLocState}(\widetilde{DC}'), \theta')), \\ (U', \theta') &\in \Delta', \\ \forall t \in \mathcal{N}_{\text{ok}}. \mathbb{C}'_t &= \text{toAbsStmnt}_{\Gamma}(\widetilde{DC}'(t).k, \widetilde{DC}'(t).C, U'(t)). \end{aligned}$$

We prove this by further case analysis on (2).

- If **INS-PRINT**, then $M' = M$, $\sigma'_a = \sigma_a$, $\Delta' = \Delta$ and there exists $\mathbf{E}, E, \widetilde{C}_s, \sigma_c, \sigma_o$ s.t.

$$\begin{aligned} e &= (t', \mathbf{out}, \llbracket E \rrbracket_{\sigma_c}), \\ \widetilde{DC}(t') &= ((\mathbf{E}[\mathbf{print}(E)], \widetilde{C}_s), (\sigma_c, \sigma_o, \circ)), \\ \widetilde{DC}'(t') &= ((\mathbf{E}[\mathbf{skip}], \widetilde{C}_s), (\sigma_c, \sigma_o, \circ)). \end{aligned}$$

As $\Delta' = \Delta$, we know $(U', \theta') \in \Delta$, we pick

$$\mathbb{W} = (\text{with } \Gamma \text{ do } \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\text{toAbsLocState}(\widetilde{DC}'), \theta')),$$

where for all $t \neq t'$, $\mathbb{C}_t = \mathbb{C}'_t$, but $\mathbb{C}_{t'} = \mathbf{E}[\text{print}(E)]$. By (1) and $\mathbb{W} \in \Omega$, we only need to prove $\mathbb{W} \xrightarrow{T}^* \mathbb{W}'$ and $\text{get-obsv}(T) = \text{get-obsv}(e)$, which by ABS-BENIGN-STEP and ABS-PRINT rule immediately.

- If INS-CALL, then there exists $x, f, E, \mathbf{E}, \widetilde{C}_s, \sigma_c, \sigma_o, v$ s.t. $M' = M, \sigma'_a = \sigma_a, v = \llbracket E \rrbracket_{\sigma_c}$ and

$$\begin{aligned} \Delta' &= \Delta\{t' \rightsquigarrow (\Gamma(f), v)\} \\ \widetilde{L}(f) &= (y, \widetilde{C}_m) \\ e &= (t', f, v) \\ \widetilde{DC}(t') &= ((\mathbf{E}[x := f(E)], \widetilde{C}_s), (\sigma_c, \sigma_o, \circ)) \\ \widetilde{DC}'(t') &= ((\widetilde{C}_m, \widetilde{C}_s), (\sigma_c, \sigma_o\{y \rightsquigarrow v\}, (x, \mathbf{E}[\text{skip}]))) \end{aligned} \quad (5)$$

We pick

$$\mathbb{W} = (\text{with } \Gamma \text{ do } \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\text{toAbsLocState}(\widetilde{DC}), \theta')),$$

where for all $t \neq t'$, $\mathbb{C}_t = \mathbb{C}'_t$ and $\mathbb{C}_{t'} = \mathbf{E}[x := f(E)]$. As $(U', \theta') \in \Delta$ and (5) we know there exists U s.t. $U' = U\{t \rightsquigarrow (\Gamma(f), v)\}$ and $(U, \theta') \in \Delta$. Thus, by definition we know $\mathbb{W}$ belongs to Ω . Then by definition of toAbsStmt we know $\mathbb{C}_{t'} = \mathbf{fexec}(f, v)$, by ABS-BENIGN-STEP and ABS-CALLS we prove the goal.

- If INS-ND-RETURN, then there exists $\mathbf{E}_i, E, \widetilde{C}_s, x, C, \sigma_c, \sigma_o, v$ s.t. $M' = M, \sigma'_a = \sigma_a, \Delta' = \Delta, v = \llbracket E \rrbracket_{\sigma_o}$ and

$$\begin{aligned} \exists \Delta''. \Delta' &= \Delta'' * \{(\{t' \rightsquigarrow (\text{end}, v)\}, \emptyset)\} \\ e &= (t, \mathbf{ok}, v) \\ \widetilde{DC}(t') &= ((\mathbf{E}_i[\text{return } E], \widetilde{C}_s), (\sigma_c, \sigma_o, (x, C))) \\ \widetilde{DC}'(t') &= ((C, \widetilde{C}_s), (\sigma_c\{x \rightsquigarrow v\}, \sigma_o, \circ)) \end{aligned} \quad (6)$$

We pick

$$\mathbb{W} = (\text{with } \Gamma \text{ do } \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\text{toAbsLocState}(\widetilde{DC}), \theta')),$$

where for all $t \neq t'$, $\mathbb{C}_t = \mathbb{C}'_t$ and $\mathbb{C}_{t'} = \mathbf{fret}(v)$. By $\Delta' = \Delta$ we know $(U', \Delta') \in \Delta$, and by definition of toAbsLocState we know

$$\begin{aligned} \text{toAbsLocState}(\widetilde{DC})(t') &= (\sigma_c\{x \rightsquigarrow v\}, (x, C)), \\ \text{toAbsLocState}(\widetilde{DC}')(t') &= (\sigma_c, \circ). \end{aligned}$$

By (6) we know $U'(t) = (\text{end}, v)$, thus by definition we know $\mathbb{W} \in \Omega$. From definition of toAbsStmt we know $\mathbb{C}_{t'} = C$. By ABS-BENIGN-STEP and ABS-FRET we prove $\mathbb{W} \xrightarrow{\epsilon}^* \mathbb{W}'$, then by definition we prove $\text{get-obsv}(\epsilon) = \text{get-obsv}((t', \mathbf{ok}, v))$.

- If INS-OBJECTMETHOD, then there exists $\widetilde{C}, \widetilde{C}_s, \sigma_c, \sigma_o, \kappa$ s.t.

$$\begin{aligned} \widetilde{C} &\neq \mathbf{E}_i[\text{return } _], \\ \kappa &\neq \circ, \\ (\widetilde{C}, ((\sigma_o, M, \sigma_a), \Delta)) &\xrightarrow{t'} (\widetilde{C}', ((\sigma'_o, M', \sigma'_a), \Delta')), \\ \widetilde{DC}(t') &= ((\widetilde{C}, \widetilde{C}_s), (\sigma_c, \sigma_o, \kappa)), \\ \widetilde{DC}'(t') &= ((\widetilde{C}', \widetilde{C}_s), (\sigma_c, \sigma'_o, \kappa)). \end{aligned}$$

By (p6) we know

$$\forall t \in \mathcal{N}_{\text{ok}}. \widetilde{DC}(t). \kappa = \circ \implies \text{finish}(t, \Delta).$$

Apply Lm. 82 we have the goal.

- If INS-SERVER, the proof is similar to the INS-OBJECTMETHOD case.

- **INS-FAULTY-STEP:** Then there exists $v, \mathcal{N}$ s.t. $e = \tau, \mathcal{N}_{\text{bad}} \neq \emptyset, v \notin \zeta(\mathcal{N}_{\text{bad}}, M), M' = M \cup \{(\mathcal{N}, v)\}, \widetilde{DC}' = \widetilde{DC}, \sigma'_a = \sigma_a$ and $\Delta' = \Delta$.

By definition we know

$$\forall \sigma. (((\sigma, M, \sigma_a), \Delta), ((\sigma, M', \sigma_a), \Delta)) \models \text{GF}(\zeta),$$

thus by definition we know

$$\forall t \in \mathcal{N}_{\text{ok}}. (((\widetilde{DC}(t). \sigma_o, M, \sigma_a), \Delta), ((\widetilde{DC}'(t). \sigma_o, M', \sigma_a), \Delta)) \models \text{nonLocal}(\text{GF}(\zeta)). \quad (7)$$

(goal1) can be proved by (p5), (p3), (p4), (7) immediately. Apply (7) on (p6) we prove (goal2), then by definition we prove $\Omega \xRightarrow{\epsilon} \Omega$, thus we prove (goal3) and (goal4). □

Lemma 82. For all $\Gamma, \widetilde{DC}, \mathcal{N}_{\text{bad}}, \sigma_c, \sigma_o, \sigma'_o, \kappa, M, M', \sigma_a, \sigma'_a, \Delta, \Delta', t$, if

1. $\kappa \neq \circ, \widetilde{C} \neq \mathbf{E_i}[\text{return } _], \sigma_a(\mathcal{N}_{\text{bad}}) = \mathcal{N}_{\text{bad}}$
2. $\text{checkFlin}_{\Gamma}(\widetilde{C}),$
3. $\widetilde{DC}(t) = (_, (\sigma_c, \sigma_o, \kappa)), \widetilde{DC}' = \widetilde{DC}\{t \rightsquigarrow (_, (\sigma_c, \sigma'_o, \kappa))\},$
4. $(\widetilde{C}, ((\sigma_o, M, \sigma_a), \Delta)) \hookrightarrow_t (\widetilde{C}', ((\sigma'_o, M', \sigma'_a), \Delta')),$
5. $\forall t. \widetilde{DC}(t). \kappa = \circ \implies \text{finish}(t, \Delta)$

then there exists T s.t. $\llbracket (\Gamma, \widetilde{DC}, \mathcal{N}_{\text{bad}}, \Delta) \rrbracket \xRightarrow{T} \llbracket (\Gamma, \widetilde{DC}', \mathcal{N}_{\text{bad}}, \Delta') \rrbracket$ and $\text{get-obsv}(T) = \epsilon$.

PROOF. Induction on (p4):

- **INS-CONCRETE:** We know $\Delta' = \Delta$, thus we only need to prove $\llbracket (\Gamma, \widetilde{DC}, \mathcal{N}_{\text{bad}}, \Delta) \rrbracket \xRightarrow{\epsilon} \llbracket (\Gamma, \widetilde{DC}', \mathcal{N}_{\text{bad}}, \Delta) \rrbracket$. By (p1) $\kappa \neq \circ$, (p3) and definition we know $\llbracket (\Gamma, \widetilde{DC}, \mathcal{N}_{\text{bad}}, \Delta) \rrbracket = \llbracket (\Gamma, \widetilde{DC}', \mathcal{N}_{\text{bad}}, \Delta) \rrbracket$, thus we have the goal immediately by definition of $\xRightarrow{\epsilon}$.
- **INS-DOT-COMM:** We know $\sigma'_o = \sigma_o, M' = M$ and

$$(\sigma_o, M) \vdash (\dot{C}, (\sigma_a, \Delta)) \hookrightarrow (\dot{C}', (\sigma'_a, \Delta')) \quad (1)$$

We further induction on (1):

- **LIN:** We know there exists t' s.t. $\Delta \rightarrow_{t'} \Delta'$, thus we know

$$\forall (U', \theta') \in \Delta'. \exists (U, \theta) \in \Delta. (U, \theta) \rightarrow_{t'} (U', \theta') \quad (2)$$

Given arbitrary $\mathbb{W}'$ in $\llbracket (\Gamma, \widetilde{DC}', \mathcal{N}_{\text{bad}}, \Delta') \rrbracket$, by definition we know there exists $(U', \theta') \in \Delta'$ s.t.

$$\begin{aligned} \forall t. \mathbb{W}'. \mathbb{C}_t &= \text{toAbsStmt}_{\Gamma}(\widetilde{DC}'(t). \kappa, \widetilde{DC}'(t). C, U'(t)), \\ \mathbb{W}'. \mathcal{N}_{\text{bad}} &= \mathcal{N}_{\text{bad}}, \\ \mathbb{W}'. \theta &= \theta'. \end{aligned}$$

Apply (2) we know there exists U, θ s.t. $(U, \theta) \in \Delta$ and $(U, \theta) \rightarrow_{t'} (U', \theta')$.

Let $\mathbb{W} = (\mathbf{with} \ \Gamma \ \mathbf{do} \ \mathbb{C}_{t_1} \parallel \dots \parallel \mathbb{C}_{t_n}, \mathcal{N}_{\text{bad}}, (\mathbb{W}'. \varrho, \theta))$, where

$$\forall t. \mathbb{C}_t = \text{toAbsStmt}_{\Gamma}(\widetilde{DC}(t). \kappa, \widetilde{DC}(t). C, U(t))$$

Now we only need to prove there exists T s.t. $\mathbb{W} \in \llbracket (\Gamma, \widetilde{DC}, \mathcal{N}_{\text{bad}}, \Delta) \rrbracket$, $\mathbb{W} \xrightarrow{T}^* \mathbb{W}'$ and $\text{get-obsv}(T) = \epsilon$. The first goal can be proved by definition and $(U, \theta) \in \Delta$ immediately. To prove the second goal, by $(U, \theta) \rightarrow_{t'} (U', \theta')$ we have the following two cases:

- * If $U(t') = (\text{end}, _)$, then $(U', \theta') = (U, \theta)$. Along with (p3) we know $\mathbb{W} = \mathbb{W}'$. We prove $\mathbb{W} \xrightarrow{\epsilon}^* \mathbb{W}'$ immediately by definition.
- * If $U(t') = (\gamma, v)$, then by definition we know $\neg \text{finish}(t', \Delta)$, and there exists γ, v' s.t. $(v', \theta') = \gamma(t')(v, \theta)$ and

$$\begin{aligned} U'(t') &= (\text{end}, v'), \\ \forall t \neq t'. U'(t) &= U(t). \end{aligned}$$

By $\neg\text{finish}(t', \Delta)$ and (p5) we know $\widetilde{DC}(t').\kappa \neq \circ$. Then by definition of toAbsStmT we know there exists f s.t. $\Gamma(f) = \gamma$, $\mathbb{W}.C_{t'} = \mathbf{fexec}(f, v)$, $\mathbb{W}'.C_{t'} = \mathbf{fret}(v')$ and $\forall t \neq t'. \mathbb{W}'.C_t = \mathbb{W}.C$. Thus by ABS-BENIGN-STEP and ABS-EXEC rule we prove $\mathbb{W} \xrightarrow{(t', f, v) :: (t', \mathbf{ok}, v') :: \epsilon} \mathbb{W}'$. By definition we know $\text{get-obsv}((t', f, v) :: (t', \mathbf{ok}, v') :: \epsilon) = \epsilon$, thus the goal is proved.

We conclude this case.

- TRYLIN: Similar to LIN case.
- COMMIT: We know $\Delta' \subseteq \Delta$, by definition we have the goal immediately.
- FLIN: We know there exists $t', \gamma, v, \mathbf{E}_i, E, E'$ s.t.

$$\begin{aligned} \widetilde{C} &= \mathbf{E}_i[\mathbf{flin}(E, \gamma, E')], & \llbracket (E, E') \rrbracket_{\sigma_o \uplus \sigma_a} &= (t', v), & t' &\in \llbracket \mathbf{Nbad} \rrbracket_{\sigma_o \uplus \sigma_a}, \\ \Delta &\xrightarrow{(t', \gamma, v)} \Delta'. \end{aligned}$$

By (p2) we know $\gamma \in \text{range}(\Gamma)$. Thus by definition we know there exists f s.t.

$$\begin{aligned} \Gamma(f) &= \gamma & (3) \\ \forall (U', \theta') \in \Delta'. \exists \theta, v'. (U', \theta) \in \Delta \wedge \gamma(t')(v, \theta) &= (v', \theta') & (4) \end{aligned}$$

Given arbitrary $\mathbb{W}'$ in $\llbracket (\Gamma, \widetilde{DC}', \mathcal{N}_{\text{bad}}, \Delta') \rrbracket$, by definition we know there exists $(U', \theta') \in \Delta'$ s.t. $\forall t. \mathbb{W}'.C_t = \text{toAbsStmT}_{\Gamma}(\widetilde{DC}(t).\kappa, \widetilde{DC}(t).C, U'(t))$, $\mathbb{W}'.\mathcal{N}_{\text{bad}} = \mathcal{N}_{\text{bad}}$ and $\mathbb{W}'.\theta = \theta'$. Apply (4) we know there exists θ, v' s.t. $(U', \theta) \in \Delta$ and $\gamma(t')(v, \theta) = (v', \theta')$.

Let $\mathbb{W} = (\mathbb{W}'.\mathbb{P}, \mathcal{N}_{\text{bad}}, (\mathbb{W}'.\varrho, \theta))$, by definition and $(U', \theta) \in \Delta$ we prove $\mathbb{W} \in \llbracket (\Gamma, \widetilde{DC}, \mathcal{N}_{\text{bad}}, \Delta) \rrbracket$.

Now we only need to prove $\mathbb{W} \xrightarrow{(t', f, v) :: (t', \mathbf{ok}, v') :: \epsilon}^* \mathbb{W}'$.

By (p1) $\sigma_a(\mathbf{Nbad}) = \mathcal{N}_{\text{bad}}$ and $t' \in \llbracket \mathbf{Nbad} \rrbracket_{\sigma_a}$ we know $t' \in \mathcal{N}_{\text{bad}}$. Apply (3), $\gamma'(t')(v, \theta) = (v', \theta')$ and $t' \in \mathcal{N}_{\text{bad}}$ on ABS-FAULTY-STEP rule we prove the goal.

- The other cases are similar to the INS-CONCRETE case as $\Delta' = \Delta$.
- INS-RETURN: Contradict (p1).
- INS-SKIP: Trivial.
- INS-ATOM: Then there exists $\widetilde{C}_1, k$ s.t. $\widetilde{C} = \langle \widetilde{C}_1 \rangle$ and

$$(\widetilde{C}_1, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{k}_t (\mathbf{skip}, ((\sigma'_o, M', \sigma'_a), \Delta')).$$

Further induction on k . The base case is trivial; If there exists k' s.t. $k = k' + 1$, then we know there exists $\widetilde{C}_1'', \sigma''_o, M'', \sigma''_a, \Delta''$ s.t.

$$(\widetilde{C}_1, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{t}_t (\widetilde{C}_1'', ((\sigma''_o, M'', \sigma''_a), \Delta'')) \quad (5)$$

$$(\widetilde{C}_1'', ((\sigma''_o, M'', \sigma''_a), \Delta'')) \xrightarrow{k'}_t (\mathbf{skip}, ((\sigma'_o, M', \sigma'_a), \Delta')) \quad (6)$$

Given arbitrary $\widetilde{C}_x, \widetilde{C}_{sx}$, let $\widetilde{DC}'' = \widetilde{DC}\{t \rightsquigarrow ((\widetilde{C}_x, \widetilde{C}_{sx}), (\sigma_c, \sigma''_o, \kappa))\}$. Apply (5) on the first induction hypothesis we know there exists T_1 s.t. $\llbracket (\Gamma, \widetilde{DC}, \mathcal{N}_{\text{bad}}, \Delta) \rrbracket \xrightarrow{T_1} \llbracket (\Gamma, \widetilde{DC}'', \mathcal{N}_{\text{bad}}, \Delta'') \rrbracket$ and $\text{get-obsv}(T_1) = \epsilon$.

By semantics and (5) we know that $\exists t. \text{finish}(t, \Delta) \wedge \neg\text{finish}(t, \Delta'')$ is impossible. Thus we know

$$\forall t. \widetilde{DC}''(t).\kappa = \circ \implies \text{finish}(t, \Delta'').$$

Apply (6) and above on the second induction hypothesis we know there exists T_2 s.t. $\llbracket (\Gamma, \widetilde{DC}'', \mathcal{N}_{\text{bad}}, \Delta'') \rrbracket \xrightarrow{T_2} \llbracket (\Gamma, \widetilde{DC}', \mathcal{N}_{\text{bad}}, \Delta') \rrbracket$ and $\text{get-obsv}(T_2) = \epsilon$, then by transitivity of $\xRightarrow{\quad}$ and definition of get-obsv we prove the goal.

□

5.3.3 Proof of Lm. 83 (Instrumented Code Removal).

Lemma 83. $\forall \tilde{\Pi}, \Pi, \Gamma, p, \varphi, \eta. \text{Er}(\tilde{\Pi}) = \Pi \wedge \text{refMap}_\eta(\varphi, p) \wedge \tilde{\Pi} \preceq_{p, \eta} \Gamma \implies \Pi \prec_{\varphi, \eta} \Gamma$

PROOF. Let $(\tilde{L}, \tilde{C}_s, \zeta) = \tilde{\Pi}$, $(L, C_s, \zeta) = \Pi$, by premise we know:

$$\begin{aligned} \text{Er}(\tilde{L}, \tilde{C}_s, \zeta) &= (L, C_s, \zeta), \\ (\tilde{L}, \tilde{C}_s, \zeta) &\preceq_{p, \eta} \Gamma. \end{aligned}$$

By definition, to prove the goal is to take arbitrary $n, \mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}, t_1, \dots, t_n, C_{t_1}, \dots, C_{t_n}$ s.t. $\mathcal{N}_{\text{ok}} = \{t_1, \dots, t_n\}$, $\mathcal{N}_{\text{ok}} \cap \mathcal{N}_{\text{bad}} = \emptyset$ and $(|\mathcal{N}_{\text{ok}}|, |\mathcal{N}_{\text{bad}}|) \in \eta$ and prove

$$\text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n} \prec_{\varphi}^{\mathcal{N}_{\text{bad}}} \text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}.$$

By definition, that is to take arbitrary $\sigma_o, \sigma_c, \theta$ s.t. $\varphi(\sigma_o) = \theta$ and prove

$$(\text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o, \theta)) \prec \{(\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta))\}.$$

By INIT rule we know there exists DC s.t. $\text{dom}(DC) = \mathcal{N}_{\text{ok}}$ and

$$\begin{aligned} \forall t \in \mathcal{N}_{\text{ok}}, DC(t) &= ((C_t, C_s), (\sigma_c, \sigma_o \{ \text{tid} \rightsquigarrow t, \text{Nodes} \rightsquigarrow \mathcal{N}_{\text{ok}} \uplus \mathcal{N}_{\text{bad}} \}, \circ)), \\ (\text{let } \Pi \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)) &\# \longrightarrow ((L, \zeta), DC, \emptyset, \mathcal{N}_{\text{bad}}). \end{aligned}$$

By ABS-INIT rule we know there exists ϱ s.t.

$$\begin{aligned} \varrho &= \{t_1 \rightsquigarrow (\sigma_c, \circ), \dots, t_n \rightsquigarrow (\sigma_c, \circ)\}, \\ (\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta)) &\# \longrightarrow (\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\varrho, \theta)). \end{aligned}$$

By definition we only need to prove

$$((L, \zeta), DC, \emptyset, \mathcal{N}_{\text{bad}}) \prec \{(\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\varrho, \theta))\}.$$

Apply $\tilde{\Pi} \preceq_{p, \eta} \Gamma$ we know:

$$\text{let } (\tilde{\Pi}, \Gamma) \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n} \preceq_p^{\mathcal{N}_{\text{bad}}} \text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n} \quad (1)$$

Apply premise $\text{refMap}_\eta(\varphi, p)$ we know there exists σ_a s.t.

$$\forall \mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}. \mathcal{N}_{\text{ok}} \cap \mathcal{N}_{\text{bad}} = \emptyset \wedge (|\mathcal{N}_{\text{ok}}|, |\mathcal{N}_{\text{bad}}|) \in \eta \implies (\sigma_o, \sigma_a, \theta, \mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}) \models_{\text{init}} p.$$

Thus we know $(\sigma_o, \sigma_a, \theta, \mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}) \models_{\text{init}} p$. By (1) we know

$$(\text{let } (\tilde{\Pi}, \Gamma) \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o, \sigma_a, \theta)) \prec \{(\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta))\}.$$

By INS-INIT rule we know there exists $\widetilde{DC}, \sigma'_a$ s.t. $\text{dom}(\widetilde{DC}) = \mathcal{N}_{\text{ok}}$ and

$$\begin{aligned} \forall t \in \mathcal{N}_{\text{ok}}, \widetilde{DC}(t) &= ((C_t, \tilde{C}_s), (\sigma_c, \sigma_o \{ \text{tid} \rightsquigarrow t, \text{Nodes} \rightsquigarrow \mathcal{N}_{\text{ok}} \uplus \mathcal{N}_{\text{bad}} \}, \circ)), \\ \sigma'_a &= \sigma_a \{ \text{Nok} \rightsquigarrow \mathcal{N}_{\text{ok}}, \text{Nbad} \rightsquigarrow \mathcal{N}_{\text{bad}} \}, \\ (\text{let } (\tilde{\Pi}, \Gamma) \text{ in } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o, \sigma_a, \theta)) &\# \longrightarrow ((\tilde{L}, \zeta, \Gamma), \widetilde{DC}, (\emptyset, \sigma'_a, \{(\emptyset, \theta)\}), \mathcal{N}_{\text{bad}}). \end{aligned}$$

Thus we know

$$((\tilde{L}, \zeta, \Gamma), \widetilde{DC}, (\emptyset, \sigma'_a, \{(\emptyset, \theta)\}), \mathcal{N}_{\text{bad}}) \prec \{(\text{with } \Gamma \text{ do } C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\varrho, \theta))\},$$

then apply Lm. 84 the goal is proved. $\square$

Definition 5.22.

$$\text{middleInsWorldConfig}(\tilde{W}) \stackrel{\text{def}}{=} \exists \tilde{L}, \zeta, \Gamma, \widetilde{DC}, M, \sigma_a, \Delta, \mathcal{N}_{\text{bad}}. \tilde{W} = ((\tilde{L}, \zeta, \Gamma), \widetilde{DC}, (M, \sigma_a, \Delta), \mathcal{N}_{\text{bad}})$$

Definition 5.23 (Erasure extension).

$$\begin{aligned} \text{Er}((\tilde{L}, \zeta, \Gamma), \widetilde{DC}, (M, \sigma_a, \Delta), \mathcal{N}_{\text{bad}})) &\stackrel{\text{def}}{=} ((\text{Er}(\tilde{L}), \zeta), \text{Er}(\widetilde{DC}), M, \mathcal{N}_{\text{bad}}) \\ \text{Er}(\{t_1 \rightsquigarrow \widetilde{NC}_{t_1}, \dots, t_n \rightsquigarrow \widetilde{NC}_{t_n}\}) &\stackrel{\text{def}}{=} \{t_1 \rightsquigarrow \text{Er}(\widetilde{NC}_{t_1}), \dots, t_n \rightsquigarrow \text{Er}(\widetilde{NC}_{t_n})\} \\ \text{Er}(\widetilde{NC}) &\stackrel{\text{def}}{=} \begin{cases} ((C, \text{Er}(\widetilde{C}_s)), (\sigma_c, \sigma_o, o)) & \text{if } \widetilde{NC} = ((C, \widetilde{C}_s), (\sigma_c, \sigma_o, o)) \\ ((\text{Er}(\widetilde{C}), \text{Er}(\widetilde{C}_s)), (\sigma_c, \sigma_o, (x, C))) & \text{if } \widetilde{NC} = ((\widetilde{C}, \widetilde{C}_s), (\sigma_c, \sigma_o, (x, C))) \end{cases} \end{aligned}$$

Definition 5.24. $\widetilde{W} \xrightarrow{T}^k \widetilde{W}$ is defined as follows:

$$\frac{}{\widetilde{W} \xrightarrow{\epsilon}^0 \widetilde{W}} \quad \frac{\widetilde{W} \xrightarrow{e} \widetilde{W}'' \quad \widetilde{W}'' \xrightarrow{T}^k \widetilde{W}'}{\widetilde{W} \xrightarrow{e::T}^{k+1} \widetilde{W}'}$$

Definition 5.25. $\widetilde{W} \xrightarrow{T}^* \widetilde{W}' \stackrel{\text{def}}{=} \exists k. \widetilde{W} \xrightarrow{T}^k \widetilde{W}'$

Note that we use $\widetilde{W} \xrightarrow{T}^* \mathbf{abort}$ as a sugar for $\nexists k, T. \widetilde{W} \xrightarrow{T}^k \mathbf{abort}$, while $(\tilde{L}, \Gamma) \vdash (\widetilde{NC}, (M, \sigma_a, \Delta)) \xrightarrow{T}^* \mathbf{abort}$ a sugar for $\nexists k. (\tilde{L}, \Gamma) \vdash (\widetilde{NC}, (M, \sigma_a, \Delta)) \xrightarrow{T}^k \mathbf{abort}$.

Lemma 84. $\forall \widetilde{W}, W, \Omega. \text{middleWorldConfig}(\widetilde{W}) \wedge \text{Er}(\widetilde{W}) = W \wedge \widetilde{W} \preceq \Omega \implies W \prec \Omega$

PROOF. By co-induction, for all W', e , if $W \xrightarrow{e} W'$, then we need to prove find Ω', T and prove

$$\begin{aligned} \Omega &\xrightarrow{T} \Omega' & (\text{goal1}) \\ \text{get-obsv}(e) &= \text{get-obsv}(T) & (\text{goal2}) \\ W' &\prec \Omega' & (\text{goal3}) \end{aligned}$$

Apply Lm. 85 we know there exists $\widetilde{W}'$ s.t. $\text{Er}(\widetilde{W}') = W'$ and $\widetilde{W} \xrightarrow{e::\epsilon}^* \widetilde{W}'$. By premise $\widetilde{W} \preceq \Omega$ we know there exists Ω', T s.t. $\Omega \xrightarrow{T} \Omega'$, $\text{get-obsv}(e) = \text{get-obsv}(T)$ and $\widetilde{W}' \preceq \Omega'$. Thus we prove (goal1) and (goal2), then by hypothesis we prove (goal3). $\square$

Lemma 85. For all $\widetilde{W}, W, W', e$, if

1. $\text{middleInsWorldConfig}(\widetilde{W})$,
2. $\text{Er}(\widetilde{W}) = W$,
3. $W \xrightarrow{e} W'$,
4. $\widetilde{W} \xrightarrow{T}^* \mathbf{abort}$,

then there exists $\widetilde{W}'$ s.t.

1. $\text{Er}(\widetilde{W}') = W'$,
2. $\widetilde{W} \xrightarrow{e::\epsilon}^* \widetilde{W}'$.

PROOF. By (p1) we know there exists $\tilde{L}, \zeta, \Gamma, \widetilde{DC}, M, \sigma_a, \Delta, \mathcal{N}_{\text{bad}}$ s.t. $\widetilde{W} = ((\tilde{L}, \zeta, \Gamma), \widetilde{DC}, (M, \sigma_a, \Delta), \mathcal{N}_{\text{bad}})$, then by (p2) we know there exists L, DC s.t. $\text{Er}(\tilde{L}) = L$, $\text{Er}(\widetilde{DC}) = DC$ and $W = ((L, \zeta), DC, M, \mathcal{N}_{\text{bad}})$. We prove the goal by case analysis on (p3), we have the following cases:

- (p3) is a CORRECT-STEP step, i.e. there exists t, NC, NC', M' s.t

$$\begin{aligned} DC(t) &= NC, \\ L \vdash (NC, M) &\xrightarrow{e}_t (NC', M'), \\ W' &= ((L, \zeta), DC\{t \rightsquigarrow NC'\}, M', \mathcal{N}_{\text{bad}}). \end{aligned}$$

By (4) and INS-BENIGN-ABORT rule we know $(\tilde{L}, \Gamma) \vdash (\widetilde{NC}, (M, \sigma_a, \Delta)) \xrightarrow{T}^* \mathbf{abort}$, then apply Lm. 86 we know there exists $\widetilde{DC}', \sigma'_a, \Delta'$ s.t.

$$\begin{aligned} \text{Er}(\widetilde{NC}') &= NC', \\ (\widetilde{L}, \Gamma) \vdash (\widetilde{NC}, (M, \sigma_a, \Delta)) &\xrightarrow{e::\epsilon}_t^* (\widetilde{NC}', (M', \sigma'_a, \Delta')). \end{aligned}$$

Let $\widetilde{W}' = ((\widetilde{L}, \zeta, \Gamma), \widetilde{DC}\{t \rightsquigarrow \widetilde{NC}'\}, (M', \sigma'_a, \Delta'), \mathcal{N}_{\text{bad}})$, by definition we prove $\text{Er}(\widetilde{W}') = W'$, by INS-BENIGN-STEP rule we prove $\widetilde{W} \xrightarrow{e::\epsilon}_t^* \widetilde{W}'$, thus we conclude this case.

- (p3) is a FAULTY-STEP step, i.e. there exists M', v s.t. $\mathcal{N}_{\text{bad}} \neq \emptyset$, $v \notin \zeta(\mathcal{N}_{\text{bad}}, M)$, $M' = M \cup \{(\mathcal{N}, v)\}$, $W' = ((L, \zeta), DC, M', \mathcal{N}_{\text{bad}})$ and $e = \tau$. let $\widetilde{W}' = ((\widetilde{L}, \zeta, \Gamma), \widetilde{DC}, (M', \sigma_a, \Delta), \mathcal{N}_{\text{bad}})$, by definition we know $\text{Er}(\widetilde{W}') = W'$, by INS-FAULTY-STEP we know $\widetilde{W} \xrightarrow{e::\epsilon}_t^1 \widetilde{W}'$, thus we conclude this case.

□

Lemma 86. For all $L, NC, NC', M, M', e, t, \Gamma, \widetilde{NC}, \sigma_a, \Delta$, if

1. $\text{Er}(\widetilde{NC}) = NC$ and $\text{Er}(\widetilde{L}) = L$,
2. $L \vdash (NC, M) \xrightarrow{e}_t (NC', M')$,
3. $(\widetilde{L}, \Gamma) \vdash (\widetilde{NC}, (M, \sigma_a, \Delta)) \xrightarrow{e::\epsilon}_t^* \text{abort}$,

then there exists $\widetilde{NC}', \sigma'_a, \Delta'$ s.t.

1. $\text{Er}(\widetilde{NC}') = NC'$,
2. $(\widetilde{L}, \Gamma) \vdash (\widetilde{NC}, (M, \sigma_a, \Delta)) \xrightarrow{e::\epsilon}_t^* (\widetilde{NC}', (M', \sigma'_a, \Delta'))$.

PROOF. Introduce $C, C_s, \sigma_c, \sigma_o, \kappa$ and let $((C, C_s), (\sigma_c, \sigma_o, \kappa)) = NC$. We prove the goal by case analysis on (p2), we have the following cases:

- (p2) is a PRINT step, i.e. there exists $\mathbf{E}, E, v$ s.t. $C = \mathbf{E}[\text{print}(E)]$, $\kappa = \circ$, $\llbracket E \rrbracket_{\sigma_c} = v$, $e = (t, \text{out}, v)$,

$$\begin{aligned} NC' &= ((\mathbf{E}[\text{skip}], C_s), (\sigma_c, \sigma_o, \circ)), \\ M' &= M. \end{aligned}$$

By (p1) we know there exists $\widetilde{C}_s$ s.t. $\widetilde{NC} = ((\mathbf{E}[\text{print}(E)], \widetilde{C}_s), (\sigma_c, \sigma_o, \circ))$ and $\text{Er}(\widetilde{C}_s) = C_s$. Let $\widetilde{NC}' = ((\mathbf{E}[\text{skip}], \widetilde{C}_s), (\sigma_c, \sigma_o, \circ))$, $\sigma'_a = \sigma_a$ and $\Delta' = \Delta$. By definition we know $\text{Er}(\widetilde{NC}') = NC'$. From INS-PRINT we prove $(\widetilde{L}, \Gamma) \vdash (\widetilde{NC}, (M, \sigma_a, \Delta)) \xrightarrow{e::\epsilon}_t^1 (\widetilde{NC}', (M, \sigma_a, \Delta))$, thus we conclude this case.

- (p2) is a CSKIP step, i.e. there exists C' s.t. $C = \text{skip}; C'$, $e = \tau$, $\kappa = \circ$,

$$\begin{aligned} NC' &= (((C', C_s), (\sigma_c, \sigma_o, \circ))), \\ M' &= M. \end{aligned}$$

By (p1) we know there exists $\widetilde{C}_s$ s.t. $\widetilde{NC} = ((\text{skip}; C', \widetilde{C}_s), (\sigma_c, \sigma_o, \circ))$ and $\text{Er}(\widetilde{C}_s) = C_s$. let $\widetilde{NC}' = ((C', \widetilde{C}_s), (\sigma_c, \sigma_o, \circ))$, $\sigma'_a = \sigma_a$ and $\Delta' = \Delta$. By definition we know $\text{Er}(\widetilde{NC}') = NC'$. By INS-CLT-SKIP we know $(\widetilde{L}, \Gamma) \vdash (((\text{skip}; C', \widetilde{C}_s), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta)) \xrightarrow{e::\epsilon}_t^1 (((C', \widetilde{C}_s), (\sigma_c, \sigma_o, \circ)), (M, \sigma_a, \Delta))$, thus we conclude this case.

- (p2) is a CALL step, i.e. there exists $\mathbf{E}, x, E, f, C_m, y, v$ s.t. $C = \mathbf{E}[x := f(E)]$, $L(f) = (y, C_m)$, $\llbracket E \rrbracket_{\sigma_c} = v$, $\kappa = \circ$, $e = (t, f, v)$,

$$\begin{aligned} NC' &= ((C_m, C_s), (\sigma_c, \sigma_o\{y \rightsquigarrow v\}, (x, \mathbf{E}[\text{skip}]))), \\ M' &= M. \end{aligned}$$

By premise $\text{Er}(\widetilde{NC}) = NC$ we know there exists $\widetilde{C}_s$ s.t. $\widetilde{NC} = ((\mathbf{E}[x := f(E)], \widetilde{C}_s), (\sigma_c, \sigma_o, \circ))$ and $\text{Er}(\widetilde{C}_s) = C_s$. By premise $\text{Er}(\widetilde{L}) = L$ we know there exists $\widetilde{C}_m$ s.t. $\widetilde{L}(f) = (y, \widetilde{C}_m)$ and $\text{Er}(\widetilde{C}_m) = C_m$. By (p3) we know $f \in \text{dom}(\Gamma)$. Let $\widetilde{NC}' = ((\widetilde{C}_m, \widetilde{C}_s), (\sigma_c, \sigma_o\{y \rightsquigarrow v\}, (x, \mathbf{E}[\text{skip}])))$, $\sigma'_a = \sigma_a$, $\Delta' = \Delta\{t \rightsquigarrow (\Gamma(f), v)\}$. by definition we know $\text{Er}(\widetilde{NC}') = NC'$.

By INS-CALL rule we prove $(\tilde{L}, \Gamma) \vdash (\widetilde{NC}, (M, \sigma_a, \Delta)) \xrightarrow{e::\epsilon}_t^1 (\widetilde{NC}', (M', \sigma'_a, \Delta'))$, thus we conclude this case.

- (p2) is a RETURN step, there exists v', E s.t. $C = \mathbf{E}[\mathbf{return} E]$, $\kappa = (x, C)$, $\llbracket E \rrbracket_{\sigma_o} = v'$, $e = (t, \mathbf{ok}, v')$,

$$\begin{aligned} NC' &= ((C, C_s), (\sigma_c\{x \rightsquigarrow v'\}, \sigma_o, \circ)), \\ M' &= M. \end{aligned}$$

By premise $\text{Er}(\widetilde{NC}) = NC$ we know there exists $\tilde{C}_s, \tilde{C}_m$ s.t. $\text{Er}(\tilde{C}_s) = C_s$ and $\text{Er}(\tilde{C}_m) = \mathbf{E}[\mathbf{return} E]$. By latter we know there exists $\mathbf{E}_i$ s.t. $\tilde{C}_m = \mathbf{E}_i[\mathbf{return} E]$. From (p3) we know there exists Δ_o s.t. $\Delta = \Delta_o * \{(\{t \rightsquigarrow (\text{end}, v')\}, \emptyset)\}$. Let $\widetilde{NC}' = ((C, \tilde{C}_s), (\sigma_c\{x \rightsquigarrow v'\}, \sigma_o, \circ))$, $\sigma'_a = \sigma_a$, $\Delta' = \Delta_o$. By definition we know $\text{Er}(\widetilde{NC}') = NC'$, then by INS-ND-RETURN rule we prove $(\tilde{L}, \Gamma) \vdash (\widetilde{NC}, (M, \sigma_a, \Delta)) \xrightarrow{e::\epsilon}_t^1 (\widetilde{NC}', (M', \sigma_a, \Delta_o))$, thus we conclude this case.

- (p2) is a OBJECTMETHOD step, i.e. there exists C'_m, σ'_o, M' s.t. $e = \tau$, $\tilde{C}_m \neq \mathbf{return} _$, $\kappa \neq \circ$,

$$\begin{aligned} (C_m, (\sigma_o, M)) &\longrightarrow_t (C'_m, (\sigma'_o, M')), \\ NC' &= ((C'_m, C_s), (\sigma_c, \sigma'_o, \kappa)). \end{aligned}$$

By (p1) we know there exists $\tilde{C}_m, \tilde{C}_s$ s.t. $\widetilde{NC} = ((\tilde{C}_m, \tilde{C}_s), (\sigma_c, \sigma_o, \kappa))$, $\text{Er}(\tilde{C}_m) = C_m$ and $\text{Er}(\tilde{C}_s) = C_s$. By (p3) we know $(\tilde{C}_m, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{\epsilon}^* \mathbf{abort}$, thus apply Lm. 87 we know there exists $\tilde{C}'_m, \sigma'_a, \Delta'$ s.t.

$$\begin{aligned} \text{Er}(\tilde{C}'_m) &= C'_m & (1) \\ (\tilde{C}_m, ((\sigma_o, M, \sigma_a), \Delta)) &\xrightarrow{\epsilon}^* (\tilde{C}'_m, (\sigma'_o, M', \sigma'_a), \Delta') & (2) \end{aligned}$$

Let $\widetilde{NC}' = ((\tilde{C}'_m, \tilde{C}_s), (\sigma_c, \sigma'_o, \kappa))$. By (1) and definition we know $\text{Er}(\widetilde{NC}') = NC'$, apply (2) on INS-OBJECT-METHOD we prove $(\tilde{L}, \Gamma) \vdash (\widetilde{NC}, (M, \sigma_a, \Delta)) \xrightarrow{\epsilon}_t (\widetilde{NC}', (M', \sigma'_a, \Delta'))$, thus we conclude this case.

- (p2) is a SERVER step: Similar to the proof of last case.

□

Lemma 87. For all $\tilde{C}, C, \sigma_o, \sigma'_o, M, M', \sigma_a, \Delta, t$, if

1. $\nexists E. C = \mathbf{return} E$,
2. $\text{Er}(\tilde{C}) = C$,
3. $(C, (\sigma_o, M)) \longrightarrow_t (C', (\sigma'_o, M'))$,
4. $(\tilde{C}, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{\epsilon}^* \mathbf{abort}$,

then there exists $\tilde{C}', \sigma'_a, \Delta'$ s.t.

1. $\text{Er}(\tilde{C}') = C'$,
2. $(\tilde{C}, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{\epsilon}^* (\tilde{C}', (\sigma'_o, M', \sigma'_a), \Delta')$.

PROOF. We prove by induction on (p2):

- $\tilde{C} = c$ and $C = c$: By (p2) we know $c \neq \mathbf{skip}$ and $C' = \mathbf{skip}$, by (p1) we know there exists x, E, E' s.t. $c = x := E$, or $c = \mathbf{send}(E, E')$, or $c = x := \mathbf{recv}()$. Let $\tilde{C}' = \mathbf{skip}$, $\sigma'_a = \sigma_a$ and $\Delta' = \Delta$. By definition we prove $\text{Er}(\tilde{C}') = C'$. By (p4) we know $\sigma'_o \perp \sigma_a$, apply $\sigma'_o \perp \sigma_a$ and (p3) on INS-CONCRETE we know $(c, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{\epsilon}_t (\mathbf{skip}, (\sigma'_o, M', \sigma_a), \Delta)$. We have $(c, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{\epsilon}_t^1 (\mathbf{skip}, (\sigma'_o, M', \sigma_a), \Delta)$, thus we conclude this case.
- $\tilde{C} = \dot{C}$ and $C = \mathbf{skip}$: Contradict (p3).
- $\tilde{C} = \langle \tilde{C}' \rangle$ and $C = \text{ErAtom}(\tilde{C}')$: We further discuss on $C = \text{ErAtom}(\tilde{C}')$:

- $\tilde{C}' = c$ and $C = c$: Then $\tilde{C} = \langle c \rangle$ and $C' = \mathbf{skip}$. Let $\tilde{C}' = \mathbf{skip}$, $\sigma'_a = \sigma_a$ and $\Delta' = \Delta$. By definition we prove $\text{Er}(\tilde{C}') = C'$. By (p3) we know $(\langle c \rangle, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{*}_t (\mathbf{skip}, (\sigma'_o, M', \sigma_a), \Delta)$. Thus we conclude this case.
- $\tilde{C}' = \dot{C}$ and $C = \mathbf{skip}$: Contradict (p3).
- $\tilde{C}' = \dot{C}; c$ and $C = c$: Then $\tilde{C} = \langle \dot{C}; c \rangle$ and $C' = \mathbf{skip}$. By (p4) we know $(\dot{C}, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{*}_t \mathbf{abort}$, thus apply Lm. 88 we know there exists σ'_a, Δ' s.t.

$$(\dot{C}, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{*}_t (\mathbf{skip}, (\sigma_o, M, \sigma'_a), \Delta') \quad (1)$$

Let $\tilde{C}' = \mathbf{skip}$, by definition we prove $\text{Er}(\tilde{C}') = C'$, by (1) and (p2) we prove $(\langle \dot{C}; c \rangle, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{*}_t (\mathbf{skip}, (\sigma'_o, M', \sigma'_a), \Delta')$. Thus we conclude this case.

- $\tilde{C}' = c; \dot{C}$ and $C = c$: Similar to the last case.
- $\tilde{C}' = \dot{C}_1; c; \dot{C}_2$ and $C = c$: Similar to the last case.
- $\tilde{C} = \tilde{C}_1; \tilde{C}_2$, $C = \text{Er}(\tilde{C}_1); \text{Er}(\tilde{C}_2)$: Let $\text{Er}(\tilde{C}_1) = C_1$, $\text{Er}(\tilde{C}_2) = C_2$, discuss on $\text{Er}(\tilde{C}_1)$:
 - $\text{Er}(\tilde{C}_1) = \mathbf{skip}$: Then by definition there exists $\dot{C}$ s.t. $\tilde{C}_1 = \mathbf{skip}$, or $\tilde{C}_1 = \dot{C}$, or $\tilde{C}_1 = \langle \dot{C} \rangle$. And by (p3) we know

$$(\mathbf{skip}; C_2, (\sigma_o, M)) \longrightarrow_t (C_2, (\sigma_o, M)).$$

Let $\tilde{C}' = \tilde{C}_2$, by definition we have $\text{Er}(\tilde{C}') = C'$.

If $\tilde{C}_1 = \mathbf{skip}$, then let $\sigma'_a = \sigma_a$ and $\Delta' = \Delta$, by semantics we know $(\mathbf{skip}; \tilde{C}_2, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{*}_t (\tilde{C}_2, (\sigma_o, M, \sigma_a), \Delta)$.

If $\tilde{C}_1 = \dot{C}$, apply (p4) on Lm. 88 we know there exists σ'_a, Δ' s.t.

$$(\dot{C}, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{*}_t (\mathbf{skip}, (\sigma_o, M, \sigma'_a), \Delta'),$$

thus we prove $(\dot{C}; \tilde{C}_2, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{*}_t (\tilde{C}_2, (\sigma_o, M, \sigma'_a), \Delta')$.

If $\tilde{C}_1 = \langle \dot{C} \rangle$, similar to the last case, we can prove $(\langle \dot{C} \rangle; \tilde{C}_2, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{*}_t (\tilde{C}_2, (\sigma_o, M, \sigma'_a), \Delta')$.

Thus we conclude this case.

- $\text{Er}(\tilde{C}_1) \neq \mathbf{skip}$: Then there exists C'_1 s.t.

$$(C_1, (\sigma_o, M)) \longrightarrow_t (C'_1, (\sigma'_o, M')).$$

Apply induction hypothesis we know there exists $\tilde{C}'_1, \sigma'_a, \Delta'$ s.t.

$$\text{Er}(\tilde{C}'_1) = C'_1 \quad (2)$$

$$(\tilde{C}_1, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{*}_t (\tilde{C}'_1, (\sigma'_o, M', \sigma'_a), \Delta') \quad (3)$$

Let $\tilde{C}' = \tilde{C}'_1; \tilde{C}_2$. By definition and (2) we prove $\text{Er}(\tilde{C}') = C'$, by (3) we prove

$$(\tilde{C}_1; \tilde{C}_2, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{*}_t (\tilde{C}'_1; \tilde{C}_2, (\sigma'_o, M', \sigma'_a), \Delta').$$

Thus we conclude this case.

- $\tilde{C} = \mathbf{if} (B) \mathbf{then} \tilde{C}_1 \mathbf{else} \tilde{C}_2$ and $C = \mathbf{if} (B) \mathbf{then} \text{Er}(\tilde{C}_1) \mathbf{else} \text{Er}(\tilde{C}_2)$: Let $\text{Er}(\tilde{C}_1) = C_1$, $\text{Er}(\tilde{C}_2) = C_2$, by (p2) we have the following cases:
 - If $\llbracket B \rrbracket_{\sigma_o \uplus \sigma_a} = \mathbf{true}$: Then we know

$$(\mathbf{if} (B) \mathbf{then} C_1 \mathbf{else} C_2, (\sigma_o, M)) \longrightarrow_t (C_1, (\sigma_o, M)).$$

Let $\tilde{C}' = \tilde{C}_1$, $\sigma'_a = \sigma_a$, $\Delta' = \Delta$, by definition we prove $\text{Er}(\tilde{C}') = C_1$, by semantics we prove $(\mathbf{if} (B) \mathbf{then} \tilde{C}_1 \mathbf{else} \tilde{C}_2, ((\sigma_o, M, \sigma_a), \Delta)) \xrightarrow{*}_t (\tilde{C}_1, (\sigma_o, M, \sigma_a), \Delta)$, thus we conclude this case.

- If $\llbracket B \rrbracket_{\sigma_o \uplus \sigma_a} = \mathbf{false}$: Similar to the last case.
- $\tilde{C} = \mathbf{while} (B) \mathbf{do} \tilde{C}_1$ and $C = \mathbf{while} (B) \mathbf{do} \text{Er}(\tilde{C}_1)$: Let $\text{Er}(\tilde{C}_1) = C_1$, by (p2) we have following case:

- If $\llbracket B \rrbracket_{\sigma_o \uplus \sigma_a} = \mathbf{true}$: Then we know

$$(\mathbf{while} (B) \mathbf{do} C_1, (\sigma_o, M)) \longrightarrow_t (C_1; \mathbf{while} (B) \mathbf{do} C_1, (\sigma_o, M)).$$

Let $\tilde{C}' = \tilde{C}_1$; $\mathbf{while} (B) \mathbf{do} \tilde{C}_1, \sigma'_a = \sigma_a, \Delta' = \Delta$, by definition we prove $\text{Er}(\tilde{C}') = C'$, by semantics we prove

$$(\mathbf{while} (B) \mathbf{do} \tilde{C}_1, ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t^* (\tilde{C}_1; \mathbf{while} (B) \mathbf{do} \tilde{C}_1, (\sigma_o, M, \sigma_a), \Delta).$$

Thus we conclude this case.

- If $\llbracket B \rrbracket_{\sigma_o \uplus \sigma_a} = \mathbf{false}$: Similar to the last case.

□

Lemma 88. For all $\dot{C}, \sigma_a, M, \sigma_a, \Delta, t$, if $(\dot{C}, ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t^* \mathbf{abort}$, then there exists σ'_a, Δ' s.t. $(\dot{C}, ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t^* (\mathbf{skip}, ((\sigma_o, M, \sigma'_a), \Delta'))$.

PROOF. This lemma can be proved by induction on $\dot{C}$ and semantics of dot commands (Fig. 22, Fig. 23). Most cases are trivial, we prove **foreach** x **in** E **do** $\dot{C}_1$ case as follows.

Let $V = \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a}$. Further induction on $|V|$. If $|V| = 0$, then we know $V = \emptyset$. By semantics we know

$$(\mathbf{foreach} x \mathbf{in} E \mathbf{do} \dot{C}_1, ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t (\mathbf{skip}, ((\sigma_o, M, \sigma_a), \Delta)).$$

Thus we conclude $|V| = 0$ case.

If there exists n' s.t. $|V| = n' + 1$, then by premise and semantics we know there exists v s.t. $v \in V$ and

$$\text{noModifyVars}(\text{FV}(E), \dot{C}_1) \tag{1}$$

$$(\mathbf{foreach} x \mathbf{in} E \mathbf{do} \dot{C}_1, ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t (\dot{C}_1; \mathbf{foreach} x \mathbf{in} E \setminus \{v\} \mathbf{do} \dot{C}_1, ((\sigma_o, M, \sigma_a\{x \rightsquigarrow v\}), \Delta)) \tag{2}$$

By premise $(\mathbf{foreach} x \mathbf{in} E \mathbf{do} \tilde{C}_1, ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t^* \mathbf{abort}$ we know $(\tilde{C}_1, ((\sigma_o, M, \sigma_a\{x \rightsquigarrow v\}), \Delta)) \longmapsto_t^* \mathbf{abort}$. Apply the first induction hypothesis we know there exists Σ'' s.t.

$$(\dot{C}_1, ((\sigma_o, M, \sigma_a\{x \rightsquigarrow v\}), \Delta)) \longmapsto_t^* (\mathbf{skip}, \Sigma'') \tag{3}$$

By (2) and (3) we know

$$(\mathbf{foreach} x \mathbf{in} E \mathbf{do} \dot{C}_1, ((\sigma_o, M, \sigma_a), \Delta)) \longmapsto_t^* (\mathbf{foreach} x \mathbf{in} E \setminus \{v\} \mathbf{do} \dot{C}_1, \Sigma'').$$

Thus we only need to prove

$$\exists \Sigma'. (\mathbf{foreach} x \mathbf{in} E \setminus \{v\} \mathbf{do} \dot{C}_1, \Sigma'') \longmapsto_t^* (\mathbf{skip}, \Sigma').$$

By the second induction hypothesis we only need to prove $|\llbracket E \setminus \{v\} \rrbracket_{\Sigma''}| \leq n'$. By (1) and Lm. 62 we know $\llbracket E \rrbracket_{\sigma_o \uplus \sigma_a} = \llbracket E \rrbracket_{\Sigma''}$, along with $v \in \llbracket E \rrbracket_{\sigma_o \uplus \sigma_a}$ and $|\llbracket E \rrbracket_{\sigma_o \uplus \sigma_a}| = n' + 1$ we prove $|\llbracket E \setminus \{v\} \rrbracket_{\Sigma''}| = n'$. Thus we conclude this case. □

5.3.4 Proof of Lm. 89 (World Simulation implies Contextual Refinement).

Lemma 89. $\forall \varphi, \Pi, \Gamma, \eta. \Pi \prec_{\varphi, \eta} \Gamma \implies \Pi \sqsubseteq_{\varphi, \eta} \Gamma$

PROOF. By definition to prove the goal is to take arbitrary $t_1, \dots, t_n, \mathcal{N}_{\text{ok}}, \mathcal{N}_{\text{bad}}, C_{t_1}, \dots, C_{t_n}, \sigma_c, \sigma_o, \theta$ and prove that if $\mathcal{N}_{\text{ok}} = \{t_1, \dots, t_n\}$, $(n, |\mathcal{N}_{\text{bad}}|) \in \eta$ and $\mathcal{N}_{\text{ok}} \cap \mathcal{N}_{\text{bad}} = \emptyset$, then

$$\begin{aligned} \forall T \in \mathcal{T} \llbracket \mathbf{let} \Pi \mathbf{in} C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o) \rrbracket. \varphi(\sigma_o) = \theta \\ \implies \exists T' \in \mathcal{T} \llbracket \mathbf{with} \Gamma \mathbf{do} C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta) \rrbracket. \text{get-obsv}(T) = \text{get-obsv}(T'). \end{aligned}$$

Apply $\Pi \prec_{\varphi, \eta} \Gamma$ we know

$$(\mathbf{let} \Pi \mathbf{in} C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)) \prec \{(\mathbf{with} \Gamma \mathbf{do} C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta))\}.$$

By concrete semantics we know there exists $\widetilde{DC}$ s.t. $(\mathbf{let} (L, C_s, \zeta) = \Pi)$:

$$\begin{aligned} \text{dom}(DC) &= \mathcal{N}_{\text{ok}}, \\ \forall t \in \mathcal{N}_{\text{ok}}, DC(t) &= ((C_t, C_s), (\sigma_c, \sigma_o \{ \text{tid} \rightsquigarrow t, \text{Nodes} \rightsquigarrow \mathcal{N}_{\text{ok}} \uplus \mathcal{N}_{\text{bad}} \}, \circ)), \\ (\mathbf{let} \Pi \mathbf{in} C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \sigma_o)) &\mapsto ((L, \zeta), DC, \emptyset, \mathcal{N}_{\text{bad}}) \end{aligned}$$

and there exists $\mathbb{S} = (\{t_1 \rightsquigarrow \sigma_c, \dots, t_n \rightsquigarrow \sigma_c\}, \theta)$ s.t.

$$(\mathbf{with} \Gamma \mathbf{do} C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, (\sigma_c, \theta)) \mapsto (\mathbf{with} \Gamma \mathbf{do} C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, \mathbb{S}) \quad (1)$$

Thus we know

$$((L, \zeta), DC, \emptyset, \mathcal{N}_{\text{bad}}) \prec \{(\mathbf{with} \Gamma \mathbf{do} C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, \mathbb{S})\} \quad (2)$$

Apply Lm. 90 we prove that $\forall T \in \mathcal{T}[(L, \zeta), DC, \emptyset, \mathcal{N}_{\text{bad}}]$. $\exists T' \in \mathcal{T}[(\mathbf{with} \Gamma \mathbf{do} C_{t_1} \parallel \dots \parallel C_{t_n}, \mathcal{N}_{\text{bad}}, \mathbb{S})]$. $\text{get-obsv}(T) = \text{get-obsv}(T')$, along with (1) and (2) we prove the goal. $\square$

Definition 5.26 (Trace set).

$$\begin{aligned} \mathcal{T}[(L, \zeta), DC, M, \mathcal{N}_{\text{bad}}] &\stackrel{\text{def}}{=} \{T \mid \exists W'. ((L, \zeta), DC, M, \mathcal{N}_{\text{bad}}) \xrightarrow{T^*} W'\} \\ \mathcal{T}[(\mathbb{P}, \mathcal{N}_{\text{bad}}, \mathbb{S})] &\stackrel{\text{def}}{=} \{T \mid \exists \mathbb{W}'. (\mathbb{P}, \mathcal{N}_{\text{bad}}, \mathbb{S}) \xrightarrow{T^*} \mathbb{W}'\} \end{aligned}$$

Lemma 90. $\forall W, \Omega. W \prec \Omega \wedge (\exists L, \zeta, DC, M, \mathcal{N}_{\text{bad}}. W = ((L, \zeta), DC, M, \mathcal{N}_{\text{bad}})) \implies \exists \mathbb{W} \in \Omega. \forall T \in \mathcal{T}[W]. \exists T' \in \mathcal{T}[\mathbb{W}]. \text{get-obsv}(T) = \text{get-obsv}(T')$

PROOF. Given arbitrary T in $\mathcal{T}[W]$, we know there exists W' s.t. $W \xrightarrow{T^*} W'$. By Lm. 91 we know there exists T', Ω' s.t. $\Omega \xrightarrow{T'} \Omega'$ and $\text{get-obsv}(T) = \text{get-obsv}(T')$, by definition of $\xRightarrow{*}$ we know

$$\forall \mathbb{W}' \in \Omega'. \exists \mathbb{W} \in \Omega. \mathbb{W} \xrightarrow{T^*} \mathbb{W}'.$$

Thus we prove the goal. $\square$

Lemma 91. $\forall W, W', k, T, \Omega. W \xrightarrow{T^*}^k W' \wedge W \prec \Omega \implies \exists T_1, \Omega'. \Omega \xrightarrow{T_1} \Omega' \wedge \text{get-obsv}(T) = \text{get-obsv}(T_1)$

PROOF. Induction on k . If $k = 0$, let $\Omega' = \Omega$ we prove the goal; if there exists k' s.t. $k = k' + 1$, we know there exists W'', T', e s.t.

$$\begin{aligned} W &\xrightarrow{e} W'', \\ W'' &\xrightarrow{T'}^{k'} W', \\ T &= e :: T'. \end{aligned}$$

Apply $W \prec \Omega$ we know there exists Ω'', T_1' s.t.

$$\begin{aligned} W'' &\prec \Omega'', \\ \Omega &\xrightarrow{T_1'} \Omega'', \\ \text{get-obsv}(e) &= \text{get-obsv}(T_1'). \end{aligned}$$

Apply induction hypothesis we know there exists T_1'', Ω' s.t. $\Omega'' \xRightarrow{T_1''} \Omega'$ and $\text{get-obsv}(T') = \text{get-obsv}(T_1'')$, thus we know $\Omega \xRightarrow{T_1'::T_1''} \Omega'$ and $\text{get-obsv}(e :: T') = \text{get-obsv}(T_1' :: T_1'')$. $\square$

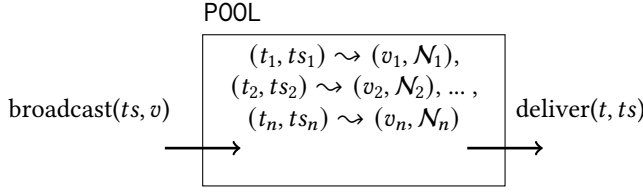


Fig. 32. Broadcast object illustration

6 Example 1: Broadcast Protocol

6.1 Broadcast Abstraction

We present a simple diagram of broadcast object illustration in Fig. 32.

$$\theta \in \{\text{POOL}\} \rightarrow (\text{NdID} \times \text{Nat} \times \text{Val} \rightarrow \mathcal{P}(\text{NdID}))$$

Definition 6.1 (Broadcast object specification).

$$\Gamma_B \stackrel{\text{def}}{=} \{\text{broadcast} \leadsto \text{BROADCAST}, \text{deliver} \leadsto \text{DELIVER}\}$$

$$\begin{aligned} \text{BROADCAST} &\stackrel{\text{def}}{=} \lambda t. \lambda((ts, v), \theta). \\ &\begin{cases} (\text{bad}, \theta) & \text{if } (t, ts) \in \text{dom}(\theta(\text{POOL})) \\ (\text{ok}, \theta\{\text{POOL} \leadsto \theta(\text{POOL})\{(t, ts) \leadsto (v, \emptyset)\}\}) & \text{otherwise} \end{cases} \end{aligned}$$

$$\begin{aligned} \text{DELIVER} &\stackrel{\text{def}}{=} \lambda t. \lambda((t', ts), \theta). \\ &\begin{cases} (v, \theta\{\text{POOL} \leadsto \theta(\text{POOL})\{(t', ts) \leadsto (v, \mathcal{N} \uplus \{t\})\}\}) & \text{if } \theta(\text{POOL})(t', ts) = \mathcal{N} \wedge t \notin \mathcal{N} \\ \text{undefined} & \text{otherwise} \end{cases} \end{aligned}$$

6.2 Specification

To verify Byzantine linearizability, $\Pi_B \preceq_{\varphi_B, \eta_B} \Gamma_B$, by Thm. 1 and Thm. 2, we need to find an object invariant p which satisfies $\text{refMap}_{\eta_B}(\varphi_B, p)$, and prove $\vdash \{p, \eta_B\}(\Pi_B, \Gamma_B)$ using our logic rules.

We show the definition of p in Fig. 35. At the core of the definition is the assertion inv , which specifies the invariant relationship between the concrete and abstract objects (defined as goodAbsPOOL) and the invariant property of the concrete message pool (defined as goodMsgPool). In addition, p_t also ensures $\text{tid} = t \wedge \text{readOnlyVarProp}(\eta_B)$, as required by the OBJ rule in Fig. 28.

The assertion goodAbsPOOL is a conjunction of the following properties. First, broadcastRelated characterizes the LP of the successful method call of broadcast . It says that whenever the current node puts a time-stamp ts into SentTs , it will simultaneously execute the corresponding abstract broadcast operation, adding the message of (tid, ts) into the abstract POOL. In addition, initHasLinearized says that if any correct node t has sent an INIT message, then the corresponding abstract broadcast operation of the node t must have been finished. Note that initHasLinearized concerns not just the current node, but any node.

Similarly, deliverRelated characterizes the LP of the deliver method. It says that if the current node has completed the abstract $\text{deliver}(t, ts)$ operation, which adds its own ID tid into the delivered set $\mathcal{N}$ for the message (t, ts, v) in the abstract POOL, then it must have already set the flag bit of the message (t, ts, v) in MailBox .

Finally, faultyRelated characterizes the helping mechanism for faulty nodes. It says that if an abstract broadcast operation of a faulty node t is completed, then there must be a quorum set of nodes which have sent the corresponding ECHO messages in the concrete side (represented by quorumECHO).

$$F \equiv \lfloor \frac{|\text{Nodes}| - 1}{3} \rfloor$$

$$\text{TH} \equiv \lfloor \frac{|\text{Nodes}| + F}{2} \rfloor$$

Fig. 33. Macros

The second key part of *inv* is the assertion *goodMsgPool* encoding the temporal ordering of concrete actions. It is a conjunction of the following properties. First, *echoHasInitd* says that every ECHO messages sent by the current node must have been recorded in its local variable *Initd*. It can help us prove the preservation of *uniqueECHO*, which says that a correct node t' never sends multiple ECHO messages with the same (t, ts) but different values v and v' . The property *uniqueECHO* is quite important, because it allows us to prove that, if two correct nodes help the same faulty node t by doing **flin**(t , broadcast, (ts, v)) and **flin**(t , broadcast, (ts, v')) respectively, then v and v' must be the same value.

The next property *mailboxHasECHO* says that every vote recorded in *MailBox* can be traced back to an ECHO message. Then *msgFlow* further says that a correct node sends an ECHO message only in response to an existing INIT message. Using these two properties and *initHasLinearized* together, we can derive that when the current node *tid* gets sufficiently many votes for (t, ts, v) in *MailBox* and linearizes itself in the *deliver*(t, ts) method (at line 29 in Alg. 1), the corresponding method call *broadcast*(ts, v) made by the node t must have been linearized.

The actions of the protocol are shown in Fig. 36. In G^m , *IdSpecExcept*(*tid*) says that the abstract pending operations from other nodes are preserved; $[MtdGs]_{inv}$ indicates a guarantee of *MtdGs* with *inv* being preserved (formal definition of $[G]_p$ in Def. 6.6). *MtdGs* specifies the transition over local state, message pool and abstract P00L. In local state, any object variables unused by client thread (defined as *sevVarSet*, formally Def. 6.7) is preserved, and variable *MailBox* only expands. In message pool, INIT messages may be sent. In abstract P00L, broadcast, deliver and assisting faulty node's broadcast operation may happen.

While the server guarantee G^s ensures the following:

- preservation of the abstract P00L and all pending operations;
- monotonic expansion of the *MailBox* (while preserving all server-unused variables);
- ECHO message may be sent.

Other nodes' behaviors R is the union of all other benign nodes guarantees and faulty nodes' guarantee.

The proof outline is shown in Alg. 1, and the proofs for assertion validity and stability are presented in the next section.

Definition 6.2. $\zeta_B \stackrel{\text{def}}{=} \lambda \mathcal{N}_{\text{bad}}, M. \{ \{ (\text{INIT}, (t, _, _)) \mid t \notin \mathcal{N}_{\text{bad}} \} \cup \{ (\text{ECHO}, _, t) \mid t \notin \mathcal{N}_{\text{bad}} \} \}$

Definition 6.3. We expand the expression with *get*(E), and define its evaluation function as follows:

$$\llbracket \text{get}(E) \rrbracket_\sigma \stackrel{\text{def}}{=} \begin{cases} v & \text{if } \llbracket E \rrbracket_\sigma = \{v\} \\ \text{None} & \text{otherwise} \end{cases}$$

Algorithm 1

```

1: var SentTs, MailBox, Initd :=  $\emptyset$ ;
2:
3: operation broadcast(ts, v) {
4:   ①  $\{p_t \wedge (\text{tid} \rightarrow (\text{BROADCAST}, (ts, v)) * \text{true})\}$ 

```

```

5: ②  $\{p_t * tid \mapsto (\text{BROADCAST}, (ts, v))\}$ 
6: if  $(ts \notin \text{SentTs})$  then
7:   ③  $\{p_t * tid \mapsto (\text{BROADCAST}, (ts, v)) \wedge ts \notin \text{SentTs}\}$ 
8:    $\langle \text{SentTs} := \text{SentTs} \cup \{ts\};$ 
9:   lin(tid) ;
10:  ④  $\{p_t * tid \mapsto (\text{end}, \text{ok}) \wedge \exists A. \text{isPool}(A) \wedge A(\text{tid}, ts) = (v, \_)\}$ 
11:  send(Nodes, (INIT, (tid, ts, v)));
12:  ⑤  $\{p_t * tid \mapsto (\text{end}, \text{ok})\}$ 
13:  return ok;
14: else
15:  ⑥  $\{p_t * tid \mapsto (\text{BROADCAST}, (ts, v)) \wedge ts \in \text{SentTs}\}$ 
16:  lin(tid) ;
17:  ⑦  $\{p_t * tid \mapsto (\text{end}, \text{bad})\}$ 
18:  return bad;
19: end if
20: ⑧  $\{p_t \wedge (\text{tid} \mapsto (\text{end}, \_) * \text{true})\}$ 
21: }
22:
23: operation deliver(t, ts) {
24:   ⑨  $\{p_t \wedge (\text{tid} \mapsto (\text{DELIVER}, (t, ts)) * \text{true})\}$ 
25:   ⑩  $\{p_t * tid \mapsto (\text{DELIVER}, (t, ts))\}$ 
26:   res := None;
27:   ⑪  $\{p_t * tid \mapsto (\text{DELIVER}, (t, ts)) \wedge \text{whileInv} \wedge \text{res} = \text{None}\}$ 
28:   while (res = None) do
29:     res := get( $\{v \mid \exists N. \text{MailBox}(t, ts, v) = (N, \text{false}) \wedge |N| > TH\}$ );
30:   end while
31:   ⑫  $\{p_t * tid \mapsto (\text{DELIVER}, (t, ts)) \wedge \text{whileInv} \wedge \text{res} \neq \text{None}\}$ 
32:   ⑬  $\{p_t * tid \mapsto (\text{DELIVER}, (t, ts)) \wedge \text{quorumECHO}(t, ts, \text{res}) \wedge \text{legalHelpFlinPrec}\}$ 
33:   MailBox := MailBox( $\{(t, ts, \text{res}) \rightsquigarrow (\text{MailBox}(t, ts, \text{res}).\text{votes}, \text{true})\}$ );
34:   ⑭  $\{p_t * tid \mapsto (\text{DELIVER}, (t, ts)) \wedge \text{quorumECHO}(t, ts, \text{res}) \wedge \text{legalHelpFlinPrec}\}$ 
35:   if  $(t \in \text{Nbad})$  then
36:     flin(t, BROADCAST, (ts, res));
37:   end if
38:   ⑮  $\{p_t * tid \mapsto (\text{DELIVER}, (t, ts)) \wedge \exists A, N. \text{isPool}(A) \wedge A(t, ts) = (\text{res}, N) \wedge \text{tid} \notin N\}$ 
39:   lin(tid) ;
40:   ⑯  $\{p_t * tid \mapsto (\text{end}, \text{res})\}$ 
41:   return res;
42:   ⑰  $\{p_t \wedge (\text{tid} \mapsto (\text{end}, \text{res}) * \text{true})\}$ 
43: }
44:
45: process server{
46:   ⑱  $\{p_t\}$ 
47:   while (true) do
48:     ⑲  $\{p_t\}$ 
49:     msg := recv();
50:     ⑳  $\{p_t \wedge \text{sent}(\{tid\}, \text{msg})\}$ 
51:     if (msg.tag = INIT) then
52:       ㉑  $\{p_t \wedge \text{sent}(\{tid\}, \text{msg}) \wedge \text{msg.tag} = \text{INIT}\}$ 
53:        $(\_, (t', ts', v')) := \text{msg};$ 
54:       ㉒  $\{p_t \wedge \text{sent}(\{tid\}, (\text{INIT}, (t', ts', v')))\}$ 

```

whileInv	$\stackrel{\text{def}}{=}$	$\text{res} \neq \text{None} \Rightarrow \text{quorumECHO}(t, \text{ts}, \text{res}) \wedge \text{MailBox}(t, \text{ts}, \text{res}) = (_, \text{false})$
legalHelpFlinPrec	$\stackrel{\text{def}}{=}$	$\exists A. \text{isPool}(A) \wedge (t \in \text{Nok} \Rightarrow A(t, \text{ts}) = (\text{res}, _))$ $\wedge (\forall N. A(t, \text{ts}) = (\text{res}, N) \Rightarrow \text{tid} \notin N)$ $\wedge (\forall v. A(t, \text{ts}) = (v, _) \Rightarrow v = \text{res})$
legalEchoPrec(t', ts')	$\stackrel{\text{def}}{=}$	$\nexists N, v. \text{sent}(N, (\text{ECHO}, (t', \text{ts}', v), \text{tid}))$
isPool(A)	$\stackrel{\text{def}}{=}$	$\text{POOL} \Rightarrow A * \text{true}$

Fig. 34. Auxiliary assertions

```

55:      if ((t', ts') ∉ Inited) then
56:          23 {pt ∧ sent({tid}, (INIT, (t', ts', v')))) ∧ (t', ts') ∉ Inited}
57:          24 {pt ∧ sent({tid}, (INIT, (t', ts', v')))) ∧ legalEchoPrec(t', ts')}
58:          Inited := Inited ∪ {(t', ts')};
59:          25 {
                    { pt ∧ sent({tid}, (INIT, (t', ts', v')))) ∧ legalEchoPrec(t', ts') }
                    ∧ (t', ts') ∈ Inited
                }
60:          send(Nodes, (ECHO, (t', ts', v'), tid));
61:          26 {pt}
62:      end if
63:      27 {pt}
64:      else if (msg.tag = ECHO) then
65:          28 {pt ∧ sent({tid}, msg) ∧ msg.tag = ECHO}
66:          (_, cont, t') := msg;
67:          29 {pt ∧ sent({tid}, (ECHO, cont, t'))}
68:          if (cont ∉ dom(MailBox)) then
69:              MailBox := MailBox{cont ↦ (∅, false)};
70:          end if
71:          30 {pt ∧ sent({tid}, (ECHO, cont, t')) ∧ cont ∈ dom(MailBox)}
72:          MailBox := MailBox{cont ↦ (MailBox(cont).votes ∪ {t'}, MailBox(cont).flag)};
73:          31 {pt}
74:      end if
75:      32 {pt}
76:  end while
77:  33 {true}
78: }

```

Definition 6.4. $\eta_B \stackrel{\text{def}}{=} \{(n, n') \mid n > 2n'\}$

Definition 6.5. $\text{IdSpecExcept}(x) \stackrel{\text{def}}{=} (x \mapsto _ \ltimes x \mapsto _) * \text{IdSpec}$

Definition 6.6. $[G]_p \stackrel{\text{def}}{=} \lambda t. G \wedge (p \ltimes p) \wedge \text{IdV}(\text{readOnlyVarSet}) \wedge (\text{tid} = t \ltimes \text{tid} = t)$

Definition 6.7 (Local variable set). Method-local variable set is defined as mtdVarSet , server-local variable set is defined as sevVarSet :

$\text{sevVarSet} \stackrel{\text{def}}{=} \{\text{msg}, t', \text{ts}', v', \text{Inited}, \text{cont}\}$
 $\text{mtdVarSet} \stackrel{\text{def}}{=} \{\text{ts}, v, \text{SentTs}, t, \text{res}\}$

6.3 Proof of Assertion Validity and Stability

In this section we show assertion validity and stability proof that is used in Alg. 1. The assertion validity is listed as follows:

p_t	$\stackrel{\text{def}}{=}$	$\text{inv} * \text{true} \wedge \text{tid} = t \wedge \text{readOnlyVarProp}(\eta_B)$
inv	$\stackrel{\text{def}}{=}$	$\exists A. \text{POOL} \Rightarrow A \wedge \text{goodAbsPOOL}(A) \wedge \text{goodMsgPool}$
$\text{goodAbsPool}(A)$	$\stackrel{\text{def}}{=}$	$\text{broadcastRelated}(A) \wedge \text{InitHasLinearized}(A) \wedge \text{deliverRelated}(A) \wedge \text{faultyRelated}(A)$
goodMsgPool	$\stackrel{\text{def}}{=}$	$\text{echoHasInitiated} \wedge \text{msgFlow} \wedge \text{mailboxHasECHO} \wedge \text{uniqueECHO}$
$\text{broadcastRelated}(A)$	$\stackrel{\text{def}}{=}$	$\forall ts. ts \in \text{SentTs} \Leftrightarrow (\text{tid}, ts) \in \text{dom}(A)$
$\text{initHasLinearized}(A)$	$\stackrel{\text{def}}{=}$	$\forall t, ts, v. t \in \text{Nok} \wedge \text{sent}(_, (\text{INIT}, (t, ts, v))) \Rightarrow A(t, ts) = (v, _)$
$\text{deliverRelated}(A)$	$\stackrel{\text{def}}{=}$	$\forall t, ts, v, N. ((t, ts, v) \notin \text{dom}(\text{MailBox}) \vee \text{MailBox}(t, ts, v).\text{flag} = \text{false}) \wedge A(t, ts) = (v, N) \Rightarrow \text{tid} \notin N$
$\text{faultyRelated}(A)$	$\stackrel{\text{def}}{=}$	$\forall t, ts, v. t \in \text{Nbad} \wedge A(t, ts) = (v, _) \Rightarrow \text{quorumECHO}(t, ts, v)$
$\text{quorumECHO}(t, ts, v)$	$\stackrel{\text{def}}{=}$	$\exists N \subseteq \text{Nodes}. (N > \text{TH}) \wedge \forall t' \in N. \text{sent}(_, (\text{ECHO}, (t, ts, v), t'))$
echoHasInitiated	$\stackrel{\text{def}}{=}$	$\forall t, ts. \text{sent}(_, (\text{ECHO}, (t, ts, _), \text{tid})) \Rightarrow (t, ts) \in \text{Initiated}$
uniqueECHO	$\stackrel{\text{def}}{=}$	$\forall t, ts, v, v', t'. t' \in \text{Nok} \wedge \text{sent}(_, (\text{ECHO}, (t, ts, v), t')) \wedge \text{sent}(_, (\text{ECHO}, (t, ts, v'), t')) \Rightarrow v = v'$
msgFlow	$\stackrel{\text{def}}{=}$	$\forall t, cont. t \in \text{Nok} \wedge \text{sent}(_, (\text{ECHO}, cont, t)) \Rightarrow \text{sent}(_, (\text{INIT}, cont))$
mailboxHasECHO	$\stackrel{\text{def}}{=}$	$\forall N, cont. \text{MailBox}(cont) = (N, _) \Rightarrow (\forall t \in N. \text{sent}(_, (\text{ECHO}, cont, t)))$

Fig. 35. Object invariant

G^m	$\stackrel{\text{def}}{=}$	$[\text{MtdGs}]_{\text{inv}} * \text{IdSpecExcept}(\text{tid})$
G^s	$\stackrel{\text{def}}{=}$	$[\text{SevGs}]_{\text{inv}} * \text{IdSpec}$
R	$\stackrel{\text{def}}{=}$	$\lambda t. \bigvee_{t' \neq t} (\text{InvokeAct}(\Pi_B.L)(t') \vee G_{t'}^m \vee G_{t'}^s) \vee \text{GF}(\zeta_B)$
MtdGs	$\stackrel{\text{def}}{=}$	$\text{IdV}(\text{sevVarSet}) \wedge \text{MailBoxGs} \wedge (\text{IdM} \vee \text{SendAct}(_, (\text{INIT}, _))) \wedge \text{AbsPoolGs}$
SevGs	$\stackrel{\text{def}}{=}$	$\text{IdV}(\text{mtdVarSet}) \wedge \text{MailBoxGs} \wedge (\text{IdM} \vee \text{SendAct}(_, (\text{ECHO}, _, \text{tid}))) \wedge \text{IdAbsPool}$
MailBoxGs	$\stackrel{\text{def}}{=}$	$\exists cont, MB. \text{MailBox} = MB \ltimes \text{MailBox} = MB \vee \text{MailBox} = MB \{ _ \rightsquigarrow _ \}$
AbsPoolGs	$\stackrel{\text{def}}{=}$	$\text{IdAbsPool} \vee \text{PoolBroadcast} \vee \text{PoolDeliver} \vee \text{PoolFaulty}$
IdAbsPool	$\stackrel{\text{def}}{=}$	$\exists A. \text{POOL} \Rightarrow A \ltimes \text{POOL} \Rightarrow A$
PoolBroadcast	$\stackrel{\text{def}}{=}$	$\exists A, ts, v. \text{POOL} \Rightarrow A \wedge (\text{tid}, ts) \notin \text{dom}(A) \ltimes \text{POOL} \Rightarrow A\{(\text{tid}, ts) \rightsquigarrow (v, \emptyset)\}$
PoolDeliver	$\stackrel{\text{def}}{=}$	$\exists A, t, ts, v, N. \text{POOL} \Rightarrow A \wedge A(t, ts) = (v, N) \wedge \text{tid} \notin N \ltimes \text{POOL} \Rightarrow A\{(t, ts) \rightsquigarrow (v, N \cup \{\text{tid}\})\}$
PoolFaulty	$\stackrel{\text{def}}{=}$	$\exists A, t, ts, v. t \in \text{Nbad} \wedge \text{POOL} \Rightarrow A \wedge (t, ts) \notin \text{dom}(A) \ltimes \text{POOL} \Rightarrow A\{(t, ts) \rightsquigarrow (v, \emptyset)\}$

Fig. 36. Rely/guarantee conditions for the broadcast protocol

- $p_{bc1} \Rightarrow p_{bc2}$ is directly from definition and Lm. 92;
- $p_{bc5} \vee p_{bc7} \Rightarrow p_{bc8}$ is directly from Lm. 93;
- $p_{bc12} \Rightarrow p_{bc13}$ is directly from Lm. 96.

The stability proofs is listed as follows:

- in Lm. 105 we prove that p_t is stable w.r.t. the local and the other nodes' actions;
- in Lm. 113 we prove p_{bc13} is stable w.r.t. the other benign nodes' actions. The stability of p_{bc13} w.r.t. local and faulty node's behaviors are trivial.

Other pre- post-condition's stability can be directly deduced from Lm. 105 and the properties of definition stable: Lm. 97, Lm. 98, Lm. 99, Lm. 100 and Lm. 101.

Definition 6.8. $p\#q \stackrel{\text{def}}{=} \forall s, \Delta, \Delta'. (s, \Delta) \models p \wedge (s, \Delta') \models q \implies \Delta\#\Delta'$

Lemma 92. $\forall p, q. p\#q \implies ((p * \text{true}) \wedge (q * \text{true}) \implies (p * q * \text{true}))$

PROOF. By definition, given arbitrary s, Δ , if $(s, \Delta) \models (p * \text{true}) \wedge (q * \text{true})$, then we need to prove $(s, \Delta) \models p * q * \text{true}$. By definition we know there exists $\Delta_a, \Delta'_a, \Delta_b, \Delta'_b$ s.t.

$$\begin{array}{llll} (s, \Delta_a) \models p & (s, \Delta_b) \models q & & \\ \Delta_a\#\Delta'_a & \Delta_b\#\Delta'_b & \Delta_a * \Delta'_a = \Delta & \Delta_b * \Delta'_b = \Delta \end{array}$$

By $p\#q$ we know $\Delta_a\#\Delta_b$. By definition we only need to prove $\exists \Delta'. \Delta_a * \Delta_b * \Delta' = \Delta$, that is to prove $\exists \Delta'. \Delta_a * \Delta_b * \Delta' \subseteq \Delta \wedge \Delta_a * \Delta_b * \Delta' \supseteq \Delta$, which by definition and premises immediately. $\square$

Lemma 93. $\forall p, q. p * q * \text{true} \implies (p * \text{true}) \wedge (q * \text{true})$

PROOF. By definition immediately. $\square$

Lemma 94. The following assertion is valid:

$$\forall N, N'. |N| > \text{TH} \wedge |N'| > \text{TH} \wedge N \subseteq \text{Nodes} \wedge N' \subseteq \text{Nodes} \implies |N \cap N'| \geq F + 1$$

PROOF. Given arbitrary $\Sigma, v_N, v_{N'}$, we prove $\Sigma|_{\sigma_o \mapsto \sigma_o \uplus \{N \rightsquigarrow v_N, N' \rightsquigarrow v_{N'}\}}$ holds for $|N| > \text{TH} \wedge |N'| > \text{TH} \wedge N \subseteq \text{Nodes} \wedge N' \subseteq \text{Nodes} \implies |N \cap N'| \geq F + 1$. We omit $\Sigma|_{\sigma_o \mapsto \sigma_o \uplus \{N \rightsquigarrow v_N, N' \rightsquigarrow v_{N'}\}}$ in the following proof writing. By $N \subseteq \text{Nodes}$ and $N' \subseteq \text{Nodes}$ we know $|N| + |N'| - |N \cap N'| \leq |\text{Nodes}|$. By $|N| > \text{TH}$ we know $|N| \geq \text{TH} + 1$, by $|N'| > \text{TH}$ we know $|N'| \geq \text{TH} + 1$, thus

$$\text{TH} + \text{TH} + 2 - |N \cap N'| \leq |\text{Nodes}|$$

Then

$$2 \cdot \text{TH} + 2 - |\text{Nodes}| \leq |N \cap N'|$$

By definition of TH we discuss on $|\text{Nodes}| + F$: If $|\text{Nodes}| + F$ is odd, then

$$|\text{Nodes}| + F - 1 + 2 - |\text{Nodes}| \leq |N \cap N'|$$

We know $F + 1 \leq |N \cap N'|$. If $|\text{Nodes}| + F$ is even, then

$$|\text{Nodes}| + F + 2 - |\text{Nodes}| \leq |N \cap N'|$$

We prove $F + 2 \leq |N \cap N'|$. Thus we conclude this lemma. $\square$

Lemma 95. The following assertion is valid:

$$\forall t, ts, v, v'. \text{readOnlyVarProp}(\eta_B) \wedge \text{quorumECHO}(t, ts, v) \wedge \text{quorumECHO}(t, ts, v') \wedge \text{uniqueECHO} \implies v = v'$$

PROOF. Given arbitrary t, ts, v, v' , from precondition by definition we know there exists N, N' :

$$|\text{Nok}| > 2|\text{Nbad}| \wedge \text{Nodes} = \text{Nok} \uplus \text{Nbad} \quad (1)$$

$$|N| > \text{TH} \wedge \forall t' \in N. \text{sent}(_, (\text{ECHO}, (t, ts, v), t')) \quad (2)$$

$$|N'| > \text{TH} \wedge \forall t' \in N'. \text{sent}(_, (\text{ECHO}, (t, ts, v'), t')) \quad (3)$$

to prove $v' = v$, by uniqueECHO and (2)(3) we only need to prove there exists some t_x s.t. $t_x \in \text{Nok} \cap N \cap N'$. by (1) we only need to prove $|N \cap N'| \geq F + 1$. which by apply $|N| > \text{TH} \wedge |N'| > \text{TH}$ on Lm. 94 immediately. $\square$

Lemma 96. The following assertion is valid:

$$\begin{aligned} & \exists A. \text{isPool}(A) \wedge \text{initHasLinearized}(A) \wedge \text{deliverRelated}(A) \wedge \text{faultyRelated}(A) \\ & \wedge \text{msgFlow} \wedge \text{uniqueECHO} \wedge \text{readOnlyVarProp}(\eta_B) \wedge \text{quorumECHO}(t, ts, res) \\ & \wedge \text{MailBox}(t, ts, res) = (_, \mathbf{false}) \Rightarrow \text{legalHelpFlinPrec} \end{aligned}$$

PROOF. By definition of legalHelpFlinPrec, given arbitrary Σ holds for precondition, we need to prove Σ holds for

$$t \in \text{Nok} \Rightarrow A(t, ts) = (res, _) \quad (\text{goal1})$$

$$\forall N. A(t, ts) = (res, N) \Rightarrow \text{tid} \notin N \quad (\text{goal2})$$

$$\forall v. A(t, ts) = (v, _) \Rightarrow v = res \quad (\text{goal3})$$

To prove (goal1), by quorumECHO(t, ts, res), definition of TH and readOnlyVarProp(η_B) we know $\exists t'. t' \in \text{Nok} \wedge \text{sent}(_, (\text{ECHO}, (t, ts, res), t'))$. Apply msgFlow we know $\text{sent}(_, (\text{INIT}, (t, ts, res)))$, then apply initHasLinearized(A) we prove $A(t, ts) = (res, _)$.

Apply MailBox(t, ts, res) = ($_, \mathbf{false}$) on deliverRelated(A) we have (goal2) immediately.

To prove (goal3), discuss on case:

- If Σ holds for $t \in \text{Nok}$, then as we have proved (goal1), we proved the goal immediately.
- If Σ holds for $t \in \text{Nbad}$, take arbitrary v , if $A(t, ts) = (v, _)$, we need to prove $res = v$. Apply faultyRelated(A) we know quorumECHO(t, ts, v). Then apply Lm. 95 we prove the goal.

□

Lemma 97. $\forall p, R, R'. \text{stable}(p, R) \wedge \text{stable}(p, R') \Rightarrow \text{stable}(p, R \vee R')$

PROOF. By definition immediately.

□

Lemma 98. $\forall p, p', R. \text{stable}(p, R) \wedge \text{stable}(p', R) \Rightarrow \text{stable}(p \wedge p', R)$

PROOF. By definition immediately.

□

Lemma 99. $\forall p, R, R'. R \Rightarrow R' \wedge \text{stable}(p, R') \Rightarrow \text{stable}(p, R)$

PROOF. By definition immediately.

□

Lemma 100. $\forall p, q, r. \text{pure}(r) \Rightarrow ((p * q) \wedge r \Leftrightarrow (p \wedge r) * q)$

PROOF. Given pure assertion r , we need to prove

$$(p * q) \wedge r \Rightarrow (p \wedge r) * q \quad (\text{goal1})$$

$$(p \wedge r) * q \Rightarrow (p * q) \wedge r \quad (\text{goal2})$$

For the first goal, given arbitrary Σ s.t. $\Sigma \models (p * q) \wedge r$, and prove

$$\Sigma \models (p \wedge r) * q.$$

From $\Sigma \models (p * q) \wedge r$ we know $\Sigma \models p * q$, then there exists Σ_1, Σ_2 s.t. $\Sigma_1 * \Sigma_2 = \Sigma$, $\Sigma_1 \models p$ and $\Sigma_2 \models q$. Now we only need to prove $\Sigma_1 \models r$. From $\Sigma \models (p * q) \wedge r$ we also know $\Sigma \models r$, apply $\Sigma_1 * \Sigma_2 = \Sigma$ and pure(r) we prove the goal.

For the second goal, given arbitrary Σ s.t. $\Sigma \models (p \wedge r) * q$, and prove

$$\Sigma \models (p * q) \wedge r$$

From $\Sigma \models (p \wedge r) * q$ we know there exists Σ_1, Σ_2 s.t. $\Sigma_1 * \Sigma_2 = \Sigma$, $\Sigma_1 \models p \wedge r$ and $\Sigma_2 \models q$. Now we only need to prove $\Sigma \models r$. This is proved by apply $\Sigma_1 * \Sigma_2 = \Sigma$ and pure(r) immediately. Thus we conclude the goal.

□

To enable compositional reasoning about stability, i.e. reasoning $\text{stable}(p * p', R * R')$ from $\text{stable}(p, R)$ and $\text{stable}(p', R')$, following Feng [2009], we define precise assertion and fence as follows, and then prove compositionality of stability in Lm. 101.

Definition 6.9 (Precise assertion). An assertion p is precise iff, for all s, Δ , there is at most one $\Delta' \subseteq \Delta$ such that $(s, \Delta) \models p$. Formally:

$$\text{precise}(p) \stackrel{\text{def}}{=} \forall \Sigma_1, \Sigma_2, \Sigma'_1, \Sigma'_2. \Sigma_1 * \Sigma_2 = \Sigma'_1 * \Sigma'_2 \wedge \Sigma_1 \models p \wedge \Sigma'_1 \models p \Rightarrow \Sigma_1.\Delta = \Sigma'_1.\Delta$$

Definition 6.10 (Fence). $I \triangleright R \stackrel{\text{def}}{=} [I] \Rightarrow R \wedge R \Rightarrow I \ltimes I \wedge \text{precise}(I)$

Lemma 101. $\forall p, p', R, R', I. p \Rightarrow I \wedge I \triangleright R \wedge \text{stable}(p, R) \wedge \text{stable}(p', R') \Rightarrow \text{stable}(p * p', R * R')$

PROOF. By definition we need to take arbitrary Σ, Σ' s.t.

$$\Sigma \models p * p' \quad (1)$$

$$(\Sigma, \Sigma') \models R * R' \quad (2)$$

and prove $\Sigma' \models p * p'$.

By (1) we know there exists Σ_1, Σ_2 s.t.

$$\Sigma_1 * \Sigma_2 = \Sigma \quad (11)$$

$$\Sigma_1 \models p \quad (12)$$

$$\Sigma_2 \models p' \quad (13)$$

By (2) we know there exists $\Sigma_a, \Sigma'_a, \Sigma_b, \Sigma'_b$ s.t.

$$\Sigma_a * \Sigma_b = \Sigma \quad (21)$$

$$\Sigma'_a * \Sigma'_b = \Sigma' \quad (22)$$

$$(\Sigma_a, \Sigma'_a) \models R \quad (23)$$

$$(\Sigma_b, \Sigma'_b) \models R' \quad (24)$$

By (11)(21) we know $\Sigma_a.s = \Sigma_1.s$, then by $I \triangleright R$ we know $R \Rightarrow I \ltimes I$, apply (23) we know $\Sigma_a \models I$; we also know $p \Rightarrow I$, apply (12) we know $\Sigma_1 \models I$. then by $\text{precise}(I)$ we know $\Sigma_1.\Delta = \Sigma_a.\Delta$. thus we know $\Sigma_1 = \Sigma_a$.

By Lm. 102 and (21) (11) we know $\Sigma_2 = \Sigma_b$, thus we only need to prove $\Sigma'_a \models p, \Sigma'_b \models p'$, which by (23) (24) (12) (13) and precondition $\text{stable}(p, R), \text{stable}(p', R')$ immediately. $\square$

Lemma 102. $\forall \Sigma_0, \Sigma_1, \Sigma_2, \Sigma. \Sigma_0 * \Sigma_1 = \Sigma \wedge \Sigma_0 * \Sigma_2 = \Sigma \Rightarrow \Sigma_1 = \Sigma_2$

PROOF. By definition and Lm. 103 immediately. $\square$

Lemma 103. $\forall \Delta_0, \Delta_1, \Delta_2, \Delta. \Delta_0 * \Delta_1 = \Delta \wedge \Delta_0 * \Delta_2 = \Delta \Rightarrow \Delta_1 = \Delta_2$

PROOF. By definition immediately. $\square$

Lemma 104. $\text{precise}(\text{inv})$

PROOF. By definition of inv immediately. $\square$

Lemma 105. $\forall t. \text{stable}(p_t, \text{InvokeAct}(\Pi_B.L)(t) \vee G_t^m \vee G_t^s \vee \text{nonLocal}(R_t))$

PROOF. By definition, Lm. 106, Lm. 98, Lm. 107, Lm. 109, Lm. 111, Lm. 111, Lm. 112 immediately. $\square$

Lemma 106. $\forall t. \text{stable}(p_t, \text{InvokeAct}(\Pi_B.L)(t))$

PROOF. By definition immediately. $\square$

Lemma 107. $\forall t. \text{stable}(p_t, G_t^m \vee G_t^s)$

PROOF. By definition we only need to prove that $\text{stable}(p_t, G_t^m)$ and $\text{stable}(p_t, G_t^s)$.
To prove $\text{stable}(p_t, G_t^m)$, by definition we only need to prove that p_t stable w.r.t.

$$(\text{MtdGs} \wedge \text{IdV}(\text{readOnlyVarSet}) \wedge (\text{inv} \times \text{inv}) \wedge (\text{tid} = t \times \text{tid} = t)) * \text{IdSpecExcept}(\text{tid}).$$

Following Lm. 104 and Lm. 101, we only need to prove inv stable w.r.t.

$$(\text{MtdGs} \wedge \text{IdV}(\text{readOnlyVarSet}) \wedge (\text{inv} \times \text{inv}) \wedge (\text{tid} = t \times \text{tid} = t)),$$

which is directly from $\text{inv} \times \text{inv}$. The proof of $\text{stable}(p_t, G_t^s)$ is similar. $\square$

Definition 6.11 (local-irrelevant).

$$\text{locallrrelevant}(p) \stackrel{\text{def}}{=} \forall \sigma_o, \sigma'_o, M, \sigma_a, \Delta. ((\sigma_o, M, \sigma_a), \Delta) \models p \iff ((\sigma'_o, M, \sigma_a), \Delta) \models p$$

Definition 6.12.

$$\Sigma|_{M \mapsto M'} \stackrel{\text{def}}{=} ((\Sigma.\sigma_o, M', \Sigma.\sigma_a), \Sigma.\Delta)$$

Lemma 108. $\forall p. \text{locallrrelevant}(p) \implies (\text{nonLocal}(p \times p) \implies (p \times p))$

PROOF. Given premise, we need to take arbitrary Σ, Σ' s.t. $(\Sigma, \Sigma') \models \text{nonLocal}(p \times p)$, then prove $(\Sigma, \Sigma') \models p \times p$. By definition of nonLocal we know

There exists σ, σ' s.t. $\Sigma.\sigma_o = \Sigma'.\sigma_o$ and

$$(\Sigma|_{\sigma_o \mapsto \sigma}, \Sigma'|_{\sigma_o \mapsto \sigma'}) \models p \times p.$$

Thus we know $\Sigma|_{\sigma_o \mapsto \sigma} \models p$ and $\Sigma'|_{\sigma_o \mapsto \sigma'} \models p$. Apply $\text{locallrrelevant}(p)$ respectively we prove $\Sigma \models p$ and $\Sigma' \models p$, thus we prove the goal. $\square$

Lemma 109. $\forall t. \text{stable}(p_t, \text{nonLocal}(\bigvee_{t' \neq t} G_{t'}^m \vee G_{t'}^s))$

PROOF. By definition, Lm. 104 and Lm. 101, we only need to prove that when $t \neq t'$, inv is stable w.r.t. the following action R^x :

$$\text{nonLocal}((\text{SevGs} \vee \text{MtdGs}) \wedge \text{IdV}(\text{readOnlyVarSet}) \wedge (\text{inv} \times \text{inv}) \wedge (\text{tid} = t' \times \text{tid} = t')).$$

By definition of inv and Lm. 98 we need to prove that

$$\begin{array}{ll} \text{stable}(\exists A. \text{POOL} \implies A \wedge \text{goodAbsPOOL}(A), R^x) & (\text{goal1}) \\ \text{stable}(\text{goodMsgPool}, R^x) & (\text{goal2}) \end{array}$$

To prove *(goal1)*, by definition of goodAbsPOOL and locallrrelevant we know the following assertion is local-irrelevant: $\exists A. \text{POOL} \implies A \wedge \text{initHasLinearized}(A)$. Thus by Lm. 108 and definition we can prove it is stable. now we only need to prove the remaining of inv is stable:

$$\begin{array}{l} \exists A. \text{POOL} \implies A \wedge \text{broadcastRelated}(A) \wedge \text{deliverRelated}(A) \wedge \text{faultyRelated}(A) \\ \wedge \text{readOnlyVarProp}(\eta_B) \end{array}$$

By Lm. 98 we discuss this separately. To illustrate that $\exists A. \text{POOL} \Rightarrow A \wedge \text{broadcastRelated}(A)$ is stable, as no local state is changed in R^x , we only need to prove that no content sent by t is added to or removed from abstract pool during R^x . As there is no action of removing messages from abstract pool in R^x , and all action of adding is impossible to be sent by t as $t \neq t'$, we prove the goal.

To illustrate that $\exists A. \text{POOL} \Rightarrow A \wedge \text{deliverRelated}(A)$ is stable, as local variable MailBox preserves during R^x , we discuss on actions that change the reception sets of abstract POOL. If t' perform the abstract broadcast, then as the newly added reception set is $\emptyset$, we prove the goal trivially. If t' perform the abstract deliver, then t' is added to reception set. As $t' \neq t$, we proved the goal. If t' helps a faulty abstract broadcast, then as the newly add reception set is $\emptyset$, the goal holds trivially.

To illustrate that $\exists A. \text{POOL} \Rightarrow A \wedge \text{faultyRelated}(A) \wedge \text{readOnlyVarProp}(\eta_B)$ is stable, which is non-trivial, we deconstruct definition and show it step by step. By definition of stable, we need to take arbitrary Σ, Σ' s.t. Σ satisfying precondition, (Σ, Σ') satisfying R^x , and prove Σ' still satisfying precondition. By definition of nonLocal we know there exists σ, σ' s.t.

$$\begin{aligned} \Sigma.\sigma_o &= \Sigma'.\sigma_o \\ (\Sigma|_{\sigma_o \mapsto \sigma}, \Sigma'|_{\sigma_o \mapsto \sigma'}) &\models (\text{SevGs} \vee \text{MtdGs}) \wedge \text{IdV}(\text{readOnlyVarSet}) \\ &\wedge (\text{inv} \ltimes \text{inv}) \wedge (\text{tid} = t' \ltimes \text{tid} = t') \end{aligned} \quad (1)$$

By definition of MtdGs and SevGs, if the action adds a new benign message into pool following PoolBroadcast, then as $t' \in \text{Nok}$ by $\text{readOnlyVarProp}(\eta_B)$, no newly message sent by faulty node is added into pool during R^x . Thus by definition of faultyRelated, we only need to prove that the concrete message pool does not shrink and $\Sigma.\sigma_o(\text{Nodes}) = \sigma(\text{Nodes}) = \sigma'(\text{Nodes})$. The former is obvious, while the latter is done by Σ and $\Sigma|_{\sigma_o \mapsto \sigma}$ both holds for $\text{readOnlyVarProp}(\eta_B)$, and (Σ, Σ') , $(\Sigma|_{\sigma_o \mapsto \sigma}, \Sigma'|_{\sigma_o \mapsto \sigma'})$ holds for $\text{IdV}(\text{readOnlyVarSet})$.

If the action adds a new faulty node's message *cont* into abstract pool following PoolFaulty, then by definition of faultyRelated we need to prove $\Sigma' \models \text{quorumECHO}(\text{cont})$. Apply *inv* we know $\Sigma'|_{\sigma_o \mapsto \sigma'} \models \text{quorumECHO}(\text{cont})$, now we only need to prove $\Sigma'.\sigma_o(\text{Nodes}) = \sigma'(\text{Nodes})$, which is similar to above case.

To prove the (*goal2*), similarly by definition of goodMsgPool and locallyIrrelevant we know msgFlow and uniqueECHO are local-irrelevant, by Lm. 108 and definition of R^x we prove they are stable. Now we only need to prove the remaining is stable:

$$\text{echoHasInitiated} \wedge \text{mailBoxHasECHO}$$

By Lm. 98 we discuss this separately. To illustrate echoHasInitiated is stable, we only need to show that no ECHO message sent by t is added to message pool during R^x . This holds because R^x only send ECHO message or INIT message, and ECHO message is always sent by t' and $t \neq t'$. mailBoxHasECHO is preserved as the local variable MailBox does not changed during R^x and no message is removed from message pool. $\square$

Lemma 110. $\forall \zeta. \text{nonLocal}(\text{GF}(\zeta)) \Rightarrow \text{GF}(\zeta)$

PROOF. Note that the Σ and Σ' we discussed here holds for p_t , thus we know M and msgCont are not mapped in object or auxiliary states of Σ and Σ' .

Given arbitrary Σ, Σ', ζ s.t. $(\Sigma, \Sigma') \models \text{nonLocal}(\text{GF}(\zeta))$, we need to prove $(\Sigma, \Sigma') \models \text{GF}(\zeta)$. That is to prove at least one of the following:

$$\begin{aligned} (\Sigma, \Sigma') &\models \text{IdVExcept}(\emptyset) \wedge \text{IdSpec} \wedge (\exists M, v. \text{Nbad} \neq \emptyset \wedge v \notin [\zeta](\text{Nbad}, M) \\ &\quad \wedge \text{msgPool}(M) \ltimes \text{msgPool}(M \cup \{(_, v)\})) \end{aligned} \quad (\text{goal1})$$

$$(\Sigma, \Sigma') \models \text{Id} \quad (\text{goal2})$$

By premise and the definition of nonLocal we know there exists σ, σ' s.t.

$$\begin{aligned}\Sigma.\sigma_o &= \Sigma'.\sigma_o & (1) \\ (\Sigma|_{\sigma_o \mapsto \sigma}, \Sigma'|_{\sigma_o \mapsto \sigma'}) &\models \text{GF}(\zeta) & (2)\end{aligned}$$

By definition of (2) we know at least one of the following holds:

$$\begin{aligned}(\Sigma|_{\sigma_o \mapsto \sigma}, \Sigma'|_{\sigma_o \mapsto \sigma'}) &\models \text{IdVExcept}(\emptyset) \wedge \text{IdSpec} \wedge (\exists M, v. \text{Nbad} \neq \emptyset \wedge v \notin [\zeta])(\text{Nbad}, M) \\ &\quad \wedge \text{msgPool}(M) \times \text{msgPool}(M \cup \{(-, v)\}) \\ (\Sigma|_{\sigma_o \mapsto \sigma}, \Sigma'|_{\sigma_o \mapsto \sigma'}) &\models \text{Id}.\end{aligned}$$

If the first case holds, along with (1) we prove (*goal1*); if the second case holds, along with (1) we prove (*goal2*). □

Lemma 111. $\forall t. \text{stable}(p_t, \text{nonLocal}(\text{GF}(\zeta_B)))$

PROOF. Given arbitrary t , by Lm. 110 and Lm. 99 we only need to prove $\text{stable}(p_t, \text{GF}(\zeta_B))$. Also by definition we know

$$\text{GF}(\zeta_B) \Leftrightarrow \text{GF}(\zeta_B) * \text{IdSpec}.$$

Thus by Lm. 104 and Lm. 101 we only need to prove $\text{stable}(\text{inv}, \text{GF}(\zeta_B))$. By definition of GF we know action $\text{GF}(\zeta_B)$ only change message pool, thus the following assertions that are irrelevant to message pool is stable:

$$\exists A. \text{POOL} \Rightarrow A \wedge \text{broadcastRelated}(A) \wedge \text{deliverRelated}(A).$$

By definition of p_t and Lm. 98 we only need to prove the remaining assertions of inv is stable:

$$\begin{aligned}(\exists A. \text{POOL} \Rightarrow A \wedge \text{faultyRelated}(A) \wedge \text{initHasLinearized}(A)) \\ \wedge \text{goodMsgPool} \wedge \text{tid} = t.\end{aligned}$$

By Lm. 98 we discuss this separately. $\exists A. \text{POOL} \Rightarrow A \wedge \text{faultyRelated}(A)$ is preserved as $\text{GF}(\zeta_B)$ does not remove existing messages from message pool.

$\exists A. \text{POOL} \Rightarrow A \wedge \text{initHasLinearized}(A)$ is preserved as $\text{GF}(\zeta_B)$ either send INIT message with faulty IDs, or send INIT message with *cont* that has been sent before. For the first case, by $\text{readOnlyVarProp}(\eta_B)$ we know $\text{Nok} \cap \text{Nbad} = \emptyset$ and thus the sender's ID is not equal to any benign IDs. This action does not add new benign INIT message to message pool, the assertion preserved; For the second case, the pre-state already holds for $A(t, ts) = (v, _)$ due to $\text{initHasLinearized}(A)$, and as abstract pool does not change during $\text{GF}(\zeta_B)$, thus the assertion preserved.

Stability illustration of goodMsgPool is similar. □

Lemma 112. $\forall t. \text{stable}(p_t, \text{nonLocal}(\bigvee_{t' \neq t} \text{InvokeAct}(\Pi_B.L)(t')))$

PROOF. By definition immediately. □

Lemma 113. $\forall t, t'. t \neq t' \Rightarrow \text{stable}(p_{\text{bc13}}(t), \text{nonLocal}(G_{t'}^s \vee G_{t'}^m))$

PROOF. By definition, given arbitrary Σ, Σ' s.t. $\Sigma \models p_{\text{bc13}}(t)$ and $(\Sigma, \Sigma') \models \text{nonLocal}(G_{t'}^s \vee G_{t'}^m)$, we need to prove $\Sigma' \models p_{\text{bc13}}(t)$. By definition of $\text{nonLocal}(G_{t'}^s \vee G_{t'}^m)$ we know

$$\text{nonLocal}(G_{t'}^s \vee G_{t'}^m) \Leftrightarrow \text{nonLocal}(G_{t'}^s \vee G_{t'}^m) * \text{IdSpec}.$$

Apply Lm. 105 and Lm. 101 we know $\Sigma' \models p_t * \text{tid} \mapsto (\text{DELIVER}, t, ts)$. As the message pool does not shrink during the action, and the evaluation of Nodes is always equal to $\text{Nok} \uplus \text{Nbad}$, we know $\Sigma' \models \text{quorumECHO}(t, ts, \text{res})$. Thus we only need to prove Σ' holds for the following:

$$\exists A. \text{isPool}(A) \wedge t \in \text{Nok} \Rightarrow A(t, ts) = (\text{res}, _) \quad (\text{goal1})$$

$$\exists A. \text{isPool}(A) \wedge \forall N. A(t, ts) = (\text{res}, N) \Rightarrow \text{tid} \notin N \quad (\text{goal2})$$

$$\exists A. \text{isPool}(A) \wedge \forall v. A(t, ts) = (v, _) \Rightarrow v = \text{res} \quad (\text{goal3})$$

To prove (*goal1*), as $\text{nonLocal}(G_t^s, \vee G_{t'}^m)$ does not change local state, discuss on t : if $\Sigma \models t \in \text{Nok}$, then we know $\Sigma \models A(t, ts) = (\text{res}, _)$, also as action does not remove message or map existing (t, ts) pair to new value in abstract pool, we know $A(t, ts) = (\text{res}, _)$ is preserved; if $\Sigma \models t \notin \text{Nok}$, then $\Sigma' \models t \notin \text{Nok}$, we prove the goal trivially.

(*goal2*) is directly from that action only add t' into reception set of abstract pool message and $t \neq t'$.

To prove (*goal3*), discuss on cases: (note that some of the deduction use condition “action does not change local state” implicitly)

- If $\Sigma \models t \in \text{Nok}$, then from (*goal1*), which we have proved, we know $\Sigma \models A(t, ts) = (\text{res}, _)$. Then as action won't add new message with t, ts into abstract pool (by definition of *PoolBroadcast*), thus the goal is proved.
- If $\Sigma \models t \in \text{Nbad}$ and $\Sigma \models (t, ts) \notin \text{dom}(A)$, then by definition of *PoolFaulty* and *faultyRelated*, the newly introduced faulty node's message also has a quorum of corresponding *ECHO* message. As we have proved that $\text{quorumECHO}(t, ts, \text{res})$ is preserved in Σ' , if the new faulty message is also marked by t, ts , then apply Lm. 95 we know its value is equal to res , thus the goal is proved.
- If $\Sigma \models t \in \text{Nbad}$ and $\Sigma \models \exists v. A(t, ts) = (v, _)$, then as $\Sigma \models \text{legalHelpFlinPrec}$ we know $\Sigma \models A(t, ts) = (\text{res}, _)$. Then as the action does not rewrite (t, ts) to new value (by definition of *PoolFaulty*), we know $\Sigma' \models A(t, ts) = (\text{res}, _)$ and it's unique.

□

7 Example 2: Single-writer / Multiple-reader Protocol

7.1 Single-writer / Multiple-reader Register Array Abstraction

SWMR protocol [Malkhi et al. 1998] implements a single-writer / multiple-reader (SWMR) register array abstraction. It has two methods: $\text{write}(v)$ allows the initiator to write v to its own slot, where the slot is identified by the writer's node ID, and $\text{read}(loc)$ allows the current node to read back the value last written to the slot loc . Note that if the writer is Byzantine, then the correct readers can read arbitrary values.

$$\theta \in \{\text{REG}\} \rightarrow (\text{Nat} \rightarrow \text{Val})$$

Definition 7.1 (Object specification of SWMR).

$$\begin{aligned} \Gamma_{\text{swmr}} &\stackrel{\text{def}}{=} \{\text{write} \leadsto \text{WR}, \text{read} \leadsto \text{RD}\} \\ \text{WR} &\stackrel{\text{def}}{=} \lambda t. \lambda v. \theta. (\text{ok}, \theta\{\text{REG} \leadsto \theta(\text{REG})\{t \leadsto v\}\}) \\ \text{RD} &\stackrel{\text{def}}{=} \lambda t. \lambda loc. \theta. (\theta(\text{REG})(loc), \theta) \end{aligned}$$

7.2 The Algorithm

The concrete implementation of the SWMR protocol is shown in Fig. 38. Each node t maintains a global view of the SWMR array by keeping a local mReg that maps node IDs to write messages (wID, v) . Initially, mReg maps all node IDs in Nodes to $(0, 0)$ (line 2). Note that we introduce $t_1, \dots, t_m$ and let $\{t_1, \dots, t_m\}$ be the evaluation of Nodes to express the initialization of mReg.

The implementation in Fig. 38 proceeds as follows:

- When a node t calls $\text{write}(v)$, it chooses a write ID (wID , initially 0) that greater than any IDs it has chosen in the past, then sends protocol-level WRITE message $(\text{WRITE}, (wID, v), t)$ to all nodes (line 6). After that, it waits until its Vote_{wr} contains sufficiently many votes for current write (line 7). The threshold TH is defined in Fig. 37, where F represents the maximum number of nodes that can be faulty in the system. Vote_{wr} is a set of nodes already aware of t writes v . A write operation will always return ok if it is finished.
- When a node receives an $(\text{WRITE}, (wID, v), t)$ message, it first compares (wID, v) with the local $\text{mReg}(t)$ in lexicographical order, and (wID, v) will be assigned to $\text{mReg}(t)$ if it is greater. This procedure is encapsulated in the inline function $\text{update}(loc, wrMsg)$ (line 67), where loc refers to the local slot that is to be compared and possibly replaced, $wrMsg$ is the competitor. Regardless of the comparison result, t then respond $(\text{WRITE-OK}, wID, t)$ to the requester.
- When a node receives an $(\text{WRITE-OK}, wID, t')$, or $(\text{WRITE-BACK-OK}, rID, t')$ message, t' is added to $\text{Vote}_{wr}/\text{Vote}_{wb}$ if wID/rID is equal to the current wID/rID .
- When a node t calls $\text{read}(loc)$, it chooses a read ID (rID , initially 0) that greater than any IDs it has chosen in the past, then send $(\text{READ}, rID, loc, t)$ to all nodes (line 16). It waits until its Vote_{rd} contains sufficiently many votes for current READ request (line 17), and pick the lexicographically maximum write message (maxMsg) from WrMsgs . Similar to Vote_{wr} , Vote_{rd} is a vote set record nodes which respond a READ-OK response, while WrMsgs is a set of write messages, recording specific contents in these responses. After that t sends $(\text{WRITE-BACK}, rID, \text{maxMsg}, loc, t)$ to all nodes (line 22), trying to re-write maxMsg . Then it waits until its Vote_{wb} contains sufficiently many votes (line 23), where node IDs Vote_{wb} show these nodes are aware of the re-write. At last the operation returns the value field of maxMsg .
- When a node t receives a $(\text{READ}, rID, loc, t')$, it sends $(\text{READ-OK}, rID, \text{mReg}(loc), loc, t)$ back. Note that the loc field in READ-OK message is not redundant, similar to the t field in a WRITE

$$\begin{aligned}
F &\equiv \lfloor \frac{|\text{Nodes}| - 1}{3} \rfloor \\
\text{TH} &\equiv |\text{Nodes}| - F \\
\text{wrMsg} \geq_{\text{lex}} \text{wrMsg}' &\equiv (\text{wrMsg.wID} = \text{wrMsg'.wID} \ \&\& \ \text{wrMsg.v} > \text{wrMsg'.v}) \\
&\quad \parallel \text{wrMsg.wID} > \text{wrMsg'.wID} \\
\text{LEGAL}(x) &\equiv \exists wID, v. x = (wID, v)
\end{aligned}$$

Fig. 37. Macros

message and loc field in a WRITE-BACK message, it provides unique identity information for $\text{mReg}(loc)$, thus preventing faulty nodes from forging READ-OK reply.

- When a node t receives a (READ-OK, rID , wrMsg , loc , t'), if rID is equal to current rID , it adds t to Vote_{rd} and wrMsg to WrMsgs ; otherwise it does nothing.
- When a node t receives a (WRITE-BACK, rID , wrMsg , loc , t'), similar to the case when receiving WRITE message, it compares wrMsg with local $\text{mReg}(loc)$, and assign former to the latter if wrMsg is greater. Regardless, it responds (WRITE-BACK-OK, rID , t) back after that.

The restrictions on messages that can be sent by faulty nodes are as defined in Def. 7.2. It means given faulty node ID set $\mathcal{N}_{\text{bad}}$ and current message pool M :

- if a faulty node send a WRITE message (WRITE, wrMsg , t), then $t \in \mathcal{N}_{\text{bad}}$ or $(\text{wrMsg}, t) \in \text{sentWrMsg}(M)$ holds, where $\text{sentWrMsg}(M)$ represents the set of write messages already present in current message pool M ;
- if a faulty node send a WRITE-BACK or READ-OK message, formally (WRITE-BACK, rID , wrMsg , loc , t) or (READ-OK, rID , wrMsg , loc , t), then at least one of the following holds:
 - $t \in \mathcal{N}_{\text{bad}} \wedge loc \in \mathcal{N}_{\text{bad}}$;
 - $t \in \mathcal{N}_{\text{bad}} \wedge (\text{wrMsg}, loc) \in \text{sentWrMsg}(M)$;
 - this message belongs to M ;
- if a faulty node send a WRITE-OK, WRITE-BACK-OK, READ message, then the message's sender must be a faulty node's ID, or this message belongs to M ;
- a faulty node can send any other arbitrary message.

Definition 7.2. $\zeta_{\text{swmr}} \stackrel{\text{def}}{=} \lambda \mathcal{N}_{\text{bad}}, M.$

$$\begin{aligned}
&\{(\text{WRITE}, \text{wrMsg}, t) \mid (t \notin \mathcal{N}_{\text{bad}} \wedge (\text{wrMsg}, t) \notin \text{sentWrMsg}(M))\} \\
&\cup \left\{ (\text{WRITE-BACK}, rID, \text{wrMsg}, loc, t) \mid \neg(t \in \mathcal{N}_{\text{bad}} \wedge (loc \in \mathcal{N}_{\text{bad}} \vee (\text{wrMsg}, loc) \in \text{sentWrMsg}(M))) \right. \\
&\quad \left. \wedge (_, (\text{WRITE-BACK}, rID, \text{wrMsg}, loc, t)) \notin M \right\} \\
&\cup \left\{ (\text{READ-OK}, rID, \text{wrMsg}, loc, t) \mid \neg(t \in \mathcal{N}_{\text{bad}} \wedge (loc \in \mathcal{N}_{\text{bad}} \vee (\text{wrMsg}, loc) \in \text{sentWrMsg}(M))) \right. \\
&\quad \left. \wedge (_, (\text{READ-OK}, rID, \text{wrMsg}, loc, t)) \notin M \right\} \\
&\cup \{(\text{WRITE-OK}, wID, t) \mid t \notin \mathcal{N}_{\text{bad}} \wedge (_, (\text{WRITE-OK}, wID, t)) \notin M\} \\
&\cup \{(\text{READ}, rID, loc, t) \mid t \notin \mathcal{N}_{\text{bad}} \wedge (_, (\text{READ}, rID, loc, t)) \notin M\} \\
&\cup \{(\text{WRITE-BACK-OK}, wID, t) \mid t \notin \mathcal{N}_{\text{bad}} \wedge (_, (\text{WRITE-BACK-OK}, wID, t)) \notin M\}
\end{aligned}$$

$$\text{where } \text{sentWrMsg}(M) \stackrel{\text{def}}{=} \left\{ (\text{wrMsg}, t) \mid (_, (\text{WRITE}, \text{wrMsg}, t)) \in M \vee (_, (\text{WRITE-BACK}, _, \text{wrMsg}, t, _)) \in M \vee (_, (\text{READ-OK}, _, \text{wrMsg}, t, _)) \in M \right\}$$

7.3 LP Location and Instrumented Code

7.3.1 LP of Write. The LP of $\text{write}(v)$ is fixed, and is always the point at which the TH_{nok} -th benign node sends corresponding WRITE-OK or WRITE-BACK-OK message, where TH_{nok} is the smallest integer that guarantees coverage of more than half the elements in set Nok , formally as follows:

$$\text{TH}_{\text{nok}} \equiv \lceil \frac{|\text{Nok}| + 1}{2} \rceil$$

```

1: var wID, rID:= 0;
2: var mReg := { $t_1 \rightsquigarrow (0, 0), \dots, t_m \rightsquigarrow (0, 0)$ };
3: operation write(v) {
4:   wID := wID + 1;
5:   Votewr :=  $\emptyset$ ;
6:   send(Nodes, (WRITE, (wID, v), tid));
7:   while (|Votewr| < TH) do
8:     skip;
9:   end while
10:  return ok;
11: }
12: operation read(loc) {
13:  rID := rID + 1;
14:  Voterd :=  $\emptyset$ ;
15:  WrMsgs :=  $\emptyset$ ;
16:  send(Nodes, (READ, rID, loc, tid));
17:  while (|Voterd| < TH) do
18:    skip;
19:  end while
20:  maxMsg := max(WrMsgs);
21:  Votewb :=  $\emptyset$ ;
22:  send(Nodes, (WRITE-BACK, rID, maxMsg, loc,
23:    tid));
24:  while (|Votewb| < TH) do
25:    skip;
26:  end while
27:  return maxMsg.v;
28: }
29: process server {
30:  while (true) do
31:    msg := recv();
32:    if (msg.tag = WRITE-OK) then
33:      ( $\_$ , wID', t) := msg;
34:      if (wID = wID') then
35:        Votewr := Votewr  $\cup$  { t };
36:      end if
37:    else if (msg.tag = READ-OK) then
38:      ( $\_$ , rID', wrMsg, loc', t) := msg;
39:      WrMsgs := WrMsgs  $\cup$  { wrMsg };
40:      Voterd := Voterd  $\cup$  { t };
41:    end if
42:    else if (msg.tag = WRITE-BACK-OK) then
43:      ( $\_$ , rID', t) := msg;
44:      if (rID' = rID) then
45:        Votewb := Votewb  $\cup$  { t };
46:      end if
47:    else if (msg.tag = WRITE) then
48:      ( $\_$ , wrMsg, t) := msg;
49:      if (LEGAL(wrMsg)) then
50:        update(t, wrMsg);
51:        send({ t }, (WRITE-OK, wrMsg.wID,
52:          tid));
53:      end if
54:    else if (msg.tag = READ) then
55:      ( $\_$ , rID', loc', t) := msg;
56:      if (loc'  $\in$  dom(mReg)) then
57:        send({ t }, (READ-OK, rID',
58:          mReg(loc'), loc', tid));
59:      end if
60:    else if (msg.tag = WRITE-BACK) then
61:      ( $\_$ , rID', wrMsg, loc', t) := msg;
62:      if (LEGAL(wrMsg)) then
63:        update(loc', wrMsg);
64:        send({ t }, (WRITE-BACK-OK, rID',
65:          tid));
66:      end if
67:    end if
68:  end while
69: }
70: inline update(loc, wrMsg) {
71:  if (wrMsg >lex mReg(loc)) then
72:    mReg := mReg { loc  $\rightsquigarrow$  wrMsg };
73:  end if
74: }

```

Fig. 38. The SWMR protocol

For example, if the size of N_{ok} equals to 4, then TH_{nok} is 3; if equals to 5, then TH_{nok} is still 3. In following discussion, we refer to this point as wLP. Now we show that the LP cannot be located before or after wLP.

- If the LP is earlier than wLP, denoted as testLP, then we construct a concrete execution example (Fig. 39) and show contradiction. In this execution, node 1 invokes write(11) in the first place, then send WRITE message to all nodes following protocol specification. The blue arrows here reveal the flow of messages. After testLP, read(1) is initiated by node 2. During

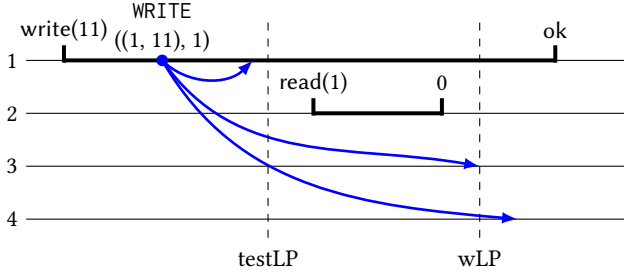


Fig. 39. LP set too early

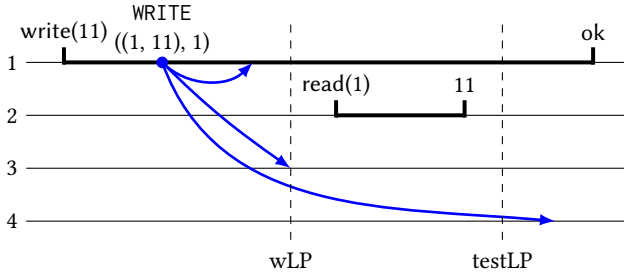


Fig. 40. LP set too late

the read operation, node 2 collect READ-OK messages from node 2, 3, 4, thus it conclude 0 (the initial value) to be its return value, as none of these nodes have been informed by the latest write. After read returns, node 1 collects response from node 3 and 4 thus reach a quorum, then it returns ok.

As the LP of a read occurs between its invocation and response, and testLP occurs before this point, we have a contradiction.

- If the LP is latter than wLP, similarly, we construct an execution (Fig. 40) and show contradiction. In this execution, node 1 first invokes write(11) and send WRITE messages to all nodes. After node 1 and 3 receive the WRITE message, node 2 initiate read(1). During the read operation, node 2 collect READ-OK messages from node 2, 3, 4, thus it conclude 11 to be its return value, as node 3 has been informed by the latest write. Then node 1 collects response from node 4 thus reach a quorum, then it returns ok.

As the LP of a read occurs between invocation and response, and testLP occurs after it, we have the contradiction.

7.3.2 LP of Read. For a read operation whose argument is a benign location (slot written by some benign node), the LP is non-fixed and could be in one of the following locations:

- at the beginning of read;
- just after a concurrent write is linearized.

We use the execution in Fig. 41 to explain the LP of read more specifically. Suppose the initial value of slot 1 is v_0 , and the vertical dashed line marked by $wLP(v)$ is the LP of write(v). Let l_0 be the time point after read(1) has started but before $wLP(v_1)$, and let l_i ($i \in \{1, \dots, n\}$) be the time point immediately after $wLP(v_i)$. Following above LP illustration, $l_0, l_1, \dots, l_n$ are all possible LP, where each l_i corresponds to the result v_i . The actual LP is ultimately determined by the return value v .

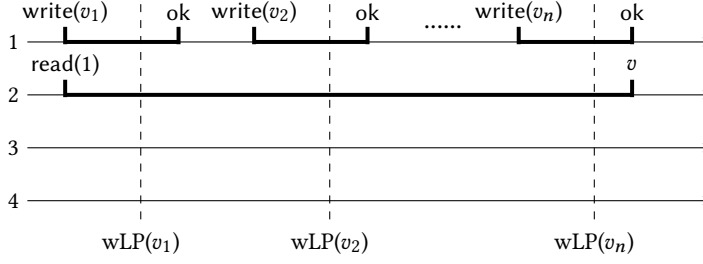


Fig. 41. A read operation occurs concurrently with multiple writes.

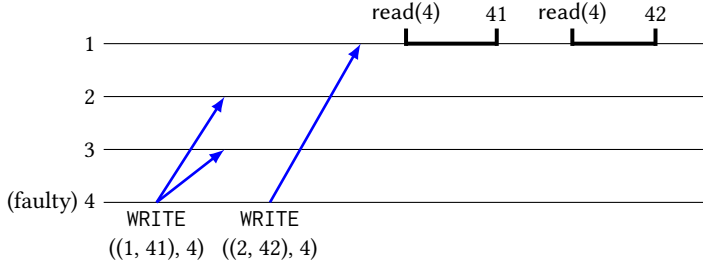


Fig. 42. Node 1 observes node 4 wrote 41 and 42 through WRITE messages indirectly.

For a read operation whose argument is a faulty location, we need to first specify what concrete behaviors performed by faulty nodes are considered write operations from a benign node's perspective. Generally, a benign node observes faulty node's potential writes through

1. the WRITE and WRITE-BACK messages sent by the faulty nodes indirectly;
2. the READ-OK responses received from faulty nodes directly.

We give execution examples draw in Fig. 42 and Fig. 43 to illustrate these two distinct situations in detail respectively.

In concrete execution shown in Fig. 42, faulty node 4 first send (WRITE, (1, 41), 4) to node 2 and 3, and send (WRITE, (2, 42), 2) to node 1. After all these nodes have received the message, node 1 initiates a read operation on 4. The READ request of this operation collects READ-OK messages from node 2,3,4 (the message flow inside the operation is omitted). They both respond (1, 41), thus the first read of 1 returns 41.

After that node 1 initiates the second read on 4. This time the READ request collects responses from node 1, 2, 3. As node 1 have been informed of (WRITE, (2, 42), 4), it responds (2, 42). The other two nodes respond (1, 41). Thus the second read of 1 returns 42.

The operations still return 41 and then 42 if the WRITE messages are substituted to WRITE-BACK messages with identical parameters. Thus the execution shows that node 1 observes node 4 wrote 41 and 42 through corresponding WRITE or WRITE-BACK messages indirectly.

In concrete execution shown in Fig. 43, node 1 initiates a read on 4. The READ request of this operation collects READ-OK messages from node 2,3,4. As node 2, 3 have not been informed by any write from 4, they respond the initial value (0, 0). Node 4 responds (1, 41). Thus the maximum write message is (1, 41). Then node 1 perform a re-write, sending WRITE-BACK message with (1, 41) to node 2,3,4 and collects responses. At last the operation returns 41. This execution shows that node 1 observes node 4 wrote 41 through a READ-OK message directly.

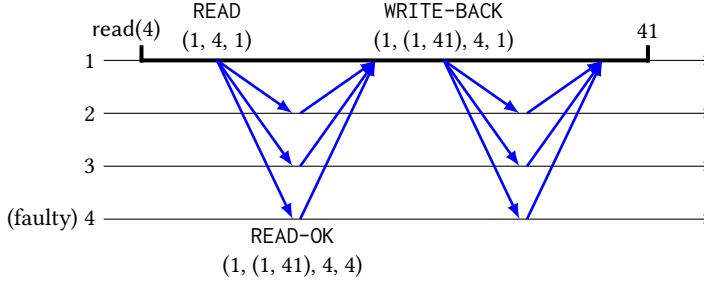


Fig. 43. Node 1 observes node 4 wrote 41 through a READ-OK message directly.

In either case, we can linearize a read of faulty location by first helping the corresponding write, then linearizing the read itself. Since the return value of a read operation can only be determined at the end of READ-OK responses collection, the help and self-linearization can only be inserted after that. We place this procedure immediately before the return point.

Readers may observe that this approach introduces redundant helps. In the concrete execution shown in Fig. 44, faulty node 4 sends (WRITE, (1, 41), 4) to other nodes. It follows that the two reads initiated by node 1 both returns 41. In our approach, each read helps a write of 41 (marked by dashed line and $\text{FLP}(41)$). While in concrete side, faulty node 4 appears to have performed only a single write. This is acceptable as the specification of SWMR provides sufficient freedom for faulty nodes to exhibit arbitrary behavior.

7.3.3 Instrumented Code. To help specify the LPs for write, we introduce global auxiliary variables wrHist and absWrCnt . wrHist records write history of each benign node. It maps benign node's ID to a set of write messages. We introduce $t_{b_1}, \dots, t_{b_n}$ and let the evaluation of Nok be $\{t_{b_1}, \dots, t_{b_n}\}$ to express the initialization of wrHist . Initially, the history sets of all benign nodes contains only the initial value (0, 0). absWrCnt counts WRITE-OK and WRITE-BACK-OK votes for each write. It maps writer's ID and write message to a set of node IDs, i.e. votes, initially empty set.

As shown by the yellow-backgrounded code in Proof outline. 2, after WRITE message being sent, all corresponding WRITE-OK or WRITE-BACK-OK responses is tracked (line 100, line 116). These votes are recorded by absWrCnt . The write linearize once absWrCnt shows that a quorum set of votes is collected. We define the following macros to judge if adding vote t to $\text{absWrCnt}(\text{loc}, \text{wrMsg})$ result in a new quorum:

$$\text{checkQuorum}(t)(\text{loc}, \text{wrMsg}) \equiv (t \notin \text{absWrCnt}(\text{loc}, \text{wrMsg}) \wedge |\text{absWrCnt}(\text{loc})(\text{wrMsg}) \cup \{t\}| = \text{TH}_{\text{nok}}) \\ \vee ((\text{loc}, \text{wrMsg}) \notin \text{dom}(\text{absWrCnt}) \wedge |\text{Nok}| \leq 1)$$

The instrumented code also ensures that the linearization happens at most once. After that we add the write message to $\text{wrHist}(t)$, where t is the writer's ID, marking the write linearized.

To help specify the LPs for read, we introduce auxiliary variables RdOps . For read of benign location, at the beginning of read we add read record to RdOps (line 31), where a read record contains reader's ID t , the target location loc and current write message of slot loc , which we also refer to it as the bottom write message of this read. Then we insert **trylin** as this is a potential LP (line 32). Then we insert **trylin** after all concurrent write LPs (line 143 - 145). At last we commit the return value and delete read record from RdOps (line 53-54).

For read of faulty location, we simply need to insert a write of return value and then linearize the read before return (line 56-57). This process does not involve any auxiliary variable.

Algorithm 2

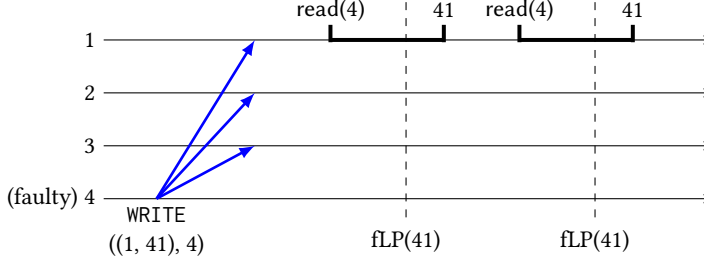


Fig. 44. Redundant helps

```

1: var wID, rID := 0;
2: var mReg := {t1 ↦ (0, 0), ..., tm ↦ (0, 0)};
3:
4: global var absWcnt, RdOps := ∅;
5: global var wrHist := {tb1 ↦ {(0, 0)}, ..., tbn ↦ {(0, 0)}};
6:
7: operation write(v) {
8:   ① {pt ∧ (tid ↦ (WR, v) * true)}
9:   wID := wID + 1;
10:  ② {pt ∧ (tid ↦ (WR, v) * true) ∧ maxSentWID(wID - 1)}
11:  Votewr := ∅;
12:  ③ {pt ∧ (tid ↦ (WR, v) * true) ∧ maxSentWID(wID - 1) ∧ writeVotesProp}
13:  send(Nodes, (WRITE, (wID, v), tid));
14:  ④ {
    {
      pt ∧ writeVotesProp ∧ sent(⌊, (WRITE, (wID, v), tid))
      ∧ (
        (unfinish(tid, (wID, v)) ∧ tid ↦ (WR, v) * true)
        ∨ (finished(tid, (wID, v)) ∧ tid ↦ (end, ok) * true)
      )
    }
  }
15:  while (|Votewr| < TH) do
16:    skip;
17:  end while
18:  ⑤ {pt ∧ (tid ↦ (end, ok) * true)}
19:  return ok;
20: }
21: operation read(loc) {
22:  ⑥ {pt ∧ (tid ↦ (RD, loc) * true)}
23:  ⑦ {pt * tid ↦ (RD, loc)}
24:  rID := rID + 1;
25:  ⑧ {pt * tid ↦ (RD, loc) ∧ maxSentRID(rID - 1)}
26:  Voterd := ∅;
27:  ⑨ {pt * tid ↦ (RD, loc) ∧ maxSentRID(rID - 1) ∧ readVotesProp}
28:  WrMsgs := ∅;
29:  ⑩ {pt * tid ↦ (RD, loc) ∧ maxSentRID(rID - 1) ∧ readRespProp}
30:  < if (loc ∈ Nok) then
31:    RdOps :=a RdOps {tid ↦ (loc, max(wrHist(loc)))};
32:    trylin(tid);
33:  end if >
34:  ⑪ {
    (readBenignLoct ∧ maxSentRID(rID - 1) ∧ bottomWrMsgProp ∧ readRespProp)
    ∨ readFaultyLoct
  }

```

```

35:   send(Nodes, (READ, rID, loc, tid));
36:   ⑫ { (readBenignLoct ∧ rIDExceedWB ∧ bottomWrMsgProp ∧ readRespProp) }
37:   while (|Voterd| < TH) do
38:     skip;
39:   end while
40:   maxMsg := max(WrMsgs);
41:   ⑬ { (readBenignLoct ∧ rIDExceedWB ∧ bottomWrMsgProp ∧ readRespProp )
42:     ∧ |Voterd| ≥ TH ∧ maxMsg = max(WrMsgs)) ∨ readFaultyLoct }
43:   ⑭ { (readBenignLoct ∧ rIDExceedWB ∧ notOutdated(maxMsg) )
44:     ∧ sent(⌊, (READ-OK, ⌊, maxMsg, loc, ⌊)) ∨ readFaultyLoct }
45:   Votewb := ⌀;
46:   ⑮ { (readBenignLoct ∧ rIDExceedWB ∧ notOutdated(maxMsg)
47:     ∧ sent(⌊, (READ-OK, ⌊, maxMsg, loc, ⌊)) ∧ writeBackVotesProp) ∨ readFaultyLoct }
48:   send(Nodes, (WRITE-BACK, rID, maxMsg, loc, tid));
49:   ⑯ { (readBenignLoct ∧ notOutdated(maxMsg) ∧ writeBackVoteSetProp )
50:     ∧ sent(⌊, (WRITE-BACK, rID, maxMsg, loc, tid)) ∨ readFaultyLoct }
51:   while (|Votewb| < TH) do
52:     skip;
53:   end while
54:   ⑰ { (readBenignLoct ∧ notOutdated(maxMsg) ∧ writeBackVoteSetProp )
55:     ∧ sent(⌊, (WRITE-BACK, rID, maxMsg, loc, tid)) ∧ |Votewb| ≥ TH }
56:   ⑱ { (readBenignLoct ∧ legalWrMsg(maxMsg)) ∨ readFaultyLoct }
57:   ⑲ {if (loc ∈ Nok) then
58:     commit(tid → (end, maxMsg.v));
59:     RdOps :=a RdOps \ {tid → RdOps(tid)};
60:   else if (loc ∈ Nbad) then
61:     flin(loc, WR, maxMsg.v);
62:     lin(tid);
63:   end if }
64:   ⑳ {pt * tid → (end, maxMsg.v)}
65:   return maxMsg.v;
66:   ㉑ {pt ∧ (tid → (end, ⌊) * true)}
67: }
68:
69: process server{
70:   ㉒ {pt}
71:   while (true) do
72:     ㉓ {pt}
73:     msg := recv();
74:     ㉔ {pt ∧ sent({tid}, msg)}
75:     if (msg.tag = WRITE-OK) then
76:       (⌊, wID', t) := msg;
77:       ㉕ {pt ∧ sent({tid}, (WRITE-OK, wID', t))}
78:       if (wID = wID') then
79:         Votewr := Votewr ∪ {t};
80:       end if
81:     ㉖ {pt}

```

```

77:   else if (msg.tag = READ-OK) then
78:     ( $\_$ , rID', wrMsg, loc', t) := msg;
79:     ②⑥ { $p_t \wedge \text{sent}(\{tid\}, (\text{READ-OK}, rID', \text{wrMsg}, \text{loc}', t))$ }
80:     if (rID' = rID && LEGAL(wrMsg) && loc' = loc) then
81:       WrMsgs := WrMsgs  $\cup$  {wrMsg};
82:       Voterd := Voterd  $\cup$  {t};
83:     end if
84:     ②⑦ { $p_t$ }
85:   else if (msg.tag = WRITE-BACK-OK) then
86:     ( $\_$ , rID', t) := msg;
87:     ②⑧ { $p_t \wedge \text{sent}(\{tid\}, (\text{WRITE-BACK-OK}, rID', t))$ }
88:     if (rID' = rID) then
89:       Votewb := Votewb  $\cup$  {t};
90:     end if
91:     ②⑨ { $p_t$ }
92:   else if (msg.tag = WRITE) then
93:     ( $\_$ , wrMsg, t) := msg;
94:     ③① { $p_t \wedge \text{sent}(\_, (\text{WRITE}, \text{wrMsg}, t))$ }
95:     if (LEGAL(wrMsg)) then
96:       update(t, wrMsg);
97:       <
98:       ③① {  $\left( \begin{array}{l} p_t \wedge \text{sent}(\_, (\text{WRITE}, \text{wrMsg}, t)) \wedge \text{mReg}(t) \geq_{\text{lex}} \text{wrMsg} \wedge \\ t \in \text{Nok} \wedge \text{checkQuorum}(tid)(t, \text{wrMsg}) \Rightarrow \\ (t \mapsto (\text{WR}, \text{wrMsg}.v) * \text{true}) \wedge \text{wrMsg} >_{\text{lex}} \max(\text{wrHist}(t)) \end{array} \right)$  }
99:       send({t}, (WRITE-OK, wrMsg.wID, tid));
100:       linWrAndHelpRd(t, wrMsg);
101:     end if
102:     ③② { $p_t$ }
103:   else if (msg.tag = READ) then
104:     ( $\_$ , rID', loc', t) := msg;
105:     ③③ { $p_t \wedge \text{sent}(\_, (\text{READ}, rID', \text{loc}', t))$ }
106:     if (loc'  $\in$  dom(mReg)) then
107:       send({t}, (READ-OK, rID', mReg(loc'), loc', tid));
108:     end if
109:     ③④ { $p_t$ }
110:   else if (msg.tag = WRITE-BACK) then
111:     ( $\_$ , rID', wrMsg, loc', t) := msg;
112:     ③⑤ { $p_t \wedge \text{sent}(\_, (\text{WRITE-BACK}, rID', \text{wrMsg}, \text{loc}', t))$ }
113:     if (LEGAL(wrMsg)) then
114:       update(loc', wrMsg);
115:       < send({t}, (WRITE-BACK-OK, rID', tid));
116:       linWrAndHelpRd(loc', wrMsg);
117:     end if
118:     ③⑥ { $p_t$ }
119:   end if
120:   ③⑦ { $p_t$ }
121: end while
122: ③⑧ {true}
123: }
124:
125: inline update(loc, wrMsg) {

```

```

126:   if (wrMsg >lex mReg(loc)) then
127:     mReg := mReg{ loc  $\rightsquigarrow$  wrMsg };
128:   end if
129: }
130:
131: inline linWrAndHelpRd(loc, wrMsg) {
132:   (39) {

$$\left( \begin{array}{l} \text{sent}(\{t\}, (\text{WRITE-OK}, \text{wrMsg.wID}, \text{tid})) \wedge \\ \left( \begin{array}{l} \text{loc} \in \text{Nok} \wedge \text{checkQuorum}(\text{tid})(\text{loc}, \text{wrMsg}) \\ \wedge (\text{loc} \rightsquigarrow (\text{WR}, \text{wrMsg.v}) * \text{true}) \wedge \text{wrMsg} >_{\text{lex}} \max(\text{wrHist}(\text{loc})) \\ \wedge p_t \setminus \{\text{absInv}\} \wedge \text{absInv} \setminus \{\text{finishedWrites}\} \wedge \text{finished}(\text{loc}, \text{wrMsg}) \\ \wedge \text{presvExcept}(\text{loc}, \text{wrMsg}) \end{array} \right) \end{array} \right) \wedge \\ \vee (\text{loc} \notin \text{Nok} \vee \neg \text{checkQuorum}(\text{tid})(\text{loc}, \text{wrMsg}) \wedge p_t) \end{array} \right)$$

133:   if (loc  $\in$  Nok) then
134:     quorumOk :=a checkQuorum(tid)(loc, wrMsg, absWrCnt);
135:     if ((loc, wrMsg)  $\notin$  dom(absWrCnt)) then
136:       absWrCnt :=a absWrCnt{ (loc, wrMsg)  $\rightsquigarrow$   $\emptyset$  };
137:     end if
138:     absWrCnt :=a absWrCnt{ (loc, wrMsg)  $\rightsquigarrow$  absWrCnt  $\cup$  { tid } };
139:     if (quorumOk) then
140:       (40) {

$$\left( \begin{array}{l} \text{loc} \in \text{Nok} \wedge (\text{loc} \rightsquigarrow (\text{WR}, \text{wrMsg.v}) * \text{true}) \wedge \text{wrMsg} >_{\text{lex}} \\ \max(\text{wrHist}(\text{loc})) \wedge p_t \setminus \{\text{absInv}\} \wedge \text{absInv} \setminus \{\text{finishedWrites}\} \\ \wedge \text{finished}(\text{loc}, \text{wrMsg}) \wedge \text{presvExcept}(\text{loc}, \text{wrMsg}) \end{array} \right)$$

141:       lin(loc);
142:       (41) {

$$\left( \begin{array}{l} \text{loc} \in \text{Nok} \wedge ((\text{REG} \Rightarrow (\bigvee_{t \in \text{Nok} \setminus \{\text{loc}\}} t \rightsquigarrow \max(\text{wrHist}(x)).v) \vee (\bigvee_{t \in \text{Nbad}} t \rightsquigarrow \\ \_)) \vee \{\text{loc} \rightsquigarrow \text{wrMsg.v}\}) * \text{pendReads}(\text{RdOps}) * \text{true}) \wedge \text{unfinishWrites} \\ \wedge \text{msgPoolInv} \wedge (\text{loc} \rightsquigarrow (\text{end}, \text{ok}) * \text{true}) \wedge \text{wrMsg} > \max(\text{wrHist}(\text{loc})) \\ \wedge \text{finished}(\text{loc}, \text{wrMsg}) \wedge \text{presvExcept}(\text{loc}, \text{wrMsg}) \end{array} \right)$$

143:       foreach t' in dom(RdOps) do
144:         if (RdOps(t').loc = loc) then trylin(t'); end if
145:       end for
146:       (42) {

$$\left( \begin{array}{l} \text{loc} \in \text{Nok} \wedge \text{unfinishWrites} \wedge \text{msgPoolInv} \wedge (\text{loc} \rightsquigarrow (\text{end}, \text{ok}) * \text{true}) \\ \wedge \text{wrMsg} > \max(\text{wrHist}(\text{loc})) \wedge \text{finished}(\text{loc}, \text{wrMsg}) \\ \wedge \text{presvExcept}(\text{loc}, \text{wrMsg}) \\ \left( \begin{array}{l} \text{REG} \Rightarrow (\bigvee_{t \in \text{Nok} \setminus \{\text{loc}\}} t \rightsquigarrow \max(\text{wrHist}(x)).v) \vee (\bigvee_{t \in \text{Nbad}} t \rightsquigarrow \_) \\ \vee \{\text{loc} \rightsquigarrow \text{wrMsg.v}\} \\ * \left( \begin{array}{l} \exists O_1, O_2. O_1 \vee O_2 = \text{RdOps} \wedge (\forall t. (t \in \text{dom}(O_1) \Rightarrow O_1(t).\text{loc} \neq \text{loc}) \\ \wedge (t \in \text{dom}(O_2) \Rightarrow O_2(t).\text{loc} = \text{loc})) \wedge \text{pendReads}(O_1) \\ * \bigotimes_{t \in \text{dom}(O_2)} R(t, \text{loc}, O_2(t).\text{wrMsg}) \oplus t \rightsquigarrow (\text{end}, \text{wrMsg.v}) \end{array} \right) \end{array} \right) \wedge \\ * \text{true} \end{array} \right)$$

147:       wrHist :=a wrHist{loc  $\rightsquigarrow$  wrHist(loc)  $\cup$  {wrMsg}};
148:       (43) {pt}
149:     end if
150:   end if
151:   (44) {pt}
152: }

```

7.4 Specification

We define the object invariant p in Fig. 45. p consists of absInv and msgPoolInv . The former mainly states how speculation set is related with concrete message pool. It primarily shows three relationship: (1) between abstract SWMR registers and message pool, (2) between pending reads

and message pool, and (3) between pending writes and message pool. While the latter mainly encodes the property of message pool itself.

`absReg` specifies that the abstract register at a benign location t is always the value of maximum pair in $\text{wrHist}(t)$, whereas the faulty node's abstract register is arbitrary value. `wrHist` records all historical abstract register states with their write IDs, thus capturing the write sequence. Additionally, `finishedWrites` help to relate `wrHist` with message pool, it states the equivalence between `wrMsg` being in $\text{wrHist}(t)$ and the message pool containing a benign quorum (of size TH_{nok}) set of responses to WRITE or WRITE-BACK request that t write `wrMsg`. Here, `finished( $t, \text{wrMsg}$ )` asserts the existence of such benign quorum of responses. In the remainder of this section, we simply refer to this as a finished write, distinguishing it from write operations that return from the call.

Not all writes are finished. After $(\text{WRITE}, \text{wrMsg}, t)$ request being sent, there is a period where we cannot find such quorum of responses in message pool. We refer to this as an unfinished write, formally `unfinish( $t, \text{wrMsg}$ )`. `unfinishWrites` says if there is an unfinished write in message pool, then a corresponding abstract unfinished write is in speculation set. Also, the write message `wrMsg` is larger than $\max(\text{wrHist}(t))$, i.e. the latest linearized write message of t .

`Rs(RdOps)` relates every read record in `RdOps` with an abstract pending read, and it specifies that such abstract reads may return any value ranging from the corresponding bottom write message up to the latest linearized write message in `wrHist`. For convenience, we introduce new expression $E \uparrow E'$ and its evaluation in defining sub-definition $R(x, \text{loc}, \text{wrMsg})$:

$$\llbracket E \uparrow E' \rrbracket_{\sigma} \stackrel{\text{def}}{=} \{v \mid v \in \llbracket E \rrbracket_{\sigma} \wedge v \geq \llbracket E' \rrbracket_{\sigma}\}$$

In `megPoolInv`, `maxSentWID(wID)/maxSentRID(rID)` says that no WRITE/READ message sent by current node contains a write/read ID larger than the local wID/rID. From the former we know that before sending WRITE message (line 13), `finished(tid, (wID, v))` is impossible. By the latter we can derive that `bottomWrMsgProp` (defined in Fig. 48) holds after READ being sent (line 35).

`bottomWrMsgProp` is crucial in reasoning that after concluding `maxMsg` (line 40), the write operation of `maxMsg` at benign location `loc` must have been initiated, and this operation is either linearized or still unfinished. The derivation is formally shown in proof of Lm. 114, and `wrMsgHasLinearizedOrLarger` mentioned above is also employed in the proof. Note that all benign locations in `REG` regard the initial value $(0, 0)$ as linearized by default.

`mRegSource` helps to prove `flowWrMsg` is preserved through execution. It says the content of local variable `mReg` originates from WRITE or WRITE-BACK messages.

`uniqueWrMsg` says that the current unfinished write initiated by benign node is unique. It is introduced to help establish stability of p_{swmr4} .

`respondHasUpdated` shows that replying with WRITE-OK/WRITE-BACK-OK for WRITE/WRITE-BACK request of `wrMsg` at `loc` guarantees `mReg(loc) > wrMsg` at the receiver side. This is introduced to assist proving `bottomWrMsgProp`'s stability w.r.t. the environment (formally Lm. 125).

`writeRespCount` relates `absWrCnt` with WRITE and WRITE-OK messages.

`msgFlow` captures the ordering of message. In the definition, `flowWROK`, `flowRDOK` and `flowWBOK` are trivial, while `flowWrMsg` shows that `wrMsg` in READ-OK and WRITE-BACK message can be traced back to an origin WRITE message.

The actions of the protocol are shown in Fig. 47. The method guarantee G^m includes the following:

- `SendOrIncOpID`: This covers the behaviors of method thread that does not involve speculation set. It says that WRITE, READ or WRITE-BACK message may be sent, and read or write operation ID may be monotonically increased, but the speculation set and server-only modifiable variables are preserved.

- TrylinOrCommitRd: This covers the first trylin and the commit of current read. During the action, $\text{pendReads}(\text{RdOps}) * \text{true}$ is preserved, and the reader may change its own slot in RdOps .
- LinRdFaulty: This covers the helping of faulty node's write and subsequent read, handled atomically within a single block.

While the server guarantee G^S includes the following:

- SendOrAddVotes: This covers the behaviors of server thread that does not involve speculation set. It says that WRITE-OK, READ-OK or WRITE-BACK-OK message may be sent. It further states that the sending of WRITE or WRITE-OK message does not result in a new quorum, and the READ-OK message must carry corresponding content from mReg . Also, votes may be added to vote set, but the speculation set, wrHist and method-only modifiable variables are preserved.
- LinWrHelpRd: This says the sending of WRITE-OK or WRITE-BACK-OK message result in a new quorum, thus the corresponding abstract write is linearized, and all related concurrent reads are helped.

For simplicity, we overload $[G]_p$ and abstract away direct node ID representation:

$$[G]_p \stackrel{\text{def}}{=} G \wedge (p \ltimes p) \wedge \text{IdV}(\text{readOnlyVarSet})$$

Other nodes behaviors R is the union of all other benign nodes guarantees and faulty nodes' guarantee.

The proof outline is shown in Proof outline. 2, and the proofs for assertion validity and stability are presented in the next section.

Definition 7.3. $\eta_{\text{swmr}} \stackrel{\text{def}}{=} \{(n, n') \mid n > 2n'\}$

Definition 7.4. Method-local modifiable variable set is defined as mtdVarSet , server-local modifiable variable set is defined as sevVarSet :

$$\text{sevVarSet} \stackrel{\text{def}}{=} \{\text{msg}, \text{wID}', \text{t}, \text{rID}', \text{wrMsg}, \text{loc}', \text{mReg}, \text{wrHist}, \text{absWrCnt}\}$$

$$\text{mtdVarSet} \stackrel{\text{def}}{=} \{\text{v}, \text{wID}, \text{rID}, \text{loc}, \text{maxMsg}, \text{RdOps}\}$$

p_t	$\stackrel{\text{def}}{=}$	$\text{absInv} \wedge \text{msgPoolInv} \wedge \text{readOnlyVarProp} \wedge \text{tid} = t$
absInv	$\stackrel{\text{def}}{=}$	$(\text{absRegs} * \text{pendReads}(\text{RdOps}) * \text{true}) \wedge \text{unfinishWrites} \wedge \text{finishedWrites}$
absRegs	$\stackrel{\text{def}}{=}$	$\text{REG} \Rightarrow (\biguplus_{t \in \text{Nok}} \{t \rightsquigarrow \max(\text{wrHist}(t)).v\}) \uplus (\biguplus_{t \in \text{Nbad}} \{t \rightsquigarrow _ \})$
$\text{pendReads}(O)$	$\stackrel{\text{def}}{=}$	$\bigotimes_{t \in \text{dom}(O)} R(t, O(t).loc, O(t).wrMsg)$
$R(t, loc, wrMsg)$	$\stackrel{\text{def}}{=}$	$(t \mapsto (\text{RD}, loc) \oplus \bigoplus_{wrMsg' \in (\text{wrHist}(loc) \upharpoonright wrMsg)} t \mapsto (\text{end}, wrMsg'.v))$ $\wedge wrMsg \in \max(\text{wrHist}(loc))$
unfinishWrites	$\stackrel{\text{def}}{=}$	$\forall loc \in \text{Nok}, wrMsg. \text{unfinish}(loc, wrMsg) \Rightarrow (loc \mapsto (\text{WR}, wrMsg.v) * \text{true})$ $\wedge wrMsg >_{\text{lex}} \max(\text{wrHist}(loc))$
$\text{unfinish}(loc, wrMsg)$	$\stackrel{\text{def}}{=}$	$\text{sent}(_, (\text{WRITE}, wrMsg, loc)) \wedge (\forall N. N \subseteq \text{Nok} \wedge$ $(\forall t \in N. \text{respondWs}(t, loc, wrMsg)) \Rightarrow N < \text{TH}_{\text{nok}})$
finishedWrites	$\stackrel{\text{def}}{=}$	$\forall loc \in \text{Nok}, wrMsg. \text{finished}(loc, wrMsg) \Leftrightarrow wrMsg \in \text{wrHist}(loc)$
$\text{finished}(loc, wrMsg)$	$\stackrel{\text{def}}{=}$	$\exists N. \text{quorum}(N) \wedge \forall t \in N. \text{respondWs}(t, loc, wrMsg)$
$\text{quorum}(N)$	$\stackrel{\text{def}}{=}$	$N \subseteq \text{Nok} \wedge N \geq \text{TH}_{\text{nok}}$
$\text{respondWs}(t, loc, wrMsg)$	$\stackrel{\text{def}}{=}$	$\text{sent}(_, (\text{WRITE}, wrMsg, loc)) \wedge \text{sent}(\{loc\}, (\text{WRITE-OK}, wrMsg.wID, t))$ $\vee \left(\begin{array}{l} \exists t_{\text{reader}}, rID. \text{sent}(_, (\text{WRITE-BACK}, rID, wrMsg, loc, t_{\text{reader}})) \\ \wedge \text{sent}(\{t_{\text{reader}}\}, (\text{WRITE-BACK-OK}, rID, t)) \end{array} \right)$
msgPoolInv	$\stackrel{\text{def}}{=}$	$\text{maxSentWID}(wID) \wedge \text{maxSentRID}(rID) \wedge \text{mRegSource}$ $\wedge \text{uniqueWrMsg} \wedge \text{respondHasUpdated} \wedge \text{writeRespCount} \wedge \text{msgFlow}$

Fig. 45. Object invariant of SWMR algorithm

$\text{maxSentWID}(wID)$	$\stackrel{\text{def}}{=}$	$\forall wID'. \text{sent}(_, (\text{WRITE}, (wID', _), \text{tid})) \Rightarrow wID' \leq wID$
$\text{maxSentRID}(rID)$	$\stackrel{\text{def}}{=}$	$\forall rID'. \text{sent}(_, (\text{READ}, rID', _, \text{tid})) \vee \text{sent}(_, (\text{WRITE-BACK}, rID', _, _, \text{tid}))$ $\Rightarrow rID' \leq rID$
mRegSource	$\stackrel{\text{def}}{=}$	$\forall loc. \text{mReg}(loc) \neq (0, 0) \Rightarrow \text{sent}(_, (\text{WRITE}, \text{mReg}(loc), loc)) \vee \text{sent}(_, (\text{WRITE-BACK},$ $_, \text{mReg}(loc), loc, _))$
uniqueWrMsg	$\stackrel{\text{def}}{=}$	$\forall loc, wID, v, v'. loc \in \text{Nok} \wedge \text{sent}(_, (\text{WRITE}, (wID, v), loc)) \wedge \text{sent}(_, (\text{WRITE}, (wID,$ $v'), loc)) \Rightarrow v = v'$
respondHasUpdated	$\stackrel{\text{def}}{=}$	$\forall loc, wrMsg. \text{respondWs}(\text{tid}, loc, wrMsg) \Rightarrow \text{mReg}(loc) \geq_{\text{lex}} wrMsg$
writeRespCount	$\stackrel{\text{def}}{=}$	$\forall t, loc, wrMsg. t \in \text{absWrCnt}(loc, wrMsg) \Rightarrow \text{respondWs}(t, loc, wrMsg)$
msgFlow	$\stackrel{\text{def}}{=}$	$\text{flowWrMsg} \wedge \text{flowWROK} \wedge \text{flowRDOK} \wedge \text{flowWBOK}$
flowWrMsg	$\stackrel{\text{def}}{=}$	$\forall loc, wrMsg. loc \in \text{Nok} \wedge wrMsg \neq (0, 0) \wedge (\text{sent}(_, (\text{READ-OK}, _, wrMsg, loc, _))$ $\vee \text{sent}(_, (\text{WRITE-BACK}, _, wrMsg, loc, _))) \Rightarrow \text{sent}(_, (\text{WRITE}, wrMsg, loc))$
flowWROK	$\stackrel{\text{def}}{=}$	$\forall t, t', wID. t' \in \text{Nok} \wedge \text{sent}(\{t\}, (\text{WRITE-OK}, wID, t')) \Rightarrow \text{sent}(_, (\text{WRITE}, (wID, _), t))$
flowRDOK	$\stackrel{\text{def}}{=}$	$\forall t, t', rID, loc. t' \in \text{Nok} \wedge \text{sent}(\{t\}, (\text{READ-OK}, rID, _, loc, t')) \Rightarrow \text{sent}(_, (\text{READ}, rID, loc, t))$
flowWBOK	$\stackrel{\text{def}}{=}$	$\forall t, t', rID. t' \in \text{Nok} \wedge \text{sent}(\{t\}, (\text{WRITE-BACK-OK}, rID, t')) \Rightarrow \text{sent}(_, (\text{WRITE-BACK}, rID,$ $_, _, t))$

Fig. 46. Auxiliary definitions for msgPoolInv

G_t^m	$\stackrel{\text{def}}{=}$	$(\text{SendOrIncOpID} \vee \text{TrylinOrCommitRd} \vee \text{LinRdFaulty}) \wedge \text{tid} = t \ltimes \text{tid} = t$
G_t^s	$\stackrel{\text{def}}{=}$	$(\text{SendOrAddVotes} \vee \text{LinWrHelpRd}) \wedge (\text{tid} = t \ltimes \text{tid} = t)$
R_t	$\stackrel{\text{def}}{=}$	$\bigvee_{t' \neq t} (\text{InvokeAct}(\Pi_{\text{SWMR}.L})(t') \vee G_{t'}^m \vee G_{t'}^s) \vee \text{GF}(\zeta_{\text{swmr}})$
SendOrIncOpID	$\stackrel{\text{def}}{=}$	$[\text{IdV}(\text{sevVarSet} \cup \{\text{RdOps}\}) \wedge \text{MonotonicOpID} \wedge \text{MtdMsgPoolGs}]_{\text{msgPoolInv}} \wedge \text{IdSpec}$
TrylinOrCommitRd	$\stackrel{\text{def}}{=}$	$[\text{IdV}(\text{sevVarSet}) \wedge \text{SetSelfRdOpsSlot} \wedge \text{IdM}]_{\text{msgPoolInv} \wedge \text{pendReads}(\text{RdOps})}$ $\ast (\text{tid} \mapsto (\text{RD}, _) \ltimes \text{emp} \vee \text{emp} \ltimes \text{tid} \mapsto (\text{end}, _)) \ast \text{IdSpec}$
LinRdFaulty	$\stackrel{\text{def}}{=}$	$[\text{IdV}(\text{sevVarSet} \cup \{\text{RdOps}\}) \wedge \text{IdM}]_{\text{msgPoolInv} \wedge \text{absRegs} \ast \text{pendReads}(\text{RdOps})}$ $\ast (\exists \text{loc}. \text{tid} \mapsto (\text{RD}, \text{loc}) \wedge \text{loc} \in \text{Nbad} \ltimes \text{tid} \mapsto (\text{end}, _)) \ast \text{IdSpec}$
SendOrAddVotes	$\stackrel{\text{def}}{=}$	$[\text{IdV}(\text{mtdVarSet} \cup \{\text{wrHist}\}) \wedge \text{AddVotesGs} \wedge \text{SevMsgPoolGs}]_{\text{msgPoolInv}} \wedge \text{IdSpec}$
LinWrHelpRd	$\stackrel{\text{def}}{=}$	$\exists \text{loc}, \text{wrMsg}. [\text{IdV}(\text{mtdVarSet}) \wedge \text{checkQuorum}(\text{tid})(\text{loc}, \text{wrMsg}) \ltimes \text{true}$ $\wedge \text{RespWr}(\text{loc}, \text{wrMsg}) \wedge \text{AddWrHist}(\text{loc}, \text{wrMsg})]_{\text{absRegs} \ast \text{pendReads}(\text{RdOps}) \wedge \text{msgPoolInv}}$ $\ast (\text{loc} \mapsto (\text{WR}, \text{wrMsg}.v) \ltimes \text{loc} \mapsto (\text{end}, \text{ok})) \ast \text{IdSpec}$
MonotonicOpID	$\stackrel{\text{def}}{=}$	$\exists wID, rID. wID = wID \wedge rID = rID \ltimes wID \geq wID \wedge rID \geq rID$
MtdMsgPoolGs	$\stackrel{\text{def}}{=}$	$\text{IdM} \vee [\text{SendAct}(_, (\text{WRITE}, _, \text{tid}))]_{\text{unfinishWrites}} \vee \text{SendAct}(_, (\text{READ}, _, _, \text{tid}))$ $\vee \text{SendAct}(_, (\text{WRITE-BACK}, _, _, _, \text{tid}))$
SetSelfRdOpsSlot	$\stackrel{\text{def}}{=}$	$\exists O, O'. \text{RdOps} = O \ltimes \text{RdOps} = O' \wedge \forall t \neq \text{tid}. O(t) = O'(t)$
SevMsgPoolGs	$\stackrel{\text{def}}{=}$	$\exists \text{loc}, \text{wrMsg}. \text{IdM} \vee \text{SendAct}(_, (\text{READ-OK}, _, \text{mReg}(\text{loc}), \text{loc}, \text{tid}))$ $\vee (\neg \text{checkQuorum}(\text{tid})(\text{loc}, \text{wrMsg}) \ltimes \text{true} \wedge \text{RespWr}(\text{loc}, \text{wrMsg}))$
AddVotesGs	$\stackrel{\text{def}}{=}$	$\text{IdV}(\{\text{Vote}_{\text{wr}}, \text{Vote}_{\text{rd}}, \text{WrMsgs}, \text{Vote}_{\text{wb}}\}) \vee \text{AddWROK} \vee \text{AddRDOK} \vee \text{AddWBOK}$
AddWROK	$\stackrel{\text{def}}{=}$	$\text{IdV}(\{\text{Vote}_{\text{rd}}, \text{WrMsgs}, \text{Vote}_{\text{wb}}\}) \wedge \exists t. (\text{sent}(\{\text{tid}\}, (\text{WRITE-OK}, wID, t)) \ltimes \text{true})$ $\wedge \text{Add}(\text{Vote}_{\text{wr}}, t)$
AddRDOK	$\stackrel{\text{def}}{=}$	$\text{IdV}(\{\text{Vote}_{\text{wr}}, \text{Vote}_{\text{wb}}\}) \wedge \exists t, \text{wrMsg}. (\text{sent}(\{\text{tid}\}, (\text{READ-OK}, rID, \text{wrMsg}, \text{loc}, t)) \ltimes \text{true})$ $\wedge \text{Add}(\text{Vote}_{\text{rd}}, t) \wedge \text{Add}(\text{WrMsgs}, \text{wrMsg})$
AddWBOK	$\stackrel{\text{def}}{=}$	$\text{IdV}(\{\text{Vote}_{\text{wr}}, \text{Vote}_{\text{rd}}, \text{WrMsgs}\}) \wedge \exists t. (\text{sent}(\{\text{tid}\}, (\text{WRITE-BACK-OK}, rID, t)) \ltimes \text{true})$ $\wedge \text{Add}(\text{Vote}_{\text{wb}}, t)$
Add(x_S, x)	$\stackrel{\text{def}}{=}$	$\exists y. x_S = y \ltimes x_S = y \cup \{x\}$
AddWrHist(loc, wrMsg)	$\stackrel{\text{def}}{=}$	$\exists O. \text{wrHist} = O \ltimes \text{wrHist} = O\{\text{loc} \rightsquigarrow \text{wrHist}(\text{loc}) \cup \{\text{wrMsg}\}\}$
RespWr(loc, wrMsg)	$\stackrel{\text{def}}{=}$	$(\text{sent}(_, (\text{WRITE}, \text{wrMsg}, \text{loc})) \ltimes \text{true} \wedge \text{SendAct}(\{\text{loc}\}, (\text{WRITE-OK}, \text{wrMsg}.wID, \text{tid})))$ $\vee (\exists t', rID. \text{sent}(_, (\text{WRITE-BACK}, rID, \text{wrMsg}, \text{loc}, t')) \ltimes \text{true} \wedge \text{SendAct}(\{t'\},$ $(\text{WRITE-BACK-OK}, rID, \text{tid})))$

Fig. 47. RG conditions of SWMR protocol

writeVotesProp	$\stackrel{\text{def}}{=}$ $\text{Vote}_{\text{wr}} \subseteq \text{Nodes} \wedge \forall t \in \text{Vote}_{\text{wr}}. \text{sent}(\{\text{tid}\}, (\text{WRITE-OK}, \text{wID}, t))$
readRespProp	$\stackrel{\text{def}}{=}$ $\text{readVotesProp} \wedge \text{readWrMsgsProp}$
readVotesProp	$\stackrel{\text{def}}{=}$ $\text{Vote}_{\text{rd}} \subseteq \text{Nodes} \wedge \forall t \in \text{Vote}_{\text{rd}}. \exists \text{wrMsg} \in \text{WrMsgs}. \text{sent}(\{\text{tid}\}, (\text{READ-OK}, \text{rID}, \text{wrMsg}, \text{loc}, t))$
readWrMsgsProp	$\stackrel{\text{def}}{=}$ $\forall \text{wrMsg} \in \text{WrMsgs}. \text{sent}(\{\text{tid}\}, (\text{READ-OK}, \text{rID}, \text{wrMsg}, \text{loc}, _))$
writeBackVotesProp	$\stackrel{\text{def}}{=}$ $\text{Vote}_{\text{wb}} \subseteq \text{Nodes} \wedge \forall t \in \text{Vote}_{\text{wb}}. \text{sent}(\{\text{tid}\}, (\text{WRITE-BACK-OK}, \text{rID}, t))$
rIDExceedWB	$\stackrel{\text{def}}{=}$ $\forall \text{rID}. \text{sent}(_, (\text{WRITE-BACK}, \text{rID}, _, _, \text{tid})) \Rightarrow \text{rID}' < \text{rID}$
readFaultyLoc _t	$\stackrel{\text{def}}{=}$ $(p_t * \text{tid} \mapsto (\text{RD}, \text{loc})) \wedge \text{loc} \in \text{Nbad}$
readBenignLoc _t	$\stackrel{\text{def}}{=}$ $p_t \wedge \text{loc} \in \text{Nok} \wedge \text{tid} \in \text{dom}(\text{RdOps})$
notOutdated(wrMsg)	$\stackrel{\text{def}}{=}$ $\text{legalWrMsg}(\text{wrMsg}) \vee \text{wrMsg} > \max(\text{wrHist}(\text{loc}))$
legalWrMsg(wrMsg)	$\stackrel{\text{def}}{=}$ $\text{wrMsg} \in \text{wrHist}(\text{loc}) \uparrow \text{RdOps}(\text{tid}).\text{wrMsg}$
bottomWrMsgProp	$\stackrel{\text{def}}{=}$ $\text{RdOps}(\text{tid}).\text{loc} \in \text{Nok} \wedge \text{RdOps}(\text{tid}).\text{wrMsg} \in \text{wrHist}(\text{loc}) \wedge \exists N. \text{quorum}(N) \wedge (\forall \text{wrMsg}, t \in N. \text{sent}(\{\text{tid}\}, (\text{READ-OK}, \text{rID}, \text{wrMsg}, \text{loc}, t)) \Rightarrow \text{wrMsg} \geq_{\text{lex}} \text{RdOps}(\text{tid}).\text{wrMsg})$
presvExcept(loc, wrMsg)	$\stackrel{\text{def}}{=}$ $\forall \text{loc}', \text{wrMsg}'. (\text{loc}' \neq \text{loc} \wedge \text{loc}' \in \text{Nok}) \vee \text{wrMsg}' \neq \text{wrMsg} \Rightarrow (\text{finished}(\text{loc}', \text{wrMsg}') \Leftrightarrow \text{wrMsg}' \in \text{wrHist}(\text{loc}'))$

Fig. 48. Auxiliary assertions in defining Pre- Post-condition

7.5 Proof of Assertion Validity and Stability

In this section we show assertion validity and stability proof that is used in Proof outline. 2. The assertion validity is listed as follows:

- $p_{\text{swmr13}} \Rightarrow p_{\text{swmr14}}$ is proved in Lm. 114;

The stability proofs is listed as follows:

- in Lm. 116 we prove p_t is stable *w.r.t.* the local and the other nodes' behaviors;
- in Lm. 125 we prove $\text{bottomWrMsgProp} \wedge \text{tid} = t$ is stable *w.r.t.* it's environment.

Other pre- post-condition's stability can be directly deduced from Lm. 116 and the properties of stable predicate.

7.5.1 Proof of $p_{\text{swmr13}} \Rightarrow p_{\text{swmr14}}$.

Lemma 114. The following assertion is valid: $p_{\text{swmr13}} \Rightarrow p_{\text{swmr14}}$

PROOF. By definition, we only need to prove the following assertion is valid:

$$\text{flowWrMsg} \wedge \text{bottomWrMsgProp} \wedge \text{readRespProp} \wedge |\text{Vote}_{\text{rd}}| \geq \text{TH} \wedge \max\text{Msg} = \max(\text{WrMsgs}) \Rightarrow \text{notOutdated}(\max\text{Msg}) \wedge \text{sent}(_, (\text{READ-OK}, _, \max\text{Msg}, \text{loc}, _))$$

Given arbitrary Σ that holds for the pre-condition, by bottomWrMsgProp we know there exists v_N s.t. $\Sigma|_{\sigma_o \mapsto \sigma_o \uplus \{N \rightsquigarrow v_N\}}$ holds for the following:

$$\begin{aligned} & \text{quorum}(N) \\ & \forall \text{wrMsg}, t \in N. \text{sent}(\{\text{tid}\}, (\text{READ-OK}, \text{rID}, \text{wrMsg}, \text{loc}, t)) \\ & \Rightarrow \text{wrMsg} \geq_{\text{lex}} \text{RdOps}(\text{tid}).\text{wrMsg} \end{aligned} \quad (1)$$

By $|\text{Vote}_{\text{rd}}| \geq \text{TH}$, the relation between Nok and Nbad , and readVotesProp we know there exists $v_{N'}$ s.t. $\Sigma|_{\sigma_o \mapsto \sigma_o \uplus \{N' \rightsquigarrow v_{N'}\}}$ holds for:

$$\begin{aligned}
& N' \subseteq \text{Vote}_{\text{rd}} \\
& \text{quorum}(N') \\
& \forall t \in N'. \exists \text{wrMsg} \in \text{WrMsgs}. \text{sent}(\{\text{tid}\}, (\text{READ-OK}, \text{rID}, \text{wrMsg}, \text{loc}, t)) \quad (2)
\end{aligned}$$

Apply Lm. 115 we know

$$\Sigma \mid_{\sigma_o \mapsto \sigma_o \uplus \{N \rightsquigarrow v_N, N' \rightsquigarrow v_{N'}\}} \models \exists t. t \in N \cap N'.$$

Thus by (1) and (2) we prove

$$\Sigma \models \exists \text{wrMsg}. \text{wrMsg} \in \text{WrMsgs} \wedge \text{wrMsg} \geq_{\text{lex}} \text{RdOps}(\text{tid}).\text{wrMsg}.$$

From p_{swmr13} we know $\Sigma \models \text{maxMsg} = \max(\text{WrMsgs})$. Thus from above we have $\Sigma \models \text{maxMsg} \geq \text{RdOps}(\text{tid}).\text{wrMsg}$.

Apply readWrMsgsProp we prove

$$\text{sent}(\{\text{tid}\}, (\text{READ-OK}, \text{rID}, \text{maxMsg}, \text{loc}, _)).$$

If $\text{maxMsg} = (0, 0)$, then the goal trivially holds as we know every mapping of wrHist contains initial value $(0, 0)$. If not, apply above and $\text{loc} \in \text{Nok}$ on flowWrMsg we know $\Sigma \models \text{sent}(_, (\text{WRITE}, \text{maxMsg}, \text{loc}))$. Then apply unfinishWrites and finishedWrites we know

$$\Sigma \models \text{maxMsg} \in \text{wrHist}(\text{loc}) \vee \text{maxMsg} > \max(\text{wrHist}(\text{loc})).$$

As $\text{RdOps}(\text{tid}).\text{wrMsg} \in \text{wrHist}(\text{loc})$ (from definition of bottomWrMsgProp) we prove $\text{notOutdated}(\text{maxMsg})$, thus we conclude the goal. $\square$

Lemma 115. The following assertion is valid: $\forall N, N'. \text{quorum}(N) \wedge \text{quorum}(N') \Rightarrow \exists t. t \in N \cap N'$

PROOF. By definition, that is to take arbitrary $N_{\text{ok}}, \mathcal{N}, \mathcal{N}'$ s.t.

$$\begin{aligned}
\mathcal{N} & \subseteq N_{\text{ok}} \wedge |\mathcal{N}| \geq \lceil \frac{|N_{\text{ok}}| + 1}{2} \rceil, \\
\mathcal{N}' & \subseteq N_{\text{ok}} \wedge |\mathcal{N}'| \geq \lceil \frac{|N_{\text{ok}}| + 1}{2} \rceil,
\end{aligned}$$

and prove $|\mathcal{N} \cap \mathcal{N}'| \geq 1$. By premise we know

$$\begin{aligned}
|\mathcal{N}| + |\mathcal{N}'| - |\mathcal{N} \cap \mathcal{N}'| & \leq |N_{\text{ok}}|, \\
|\mathcal{N}| + |\mathcal{N}'| & \geq 2 \cdot \lceil \frac{|N_{\text{ok}}| + 1}{2} \rceil.
\end{aligned}$$

Thus we have

$$|\mathcal{N} \cap \mathcal{N}'| \geq 2 \cdot \lceil \frac{|N_{\text{ok}}| + 1}{2} \rceil - |N_{\text{ok}}|.$$

Discuss on $|N_{\text{ok}}|$: if $|N_{\text{ok}}|$ is odd, then we prove $|\mathcal{N} \cap \mathcal{N}'| \geq 1$; if $|N_{\text{ok}}|$ is even, then we prove $|\mathcal{N} \cap \mathcal{N}'| \geq 2$. Thus we conclude the goal. $\square$

7.5.2 Stability of p_t .

Lemma 116. $\forall t. \text{stable}(p_t, \text{InvokeAct}(\tilde{\Pi}_{\text{swmr}}.L)(t) \vee G_t^m \vee G_t^s \vee \text{nonLocal}(\bigvee_{t' \neq t} \text{InvokeAct}(\tilde{\Pi}_{\text{swmr}}.L)(t') \vee G_{t'}^m \vee G_{t'}^s \vee \text{GF}(\zeta_{\text{swmr}})))$

PROOF. By definition, Lm. 117, Lm. 118, Lm. 121, Lm. 122 and Lm. 124 immediately. $\square$

Lemma 117. $\forall t. \text{stable}(\text{absInv}, \text{InvokeAct}(\tilde{\Pi}_{\text{swmr}}.L)(t) \vee G_t^m \vee G_t^s)$

PROOF. Take arbitrary t , first from Lm. 119 and definition of G_t^m and G_t^s we know $\text{stable}(\text{absInv}, G_t^m \vee G_t^s)$. By definition of InvokeAct now we only need to prove absInv is stable *w.r.t.* $\text{IdV}(\{\text{RdOps}, \text{wrHist}\}) \wedge \text{IdM} \wedge (\text{emp} \vee t \mapsto (\text{end}, _) \ltimes t \mapsto _) * \text{IdSpec}$.

Destruct absInv and prove stability one by one. For stability of $\text{absRegs} * \text{pendReads}(\text{RdOps}) * \text{true}$, the action does not change abstract object, and the pre-state holds for $t \notin \text{dom}(\text{RdOps})$, thus the disjoint parts described by $\text{absRegs} * \text{pendReads}(\text{RdOps})$ preserves; for unfinishWrites , as the action does not change message pool, and no abstract unfinished write operation is executed during the action, we know it preserves; for finishWrites , it preserves because wrHist and the message pool are not changed. $\square$

Lemma 118. $\forall t. \text{stable}(\text{absInv}, \text{nonLocal}(\text{InvokeAct}(\tilde{\Pi}_{\text{swmr}}.L)(t) \vee G_t^m \vee G_t^s))$

PROOF. By $\text{locallIrrelevant}(\text{absInv})$, Lm. 120 and Lm. 117 immediately. $\square$

Lemma 119. $\text{stable}(\text{absInv}, \text{SendOrIncOpID} \vee \text{TrylinOrCommitRd} \vee \text{LinRdFaulty} \vee \text{SendOrAddVotes} \vee \text{LinWrHelpRd})$

PROOF. Destruct absInv and prove each stability as follows:

- $(\text{absRegs} * \text{pendReads}(\text{RdOps}) * \text{true}) \wedge \text{finishedWrites}$: we destruct action and prove them one by one:
 - For SendOrIncOpID , as this action does not change auxiliary variable wrHist , RdOps and speculation set, thus $\text{absRegs} * \text{pendReads}(\text{RdOps}) * \text{true}$ preserves trivially. For stability of finishedWrites , we only need to prove the possible message send behavior does not introduce newly finished write. By definition of MtdMsgPoolGs we know that a new WRITE or WRITE-BACK message may be sent during the action.
If a new $(\text{WRITE}, \text{wrMsg}, t)$ message is sent, as the message is not contained in message pool of pre-state, and msgPoolInv holds for both pre- and post-state, we know both message pool of the pre- and post-state does not contain any corresponding WRITE-OK message, or any WRITE-BACK , WRITE-BACK-OK message talks about (t, wrMsg) . Thus there is no new finished write in post-state.
If a new $(\text{WRITE-BACK}, _, \text{wrMsg}, t, _)$ message is sent, by msgPoolInv we know the post-state does not contain any corresponding WRITE-BACK-OK message, thus by definition we conclude the goal.
 - For TrylinOrCommitRd , as this action does not change message pool and wrHist , finishedWrites trivially preserves. To prove the rest, by definition we only need to prove $\text{absRegs} * \text{true}$ is stable *w.r.t.* $(\text{tid} \mapsto (\text{RD}, _) \ltimes \text{emp} \vee \text{emp} \ltimes \text{tid} \mapsto (\text{end}, _)) * \text{IdSpec}$, which is obvious.
 - For LinRdFaulty , trivial.
 - For SendOrAddVotes , as the action does not change speculation set or variable wrHist , RdOps , we know $\text{absRegs} * \text{pendReads}(\text{RdOps}) * \text{true}$ trivially preserves. For stability of finishedWrites , similar to the SendOrIncOpID case, we only need to prove the possible message sent does not introduce new finished write. This is shown by definition

- of SevMsgPoolGs immediately. It says a WRITE-OK or WRITE-BACK-OK message that talks about $(t, wrMsg)$ is sent by tid only if the pre-state holds for $\neg \text{checkQuorum}(\text{tid})(t, wrMsg)$, which means this sent does not turn an unfinished write into a finished one.
- For LinWrHelpRd, this action turns an unfinished write of $(t, wrMsg)$ to a finished one, $wrMsg$ is added to $wrHist(t)$ along with it. Thus finishedWrites preserves. $\text{absInv} * \text{pendReads}(\text{RdOps}) * \text{true}$ preserves by definition immediately.
 - unfinishedWrites:
 - For SendOrIncOpID or SendOrAddVotes, as speculation set and wrHist is not changed by the action, we only need to prove no newly unfinished write is introduced. By definition of MtdMsgPoolGs, a send of WRITE message guarantees unfinishWrites in post-state. As msgPoolInv holds in both pre- and post-state, we know a send of new WRITE-OK message indicate that a corresponding WRITE message in pre-state, thus the unfinished write is already in pre-state, no new unfinished write is introduced. The case discussion on SevMsgPoolGs is similar.
 - For TrylinOrCommitRd or LinRdFaulty, unfinishWrites preserves trivially as message pool, wrHist and the part of speculation set that describe pending writes is not changes.
 - For LinWrHelpRd, trivial. As the concrete unfinished write (indicated by message pool) is turned in to a finished one, and the abstract side execute the write operation correspondingly.

□

Lemma 120. $\forall p, R. \text{localIrrelevant}(p) \wedge \text{stable}(p, R) \implies \text{stable}(p, \text{nonLocal}(R))$

PROOF. By definition, given arbitrary Σ, Σ' s.t. $\Sigma \models p$ and $(\Sigma, \Sigma') \models \text{nonLocal}(R)$, we need to prove $\Sigma' \models p$. By definition of nonLocal, there exists σ, σ' s.t. $(\Sigma|_{\sigma_0 \mapsto \sigma}, \Sigma'|_{\sigma_0 \mapsto \sigma'}) \models R$. By premise $\text{localIrrelevant}(p)$ we know $\Sigma|_{\sigma_0 \mapsto \sigma} \models p$. Then by premise $\text{stable}(p, R)$ we know $\Sigma'|_{\sigma_0 \mapsto \sigma'} \models p$. At last by $\text{localIrrelevant}(p)$ we prove $\Sigma' \models p$.

□

Lemma 121. $\forall t. \text{stable}(\text{msgPoolInv} \wedge \text{tid} = t, \text{InvokeAct}(\Pi_{\text{swmr}.L})(t') \vee G_t^m \vee G_t^s)$

PROOF. Given arbitrary t , we can prove $\text{stable}(\text{msgPoolInv} \wedge \text{tid} = t, G_t^m \vee G_t^s)$ directly from definition, as the action always guarantee msgPoolInv in both pre- and post-state. For $\text{InvokeAct}(\Pi_{\text{swmr}.L})$, the stability is trivial by definition as message pool and relative variables is not changed during it.

□

Lemma 122. $\forall t, t'. t' \neq t \implies \text{stable}(\text{msgPoolInv} \wedge \text{tid} = t, \text{nonLocal}(\text{InvokeAct}(\Pi_{\text{swmr}.L})(t') \vee G_{t'}^m \vee G_{t'}^s))$

PROOF. By Lm. 123 and Lm. 120 we know

$$\forall t. \text{stable}(\text{uniqueWrMsg} \wedge \text{writeRespCount} \wedge \text{msgFlow}, \text{nonLocal}(G_t^m \vee G_t^s)).$$

Also by definition we know

$$\forall t. \text{stable}(\text{uniqueWrMsg} \wedge \text{writeRespCount} \wedge \text{msgFlow}, \text{nonLocal}(\text{InvokeAct}(\Pi_{\text{swmr}.L})(t))).$$

Thus we only need to prove the rest is stable. Take arbitrary t' , to prove the rest is stable w.r.t. $\text{nonLocal}(\text{InvokeAct}(\Pi_{\text{swmr}.L})(t'))$ is trivial since the action does not change message pool and local state, now we focus on the stability under $\text{nonLocal}(G_{t'}^m \vee G_{t'}^s)$ when $t' \neq t$:

- $\text{maxSentWID}(\text{wID})$ and $\text{maxSentRID}(\text{rID})$: By definition of nonLocal we know local variable wID and rID is not changed during the action, thus we only need to prove there is no WRITE, WRITE-BACK or READ message sent by t during the action, which by definition and $t \neq t'$ immediately.
- mRegSource : By definition of nonLocal we know local variable mReg is not changed, as the message pool never shrink, we prove the goal.
- respondHasUpdate : By definition we know mReg is not changed, thus we only need to prove that there is no newly WRITE-OK or WRITE-BACK-OK message sent by t during the action, which by definition and $t \neq t'$ immediately.

□

Lemma 123. $\text{stable}(\text{uniqueWrMsg} \wedge \text{writeRespCount} \wedge \text{msgFlow}, \text{SendOrIncOpID} \vee \text{TrylinOrCommitRd} \vee \text{LinRdFaulty} \vee \text{SendOrAddVotes} \vee \text{LinWrHelpRd})$

PROOF. The goal can be proved directly from definition, as the action always guarantee msg-PoolInv in both pre- and post-state. □

Lemma 124 (Object invariant stable *w.r.t* faulty nodes' behavior).

$\forall t. \text{stable}(p_t, \text{nonLocal}(\text{GF}(\zeta_{\text{swmr}})))$

PROOF. Given arbitrary t , by Lm. 110 we only need to prove $\text{stable}(p_t, \text{GF}(\zeta_{\text{swmr}}))$. By definition we know the action only change message pool, thus we only need to prove the following assertion is stable:

- For unfinishWrites : as the speculation set is not changed during the action, we only need to prove that there is no new WRITE message sent by benign nodes during the action, which by definition of ζ_{swmr} immediately.
- For finishedWrites , as auxiliary variable wrHist is not change during the action, we only need to prove that there does not exists any wrMsg and loc s.t. $\text{wrMsg} \notin \text{wrHist}(\text{loc})$ and $\text{loc} \in \text{Nok}$ holds for the pre-state, and $\text{finished}(\text{loc}, \text{wrMsg})$ holds for the post-state. We prove this by contradiction, assume there exists such wrMsg, loc so that the first condition holds, then prove that the second is false.

By assumption we know $\text{finished}(\text{loc}, \text{wrMsg})$ does not holds for pre-state, thus by definition we only need to prove that take arbitrary $t \in \text{Nok}$, if $\text{respondWs}(t, \text{loc}, \text{wrMsg})$ is not holds for the pre-state, then it's also not holds for the post-state. This can be proved by definition of ζ_{swmr} immediately.

- For $\text{maxSentWID}(\text{wID}) \wedge \text{maxSentRID}(\text{rID})$, if a WRITE, WRITE-BACK or READ message tagged by a benign ID tid is sent during $\text{GF}(\zeta_{\text{swmr}})$, then we need to prove this message is already in pre-state's message pool, which by definition of ζ_{swmr} immediately.
- For mRegSource , as mReg does not change during action, this preserves trivially in post-state.
- For uniqueWrMsg , if the action send $(\text{WRITE}, \text{wrMsg}, \text{loc})$ and $\text{loc} \in \text{Nok}$, then by definition of ζ_{swmr} we know at least one of the following has been sent in pre-state:
 - $(\text{WRITE}, \text{wrMsg}, \text{loc})$: then uniqueWrMsg preserves trivially.
 - $(\text{WRITE-BACK}, _, \text{wrMsg}, \text{loc}, _)$ or $(\text{READ-OK}, _, \text{wrMsg}, \text{loc}, _)$: then by flowWrMsg we know $(\text{WRITE}, \text{wrMsg}, \text{loc})$ has been sent, uniqueWrMsg preserves trivially.

Thus we conclude the case.

- For respondHasUpdated , as $\text{tid} \in \text{Nok}$, if a new WRITE or WRITE-BACK message is sent during $\text{GF}(\zeta_{\text{swmr}})$, we need to prove that no corresponding WRITE-OK or WRITE-BACK-OK message sent by tid is already in pre-state's message pool, which can be proved by flowWROK and flowWBOK immediately.

- For writeRespCount, as absWrCnt does not change during action, this preserves trivially.
- For msgFlow, by definition flowWROK, flowWBOK, flowRDOK stable trivially due to definition of ζ_{swmr} . We only need to prove stability of flowWrMsg, i.e. introduce arbitrary $loc, rID, wrMsg, t$, if $loc \in \text{Nok}$, a (READ-OK, $rID, wrMsg, loc, t$) or a (WRITE-BACK, $rID, wrMsg, loc, t$) is sent during $\text{GF}(\zeta_{\text{swmr}})$, we need to prove $\text{sent}(_, (\text{WRITE}, wrMsg, loc))$ holds for the post-state. By definition of ζ_{swmr} , we discuss on cases:
 - If $t \in \mathcal{N}_{\text{bad}} \wedge loc \in \text{Nbad}$, contradict to condition $loc \in \text{Nok}$, thus we have the goal immediately.
 - If $t \in \mathcal{N}_{\text{bad}} \wedge (wrMsg, loc) \in \text{sentWrMsg}(M)$, then by definition of $\text{sentWrMsg}(M)$ we discuss on cases:
 - * If $(_, (\text{WRITE}, wrMsg, loc)) \in M$, then as action does not remove message from message pool, we have our goal.
 - * If $(_, (\text{READ-OK}, _, wrMsg, loc, _)) \in M \vee (_, (\text{WRITE-BACK}, _, wrMsg, loc, _)) \in M$, then apply flowWrMsg on it we have the goal.
 - If the message already in pre-state's message pool, then apply flowWrMsg we have the goal immediately.

□

7.5.3 Stability of bottomWrMsgProp.

Lemma 125. $\forall t. \text{stable}(\text{absInV} \wedge \text{bottomWrMsgProp} \wedge \text{tid} = t, \text{InvokeAct}(\Pi_{\text{swmr}}.L)(t) \vee G_t^s \vee \text{nonLocal}(R_t))$

PROOF. Take arbitrary t, t', Σ, Σ' s.t. $\Sigma \models \text{bottomWrMsgProp} \wedge \text{tid} = t$ and at least one of the following holds

$$\begin{array}{ll}
 (\Sigma, \Sigma') \models \text{InvokeAct}(\Pi_{\text{swmr}}.L)(t) & (case1) \\
 (\Sigma, \Sigma') \models G_t^s & (case2) \\
 t' \neq t \wedge (\Sigma, \Sigma') \models \text{nonLocal}(G_{t'}^m) & (case3) \\
 t' \neq t \wedge (\Sigma, \Sigma') \models \text{nonLocal}(G_{t'}^s) & (case4) \\
 t' \neq t \wedge (\Sigma, \Sigma') \models \text{nonLocal}(\text{InvokeAct}(\Pi_{\text{swmr}}.L)(t')) & (case5) \\
 (\Sigma, \Sigma') \models \text{GF}(\zeta_{\text{swmr}}) & (case6)
 \end{array}$$

we need to prove

$$\Sigma' \models \text{absInV} \wedge \text{bottomWrMsgProp} \wedge \text{tid} = t.$$

As we have proved stability of absInV w.r.t. environment in Lm. 117, Lm. 118 and Lm. 124, we focus on proving $\Sigma' \models \text{bottomWrMsgProp}$ as follows:

- If (case1) holds, the goal holds immediately as the action does not change RdOps, wrHist, or add new messages to original message pool.
- If (case2) holds, by definition, as G_t^s does not change RdOps, loc, and they only add write messages into write history set wrHist, we know $\Sigma' \models \text{RdOps}(\text{tid}).wrMsg \in \text{wrHist}(\text{loc})$. Now we only need to prove the remaining of bottomWrMsgProp is preserved in Σ' . As $\Sigma \models \text{absInV}$, apply $\Sigma \models \text{RdOps}(\text{tid}).wrMsg \in \text{wrHist}(\text{loc})$ on $\Sigma \models \text{finishedWrites}$ we know $\Sigma \models \text{finished}(\text{loc}, \text{RdOps}(\text{tid}).wrMsg)$, that is there exists v_N s.t. $\Sigma|_{\sigma_o \mapsto \sigma_o \cup \{N \rightsquigarrow v_N\}}$ holds for

$$\text{quorum}(N) \wedge \forall t' \in N. \text{respondWs}(t', \text{loc}, \text{RdOps}(\text{tid}).wrMsg) \quad (1)$$

As the message pool never shrink, loc and tid does not change during G_t^s , then we know $\Sigma'|_{\sigma_o \mapsto \sigma_o \cup \{N \rightsquigarrow v_N\}}$ also holds for above.

Now we only need to prove $\Sigma' |_{\sigma_o \mapsto \sigma_o \uplus \{N \leadsto v_N\}}$ holds for

$$\begin{aligned} & \forall wrMsg, t' \in N. \text{sent}(\{tid\}, (\text{READ-OK}, rID, wrMsg, loc, t')) \\ & \Rightarrow wrMsg \geq_{\text{lex}} \text{RdOps}(tid).wrMsg. \end{aligned}$$

Discuss on t , if $t \notin v_N$, then this preserve trivially; if $t \in v_N$, and if G_t^s send READ-OK message following the definition of SevMsgPoolGs, s.t. $\Sigma' |_{\sigma_o \mapsto \sigma_o \uplus \{N \leadsto v_N\}} \models \text{sent}(_, (\text{READ-OK}, rID, mReg(loc), loc, t))$, then we need to prove

$$\Sigma' \models mReg(loc) \geq_{\text{lex}} \text{RdOps}(tid).wrMsg.$$

As $t \in v_N$, from (1) we know Σ holds for

$$\text{respondWs}(t, loc, \text{RdOps}(tid).wrMsg)$$

As RdOps, loc is not changed during G_t^s , we know the above is preserved in Σ' .

Then apply respondHasUpdated we know $\Sigma' \models mReg(loc) \geq_{\text{lex}} \text{RdOps}(tid).wrMsg$, thus the goal is proved.

- If (*case3*) holds, then as the method thread from the other nodes only record bottom write message to their own field in $\text{RdOps}(tid)$, and it does not change $wrHist$, we know $\models \text{RdOps}(tid).wrMsg \in wrHist(loc)$ is preserved in Σ' . The rest of it preserve as $\text{nonLocal}(G_{t'}^s)$ does not add new READ-OK message into message pool or change RdOps (since $t \neq t'$).
- If (*case4*) holds, from definition as $\text{nonLocal}(G_{t'}^s)$ does not change RdOps , $wrHist$, and only adds write messages into $wrHist$, we know $\text{RdOps}(tid).wrMsg \in wrHist(loc)$ is preserved in Σ' .

From the definition of nonLocal we know there exists σ, σ' s.t.

$$\begin{aligned} \Sigma. \sigma_o &= \Sigma'. \sigma_o \\ (\Sigma |_{\sigma_o \mapsto \sigma}, \Sigma' |_{\sigma_o \mapsto \sigma'}) &\models G_{t'}^s \quad (2) \end{aligned}$$

As absInv is local-irrelevant, we know $\Sigma |_{\sigma_o \mapsto \sigma} \models \text{finishedWrites}$. Apply $\Sigma \models \text{RdOps}(tid).wrMsg \in wrHist(loc)$ we know there exists v_N s.t. $\Sigma |_{\sigma_o \mapsto \sigma_o \uplus \{N \leadsto v_N\}}$ holds for

$$\text{quorum}(N) \wedge \forall t' \in N. \text{respondWs}(t', loc, \text{RdOps}(tid).wrMsg)$$

As the message pool never shrink, loc and tid does not change during $\text{nonLocal}(G_{t'}^s)$, we know

$$\Sigma' |_{\sigma_o \mapsto \sigma_o \uplus \{N \leadsto v_N\}} \models \text{quorum}(N) \wedge \forall t' \in N. \text{respondWs}(t', loc, \text{RdOps}(tid).wrMsg) \quad (3)$$

Now we only need to prove $\Sigma' |_{\sigma_o \mapsto \sigma_o \uplus \{N \leadsto v_N\}}$ holds for

$$\begin{aligned} & \forall wrMsg, t' \in N. \text{sent}(\{tid\}, (\text{READ-OK}, rID, wrMsg, loc, t')) \\ & \Rightarrow wrMsg \geq_{\text{lex}} \text{RdOps}(tid).wrMsg \end{aligned}$$

Let $t = \Sigma. \sigma_o, rID = \Sigma. \sigma_o(rID)$ and $loc = \Sigma. \sigma_o(loc)$. Discuss on t' : if $t' \notin v_N$, then this preserve trivially; if $t' \in v_N$, and if t' send READ-OK message in response for READ request identified by loc and rID to t following the definition of SevMsgPoolGs, i.e.

$$(\Sigma |_{\sigma_o \mapsto \sigma}, \Sigma' |_{\sigma_o \mapsto \sigma'}) \models \text{SendAct}(\{t\}, (\text{READ-OK}, rID, mReg(loc), loc, t')).$$

Let $wrMsg = \sigma'(mReg(loc))$ we have:

$$(\Sigma |_{\sigma_o \mapsto \sigma}, \Sigma' |_{\sigma_o \mapsto \sigma'}) \models \text{SendAct}(_, (\text{READ-OK}, rID, wrMsg, loc, t')) \quad (4)$$

Thus we know $(\Sigma, \Sigma') \models \text{SendAct}(_, (\text{READ-OK}, rID, wrMsg, loc, t'))$. Now we need to prove

$$\Sigma' \models wrMsg \geq_{\text{lex}} \text{RdOps}(t).wrMsg.$$

From (3) and $t' \in v_N$ we know

$$\begin{aligned} \Sigma' |_{\sigma_o \mapsto \sigma'} &\models \text{respondWR}(t', loc, \text{RdOps}(t).wrMsg) \\ &\vee \text{respondWB}(t', loc, \text{RdOps}(t).wrMsg) \end{aligned} \quad (5)$$

By (4) we know $\Sigma' |_{\sigma_o \mapsto \sigma'} \models \text{sent}(_, (\text{READ-OK}, rID, \text{mReg}(loc), loc, t'))$. By (2) we know $\Sigma' |_{\sigma_o \mapsto \sigma'} \models \text{inv}$, then apply $\Sigma' |_{\sigma_o \mapsto \sigma'} \models \text{respondHasUpdated}$ on (5) we prove

$$\Sigma' |_{\sigma_o \mapsto \sigma'} \models \text{mReg}(loc) \geq_{\text{lex}} \text{RdOps}(t).wrMsg.$$

Thus we prove $\Sigma' \models wrMsg \geq_{\text{lex}} \text{RdOps}(t).wrMsg$.

- If (case5) holds, the proof is similar to the proof of (case1).
- If (case6) holds, as $\text{GF}(\zeta_{\text{swmr}})$ does not change local state or auxiliary state, thus $\text{RdOps}(tid).wrMsg \in \text{wrHist}(loc)$ preserve trivially. As faulty nodes can only send READ-OK message with their own ID, which did not add new benign READ-OK message into message pool, the remaining preserve trivially, or send READ-OK message with benign ID that in original message pool, the remaining preserves trivially.

□

8 Related Work and Conclusion

Correctness criteria for BFT protocols. Byzantine linearizability has been used as the correctness criterion in many works about BFT protocol design (e.g. [Cohen and Keidar 2021; Kowalski et al. 2024]). It also serves as the basis for many other stronger correctness criteria. For instance, Malkhi et al. [1998] propose Byznearizability, which is Byzantine linearizability plus an additional liveness condition called “eventual recovery”: if faulty nodes stop their concrete execution, their abstract effects should be finite and the system can eventually recover. Liskov and Rodrigues [2006] propose BFT-linearizability, which further strengthens Byznearizability by requiring “bounded recovery”: the number of abstract operations that model the faulty nodes’ effects after the faulty nodes stop is bounded by some constant. Hendricks [2009] takes Byzantine linearizability as the base criterion, and adds various kinds of recovery conditions. Kshemkalyani et al. [2025] propose Byzantine register linearizability that strengthens Byzantine linearizability with some conditions designed specifically for registers. Our work focuses on verifying Byzantine linearizability. It may serve as the starting point for us to support these stronger correctness criteria in the future.

Verification of BFT protocols. Castro and Liskov [1999] prove the correctness of the PBFT algorithm, by modeling the concrete implementation and the abstract specification using automata and constructing simulation relations between the automata. In a sense, our program logic can be viewed as a proof method with higher-level abstraction where the details of constructing simulation relations are hidden by the logic soundness proofs. It is syntax-directed, and does not need users to understand the underlying semantic models. It is also more general, and is not tied to a specific BFT protocol. We are interested in applying our logic to verify the PBFT algorithm in the future.

Charron-Bost et al. [2011] verify several BFT consensus algorithms in Isabelle/HOL. They define two semantic models of executions, and prove a kind of contextual refinement between the two execution models, which enables them to verify the algorithms in the coarse-grained execution model only. In our work, we prove a contextual refinement between the concrete algorithms and their atomic specifications. This would also enable us to replace the concrete component algorithms by their atomic specifications when verifying a larger composite algorithm.

Honoré et al. [2024] propose an abstract model called AdoB that can represent both benign-fault and Byzantine-fault-tolerant protocols. They prove safety and liveness of AdoB, and also prove refinement between concrete protocols and AdoB. Later Qiu et al. [2024] propose the LiDO model which is an improvement over the AdoB model and provides better support for liveness refinement. Their refinement is largely similar to Byzantine linearizability, but against a more concrete abstraction – one expressive enough to capture behaviors for both benign-fault and Byzantine-fault-tolerance. Unlike these works, our work focus on proposing a program logic

that supports syntax-directed verification, for verifying Byzantine linearizability (equivalent to a contextual refinement) of the protocols w.r.t. atomic specifications. When the η parameter is restricted to require the number of (Byzantine) faulty nodes to be zero, Byzantine linearizability degenerates to linearizability that applies to benign-fault tolerant protocols. That said, we do not support liveness reasoning as in their works. To reason about liveness, we need to extend our work to support infinite traces, which is left as future work.

Cirisci et al. [2023] share similar goals with Honoré et al. [2024] and Qiu et al. [2024], and present another abstract model called QTree. They prove the refinement between protocol executions and QTree executions by instrumenting the protocols with LPs, as in the linearizability proofs for concurrent objects with *fixed* LPs. In our work, we also rely on code instrumentation for proving refinement, but our full logic supports non-fixed LPs as well as fixed ones. It would be interesting to adapt our logic to verify refinement with QTree specifications.

There are also a huge number of works that verify BFT protocols but do not prove refinement. For instance, Rahli et al. [2018] prove the agreement property of the PBFT algorithm in Coq, by modeling the executions of the algorithm as collections of ordered events. Berkovits et al. [2019] propose a deductive verification method for BFT protocols by modeling the protocols in first-order logic. Losa and Dodds [2020] also use first-order logic to model and verify the Stellar consensus protocol. Bertrand et al. [2022] verify the safety and liveness properties of the DBFT consensus algorithm using model checking techniques. Carr et al. [2022] present an abstract model for the HotStuff/LibraBFT protocol, and prove correctness properties on the abstract model in Agda. Zhao et al. [2024] present a compositional verification framework in Coq, supporting invariant-based safety proofs and TLA-based liveness proofs for BFT protocols. These works do not prove (Byzantine) linearizability or refinement w.r.t. abstract specifications. Our program logic supports syntax-directed verification of refinement, but we have not automated or mechanized the proofs.

Conclusion. We propose a program logic for verifying Byzantine linearizability of BFT protocols. We reason about the effects of faulty nodes by introducing a helping mechanism. We prove that Byzantine linearizability is equivalent to a contextual refinement, so our logic is also a sound proof method for refinement in the presence of Byzantine faulty nodes. We have applied our logic to verify two typical BFT protocols.

Acknowledgments

We thank anonymous referees for their suggestions and comments on earlier versions of this paper. This work is supported in part by National Natural Science Foundation of China (NSFC) under Grant No. 62232015.

References

- Ittai Abraham, Guy Gueta, Dahlia Malkhi, Lorenzo Alvisi, Rama Kotla, and Jean-Philippe Martin. 2017. Revisiting Fast Practical Byzantine Fault Tolerance. arXiv:1712.01367 [cs.DC]
- Idan Berkovits, Marijana Lazić, Giuliano Losa, Oded Padon, and Sharon Shoham. 2019. Verification of Threshold-Based Distributed Algorithms by Decomposition to Decidable Logics. In *Computer Aided Verification*. Springer International Publishing, 245–266.
- Nathalie Bertrand, Vincent Gramoli, Igor Konnov, Marijana Lazić, Pierre Tholoniati, and Josef Widder. 2022. Holistic Verification of Blockchain Consensus. In *36th International Symposium on Distributed Computing (DISC 2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 246)*, Christian Scheidele (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 10:1–10:24. doi:10.4230/LIPIcs.DISC.2022.10
- Harold Carr, Christa Jenkins, Mark Moir, Victor Cacciari Miraldo, and Lisandra Silva. 2022. Towards Formal Verification of HotStuff-Based Byzantine Fault Tolerant Consensus in Agda. In *NASA Formal Methods: 14th International Symposium, NFM 2022, Pasadena, CA, USA, May 24–27, 2022, Proceedings* (Pasadena, CA, USA). Springer-Verlag, Berlin, Heidelberg, 616–635. doi:10.1007/978-3-031-06773-0_33

- Miguel Castro and Barbara H. Liskov. 1999. *A Correctness Proof for a Practical Byzantine-Fault-Tolerant Replication Algorithm*. Technical Report. USA. <https://publications.csail.mit.edu/lcs/pubs/pdf/MIT-LCS-TM-597.pdf>
- Bernadette Charron-Bost, Henri Debrat, and Stephan Merz. 2011. Formal Verification of Consensus Algorithms Tolerating Malicious Faults. In *Stabilization, Safety, and Security of Distributed Systems*. 120–134.
- Berk Cirisci, Constantin Enea, and Suha Orhun Mutluergil. 2023. Quorum Tree Abstractions of Consensus Protocols. In *Proceedings of 32nd European Symposium on Programming (ESOP 2023)* (Paris, France). Springer-Verlag, Berlin, Heidelberg, 337–362. doi:10.1007/978-3-031-30044-8_13
- Shir Cohen and Idit Keidar. 2021. Tame the Wild with Byzantine Linearizability: Reliable Broadcast, Snapshots, and Asset Transfer. In *35th International Symposium on Distributed Computing (DISC 2021) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 209)*, Seth Gilbert (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 18:1–18:18. doi:10.4230/LIPIcs.DISC.2021.18
- Xinyu Feng. 2009. Local Rely-Guarantee Reasoning. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Savannah, GA, USA) (POPL '09). Association for Computing Machinery, New York, NY, USA, 315–327. doi:10.1145/1480881.1480922
- Danny Hendler, Nir Shavit, and Lena Yerushalmi. 2004. A scalable lock-free stack algorithm. In *SPAA'04*. 206–215.
- James Hendricks. 2009. *Efficient Byzantine Fault Tolerance for Scalable Storage and Services*. Ph. D. Dissertation. Carnegie Mellon University.
- Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Trans. Program. Lang. Syst.* 12, 3 (jul 1990), 463–492. doi:10.1145/78969.78972
- Wolf Honoré, Longfei Qiu, Yoonseung Kim, Ji-Yong Shin, Jieung Kim, and Zhong Shao. 2024. AdoB: Bridging Benign and Byzantine Consensus with Atomic Distributed Objects. *Proc. ACM Program. Lang.* 8, OOPSLA1 (2024), 419–448.
- Cliff B. Jones. 1983. Tentative Steps Toward a Development Method for Interfering Programs. *ACM Trans. Program. Lang. Syst.* 5, 4 (oct 1983), 596–619. doi:10.1145/69575.69577
- Ramakrishna Kotla, Lorenzo Alvisi, Mike Dahlin, Allen Clement, and Edmund Wong. 2010. Zyzzyva: Speculative Byzantine Fault Tolerance. *ACM Trans. Comput. Syst.* 27, 4, Article 7 (jan 2010), 39 pages. doi:10.1145/1658357.1658358
- Vincent Kowalski, Achour Mostéfaoui, and Matthieu Perrin. 2024. Atomic Register Abstractions for Byzantine-Prone Distributed Systems. In *27th International Conference on Principles of Distributed Systems (OPODIS 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 286)*, Alysson Bessani, Xavier Défago, Junya Nakamura, Koichi Wada, and Yukiko Yamauchi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 35:1–35:20. doi:10.4230/LIPIcs.OPODIS.2023.35
- Ajay D. Kshemkalyani, Manaswini Piduguralla, Sathya Peri, and Anshuman Misra. 2025. On Constructing a Byzantine Linearizable SWMR Atomic Register from SWSR Atomic Registers. In *Proceedings of the 26th International Conference on Distributed Computing and Networking (ICDCN '25)*. Association for Computing Machinery, New York, NY, USA, 239–243. doi:10.1145/3700838.3700839
- Hongjin Liang and Xinyu Feng. 2013. Modular Verification of Linearizability with Non-Fixed Linearization Points. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Seattle, Washington, USA) (PLDI '13). Association for Computing Machinery, New York, NY, USA, 459–470. doi:10.1145/2491956.2462189
- Barbara Liskov and Rodrigo Rodrigues. 2006. Tolerating Byzantine Faulty Clients in a Quorum System. In *Proceedings of the 26th IEEE International Conference on Distributed Computing Systems (ICDCS 2006)*. IEEE Computer Society.
- Giuliano Losa and Mike Dodds. 2020. On the Formal Verification of the Stellar Consensus Protocol. In *2nd Workshop on Formal Methods for Blockchains (FMBC 2020) (Open Access Series in Informatics (OASISs), Vol. 84)*, Bruno Bernardo and Diego Marmosoler (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 9:1–9:9. doi:10.4230/OASISs.FMBC.2020.9
- Dahlia Malkhi, Mike Reiter, and Nancy Lynch. 1998. A correctness condition for memory shared by byzantine processes. *Unpublished manuscript*, Sept 98 (1998).
- Longfei Qiu, Yoonseung Kim, Ji-Yong Shin, Jieung Kim, Wolf Honoré, and Zhong Shao. 2024. LiDO: Linearizable Byzantine Distributed Objects with Refinement-Based Liveness Proofs. *Proc. ACM Program. Lang.* 8, PLDI (2024), 1140–1164.
- Vincent Rahli, Ivana Vukotic, Marcus Völpl, and Paulo Esteves Verissimo. 2018. Velisarios: Byzantine Fault-Tolerant Protocols Powered by Coq. In *European Symposium on Programming (ESOP 2018)*. Springer-Verlag, 619–650.
- Sam Toueg. 1984. Randomized Byzantine Agreements. In *Proceedings of the Third Annual ACM Symposium on Principles of Distributed Computing (PODC '84)*. Association for Computing Machinery, New York, NY, USA, 163–178. doi:10.1145/800222.806744
- Aaron Joseph Turon, Jacob Thamsborg, Amal Ahmed, Lars Birkedal, and Derek Dreyer. 2013. Logical relations for fine-grained concurrency. In *POPL*. ACM, 343–356.
- Viktor Vafeiadis. 2008. *Modular fine-grained concurrency verification*. Ph. D. Dissertation. University of Cambridge, Computer Laboratory.

Qiyuan Zhao, George Pirlea, Karolina Grzeszkiewicz, Seth Gilbert, and Ilya Sergey. 2024. Compositional Verification of Composite Byzantine Protocols. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (Salt Lake City, UT, USA) (CCS'24). Association for Computing Machinery, New York, NY, USA, 34–48. doi:10.1145/3658644.3690355